

# NetCore Tetra Systemhandbuch

Architektur Installation Konfiguration Betrieb und Instandhaltung

Projektstand aus dem öffentlichen Repository und den bereitgestellten ETSI Unterlagen

27. September 2026

# Inhaltsverzeichnis

## Kapitelübersicht

- 1 Projektstand und Systemgrenzen
- 2 Architektur und Datenwege
- 3 Funktionen der Basisstation und des Netzes
- 4 Hardware und Funkaufbau
- 5 Netzplanung und Ports
- 6 Installation der Basisstation
- 7 Installation der Core Dienste
- 8 Konfiguration
- 9 Normalbetrieb und Bedienabläufe
- 10 Wartung Reparatur und Wiederherstellung
- 11 Fehlersuche
- 12 Tests und Abnahme
- 13 Bekannte Grenzen und Ausbau
- 14 Diensthandbuch und Konfigurationsindex
- 15 Protokolle Pfade Quellen und Checklisten
- 16 Schnittstellenatlas und API-Verträge
- 17 Backend-Konfiguration im Detail
- 18 TBS-Konfiguration und RF-Parameter
- 19 Dienstbetrieb: 25 konkrete Arbeitsblätter
- 20 Fehlersuche und Reparatur nach Symptomen
- 21 Protokollinventur, Zustände und Konformitätsgrenzen
- 22 Integrationstests und On-Air-Abnahme
- 23 Wartung, Änderungsnachweis und Quellnavigation
- 24 Praxisabläufe: vom Aufbau bis zur Wiederherstellung
- 25 Installationsbuch und Standortanpassung
- 26 Technische Datenblätter für Schnittstellen und Zustände
- 27 Kopierbare Prüf- und Einsatzblätter
- 28 Bekannte Abweichungen im untersuchten Quellstand
- 29 Einsatzreferenz für die ersten fünf Minuten
- 30 Aktualisierung auf v1.9.0
- 31 Roadmap und Releases

## Leseschlüssel

Kapitel 30 und 31 ergänzen den aktuellen Releasebetrieb und die vollständige Release-/Roadmapübersicht. Die Kapitel 1 bis 13 führen durch System, Aufbau, Installation und Betrieb. Kapitel 14 bis 18 sind die ausführliche Dienst-, API- und Schlüsselreferenz. Kapitel 19 bis 29 verbinden diese Daten mit Arbeitsabläufen, Tests, Reparaturen und kurzen Einsatzblättern. Ein Port oder Beispielwert ist immer gegen die aktive Standortkonfiguration zu prüfen. Ein angegebener Quellpfad bezieht sich auf den festgehaltenen Commit und lässt sich im Repository direkt aufsuchen.

| Kennzeichnung              | Lesart                                                                                |
|----------------------------|---------------------------------------------------------------------------------------|
| GET, POST und Pfad         | deklarierter API-Vertrag des Quellstands;<br>Requestschema am laufenden Dienst prüfen |
| <HOST>, <PORT>, <UNIT>     | vor einem Kommando durch reale Standortwerte<br>ersetzen                              |
| Beispielwert               | aus einer eingetragenen Vorlage; kein<br>zusicherter Runtime-Default                  |
| <i>On Air nachgewiesen</i> | nur mit Funkgerät, SDR-/Messdaten und<br>dokumentiertem Build                         |
| ready=503                  | Prozess kann laufen, während eine fachliche<br>Abhängigkeit fehlt                     |

Für eine akute Störung bei Kapitel 29 beginnen; für einen Neuaufbau Kapitel 25 und das Arbeitsblatt in Kapitel 27 verwenden. Die Kapitelüberschriften dienen als Navigation; PDF und Word besitzen eigene Seitenumbrüche.

### **Ausgabe 2.1 · Quellenstand 27. September 2026 · Repository** main bei 086a81fa8820ef579c475a65a38e3d23644c52f0

Dieses Handbuch beschreibt NetCore Tetra als Gesamtsystem: Basisstation, Funkseite, verteilte Backend-Dienste, Leitstellenoberfläche, Audio, Paketdaten, SIP, MQTT, Provisionierung sowie Betrieb und Fehlersuche. Es richtet sich an Menschen, die die Anlage aufbauen, konfigurieren, testen, überwachen oder reparieren. Der Text führt zuerst durch die Systemgrenzen und den Aufbau, danach durch die Inbetriebnahme und schließlich durch den laufenden Betrieb. Die Konfigurationsreferenz am Ende dient als Nachschlagewerk.

Die belegbare Grundlage ist der öffentlich abrufbare Repository-Stand vom 27. September 2026. Die bereitgestellten ETSI-PDFs sind normative Hintergrundquellen; ihre Existenz belegt keine vollständige Implementierung oder Konformität. Beobachtungen aus bisherigen Projektgesprächen werden als Betriebsbeobachtung bezeichnet. IP-Adressen, ISSIs, Frequenzen und Zugangsdaten in Beispielen sind zu ersetzen und gegen die reale Installation zu prüfen. Dieses Buch enthält bewusst keine übernommenen Geheimnisse aus Beispielkonfigurationen.

**Statusbegriffe.** *Im Code vorhanden* bedeutet, dass Modul, Konfiguration oder Dienst im untersuchten Commit liegt. *Im Labortest prüfbar* bezeichnet eine vorgesehene Teststrecke. *On Air nachgewiesen* setzt ein dokumentiertes Ergebnis mit realem Funkgerät, SDR, Messung und Softwarestand voraus. *Geplant* ist ein Architekturwunsch ohne behaupteten Betriebsnachweis. Diese Begriffe gelten durchgehend.

### Dokumentsteuerung

| Merkmal            | Festlegung                                                                                                                        |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Quellbasis         | <a href="https://github.com/JanHG98/netcore-tetra">https://github.com/JanHG98/netcore-tetra</a> ,<br>Branch main, Commit 086a81fa |
| Taglage beim Abruf | v1.9.0 aktuell; nur Remote-Branch main;<br>historische Stände als Archiv-Tags erhalten                                            |

| Merkmal               | Festlegung                                                                                                    |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Referenzkonfiguration | Docs/<br>basisstation.config.sanitized.example.toml<br>und deploy/open-lab/inventory.example.toml             |
| Runtime-Inventory     | 25 Dienste im Open Lab; Provisioning Core<br>zusätzlich außerhalb des Deploy-Inventory                        |
| Statische Prüfung     | Inventory-Validierung erfolgreich; Full-System-<br>Audit mit drei dokumentierten Befunden<br>fehlgeschlagen   |
| Echte Gesamtanlage    | Kein Zugriff auf laufende TBS, LXC, SDR,<br>Funkgeräte oder Live-Logs für diese Ausgabe                       |
| Änderungsprinzip      | Bei jedem neuen Commit zuerst Version,<br>Inventory, Konfigurationsschema und<br>Abnahmebefunde aktualisieren |



# 1 Projektstand und Systemgrenzen

NetCore Tetra verbindet eine lokale, selbstständig laufende TETRA-Basisstation mit einer verteilten Steuerungs- und Anwendungsebene. Die TBS besitzt PHY, MAC, LLC, MLE, lokale MM- und CMCE-Verfahren, SDR-Ansteuerung, die unmittelbare Luftschnittstelle und die zeitkritische Rufabwicklung. Backend-Dienste übernehmen netzweite Teilnehmer-, Gruppen-, Mobilitäts-, Medien-, SDS-, Paketdaten- und Verwaltungsfunktionen. Der Node Gateway vermittelt den Zustand zwischen TBS und Diensten. Ein Ausfall des zentralen Netzes soll die lokale Funkzelle nicht beenden.

Der Status bleibt Open Lab. Viele Management-WebUIs sind ohne Login, Management-Token und TLS vorgesehen. Die neue Warnzentrale verlangt dagegen standardmäßig ein Token für ihre Fach-API. Das geschützte Managementnetz bleibt erforderlich; ein produktiver Sicherheitsnachweis liegt nicht vor.

Diese Ausgabe dokumentiert Release **v1.9.0** auf `main` bei `086a81fa8820ef579c475a65a38e3d23644c52f0`. Stichtag ist der **27. September 2026 in Europe/Berlin**; die Veröffentlichung erfolgte am 26. September um 23:02 UTC. Die neue Warnzentrale erweitert das Open-Lab-Inventory auf **25 deploybare Dienste**. `system-backend/services.toml` zählt **26 Einträge einschließlich der Bibliothek shared**. Provisioning Core ist ein zusätzlicher Workspace-Dienst und steht in keiner dieser beiden Listen. Für die konkrete Anlage sind die tatsächlich ausgerollten Dateien zu prüfen.

**Aktueller Audit-Befund.** Die Referenzprüfung ergab OK: 25 services, contract=netcore.v1, mode=open\_lab. Der Full-System-Audit ergab **FAIL mit drei Befunden**: Die Prüfliste erwartet noch 24 Dienste; alert-service bietet die erwarteten /metrics- und /openapi.json-Marker nicht; die Root-TBS-Konfiguration verwendet für das Gateway 10.0.1.179, das Inventory 10.0.20.10. Der separat bestätigte GitHub-Workflow **Warning service** für diesen Commit ist erfolgreich. Diese Aussagen betreffen Quellstand, lokale statische Prüfung und CI; laufende TBS, LXC und Funkgeräte wurden nicht geprüft.

## 1.1 Was diese Ausgabe belegt

| Gegenstand              | Befund                                                              | Grenze                                                                 |
|-------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| 25 Backend-Dienste      | Inventory validiert mit Vertrag netcore.v1 und mode=open_lab        | Keine laufenden LXC aus dieser Umgebung geprüft                        |
| Basisstation            | Rust-Workspace und TBS-Konfiguration vorhanden                      | Kein aktueller SDR- oder Funkgerätetest durchgeführt                   |
| Mehrträger              | Haupt- und Sekundärträger sowie Mittenfrequenz konfigurierbar       | Zweiter Kontrollkanal oder realer Handover nicht pauschal belegt       |
| SIP und Brew            | Beide Pfade und ein separater Brew-Server im Repository             | Rufrouting und Medienpfad je Standort einzeln abnehmen                 |
| MQTT und Home Assistant | IoT Gateway mit Discovery, State-Ingress und Sandbox-Command-Policy | Reale Aktorbefehle standardmäßig gesperrt; bisherige SDS-Ankunft in HA |

| Gegenstand       | Befund                                                  | Grenze                                                                  |
|------------------|---------------------------------------------------------|-------------------------------------------------------------------------|
|                  |                                                         | offen                                                                   |
| ETSI-Konformität | Quellenregister und eigene Konformitätsmatrix vorhanden | Kein pauschales Zertifikat; viele unvollständige SAP/PDU-Pfade gelistet |

## 1.2 Sicherheits- und Zulassungsgrenze

Senden, Frequenzpaar, Identitäten, Leistung, Antennenanlage und EMV müssen zur jeweiligen Berechtigung und Installation passen. Beispielwerte wie 418,000 MHz Downlink und 408,000 MHz Uplink sind Projektdaten, keine allgemeine Freigabe. Änderungen am 230-V-Versorgungsteil oder an PA/Duplexer/Blitzschutz erfolgen nach den Unterlagen des konkreten Geräts und durch dafür geeignete Fachkräfte. Ein Software-Health-Status ersetzt weder HF-Messung noch elektrische Prüfung.

## 2 Architektur und Datenwege

### 2.1 Ebenen und Zuständigkeiten

| Ebene                | Komponenten                                                                                  | Autorität und Hauptaufgabe                                                    |
|----------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Funkkante            | TBS, SDR, lokale Dienste, Audio-Cache                                                        | Zeitkritische Luftschnittstelle, lokale Calls/SDS, Registrierung und Fallback |
| Vermittlung          | Node Gateway, Mobility, Subscriber, Group, Call Control, Media Switch, SDS Router            | Netzweite Zustände, Zulassung und Weiterleitung                               |
| Daten und Sicherheit | Packet Core, IP Gateway, Security Core, KMF, Transit                                         | Paketdaten, Netzübergang, Schlüssel- und Policy-Lebenszyklus, Regionen        |
| Anwendungen          | IoT Gateway, Hardware Gateway, RF Monitor, Alarm/Task Workflow, Asset Management, SIP Switch | Externe Systeme, Geräte, Aufgaben, Alarmierung und Telefonie                  |
| Bedienung            | Provisioning Core, Control Room, Media Library, Directory, Observability                     | Verwaltung, Einsatzoberfläche, Medien, Namen und Betriebsdaten                |

Ein TETRA-Funkgerät registriert sich über die Luftschnittstelle bei der TBS. Die TBS bewertet den unmittelbar nötigen Funkzustand; zentrale Subscriber- und Group-Policies können über Node Gateway einfließen. Für einen netzweiten Gruppenruf verwaltet Call Control den logischen Ruf und den Floor, während Media Switch codierte Sprachframes zwischen TBS-Call-Legs verteilt. Recorder hängt passiv am Medienpfad. Der Control Room zeigt und bedient diese Zustände, ist aber keine zweite autoritative Teilnehmerdatenbank.

Der TBS-Kontrollpfad verbindet [control\_room] der Basisstation per WebSocket mit `ws://<node-gateway>:8080/ws/node`. Die Backends verbinden sich über `/ws/backend` zum Gateway oder sprechen ihre jeweiligen HTTP-APIs. Der Gateway veröffentlicht eine Health-Matrix. Die TBS schaltet bei fehlenden Fachkernen einzelne lokale Fallbacks und bei fehlendem Gateway in die lokale Autorität. Die Zustände heißen `online`, `degraded`, `isolated` und `recovering`.

### 2.2 Identitäten und Datenobjekte

**MCC** und **MNC** identifizieren das TETRA-Netz, **LA** das Location Area, **Colour Code** die Zelle. **ISSI** ist die individuelle Funkteilnehmerkennung, **GSSI** die Gruppenkennung. **SNEI/NSAPI** gehören zur SndCP/PDP-Verwaltung. `node_id` identifiziert die TBS zum Gateway; Namen im Directory dienen der Anzeige, ersetzen aber keine Funkkennung. Nummern im SIP-Wählplan sind keine ISSI-Einträge im Subscriber Core.

Im Normalbetrieb legt Provisioning Core Teilnehmerprofile im Subscriber Core und Gruppen/Mitgliedschaften im Group Core an. Eine Gerätefreigabe, Gruppenmitgliedschaft, aktuelle Registrierung und tatsächliche Erreichbarkeit sind verschiedene Zustände. Fehlt einer

davon, kann die Anzeige plausibel aussehen und der Ruf trotzdem scheitern. Bei Fehlersuche diese vier Ebenen getrennt kontrollieren.

## 2.3 Medien- und Nachrichtenfluss

Ein Uplink-Sprachframe entsteht im Funkgerät, läuft durch SDR/PHY/MAC und CMCE der TBS und wird lokal oder über Call Control/Media Switch zu anderen TBS weitergereicht. Das im Repository verwendete TETRA-Medienformat umfasst für einen 60-ms-Verbund 35 Byte Sprachdaten; Brew transportiert nach seiner implementierten Rahmung einen 36-Byte-Payload aus Steuerbyte und Sprachdaten. SIP-Audio hat einen eigenen RTP/G.711-Pfad mit Transcoding. Eine WebSocket-Verbindung allein beweist deshalb noch keine hörbare Sprache.

SDS/Status laufen lokal oder über SDS Router; externe Automationen gelangen über IoT Gateway und MQTT in Anwendungspfade. Packet Data nutzt SNDCCP, Packet Core und IP Gateway; WAP ist ein eigener Anwendungspfad. Aufzeichnungen liegen zuerst lokal, die Media Library kann sie per URL holen, katalogisieren und archivieren. Audio-Dispatch holt freigegebene Assets in den lokalen Cache, codiert sie vor dem Payout und sendet erst danach über die Funkgruppe.

## 2.4 Ausfallverhalten

Der Gateway versendet eine versionierte Service-Matrix, deren Lease in der TBS standardmäßig 60 Sekunden gilt. Ein veralteter Zustand wird nicht dauerhaft als gesund interpretiert. `enter_after_secs` und `recover_after_secs` dämpfen kurze Unterbrechungen. Lokale Calls, Registrierung, SDS und installierte Schlüssel können weiterlaufen. Das Wiederabspielen alter Sprachframes ist ausgeschlossen; SDS- und Steuerereignisse können begrenzt aus dem lokalen JSONL-Spool nachgereicht werden. Ein bereits lokal zugestelltes Gruppen-SDS wird beim Replay markiert, um eine doppelte lokale Aussendung zu vermeiden.

## 3 Funktionen der Basisstation und des Netzes

### 3.1 Zelle Registrierung und Gruppen

Die TBS sendet SYNC/SYSINFO, bedient den Kontrollträger und verarbeitet Location Update, Deregistration, Gruppenaffiliation und lokale Rufsignalisierung. Konfigurationen für `main_carrier`, `secondary_carrier`, `freq_band`, `duplex_spacing`, `location_area`, `colour_code` und optionale Nachbarzellen liegen unter `[cell_info]`. Ein zweiter HF-Träger erhöht mögliche Ressourcenkapazität, ist aber nicht automatisch ein zweiter unabhängig zu programmierender Kontrollkanal. Der tatsächliche Kontroll-/Traffic-Slot-Plan ist im Laufzeitlog und mit einem Gerät nachzuweisen.

Für die bisherige Betriebsbeobachtung waren Träger 720/721 mit Downlink 418,000/418,025 MHz und Uplink 408,000/408,025 MHz vorgesehen; das SDR-Zentrum lag bei 418,0125/408,0125 MHz. Diese Werte beschreiben einen beobachteten Stand, nicht den Nachweis, dass jedes Gerät beide Träger als Kontrollkanal benötigt. Für ein MS zunächst die belegte Kontrollträgerfrequenz und Netz-/Zellparameter programmieren; Sekundärträger und Nachbarzelllogik anhand Air-Logs und MS-Verhalten prüfen.

Bei periodischer Registrierung ist `periodic_registration_secs` maßgeblich. Der dokumentierte aktuelle Pfad zieht ein verschwundenes Gerät wieder an, statt es mit `ExpiryOfTimer` abzuweisen und Gruppenaffiliationen zu verlieren. Insbesondere bei Motorola/Sepura ist das reale Verhalten versionsabhängig zu testen. Bei einem PTT-Problem nach Re-Registration zuerst Registrierung, Affiliation und Call Admission getrennt lesen.

### 3.2 Gruppenruf Einzelruf und Floor

Ein Gruppenruf beginnt über CMCE-Setup und erhält eine Traffic-Ressource. Der Floor wechselt zwischen Sprechern; nach `U-TX-CEASED` folgt die Hangtime und später `D-RELEASE`. Für ältere Motorola-Geräte gibt es einen optionalen Workaround beim erneuten Floor-Take desselben Sprechers. Die v21-Kompatibilitätsnotiz stellt den Main-Lebenszyklus wieder her; zusätzliche aggressive Sepura-Release-Signalisierung aus einer früheren Variante wurde entfernt. Ein Anruf darf nur nach vollständiger Signalisierungs- und Audio-Abnahme als stabil gelten.

Provisioning Core verwaltet keine Freigabeliste für Simplex/Duplex-Einzelrufe. SIP-Serviceziele werden durch `[asterisk]` und den Wählplan der TBS geroutet, nicht als Funkgeräte im Provisioning Core angelegt. Bei identischen Nummern gewinnt laut Projektkonfiguration ein explizites Asterisk-Präfix gegenüber lokaler ISSI-Routingentscheidung.

### 3.3 SDS Status Position und Alarmer

SDS Router vermittelt Einzel- und Gruppennachrichten, behandelt Offline-Zustellung und Anwendungsrouten. Statusmeldungen, LIP-Positionen, TPG2200-Call-Out, DAPNET und GeoAlarm sind im TBS-Code bzw. dokumentierten optionalen Integrationspfaden vertreten; jeder Pfad benötigt seine eigene Freigabe und Teststrecke. Der Alarm Workflow dedupliziert `netcore-event-v1`, führt Alarmakten und Eskalationen über SDS. Task Workflow kann Aufträge per REST/MQTT/SDS und WAP ausgeben. Keine Aussage über eine erfolgreich an Home Assistant zugestellte Nachricht lässt sich allein aus einem aktiven Broker-Connect ableiten.

### 3.4 Paketdaten WAP und IP

Die TBS enthält SNDCP/PDP-Pfade für IPv4, NSAPI-Kontexte, dynamische Adressen, einen lokalen WAP/WTP/WSP-Pfad und einen Packet-Data-Gateway. Die Dokumentation Docs/SNDCP\_COMPLETE.md begrenzt die alte lokale Stufe ausdrücklich: keine allgemeine IPv6- oder Kompressionsunterstützung und kein pauschaler Internetzugang. Spätere Packet-Core-/IP-Gateway-Dokumente erweitern die verteilte Datenebene. Vor einer Freigabe deshalb lokalen TBS-Modus, zentrale packet.mode, TUN, NAT/Firewall, Routing und Funkgerät-Profil zusammen prüfen. Die Aktivierung des angezeigten sndcp\_service-Flags ist kein erfolgreicher End-to-End-IP-Test.

### 3.5 Mobilität und Mehrzellenbetrieb

Mobility Core pflegt den Serving Node und kann MM-Kontexte zwischen TBS übertragen. Nachbarzell-Information und Zellwechsel sind in der TBS vorhanden; Call Restore hat eigene Laufzeit- und Testpfade. Eine automatische RF-Entscheidung, nahtloser Übergang eines laufenden Gesprächs und der SIP-Medien-Handover sind gesonderte Testziele. Die Phase-11-SIP-Dokumentation nennt unterbrechungsfreies Live-Handover für bestehende SIP-Rufe ausdrücklich als spätere Stufe.

### 3.6 SIP Brew Medien und TTS

Die native TBS-Asterisk-Brücke erfordert den Feature-Build --features asterisk und die passende native Codec-Bibliothek; ein TOML-Block allein aktiviert keinen fehlenden Build-Feature. Phase 11c verwendet einen lokalen Asterisk je TBS, einen zentralen SIP Switch und einen exklusiven PBX-Fallback. Der SIP Switch entscheidet anhand Mobility Core über die Serving-TBS; RTP bleibt in der ersten Stufe am Edge-Medienendpunkt. Brew ist ein paralleler Netzverbundpfad. Der separate experimentelle Brew-Server hat seine eigene Konfiguration und Ports 9000 bis 9003; er ist nicht automatisch Teil des 25-LXC-Deployments.

Aufnahmen, Audio Player, Media Library und TTS sind getrennte Komponenten. Lokales WAV/MP3-Playout soll vollständig vorbereitet sein, bevor die Aussendung beginnt; die Konfiguration nennt 720 ms Lead-in und 6 s Gruppen-Release-Guard. Die Media Library übernimmt Freigabe, Archiv und zentralen Asset-Status, während die TBS einen lokalen Cache hält. Piper kann als HTTP-Dienst unter Port 5005 laufen. Ob lokale oder zentrale TTS der gewünschte einzige Produktionsweg ist, muss pro Standort verbindlich festgelegt werden.

### 3.7 MQTT Home Assistant Homematic und physische I O

IoT Gateway veröffentlicht Zustände auf MQTT mit Präfix netcore/v1 und Home-Assistant-Discovery unter homeassistant. Die Beispielkonfiguration setzt QoS 1, retained Zustände, Discovery und State-Ingress; reale Command-Egress sowie Homematic-Schreibzugriff sind standardmäßig ausgeschaltet. Die Command-Policy ist Default-Deny, ignoriert retained Befehle und lässt in der Open-Lab-Vorlage nur virtuelle Lab-Geräte zu. Hardware Gateway sammelt Telemetrie; physische Ausgänge bleiben standardmäßig deaktiviert. RF Monitor liefert DSP-Werte vor der PA; VSWR und Vor-/Rücklaufleistung erfordern kalibrierte externe Sensorik.

### 3.8 Bedienung und Sichtbarkeit

Provisioning Core verwaltet ISSI, GSSI und Mitgliedschaften. Control Room bündelt Operator-, Incident- und Lageansichten; Native-UI- und RBAC-Dokumente beschreiben verschiedene Entwicklungsstände. Directory auf 8095 liefert sprechende Gerätenamen, Gruppen und Statuslabels. Observability sammelt Logs, Metriken, Alerts und Diagnosepakete. Dashboard-Basic-Auth der TBS schützt nur deren Oberfläche; sie ersetzt nicht die fehlende Authentisierung der Open-Lab-Backend-Dienste.

## 4 Hardware und Funkaufbau

### 4.1 Mindestbaugruppe

Eine funktionsfähige TBS braucht einen 64-Bit-Linux-Rechner, unterstütztes SDR samt SoapySDR-Treiber, stabile Taktung, Kühlung, korrekt dimensionierte Versorgung, definierte RX/TX-Führung, geeigneten Duplexer/Filter, Last und Antenne sowie ein Verwaltungsnetz. Das Repository enthält `sxxcvr-main/` für SoapySX und HAT-nahe Dateien. Raspberry Pi 4/5 ist in der Projektanleitung genannt. Ob ein zusätzliches AI-HAT Rechenvorteile bringt, ist für PHY/Codec nicht nachgewiesen; CPU- und USB-/GPIO-Budget sind vor Beschaffung zu messen.

Für Jans geplanten Aufbau gehören SXceiver, SMA-Brücken, BNC-Patchfeld, Duplexer und N-Antennenweg zusammen. Die Steckverbinderfolge bestimmt nicht allein Dämpfung und Belastbarkeit: Kabel, Adapter, Duplexer, Schutzadapter und Dummyload brauchen eine gemeinsame 50-Ohm- und Leistungsprüfung. Die tatsächliche TX-Leistung und Rückflusssdämpfung gehören auf ein Messprotokoll. Die geplante 230-V-/GPIO-/LCD-/Watchdog-Platine ist keine durch dieses Repository freigegebene Serienbaugruppe; mechanischer Plan und KiCad-Stand müssen separat validiert werden.

### 4.2 HF-Inbetriebnahme

1. SDR ohne Senderlast prüfen: Treiber, Kanalnummer, Sample-Rate, Takt und Gains aus `SoapySDRUtil --probe` lesen.
2. Vor erstem Sendetest 50-Ohm-Dummyload und geeignete Dämpfung/Messgeräte verwenden; RX und TX nicht vertauschen.
3. TX- und RX-Mitte sowie beide 25-kHz-Träger rechnerisch und mit SDR-/Spektrummessung prüfen.
4. Duplexer auf die tatsächlich verwendeten Frequenzpaare abstimmen; PA-Ausgang, Oberwellen und Isolation messen.
5. Antennen- und Überspannungsschutz nach Standortkonzept prüfen. Erdung, Blitzschutz und 230-V-Teil nicht aus Softwarekonfiguration ableiten.

Die Musterkonfiguration nennt `freq_band=4, main_carrier=720, secondary_carrier=721, tx_freq=418000000, rx_freq=408000000`, Zentren 418012500/408012500 Hz. Der Kommentar zu `duplex_spacing=0` im Muster spricht zugleich von 5 MHz, während die eingetragenen RX/TX-Werte 10 MHz auseinanderliegen. Die abgeleiteten Carrier aus Laufzeitlog und MS-Codeplug sind vor HF-Betrieb maßgeblich; die Kommentarzeile allein ist keine Frequenzrechnung.

### 4.3 GPIO Sensorik und Rack

Das SXceiver-HAT belegt einen Teil des Pi-Headers. Bevor freie GPIO auf eine eigene Leiterplatte gehen, für **jeden Pin** Belegung, Pegel (3,3 V), Startzustand, Pull-up, Strombudget und Störanfälligkeit gegen die konkret verwendete HAT-Version prüfen. I<sup>2</sup>C kann Temperatur-/Spannungssensorik und kleine Anzeigen bündeln; leistungsfähige Aktoren benötigen Treiber, separate Versorgung und fail-safe Verhalten. Lüfter, Temperatur und Watchdog gehören in ein Betriebs- und Alarmschema, nicht unmittelbar in die HF-Freigabe. RF Monitor sieht ohne echte Messsonden nur die Vor-PA-DSP-Welt.



## 5 Netzplanung und Ports

### 5.1 Adressierung

Das Beispiel-Inventory verwendet 10.0.20.0/24 mit 25 festen Hostadressen. Jans jüngste beobachtete Anlage benutzte einzelne Adressen in 10.0.1.0/24. Beide sind Beispiele aus verschiedenen Ständen. Vor Deployment ein einziges standortbezogenes Inventory festlegen und daraus TBS-Adresse, Gateway, Dienst-URLs, DNS/Hosts und Firewall ableiten. DHCP-Reservierungen sind möglich; ein LXC-Installer erkennt seine eigene Adresse, aber nicht automatisch alle anderen Dienste. Die gewünschte gegenseitige Auto-Discovery ist ein Ausbaziel und ersetzt die aktuellen TOML-Abhängigkeiten noch nicht.

Für das Open Lab gilt: LXC-Verwaltungsports und MQTT-Broker nur im isolierten Verwaltungsnetz erreichbar, keine öffentliche Weiterleitung. Security Core und KMF besonders eng einschränken. Zeitquelle/NTP, DNS oder /etc/hosts, Backup-Ziel und Medien/NFS getrennt planen. Die Beispiel-IP einer TBS ist nicht dieselbe wie der Gateway-Host. Der TBS-Dashboard-Port 8080 kann auf **einem anderen Host** gleichzeitig mit Gateway 8080 existieren.

### 5.2 Protokollübersicht

| Verbindung               | Transport                                  | Typischer Zweck                       | Prüfpunkt                      |
|--------------------------|--------------------------------------------|---------------------------------------|--------------------------------|
| TBS zu Node Gateway      | WebSocket über TCP 8080 /ws/node           | Steuerung, Health und Ereignisse      | Node online, Matrix frisch     |
| Backends zu Node Gateway | WebSocket über TCP 8080 /ws/backend        | Fachkern-Ereignisse und Kommandos     | Reconnect, Ack, Correlation ID |
| Fachdienst Management    | HTTP/TCP auf Dienstport                    | WebUI, REST, Health, Metrics, OpenAPI | /health/live, /health/ready    |
| IoT Gateway zu Broker    | MQTT/TCP 1883 als Beispiel                 | Events, State, Commands, HA Discovery | Topic, QoS, Retain, Ack        |
| SIP Switch und PBX       | SIP UDP/TCP 5060 als Vorlage               | Registrierung und Signalisierung      | Registrar/Contact/Route        |
| SIP Media                | RTP/UDP 10000 bis 20000 am Switch          | Sprachpakete                          | SDP-Adresse, Richtung, Codec   |
| Native TBS SIP           | SIP 5062, RTP 30000 bis 30100 als Beispiel | Lokaler Asterisk-Pfad                 | Feature-Build und Audio        |
| Brew                     | WebSocket, konfigurierbarer Port           | Ruf/SDS/ Positionsverbund             | Auth, Frameformat, Routing     |
| TTS                      | HTTP/TCP 5005 als Beispiel                 | Piper Synthese                        | /voices und Ausgabe            |
| Directory                | HTTP/TCP 8095 als Beispiel                 | Namen und Statuslabel                 | API und Datenbank              |

Ports im konkreten Betrieb immer gegen die ausgerollte TOML und `ss -lntup` abgleichen. Der Portkatalog im Anhang unterscheidet Managementports von zusätzlichen Datenports; ein offener TCP-Port sagt nichts über Readiness oder fachliche Funktion.

## 6 Installation der Basisstation

### 6.1 Vorbereitung

Ein 64-Bit Debian/Raspberry-Pi-OS mit systemd, stabiler Stromversorgung und Kühlung vorbereiten. Die Repository-Dokumentation nennt git, Compiler, CMake/Clang, libsoapysdr-dev, SoapysDR-Tools, ffmpeg, jq, sqlite3, nfs-common und den gerätespezifischen SDR-Treiber. Rust-Toolchain-Version und nativer Codec müssen zum Build passen. Installation aus vertrauenswürdigen Paketquellen sowie die genaue Feature-Auswahl in einem Buildprotokoll festhalten. Ein Betriebssystem-Image ist derzeit als zentraler Imagebuilder geplant, im 25-LXC-Deployment nicht als fertige TBS-Imager-Datei enthalten.

```
sudo apt update
sudo apt install -y git curl ca-certificates build-essential pkg-config cmake clang \
    libsoapysdr-dev soapysdr-tools ffmpeg jq sqlite3 nfs-common
SoapysDRUtil --info
SoapysDRUtil --find
SoapysDRUtil --probe="driver=<TREIBER>"
```

### 6.2 Quellstand und Build

Einen konkreten Commit oder Tag ausrollen und ihn in der Anlagenakte notieren. Im bestehenden Repository ist bluestation-bs das TBS-Binary; die Asterisk-Bridge ist feature-gated. Vor dem Kopieren eines Builds cargo build und die benötigten nativen Bibliotheken prüfen. Das folgende Beispiel ist ein **Buildmuster**, keine Aussage, dass es in dieser Umgebung ausgeführt wurde.

```
git clone https://github.com/JanHG98/netcore-tetra.git /opt/netcore-tetra
cd /opt/netcore-tetra
git checkout 086a81fa8820ef579c475a65a38e3d23644c52f0
cargo build --locked --release -p bluestation-bs
# Nur wenn Asterisk und Codec-Bibliothek tatsächlich eingerichtet sind:
# cargo build --locked --release -p bluestation-bs --features asterisk
sudo install -m 0755 target/release/bluestation-bs /usr/local/bin/bluestation-bs
```

Docs/basisstation.config.sanitized.example.toml als Ausgangspunkt in /etc/netcore/config.toml kopieren. Die Datei vor dem Start auf echte IP-Adressen, RF-Parameter, Dienstschalter und alle Platzhalter prüfen. Eine unveränderte zweite Datei /etc/netcore/config.toml.fallback als bekannten guten Stand ablegen. Die Laufzeit kann nach einem Parserfehler darauf zurückfallen; ein rotes Dashboard-Banner zeigt diesen Zustand an. Fallback ist Rettungsanker, keine automatische Freigabe einer fehlerhaften neuen Konfiguration.

```
sudo install -d -m 0750 /etc/netcore /var/lib/netcore
sudo install -m 0640 Docs/basisstation.config.sanitized.example.toml /etc/netcore/config.toml
sudo cp /etc/netcore/config.toml /etc/netcore/config.toml.fallback
sudo -u netcore RUST_LOG=info /usr/local/bin/bluestation-bs /etc/netcore/config.toml
```

Vor dem letzten Befehl den Systembenutzer netcore, SDR-Gerätezugriff und Eigentümer der State-/Medienverzeichnisse einrichten. Erst manuell mit passender HF-Last testen; danach die lokale tetra.service mit diesem Binary und genau dieser Konfigurationsdatei anlegen oder bestehende Unit anpassen. service\_name="tetra" muss mit der Unit für Dashboard-/SDS-Neustartfunktionen übereinstimmen.

### 6.3 Erststart und Abnahme am Pi

Prüfsequenz: TOML-Parsing, Soapy-Gerät, RX/TX-Kanäle, Sample-Rate und Center-Frequenz, SYSINFO, Downlink-Kontinuität, Dashboard-Erreichbarkeit, reale Registration und Gruppenruf. Danach erst zentrale Dienste zuschalten. Bei Dual Carrier die abgeleiteten Frequenzen im Log mit Codeplug und Messgerät vergleichen. Eine initiale RX-Buffer-Overrun- oder TX-Late-Meldung ist einzeln zu bewerten; anhaltende Drops, PAPR/EVM-Abweichungen oder hörbare Aussetzer sind keine normalen Betriebswerte.

```
systemctl status tetra.service --no-pager --full
journalctl -u tetra.service -b -n 200 --no-pager
ss -lntup
curl -fsS http://127.0.0.1:8080/api/edge-fallback
```

Der letzte Endpunkt gilt für einen entsprechend aktivierten TBS-Dashboard-Stand. Falls ein anderer Dashboard-Port konfiguriert ist, ihn ersetzen. Keine API-Antwort als Beweis für realen Funkbetrieb lesen.

## 7 Installation der Core Dienste

### 7.1 LXC-Grundlage

Das Open-Lab-Deployment ist für getrennte Debian-13-kompatible LXC mit systemd ausgelegt. Jeder LXC braucht Netzadresse, Buildwerkzeuge, Rust-Toolchain und schlüsselbasierten SSH-Zugriff vom Deployment-Host; IP Gateway braucht /dev/net/tun und die nötigen Netzwerkfähigkeiten. Recorder/Media Library brauchen bei Nutzung der Archive separat vorbereitetes NFS. Vor apply die 25 Beispielhosts ersetzen und Speicher-/CPU-Budget für lokale Rust-Builds bereitstellen.

### 7.2 Reproduzierbares Deployment

Die folgenden Unterbefehle stammen aus deploy/open-lab/README.md. validate, plan und render vorbereiten und schreiben lokale Artefakte; apply --dry-run zeigt die vorgesehenen SSH/SCP-Schritte; erst apply verändert LXC. Die generierte Konfiguration muss auf echte Hosts zeigen. Geheimnisse, TLS-Schlüssel und KMF-Mastermaterial werden vom Tool ausdrücklich nicht automatisch verteilt.

```
cd /opt/netcore-tetra
cp deploy/open-lab/inventory.example.toml deploy/open-lab/inventory.toml
${EDITOR:-nano} deploy/open-lab/inventory.toml
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml plan
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml render
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply --dry-run
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml status
```

render erzeugt unter anderem Dienstkatalog, Konfigurationen, Hosts-Beispiel, CSV-Portliste und Dependency-Graph. Die Installer bauen auf den jeweiligen LXC und starten in Abhängigkeitsreihenfolge. Eine einzelne manuelle Installation verwendet system-backend/<dienst>/install/install.sh und die passende /etc/netcore/<dienst>.toml; der tatsächliche Pfad steht für jeden Dienst im Inventory.

### 7.3 TBS an den Core binden

Die TBS verbindet sich mit dem **Node Gateway**, nicht direkt mit dem Control Room. [control\_room] enthält Gateway-Host, Port 8080, Pfad /ws/node, eindeutige node\_id und einen Standortnamen. Erst nach erfolgreicher lokaler Funkabnahme central\_sds\_routing und weitere zentrale Autoritäten einschalten. Eine aktive Verbindung plus frische Service-Matrix ist für zentralen Betrieb nötig; bei degraded die betroffenen Fachkerne ansehen.

### 7.4 Zusatzdienste außerhalb der 25er-Liste

**Provisioning Core** wird separat über system-backend/provisioning-core/install/install.sh installiert, läuft beispielhaft auf TCP 8125 und spricht Subscriber Core 8100 sowie Group Core 8110. Er ist im Workspace und in services.toml, aber nicht in deploy/open-lab/inventory.example.toml. Sein älterer Installationsguide nennt Branches swmi und

feature/provisioning-core, die in der aktuellen Remote-Branchliste nicht vorlagen; für diese Ausgabe den geprüften `main`-Commit verwenden und die Installerdatei dort kontrollieren.

**Directory** nutzt im Repository einen Python-Dienst auf TCP 8095 mit Datenbank/Seed und eigenen CRUD-APIs. **Piper/TTS** liegt unter `system-backend/tts/` und ist eine lokale Sprachsynthese auf Beispielport 5005. **Brew-Server** unter `misc/brew-server/` ist ein eigenes experimentelles Rust-Projekt mit separater Cargo-Datei und eigener TLS/Auth-Konfiguration. **Lokaler TBS-SIP-Failover** wird nach Phase 11c auf jeder TBS installiert und gehört nicht als zusätzlicher Core-LXC ins Inventory.

## 7.5 Startreihenfolge

Gateway und fachliche Autoritäten zuerst, Anwendungsebene danach, Control Room und Observability zuletzt. Die generierte Audit-Reihenfolge lautet: node-gateway, mobility-core, subscriber-core, group-core, packet-core, security-core, call-control, sds-router, ip-gateway, kmf, media-switch, application-gateway, iot-gateway, recorder, transit, hardware-gateway, rf-monitor, task-workflow, sip-switch, media-library, alarm-workflow, asset-management, control-room, observability. Ein Service mit `ready=503` kann wegen einer noch fehlenden Abhängigkeit laufen, aber fachlich nicht bereit sein.

## 8 Konfiguration

### 8.1 Grundregeln

Konfigurationen werden als textuelle TOML-Dateien geführt. Vor Änderungen eine Kopie des laufenden Standes erstellen, den betroffenen Dienst und seine Abhängigkeiten benennen, genau einen Satz an Werten verändern und danach Parsing, Unit-Status, Health, Funktionstest und Rückfall prüfen. Beispielwerte aus 10.0.1.0/24 und 10.0.20.0/24 niemals unbemerkt mischen. Passwörter, Bot-Token, SIP-Secrets, KMF-Material und Zertifikate gehören nicht in ein Handbuch, Commit oder Diagnosepaket.

### 8.2 Basisstation Abschnitt für Abschnitt

| TOML-Bereich                    | Aufgabe                                         | Prüfung vor Neustart                            |
|---------------------------------|-------------------------------------------------|-------------------------------------------------|
| [phy_io.soapysdr]               | SDR, RX/TX, Zentren, Sample-Rate, Kanäle, Gains | SoapySDRUtl --probe; Passband und Duplexer      |
| [net_info]                      | MCC und MNC                                     | Netzkennung mit Endgeräteprofil und Genehmigung |
| [cell_info]                     | Carrier, LA, CC, Dienste, Nachbarn, Timer       | abgeleitete Frequenzen und SYSINFO              |
| [cell_info.wap_ip]              | WAP/IP-Pfad                                     | Geräteprofil, DNS/Gateway, WSP-Port             |
| [cell_info.packet_data_gateway] | TUN/NAT/Forwarding                              | Adressen, Kernelrechte, Firewall                |
| [cell_info.sds_command_control] | Steuerbefehle über SDS                          | Absenderzulassung und Wirkbereich               |
| [recovery] / [health]           | Erholung und Health-Snapshots                   | Limits und Log-Frequenz                         |
| [dashboard]                     | Lokale WebUI                                    | Bind, Port, Benutzer, Zugriffszonen             |
| [recording] / [audio_player]    | Lokalaufnahme und Audioauspielung               | Disk, NFS, Cache, Quelle und Gruppe             |
| [tts] / [media_library]         | Synthese und Asset-Austausch                    | HTTP, Freigabe und Archivpfade                  |
| [control_room]                  | Gateway-WebSocket                               | Host aus Inventory, /ws/node, node_id           |
| [edge_fallback]                 | Health-Lease, Policy-Cache, Replay-Spool        | Schreibrechte, Alter, Hysterese                 |
| [brew] / [asterisk]             | Netzverbund und SIP                             | Auth, Routen, Codec, Feature-Build              |

Nur tatsächlich benötigte Integrationen aktivieren. Die Repository-Datei `config.toml` enthält demonstrative Werte und ist keine vollständig sanitisierte Vorlage; die bereinigte Beispiel-

TOML aus Docs/ ist der geeignetere Startpunkt. Der vollständige Feldindex im Anhang zeigt Schlüssel der geprüften Beispiele, nicht automatisch jede gültige Runtime-Option.

### 8.3 Shadow und authoritative

Mehrere Fachkerne haben shadow/authoritative oder ähnliche Aktivierungsmodi. Shadow kann Datenfluss und WebUI prüfen, ohne sofort Funkinjektion, Kerneländerung, OTAR oder externe Aktionen auszulösen. Umschalten erst nach bewusstem Test der Abhängigkeiten und der Fallbackreaktion. Die einzelnen Beispielwerte stehen in den dienstspezifischen TOML-Dateien. Security Core und KMF niemals implizit auf offenen Betrieb zurückfallen lassen; IoT Command-Egress und reale Relais bleiben ohne authentifizierte Betriebsarchitektur deaktiviert.

### 8.4 Beispiel einer konsistenten TBS Gateway Bindung

```
[control_room]
enabled = true
host = "10.0.20.10"      # nur für das Beispiel-Inventory
port = 8080
endpoint_path = "/ws/node"
node_id = "TBS-LAB-01"
station_name = "TBS-LAB-01"
central_sds_routing = false
```

Bei der realen Anlage die Host-IP durch den Node-Gateway-Eintrag im ausgerollten Inventory ersetzen. central\_sds\_routing erst nach dem SDS-E2E-Test aktivieren. Ein Test von GET /api/v1/core-services am Gateway und GET /api/edge-fallback an der TBS zeigt beide Seiten der Zustandsentscheidung.



## 9 Normalbetrieb und Bedienabläufe

### 9.1 Tageskontrolle

Beim Schichtbeginn oder nach Neustart: Versionsstand und Bootzeit der TBS, SDR/Carrier, Temperatur und RF-Messwerte, Gateway-Verbindung, Health-Matrix, 25 Dienst-Readiness-Werte, Broker- und SIP-Registrierung, Media-Cache, Speicherbelegung sowie Fehlerzähler prüfen. Ein Dashboard-Feld CONNECTED ist nur ein Transportstatus; für jede kritische Funktion mindestens einen fachlichen Prüfpunkt ergänzen. Uhrzeit und Testergebnis in einer Betriebsakte festhalten.

### 9.2 Teilnehmer aufnehmen

1. ISSI, Netzkennung, Gerät/Codeplug, Besitzer und vorgesehene Gruppen erfassen.
2. Im Provisioning Core Teilnehmer freigeben und in Subscriber Core synchronisierten Status kontrollieren.
3. Gruppen im Group Core anlegen und Mitgliedschaft/Auto-Attach prüfen.
4. Gerät einschalten, Registrierung an der TBS und Serving Node im Mobility Core kontrollieren.
5. Erst Einzel-SDS, dann Gruppen-PTT, dann gegebenenfalls Packet Data und SIP testen.

Der Directory-Name dient der Anzeige; er ersetzt kein Profil im Subscriber Core. Bei Fehlern zuerst Admission, dann Affiliation, dann Funkkanal trennen. Eine frisch registrierte ISSI ohne Gruppenreport kann sonst den Eindruck eines allgemeinen PTT-Ausfalls erzeugen.

### 9.3 Gruppenruf und Leitstellenarbeit

GSSI, Teilnehmerrechte und aktuellen Serving Node prüfen. Einen kurzen Sprechtest mit erstem und zweitem Sprecher durchführen, Floor-Abgabe/Hangtime beobachten und Call Control/Media Switch mit TBS-Log korrelieren. Ein Control-Room-PTT braucht eine eindeutig definierte Absenderidentität, Rolle und Medienroute; die lokale Oberfläche allein erzeugt keine autoritative Netz Wahrheit. Für eine reproduzierbare Abnahme je Ruf Call-ID, TBS-IDs, GSSI, Zeitstempel, Audio und Release festhalten.

### 9.4 SDS zu MQTT und Home Assistant

Eine SDS-Nachricht an eine Service-ISSI durchläuft mehrere Stellen: MS-Uplink an TBS, CMCE/SDS, Node Gateway/SDS Router, IoT Gateway, MQTT-Broker, Home-Assistant-Topic/Automation. Jeder Übergang braucht einen eigenen Nachweis. Für die zuletzt offene Nachricht an 4010001: zuerst den Uplink im TBS-Log suchen, dann Routing-Entscheid und Ack im SDS Router, dann IoT-Event, dann `mosquitto_sub -v -t 'netcore/v1/#'`, dann Home-Assistant-MQTT-Ereignis. Sind fünf Dienste verbunden, folgt daraus noch keine SDS-Route. Die HA-automations.yaml muss nur auf tatsächlich publizierte Topics und Payloads reagieren.

### 9.5 Audio TTS und Recording

Eine Datei lokal aufnehmen oder hochladen, Metadaten und Freigabestatus in der Media Library prüfen, Asset auf die TBS cachen und erst nach vollständigem Encode zur Gruppe

ausspielen. Bei TTS den HTTP-Dienst, Stimme, Synthesis-Ergebnis, lokale WAV-Datei und das tatsächlich gehörte erste Wort prüfen. Recorder-Zustand, freier Speicher, Sidecar und optionales NFS-Archiv nach einem echten Ruf kontrollieren. Ein fehlendes NFS darf die lokale Aufnahme nicht stillschweigend als archiviert markieren.

## 9.6 SIP und Brew

Für Phase 11c muss die TBS im Normalbetrieb ausschließlich über den zentralen Switch registrieren; direkter PBX-Kontakt ist erst im bestätigten Fallback aktiv. `netcore-tbs-sip-failover.service`, lokale Asterisk-Registrierung, Switch-Serving-Node und RTP/SDP getrennt prüfen. Ausgehender Ruf, eingehender Ruf, Klingeln, Annahme, beide Audiorichtungen, DTMF und Auflegen gehören in denselben Test. Brew parallel betreiben nur mit eindeutigen Nummern-/Gruppenrouten, sonst drohen Schleifen und doppelte Zustellung. Ein SIP-180 Ringing belegt noch keine Audioverbindung.

## 9.7 Edge-Fallback

Bei Gateway-Ausfall lokale Registrierung, Gruppenruf und SDS prüfen; gleichzeitig muss die TBS `isolated` statt fälschlich `online` anzeigen. Bei Ausfall eines einzigen Fachkerns `degraded` und dessen spezifischen Fallback verifizieren. Nach Recovery den Replay-Spool, Dienst-Revisionen, offene Calls und mögliche Doppelsendungen kontrollieren. KMF-Schlüsselrotation und OTAR während einer Isolation nicht simulieren oder mit offenen Default-Keys ersetzen.

## 10 Wartung Reparatur und Wiederherstellung

### 10.1 Routineplan

| Intervall         | Maßnahme                                                                                       | Nachweis                                          |
|-------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Täglich           | Health, Disk, Temperatur, Broker/SIP, RF-Alarm, TBS-Logs                                       | kurze Betriebsnotiz mit UTC-Zeit                  |
| Wöchentlich       | Testgerät registrieren, SDS und kurzen Ruf, Backup-Job, NFS-Archiv und Alert-Zustellung prüfen | Testprotokoll und Referenz-IDs                    |
| Monatlich         | Patchstand, Ersatz-SD/SSD, Dummyload-/Duplexer-/Antennenpfad, Lüfter und Stecker sichten       | Mess- und Versionsblatt                           |
| Vor jedem Update  | Konfig/DB/State sichern, Commit notieren, Rollback-Dateien bereitstellen                       | Backup restaurierbar getestet                     |
| Nach jedem Update | Parser, Liveness, Readiness, On-Air-Kernpfad und Fallback                                      | Abnahme mit zwei Geräten, wenn Funktion betroffen |

### 10.2 Sicherungen

TBS: /etc/netcore/, Unit und Drop-ins, Policy-Cache, Spool, lokale Aufnahmen, Audio-/TTS-Cache nach Bedarf und den exakten Git-Commit sichern. LXC: /etc/netcore/<dienst>.toml, Datenverzeichnisse aus der jeweiligen Beispielkonfiguration und echte Datenbankdateien. Recorder und Media Library benötigen getrennte Archiv- und Indexsicherung; eine NFS-Freigabe allein ist kein Backup. KMF-Mastermaterial getrennt und zugriffsgeschützt aufbewahren. Geheimnisse vor Support-Exporten entfernen.

### 10.3 Update und Rollback

Neue Version zuerst in einer Testumgebung oder an einer nicht kritischen Zelle bauen. Diff von config.toml, Inventory und Beispielkonfigurationen prüfen; Datenmigrationen und Backward-Compatibility der Verträge beachten. Dann Backend-Abhängigkeiten kontrolliert aktualisieren und TBS zuletzt, oder nach der konkreten Release-Anleitung vorgehen. Einen laufenden Call nicht als Updatefenster verwenden. Bei Fehler Dienst stoppen, Binärstand und Konfiguration auf dokumentierten letzten guten Zustand zurückführen, Datenbank-Schema-Kompatibilität prüfen und danach erneut End-to-End testen. Git-Reset allein repariert keine migrierte State-Datei.

### 10.4 Reparatur an Hardware und RF

Eine ausgefallene SD-Karte durch ein geprüftes Image plus standortspezifische Konfiguration/Secrets ersetzen. SDR- und Antennenfehler anhand lokalem Loopback, Treiber-Probe, RX-Rauschboden, TX-Last, Kabel und Duplexer einkreisen. Fehlersuche zuerst bei geringer, kontrollierter Leistung an Last; der reale Antennenpfad folgt nach Messung. Bei

Überspannung oder Feuchte mehrere Komponenten als mögliche gemeinsame Fehlerquelle behandeln. Ein PA-/Duplexer-Austausch erfordert erneute Kalibrierung, Leistungs- und Spektrummessung.

### 10.5 Geplanter Imagebuilder und Auto Discovery

Eine zentrale VM/LXC mit Imagebuilder, Site-Provisioning, VPN-Schlüsselvergabe und automatischer Dienstsuche ist als Projektziel beschrieben, im untersuchten Inventory aber nicht als fertiger End-to-End-Workflow nachgewiesen. Bis dahin Image und Standortdaten kontrolliert getrennt halten. Für eine spätere automatische Erkennung braucht jeder Dienst eindeutige Identität, Diensttyp, Version, Adresse, Vertrag, Health, Authentisierung und Ablaufzeit; bloßes UDP-Broadcast im gemeinsamen VLAN kann keine Vertrauensentscheidung ersetzen.

## 11 Fehlersuche

Bei jeder Störung zuerst Zeitpunkt, Softwarestand, TBS-ID, ISSI/GSSI, Call-/Command-ID und letzte funktionierende Änderung erfassen. Dann von unten nach oben prüfen: Strom/Netz/HF, Prozess, Health, Gateway, Policy, fachlicher Pfad, Endgerät. Keine State-Datei löschen, nur weil ready=503 erscheint. Die folgende Matrix nennt erste konkrete Schritte; sie ersetzt kein Log des betroffenen Geräts.

| Symptom                                       | Wahrscheinliche Ebene       | Erste Prüfung                                         | Nächster Schritt                                          |
|-----------------------------------------------|-----------------------------|-------------------------------------------------------|-----------------------------------------------------------|
| TBS startet nach TOML-Edit nicht              | Parser/Fallback             | journalctl -u tetra -b; rotes Fallback-Banner         | Diff gegen .fallback, Schlüssel/Typ korrigieren           |
| Gateway verbunden, Fallback trotzdem isoliert | Health-Matrix-Lease         | TBS /api/edge-fallback, Gateway /api/v1/core-services | Revision/Lease, Host/IP und /ws/node prüfen               |
| MS findet die Zelle nicht                     | RF/Codeplug                 | Downlink, MCC/MNC/CC, Hauptträger, SDR-Log            | Dummyload/Spektrum, Codeplug und Carrier-Abgleich         |
| Registrierung klappt, Gruppen-PTT nicht       | Group Policy/Affiliation    | ISSI-Admission und GSSI-Mitgliedschaft                | Gruppenreport, Call Admission und CMCE-Log vergleichen    |
| Wiederregistrierung führt zu PTT-Verlust      | MM/Group Re-Affiliation     | T351-/Location-Update-Logs und Affiliation            | Firmware/Workaround und Core-Sync getrennt testen         |
| Gruppenruf hängt nach Loslassen               | CMCE/FACCH/Release          | U-TX-CEASED, D-TX-CEASED, Hangtime, D-RELEASE         | Buffer-/Late-Zähler und MS-Firmware prüfen                |
| SDS an 4010001 erreicht HA nicht              | SDS-Router/IoT/MQTT         | Uplink, Route, Ack, Topic, Broker-Subscriber          | Erst Payload-Topic, dann HA-YAML anpassen                 |
| HA sieht Geräte, Befehle wirken nicht         | Default-Deny/Command-Egress | Policy, execute_commands, Ack-Ledger                  | virtuelles Lab-Ziel testen; reale Aktoren nicht erzwingen |
| Audio startet ohne erstes Wort                | Playout/Timing              | Asset ready, Cache/Encode, Lead-in                    | 720-ms-Leadin, Guard und TCH-Zuteilung prüfen             |
| SIP klingelt, Audio fehlt                     | RTP/SDP/Codec               | beide RTP-Richtungen, SDP-Contact, RTP-Zähler         | Feature-Build/Codec und Asterisk-Routing prüfen           |
| SIP doppelt registriert                       | Phase-11c-Failover          | aktiver Include und Asterisk Contacts                 | Failover-State/Expiry/Hysterese reparieren                |

| Symptom                        | Wahrscheinliche Ebene | Erste Prüfung                              | Nächster Schritt                                        |
|--------------------------------|-----------------------|--------------------------------------------|---------------------------------------------------------|
| Brew-Audio verzerrt oder stumm | Frameformat/Jitter    | 36-Byte-STE-Payload, 60-ms-Pacing          | Server-/TBS-Version und Transcoder-Zähler               |
| Paketdaten verbunden, kein IP  | SND/CP/TUN/NAT        | NSAPI/PDP, TUN, Route, Firewall            | ICMP/DNS/WAP einzeln testen                             |
| Recorder fehlt im Archiv       | Disk/NFS/Import       | lokale WAV/JSON, Mount, Media-Asset-Status | Retry-Log und Schreibrechte, nicht Original löschen     |
| VSWR-Alarm trotz guter DSP-EVM | externe Sensorik      | Kalibrierung/Probe-JSON                    | Kabel, Last, Duplexer und Antennenpfad messen           |
| Deployment-Audit fällt fehl    | Konfigurationsdrift   | TBS [control_room] gegen Inventory         | Gateway-IP konsistent setzen und Audit erneut ausführen |

### 11.1 Diagnosereihenfolge als Befehlssatz

```
systemctl status <unit> --no-pager --full
journalctl -u <unit> -b -n 200 --no-pager
ss -lntup
curl -fsS http://<dienst-host>:<port>/health/live
curl -i http://<dienst-host>:<port>/health/ready
curl -fsS http://<dienst-host>:<port>/metrics
```

/health/live sagt nur, dass der Prozess antwortet. Eine 503 bei /health/ready kann eine bewusst fehlende Abhängigkeit signalisieren. Im Dashboard die dazugehörige Begründung lesen. Bei Funkfehlern zusätzlich TBS-Systemlog und Endgeräteereignis mit derselben UTC-Zeit sammeln. Für Audio SIP/RTP nur in autorisierter Testumgebung mitschneiden und personenbezogene Inhalte minimieren.

## 12 Tests und Abnahme

### 12.1 Statische Prüfung

Das geprüfte Inventory akzeptiert 25 Dienste. Der Full-System-Audit ist wegen der alten 24er-Prüfliste, fehlender Warn-API-Marker und der Gateway-Hostabweichung rot. Eine gespeicherte ältere PASS-Datei belegt den aktuellen Quellstand nicht. Nach bewusster Korrektur dieser drei Punkte erneut prüfen und die tatsächliche Ausgabe aufheben.

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 tools/check_full_system_integration.py
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile full --validate-only
```

### 12.2 Integrationstests

Das Smoke-Profil prüft Verträge und Mock-TBS. `full --allow-mutations` verändert Labordaten und prüft unter anderem Subscriber/Group, Call, SDS und Paketdaten. `fault --allow-mutations --allow-restarts` stoppt absichtlich Dienste. Letzteres nur in einem freigegebenen Testfenster ohne produktive Teilnehmer. Artefakte landen unter `tests/e2e/artifacts/<run-id>/` mit JSON, JUnit und Summary. Erfolg verlangt `failed=0`, nachvollziehbare Skips und wieder aktive Dienste.

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile smoke
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile full --allow-mutations
# Nur im eigens dafür vorgesehenen Open Lab:
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile fault --allow-mutations --allow-restarts
```

### 12.3 Funkabnahme

Mock- und Unit-Tests beweisen keine HF-Konformität. Für reale Abnahme mindestens zwei passende Funkgeräte, möglichst verschiedene Hersteller/Firmwarestände, dokumentierte TBS-Config und Messmittel verwenden. Erst Registrierung und Einzel-SDS, dann Gruppenruf/Floor/Release, Einzelruf/SIP, Packet Data/WAP, Isolation/Recovery, Dual Carrier und Mehrzellenwechsel testen. Jeder Fall bekommt Ergebnis, UTC, Geräte, Frequenzen, Logs/PCAP/Messbild und Tester. `tests/e2e/on_air_template.json` und `validate_on_air_evidence.py --require-complete --require-two-vendors` strukturieren die Evidenz, ersetzen die Messung aber nicht.

### 12.4 Abnahmekriterien für neue Funktionen

Eine Funktion gilt erst als betriebsbereit, wenn Konfiguration und Codeversion reproduzierbar, negative Fälle definiert, Logs korrelierbar, Fallback sicher und End-to-End-Verhalten mit realen Geräten dokumentiert sind. Für sicherheitsrelevante Bereiche zusätzlich Berechtigungen, Schlüsseltransport, Datenhaltung und Wiederherstellung prüfen. Eine erfolgreich geladene WebUI oder ein grünes Static-Script genügt dafür nicht.

## 13 Bekannte Grenzen und Ausbau

### 13.1 Offene technische Risiken

Die generierte Docs/IMPLEMENTATION\_GAPS.md listet zahlreiche TODO-/unimplemented!-/Panic-Pfade und unvollständig verdrahtete SAP-Primitiven. Die Zahlen sind Inventurtreffer, keine Aussage, dass jeder davon im normalen TBS-Pfad erreichbar ist. Sie verbieten aber die Behauptung einer vollständig implementierten ETSI-Luftschnittstelle. Die eigene Docs/ETSI\_CONFORMANCE\_MATRIX.md unterscheidet Parser, Encoder, Runtime-Nutzung und On-Air-Nachweis. Besonders TLMC/TLPD, seltene PDU-Varianten und Herstellerinteroperabilität brauchen gezielte Tests.

### 13.2 Projektziele gegenüber heutigem Stand

| Ziel                               | Heutige Grundlage                         | Fehlender Nachweis oder Arbeit                                    |
|------------------------------------|-------------------------------------------|-------------------------------------------------------------------|
| Mehrzellen-Handover laufender Rufe | Mobility/Core und Call-Restore-Module     | End-to-End-On-Air mit zwei TBS und aktiver Sprache                |
| Zentraler Imagebuilder             | Deployment-Skripte und Installer          | signiertes, standortspezifisches Pi-Image samt Rückrollung        |
| Gegenseitige Auto Discovery        | zentrales Inventory und Health            | authentifizierte Erkennung, Verträge und Konfliktauflösung        |
| Vollständige Leitstelle IDECS      | Control Room und Native UI                | Arbeitsplatz-Audio, Rollen, AD/NFC und skalierbare UI abnehmen    |
| Eigene 230-V-/GPIO-Platine         | KiCad-Arbeitsstand außerhalb dieses Repos | Schaltplan, Footprints, Layout, EMV und elektrische Abnahme       |
| Produktive Security                | Security Core/KMF und Labortests          | Auth/TLS/RBAC für alle 25 Managementdienste und Schlüsselbetrieb  |
| Vollständige virtuelle TBS/MS      | Mock-TBS und E2E-Runner                   | interaktive Funkgeräte, Funkkanalmodell und Mehrzellen-Simulation |

### 13.3 Umgang mit Normen

Die beigegefügteten ETSI-Dateien reichen von historischen ETS/ES/TS-Texten bis zu Entwürfen aus 2026. Für konkrete Implementierungsprüfungen eine feste Fassung je Feature nennen, Klauseln nicht zwischen Editionen mischen und Drafts als Entwurf markieren. Die 2016er EN 300 392-2 V3.8.1 ist laut projektspezifischem Quellenregister die feste Referenz vieler Code-Klauseln. EN 300 394-1 unterstützt eine spätere RF-Konformitätsabnahme. PEI, ISI, Security, Supplementary Services und TSIM haben eigene Dokumente und sind nicht automatisch vollständig realisiert.



## 14 Diensthandbuch und Konfigurationsindex

Die folgenden 24 Profile und das zusätzliche Warn-Dienstprofil in Kapitel 30 werden direkt aus dem aktuellen Deployment-Inventory abgeleitet. Alle IP-Adressen sind Labormuster. Für jeden Dienst nennt das Inventory einen eigenen Installer, ein Konfigurationsziel, eine systemd-Unit und seine Abhängigkeiten. Die Feldlisten stammen aus den jeweils geparsten TOML-Beispielen. Ein vorhandener Schlüssel beweist nicht, dass die Option in jedem Buildpfad aktiv ist.

### 14.1 node-gateway

Verbindet TBS-Knoten und Fachkerne, bestätigt Kommandos und veröffentlicht die Health-Matrix.

| Eigenschaft           | Wert                                                         |
|-----------------------|--------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.10:8080/tcp für Management                           |
| Abhängigkeiten        | keine                                                        |
| Konfigurationsziel    | /etc/netcore/node-gateway.toml                               |
| Vorlage               | system-backend/node-gateway/config/node-gateway.example.toml |
| Unit                  | netcore-node-gateway.service                                 |
| Installer             | system-backend/node-gateway/install/install.sh               |

**Fachtest.** TBS an /ws/node anmelden, /api/v1/core-services und Ack/Revision prüfen.

**Ausfall.** Ohne Gateway läuft die TBS lokal weiter; netzweite Vermittlung ist nicht verfügbar.

**Erstdiagnose.** `systemctl status netcore-node-gateway.service` und `journalctl -u netcore-node-gateway.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt         | Schlüssel im Beispiel                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| [server]          | bind, node_path, backend_path, history_limit, stale_after_secs, hello_timeout_secs, application_ping_secs |
| [security]        | mode, allow_remote_management                                                                             |
| [limits]          | max_message_bytes, max_http_body_bytes                                                                    |
| [service_monitor] | enabled, interval_secs, timeout_ms, failure_threshold, recovery_threshold, targets                        |

## 14.2 mobility-core

Führt Serving-Node-Lage und überträgt MM-Kontexte zwischen Basisstationen.

| Eigenschaft           | Wert                                                           |
|-----------------------|----------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.11:8090/tcp für Management                             |
| Abhängigkeiten        | node-gateway                                                   |
| Konfigurationsziel    | /etc/netcore/mobility-core.toml                                |
| Vorlage               | system-backend/mobility-core/config/mobility-core.example.toml |
| Unit                  | netcore-mobility-core.service                                  |
| Installer             | system-backend/mobility-core/install/install.sh                |

**Fachtest.** Zwei TBS registrieren, Serving Node und Export/Import-Ack mit derselben ISSI verfolgen. **Ausfall.** Ausfall lässt lokale Registrierung bestehen, verhindert verlässliche zentrale Übergabe.

**Erstdiagnose.** `systemctl status netcore-mobility-core.service` und `journalctl -u netcore-mobility-core.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                          |
|----------------|------------------------------------------------|
| [server]       | bind, history_limit, transfer_timeout_secs     |
| [node_gateway] | url, reconnect_secs                            |
| [security]     | mode, allow_remote_management                  |
| [limits]       | max_body_bytes, max_transfers, max_subscribers |

## 14.3 subscriber-core

Hält Teilnehmerprofile, Zulassung und Synchronisation zur Funkkante.

| Eigenschaft           | Wert                                                               |
|-----------------------|--------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.12:8100/tcp für Management                                 |
| Abhängigkeiten        | node-gateway                                                       |
| Konfigurationsziel    | /etc/netcore/subscriber-core.toml                                  |
| Vorlage               | system-backend/subscriber-core/config/subscriber-core.example.toml |

| Eigenschaft | Wert                                              |
|-------------|---------------------------------------------------|
| Unit        | netcore-subscriber-core.service                   |
| Installer   | system-backend/subscriber-core/install/install.sh |

**Fachtest.** Test-ISSI freigeben und sperren, TBS-Policy-Sync und Admission beobachten.

**Ausfall.** Last-known Policy und lokale statische Konfiguration regeln den Rückfall; keine stille Öffnung.

**Erstdiagnose.** `systemctl status netcore-subscriber-core.service` und `journalctl -u netcore-subscriber-core.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt       | Schlüssel im Beispiel                                       |
|-----------------|-------------------------------------------------------------|
| [server]        | bind, history_limit                                         |
| [node_gateway]  | url, reconnect_secs                                         |
| [storage]       | database_path, backup_path                                  |
| [access_policy] | mode, auto_sync, disconnect_unauthorized, sync_timeout_secs |
| [security]      | mode, allow_remote_management                               |
| [limits]        | max_body_bytes, max_subscribers, max_groups_per_subscriber  |

## 14.4 group-core

Verwaltet GSSI, Mitgliedschaften, Affiliation und DGNA-Policy.

| Eigenschaft           | Wert                                                     |
|-----------------------|----------------------------------------------------------|
| Beispielhost und Port | 10.0.20.13:8110/tcp für Management                       |
| Abhängigkeiten        | node-gateway, subscriber-core                            |
| Konfigurationsziel    | /etc/netcore/group-core.toml                             |
| Vorlage               | system-backend/group-core/config/group-core.example.toml |
| Unit                  | netcore-group-core.service                               |
| Installer             | system-backend/group-core/install/install.sh             |

**Fachtest.** GSSI und Mitgliedschaft setzen, Attach-Report und PTT-Zulassung kontrollieren.

**Ausfall.** Lokale Affiliationen und Cache bleiben maßgeblich, neue zentrale Änderungen fehlen.

**Erstdiagnose.** `systemctl status netcore-group-core.service` und `journalctl -u netcore-group-core.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                                             |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| [server]       | bind, history_limit                                                                                               |
| [node_gateway] | url, reconnect_secs                                                                                               |
| [storage]      | database_path, backup_path                                                                                        |
| [policy]       | allow_unlisted_groups, enforce_memberships, reconcile_registered, auto_sync, sync_timeout_secs, dgna_timeout_secs |
| [security]     | mode, allow_remote_management                                                                                     |
| [limits]       | max_body_bytes, max_groups, max_memberships                                                                       |

## 14.5 call-control

Verwaltet netzweite logische Calls, Call Legs, Floor und Restore.

| Eigenschaft           | Wert                                                         |
|-----------------------|--------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.14:8120/tcp für Management                           |
| Abhängigkeiten        | node-gateway, subscriber-core, group-core, mobility-core     |
| Konfigurationsziel    | /etc/netcore/call-control.toml                               |
| Vorlage               | system-backend/call-control/config/call-control.example.toml |
| Unit                  | netcore-call-control.service                                 |
| Installer             | system-backend/call-control/install/install.sh               |

**Fachtest.** Gruppenruf über zwei TBS aufbauen, Call-ID, Floor, Release und Restore prüfen.

**Ausfall.** Nur lokale Zellenrufe; keine zentrale Call-Autorität.

**Erstdiagnose.** `systemctl status netcore-call-control.service` und `journalctl -u netcore-call-control.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt | Schlüssel im Beispiel |
|-----------|-----------------------|
| [server]  | bind, history_limit   |

| Abschnitt       | Schlüssel im Beispiel                                                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [node_gateway]  | url, reconnect_secs                                                                                                                                             |
| [mobility_core] | enabled, base_url, timeout_ms, allow_local_fallback, accept_stale_route                                                                                         |
| [storage]       | database_path, backup_path                                                                                                                                      |
| [calls]         | command_timeout_secs, restore_timeout_secs, reconcile_interval_secs, auto_target_affiliated_nodes, release_partial_start_on_failure, allow_operator_force_floor |
| [security]      | mode, allow_remote_management                                                                                                                                   |
| [limits]        | max_body_bytes, max_calls, max_legs_per_call, max_pending_commands                                                                                              |

## 14.6 media-switch

Verteilt codierte TETRA-Sprachframes zwischen den Call Legs.

| Eigenschaft           | Wert                                                         |
|-----------------------|--------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.15:8130/tcp für Management                           |
| Abhängigkeiten        | node-gateway, call-control                                   |
| Konfigurationsziel    | /etc/netcore/media-switch.toml                               |
| Vorlage               | system-backend/media-switch/config/media-switch.example.toml |
| Unit                  | netcore-media-switch.service                                 |
| Installer             | system-backend/media-switch/install/install.sh               |

**Fachtest.** Frames in beiden Richtungen, Drop-/Jitter-Zähler und hörbaren Inhalt prüfen.

**Ausfall.** Lokales Luftschnittstellen-Audio läuft weiter, zentrale Frames entfallen.

**Erstdiagnose.** `systemctl status netcore-media-switch.service` und `journalctl -u netcore-media-switch.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel             |
|----------------|-----------------------------------|
| [server]       | bind, history_limit               |
| [node_gateway] | url, reconnect_secs               |
| [call_control] | url, events_url, route_ready_url, |

| Abschnitt  | Schlüssel im Beispiel                                                                                                                                                                                                                                                                                                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | reconcile_secs, reconnect_secs,<br>request_timeout_secs                                                                                                                                                                                                                                                                                                                                   |
| [media]    | frame_duration_ms, jitter_buffer_frames,<br>min_jitter_buffer_frames,<br>max_jitter_buffer_frames, adaptive_jitter,<br>adaptive_jitter_up_threshold_ms,<br>adaptive_jitter_down_stable_frames,<br>cold_start_buffer_frames,<br>cold_start_buffer_max_age_ms,<br>session_idle_secs, max_frames_per_tick,<br>allow_same_leg_loopback,<br>tap_history_frames,<br>recorder_tap_history_frames |
| [security] | mode, allow_remote_management                                                                                                                                                                                                                                                                                                                                                             |
| [limits]   | max_body_bytes, max_sessions, max_streams,<br>max_pending_frames                                                                                                                                                                                                                                                                                                                          |

## 14.7 recorder

Zeichnet den Media-Switch-Tap außerhalb des zeitkritischen Rufpfades auf.

| Eigenschaft           | Wert                                                     |
|-----------------------|----------------------------------------------------------|
| Beispielhost und Port | 10.0.20.16:8140/tcp für Management                       |
| Abhängigkeiten        | media-switch                                             |
| Konfigurationsziel    | /etc/netcore/recorder.toml                               |
| Vorlage               | system-backend/recorder/config/<br>recorder.example.toml |
| Unit                  | netcore-recorder.service                                 |
| Installer             | system-backend/recorder/install/install.sh               |

**Fachtest.** Echten Ruf aufzeichnen, Asset, Länge, Integrität und Archivziel kontrollieren.

**Ausfall.** Lokale TBS-Aufnahme ist ein eigener Pfad; der Core-Recorder kann fehlen.

**Erstdiagnose.** `systemctl status netcore-recorder.service` und `journalctl -u netcore-recorder.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                   |
|----------------|-----------------------------------------------------------------------------------------|
| [server]       | bind, history_limit                                                                     |
| [media_switch] | tap_url, sessions_url, poll_interval_ms,<br>session_reconcile_ms, request_timeout_secs, |

| Abschnitt  | Schlüssel im Beispiel                                                                                                                                                      |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | batch_limit                                                                                                                                                                |
| [storage]  | root, export_root, frame_duration_ms, session_absent_grace_secs, maximum_idle_secs, default_retention_days, retention_scan_secs, fsync_every_frames, minimum_free_space_mb |
| [security] | mode, allow_remote_management, allow_delete                                                                                                                                |
| [limits]   | max_body_bytes, max_active_recordings, max_recordings                                                                                                                      |

## 14.8 sds-router

Vermittelt SDS/Status und Store-and-forward zwischen Nodes und Anwendungen.

| Eigenschaft           | Wert                                                     |
|-----------------------|----------------------------------------------------------|
| Beispielhost und Port | 10.0.20.17:8150/tcp für Management                       |
| Abhängigkeiten        | node-gateway, subscriber-core, group-core, mobility-core |
| Konfigurationsziel    | /etc/netcore/sds-router.toml                             |
| Vorlage               | system-backend/sds-router/config/sds-router.example.toml |
| Unit                  | netcore-sds-router.service                               |
| Installer             | system-backend/sds-router/install/install.sh             |

**Fachtest.** Einzel-SDS, Gruppenzustellung, Offline-Ack und Replay testweise durchlaufen.

**Ausfall.** Lokale Zustellung und begrenzter dauerhafter Spool auf der TBS.

**Erstdiagnose.** `systemctl status netcore-sds-router.service` und `journalctl -u netcore-sds-router.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------|
| [server]       | bind, history_limit                                                                                   |
| [node_gateway] | url, reconnect_secs                                                                                   |
| [storage]      | database_path, backup_path                                                                            |
| [routing]      | default_ttl_secs, max_ttl_secs, max_attempts, initial_retry_secs, max_retry_secs, dedupe_window_secs, |

| Abschnitt  | Schlüssel im Beispiel                                          |
|------------|----------------------------------------------------------------|
|            | presence_timeout_secs,<br>authoritative_ingress                |
| [security] | mode, allow_remote_management,<br>mask_payload_in_list         |
| [limits]   | max_body_bytes, max_payload_bytes,<br>max_messages, max_routes |

## 14.9 packet-core

Hält PDP/NSAPI-Zustände, Adressen und Paketdaten-Flow-Control.

| Eigenschaft           | Wert                                                       |
|-----------------------|------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.18:8160/tcp für Management                         |
| Abhängigkeiten        | node-gateway, subscriber-core, mobility-core               |
| Konfigurationsziel    | /etc/netcore/packet-core.toml                              |
| Vorlage               | system-backend/packet-core/config/packet-core.example.toml |
| Unit                  | netcore-packet-core.service                                |
| Installer             | system-backend/packet-core/install/install.sh              |

**Fachtest.** PDP aktivieren, IPv4-Lease, Fragmentierung und Release bei Abmeldung prüfen.

**Ausfall.** Die lokale TBS kann ihre konfigurierten SMDCP-Kontexte weiter bedienen.

**Erstdiagnose.** `systemctl status netcore-packet-core.service` und `journalctl -u netcore-packet-core.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                                                                                                                                                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]       | bind, history_limit                                                                                                                                                                                                                    |
| [node_gateway] | url, reconnect_secs                                                                                                                                                                                                                    |
| [storage]      | database_path, backup_path                                                                                                                                                                                                             |
| [packet]       | mode, ready_timer_secs, standby_timer_secs,<br>response_wait_secs, context_ready_secs,<br>default_mtu, max_n_pdu_bytes,<br>max_contexts_per_subscriber,<br>max_total_contexts, strict_source_address,<br>preserve_context_on_node_loss |
| [address_pool] | network_prefix, first_host, last_host,                                                                                                                                                                                                 |



| Abschnitt       | Schlüssel im Beispiel                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
|                 | gateway, allow_static                                                                                                       |
| [fragmentation] | timeout_secs, max_datagrams,<br>max_total_bytes,<br>max_fragments_per_datagram, reject_overlaps                             |
| [flow_control]  | max_queue_packets_per_context,<br>max_queue_bytes_per_context,<br>queue_ttl_secs, action_retry_secs,<br>action_max_attempts |
| [security]      | mode, allow_remote_management,<br>expose_payloads                                                                           |
| [limits]        | max_body_bytes, max_events, max_actions,<br>max_payload_bytes                                                               |

### 14.10 ip-gateway

Verbindet Packet Core über TUN, Routing, DNS, Firewall und optionales NAT mit IPv4.

| Eigenschaft           | Wert                                                     |
|-----------------------|----------------------------------------------------------|
| Beispielhost und Port | 10.0.20.19:8170/tcp für Management                       |
| Abhängigkeiten        | packet-core                                              |
| Konfigurationsziel    | /etc/netcore/ip-gateway.toml                             |
| Vorlage               | system-backend/ip-gateway/config/ip-gateway.example.toml |
| Unit                  | netcore-ip-gateway.service                               |
| Installer             | system-backend/ip-gateway/install/install.sh             |

**Fachtest.** TUN-Adresse, Route, NAT-Regeln, DNS und einen echten IP-Test prüfen. **Ausfall.** Nur lokal konfigurierter Gateway-Pfad bleibt verfügbar; keine pauschale Internetzusage.

**Erstdiagnose.** `systemctl status netcore-ip-gateway.service` und `journalctl -u netcore-ip-gateway.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt     | Schlüssel im Beispiel                                                          |
|---------------|--------------------------------------------------------------------------------|
| [server]      | bind                                                                           |
| [packet_core] | url, poll_interval_ms, context_refresh_ms,<br>request_timeout_ms, outbox_batch |
| [storage]     | database_path, backup_path                                                     |

| Abschnitt     | Schlüssel im Beispiel                                                                             |
|---------------|---------------------------------------------------------------------------------------------------|
| [interface]   | mode, name, address, network, mtu, owner_user, delete_on_exit                                     |
| [routing]     | enable_ipv4_forwarding, reconcile_interval_secs, install_connected_route                          |
| [nat]         | enabled, masquerade, egress_interface                                                             |
| [firewall]    | enabled, default_forward_policy, allow_established, allow_general_internet, allow_icmp, log_drops |
| [dns]         | enabled, bind, upstream, local_domain, ttl_secs, query_timeout_ms                                 |
| [test_server] | enabled, bind, udp_echo_bind                                                                      |
| [capture]     | directory, max_captures, max_file_bytes, snaplen                                                  |
| [security]    | mode, allow_remote_management                                                                     |
| [limits]      | max_body_bytes, max_events, max_flows, max_packet_bytes                                           |

### 14.11 security-core

Verwaltet Security-Class-Policy, Auth-Kontexte, Sperren und Audit im Laborausbau.

| Eigenschaft           | Wert                                                           |
|-----------------------|----------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.20:8180/tcp für Management                             |
| Abhängigkeiten        | node-gateway, subscriber-core                                  |
| Konfigurationsziel    | /etc/netcore/security-core.toml                                |
| Vorlage               | system-backend/security-core/config/security-core.example.toml |
| Unit                  | netcore-security-core.service                                  |
| Installer             | system-backend/security-core/install/install.sh                |

**Fachtest.** Admission/Sperre und Security-Event protokolliert testen, Policy-Modus dokumentieren. **Ausfall.** Letzte bekannte Policy beibehalten; niemals unbemerkt herabstufen.

**Erstdiagnose.** `systemctl status netcore-security-core.service` und `journalctl -u netcore-security-core.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

**Konfigurationsfelder**

| Abschnitt        | Schlüssel im Beispiel                                                                                                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]         | bind, history_limit                                                                                                                                                               |
| [node_gateway]   | url, reconnect_secs, observe_nodes                                                                                                                                                |
| [storage]        | database_path, backup_path, lab_seed_path                                                                                                                                         |
| [policy]         | operating_mode, default_security_class,<br>minimum_security_class,<br>authentication_required,<br>allow_class1_fallback,<br>reject_unknown_subscribers,<br>disable_after_failures |
| [authentication] | provider, challenge_bytes, response_bytes,<br>challenge_ttl_secs, max_attempts,<br>lockout_secs, issue_dck_on_success                                                             |
| [dck]            | key_bytes, ttl_secs, rotate_before_secs,<br>max_active_per_subscriber                                                                                                             |
| [security]       | mode, allow_remote_management,<br>expose_ephemeral_edge_material                                                                                                                  |
| [limits]         | max_body_bytes, max_profiles, max_contexts,<br>max_actions, max_alarms, max_audit                                                                                                 |

**14.12 kmf**

Verwaltet Schlüssel-Lebenszyklus, Rotation und OTAR-Orchestrierung.

| Eigenschaft           | Wert                                       |
|-----------------------|--------------------------------------------|
| Beispielhost und Port | 10.0.20.21:8190/tcp für Management         |
| Abhängigkeiten        | security-core                              |
| Konfigurationsziel    | /etc/netcore/kmf.toml                      |
| Vorlage               | system-backend/kmf/config/kmf.example.toml |
| Unit                  | netcore-kmf.service                        |
| Installer             | system-backend/kmf/install/install.sh      |

**Fachtest.** Nur im Testnetz Key-Metadaten, Job-Status, Revision und Audit kontrollieren.

**Ausfall.** Installierte Schlüssel bleiben, neue OTAR-Aktionen sind nicht verfügbar.

**Erstdiagnose.** `systemctl status netcore-kmf.service` und `journalctl -u netcore-kmf.service -b;` danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

## Konfigurationsfelder

| Abschnitt  | Schlüssel im Beispiel                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]   | bind, history_limit                                                                                                                                                 |
| [storage]  | database_path, vault_path, master_key_path, backup_dir, bootstrap_dir                                                                                               |
| [policy]   | operating_mode, default_key_bytes, default_crypto_period_secs, rotation_lead_secs, require_dual_approval, allow_overlapping_crypto_periods, auto_retire_predecessor |
| [vault]    | provider, master_key_bytes, fsync                                                                                                                                   |
| [otar]     | action_ttl_secs, max_attempts, retry_backoff_secs, max_claim_batch                                                                                                  |
| [security] | mode, allow_remote_management, expose_raw_keys                                                                                                                      |
| [limits]   | max_body_bytes, max_keys, max_nodes, max_jobs, max_actions, max_audit                                                                                               |

## 14.13 transit

Verbindet regionale NetCore-Domänen über Peer-/Routen-/Sessionzustände.

| Eigenschaft           | Wert                                                  |
|-----------------------|-------------------------------------------------------|
| Beispielhost und Port | 10.0.20.22:8200/tcp für Management                    |
| Abhängigkeiten        | mobility-core, call-control, media-switch, sds-router |
| Konfigurationsziel    | /etc/netcore/transit.toml                             |
| Vorlage               | system-backend/transit/config/transit.example.toml    |
| Unit                  | netcore-transit.service                               |
| Installer             | system-backend/transit/install/install.sh             |

**Fachtest.** Routen- und Peerstatus, Schleifenschutz und Rückfall bei Peer-Ausfall testen.

**Ausfall.** Keine interregionale Vermittlung; lokale und regionale Basispfade bleiben getrennt.

**Erstdiagnose.** `systemctl status netcore-transit.service` und `journalctl -u netcore-transit.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

**Konfigurationsfelder**

| Abschnitt   | Schlüssel im Beispiel                                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]    | bind, history_limit                                                                                                                                               |
| [storage]   | database_path, backup_path                                                                                                                                        |
| [region]    | region_id, swmi_id, display_name, advertised_endpoint, protocol_version, operating_mode, capabilities                                                             |
| [routing]   | max_hops, dedupe_ttl_secs, session_idle_ttl_secs, route_stale_secs, prefer_direct_region_peer, allow_transitive_routing, allow_dynamic_peers, fail_closed_on_loop |
| [transport] | connect_timeout_ms, io_timeout_ms, heartbeat_interval_secs, peer_timeout_secs, retry_backoff_secs, max_attempts, max_batch                                        |
| [security]  | mode, allow_remote_management, tls, token_auth                                                                                                                    |
| [limits]    | max_body_bytes, max_peers, max_routes, max_sessions, max_envelopes, max_local_deliveries, max_events                                                              |

**14.14 application-gateway**

Bündelt Connectoren, Webhooks, Regeln, Vorlagen und TTS-Anbindungen.

| Eigenschaft           | Wert                                                                           |
|-----------------------|--------------------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.23:8220/tcp für Management                                             |
| Abhängigkeiten        | sds-router                                                                     |
| Konfigurationsziel    | /etc/netcore/application-gateway.toml                                          |
| Vorlage               | system-backend/application-gateway/<br>config/application-gateway.example.toml |
| Unit                  | netcore-application-gateway.service                                            |
| Installer             | system-backend/application-gateway/<br>install/install.sh                      |

**Fachtest.** Ein Lab-Event mit Correlation ID durch Connector und Response führen. **Ausfall.** Nur lokal erreichbare Integrationen sind verfügbar.

**Erstdiagnose.** `systemctl status netcore-application-gateway.service` und `journalctl -u netcore-application-gateway.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

## Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                                                                                                                                                                                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]       | bind, public_base_url, max_body_bytes, history_limit                                                                                                                                                                                                                                                                       |
| [storage]      | state_path, state_backup_path, secrets_path, spool_dir, backup_dir                                                                                                                                                                                                                                                         |
| [security]     | mode, management_token_auth, management_tls, allow_remote_management, connector_secrets_allowed, warning_banner                                                                                                                                                                                                            |
| [runtime]      | operating_mode, worker_interval_ms, probe_interval_secs, default_ttl_secs, max_attempts, base_backoff_secs, max_backoff_secs, dedupe_window_secs, max_response_bytes, max_artifact_bytes, max_events, max_deliveries, max_tts_jobs, max_audit_records, event_retention_secs, delivery_retention_secs, audit_retention_secs |
| [[connectors]] | connector_id, display_name, kind, direction, endpoint, health_endpoint, enabled, timeout_ms, rate_limit_per_minute, circuit_failure_threshold, circuit_open_secs, required_secrets, settings; 12 Beispieleinträge                                                                                                          |
| [[rules]]      | rule_id, name, enabled, priority, source_connector, event_type, target_connector, template_id, stop_processing; 2 Beispieleinträge                                                                                                                                                                                         |
| [[templates]]  | template_id, name, kind, body, content_type, enabled, target_connector, description; 3 Beispieleinträge                                                                                                                                                                                                                    |

## 14.15 media-library

Importiert, verarbeitet, genehmigt und archiviert Medien für Aufnahme und Playout.

| Eigenschaft           | Wert                                                           |
|-----------------------|----------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.24:8230/tcp für Management                             |
| Abhängigkeiten        | media-switch, recorder, application-gateway                    |
| Konfigurationsziel    | /etc/netcore/media-library.toml                                |
| Vorlage               | system-backend/media-library/config/media-library.example.toml |
| Unit                  | netcore-media-library.service                                  |

| Eigenschaft | Wert                                            |
|-------------|-------------------------------------------------|
| Installer   | system-backend/media-library/install/install.sh |

**Fachtest.** WAV per URL importieren, ready/approved, Preview, NFS und TBS-Cache prüfen.

**Ausfall.** Freigegebene lokal gecachte Medien können an der TBS weiter genutzt werden.

**Erstdiagnose.** `systemctl status netcore-media-library.service` und `journalctl -u netcore-media-library.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                                                                                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]       | bind, public_base_url, max_body_bytes                                                                                                                                                                                           |
| [security]     | mode, token_auth, tls, allow_remote_management, allow_delete, allow_url_import, allow_private_import_urls                                                                                                                       |
| [storage]      | root, state_file, temp_root, backup_root, archive_root, recording_archive_root, tts_archive_root, max_asset_bytes, max_total_bytes, fsync_imports                                                                               |
| [runtime]      | operating_mode, worker_interval_ms, probe_interval_secs, import_timeout_secs, max_assets, max_jobs, max_events, max_audit_records, max_attempts, frame_interval_ms, auto_approve_tts, auto_archive_recordings, auto_archive_tts |
| [playout]      | mode, default_station, request_timeout_secs, completion_timeout_secs, poll_interval_ms, stations                                                                                                                                |
| [codec]        | frame_bytes, ffmpeg_command, encoder_command, decoder_command                                                                                                                                                                   |
| [tts]          | enabled, endpoint, template_directory, default_voice, default_speed, max_text_characters, synthesis_timeout_secs, max_output_file_mb, voices                                                                                    |
| [dependencies] | media_switch_base_url, recorder_base_url, application_gateway_base_url                                                                                                                                                          |

## 14.16 control-room

Aggregiert Lage, Operator-Aktionen, Incidents, Status und Schichtbuch.

| Eigenschaft           | Wert                                                                                                                                                                                                                                                                                                         |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.25:9010/tcp für Management                                                                                                                                                                                                                                                                           |
| Abhängigkeiten        | node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, iot-gateway, hardware-gateway, rf-monitor, alarm-workflow, task-workflow, asset-management, sip-switch |
| Konfigurationsziel    | /etc/netcore/control-room.toml                                                                                                                                                                                                                                                                               |
| Vorlage               | system-backend/control-room/config/control-room.example.toml                                                                                                                                                                                                                                                 |
| Unit                  | netcore-control-room.service                                                                                                                                                                                                                                                                                 |
| Installer             | system-backend/control-room/install/install.sh                                                                                                                                                                                                                                                               |

**Fachtest.** Federation, Operator-Ansicht, Rechte und eine rückverfolgbare Aktion prüfen.

**Ausfall.** Lokales TBS-Dashboard bleibt sichtbar; zentrale Bedienung kann fehlen.

**Erstdiagnose.** `systemctl status netcore-control-room.service` und `journalctl -u netcore-control-room.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt     | Schlüssel im Beispiel                                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------------------------------------|
| [server]      | bind, node_path, ui_path, history_limit                                                                                         |
| [persistence] | enabled, database_path, persist_events, persist_noisy_events, load_recent_limit                                                 |
| [auth]        | enabled, allow_health_unauthenticated, node_token_env, bootstrap_username_env, bootstrap_password_env, bootstrap_role           |
| [federation]  | enabled, poll_interval_secs, request_timeout_ms, failure_threshold, fetch_summaries                                             |
| [operations]  | state_path, backup_path, auto_service_incidents, incident_limit, shift_log_limit                                                |
| [[services]]  | name, display_name, kind, base_url, health_live, health_ready, summary_path, webui_path, critical, enabled; 20 Beispieleinträge |
| [directory]   | hide_infrastructure                                                                                                             |



| Abschnitt         | Schlüssel im Beispiel |
|-------------------|-----------------------|
| [service_monitor] | targets               |

### 14.17 observability

Sammelt Metrics, Logs, Traces, Alerts, Silences und Diagnosepakete.

| Eigenschaft           | Wert                                                                                                                                                                                                                                                                                                                       |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.26:8210/tcp für Management                                                                                                                                                                                                                                                                                         |
| Abhängigkeiten        | node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, iot-gateway, hardware-gateway, rf-monitor, alarm-workflow, task-workflow, asset-management, sip-switch, control-room |
| Konfigurationsziel    | /etc/netcore/observability.toml                                                                                                                                                                                                                                                                                            |
| Vorlage               | system-backend/observability/config/observability.example.toml                                                                                                                                                                                                                                                             |
| Unit                  | netcore-observability.service                                                                                                                                                                                                                                                                                              |
| Installer             | system-backend/observability/install/install.sh                                                                                                                                                                                                                                                                            |

**Fachtest.** Scrape-Zeit, Trace-ID, Alert und Retention nach einem Testevent prüfen. **Ausfall.** Lokale Journale und Health-Endpunkte bleiben die Diagnosequelle.

**Erstdiagnose.** `systemctl status netcore-observability.service` und `journalctl -u netcore-observability.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt    | Schlüssel im Beispiel                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------------|
| [server]     | bind, history_limit, max_body_bytes                                                                       |
| [storage]    | state_path, backup_path, diagnostic_dir                                                                   |
| [security]   | mode, token_auth, tls, allow_remote_management, warning_banner                                            |
| [collection] | scrape_interval_secs, request_timeout_ms, max_response_bytes, scrape_on_start, ingest_logs, ingest_traces |
| [retention]  | metric_retention_secs, log_retention_secs,                                                                |

| Abschnitt       | Schlüssel im Beispiel                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|                 | trace_retention_secs, audit_retention_secs, max_series, max_samples_per_series, max_logs, max_spans, max_alerts, max_audit_records           |
| [stack]         | prometheus_url, grafana_url, loki_url, alertmanager_url, prometheus_ready_path, grafana_ready_path, loki_ready_path, alertmanager_ready_path |
| [[targets]]     | target_id, display_name, service, base_url, metrics_path, live_path, ready_path, enabled, labels; 21 Beispieleinträge                        |
| [[alert_rules]] | rule_id, name, description, metric, comparator, threshold, for_secs, severity, enabled, labels, annotations; 3 Beispieleinträge              |

### 14.18 iot-gateway

Normalisiert TETRA-Ereignisse auf MQTT und integriert HA/Homematic im Sandboxmodus.

| Eigenschaft           | Wert                                                       |
|-----------------------|------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.27:8240/tcp für Management                         |
| Abhängigkeiten        | node-gateway, mobility-core, call-control, sds-router      |
| Konfigurationsziel    | /etc/netcore/iot-gateway.toml                              |
| Vorlage               | system-backend/iot-gateway/config/iot-gateway.example.toml |
| Unit                  | netcore-iot-gateway.service                                |
| Installer             | system-backend/iot-gateway/install/install.sh              |

**Fachtest.** Broker-Topic, Discovery, State-Ingress, virtuelles Command und Ack testen.

**Ausfall.** MQTT-Brücke fehlt; lokale Funk-/Core-Funktionen laufen weiter.

**Erstdiagnose.** `systemctl status netcore-iot-gateway.service` und `journalctl -u netcore-iot-gateway.service -b`; danach GET /health/live, GET /health/ready und den fachlichen Test. Bei ready=503 die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt  | Schlüssel im Beispiel               |
|------------|-------------------------------------|
| [server]   | bind, history_limit, max_body_bytes |
| [security] | mode, allow_remote_management       |

| Abschnitt            | Schlüssel im Beispiel                                                                                                                                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [mqtt]               | host, port, client_id, topic_prefix, keep_alive_secs, clean_session, reconnect_secs, publish_timeout_secs, qos, event_retain, state_retain, observe_commands, execute_commands                                                                      |
| [home_assistant]     | enabled, discovery_enabled, discovery_prefix, status_topic, node_id, discovery_qos, discovery_retain, expose_gateway, expose_sources, expose_virtual_devices, accept_state_ingress, state_ingress_topic, allow_command_egress, command_egress_topic |
| [homematic]          | enabled, mode, ccu_host, ccu_port, poll_interval_ms, request_timeout_ms, allow_writes                                                                                                                                                               |
| [commands]           | enabled, mode, default_deny, allow_retained, default_ttl_secs, max_ttl_secs, max_future_skew_secs, publish_lifecycle_acks, ack_qos, ack_retain                                                                                                      |
| [storage]            | state_dir, outbox_dir, dedup_file, command_inbox_file, command_ledger_file, command_audit_file, virtual_state_file, external_state_file, homematic_state_file, dedup_limit, outbox_limit, command_ledger_limit                                      |
| [polling]            | interval_ms, batch_limit, request_timeout_ms                                                                                                                                                                                                        |
| [[command_policies]] | id, enabled, effect, command_types, target_types, target_prefixes, max_ttl_secs, allow_dry_run; 5 Beispieleinträge                                                                                                                                  |
| [[sources]]          | id, url, enabled; 4 Beispieleinträge                                                                                                                                                                                                                |

### 14.19 hardware-gateway

Sammelt Edge-/Rack-Telemetrie und erzeugt Schwellwert-Ereignisse.

| Eigenschaft           | Wert                                                                 |
|-----------------------|----------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.28:8250/tcp für Management                                   |
| Abhängigkeiten        | iot-gateway                                                          |
| Konfigurationsziel    | /etc/netcore/hardware-gateway.toml                                   |
| Vorlage               | system-backend/hardware-gateway/config/hardware-gateway.example.toml |
| Unit                  | netcore-hardware-gateway.service                                     |

| Eigenschaft | Wert                                               |
|-------------|----------------------------------------------------|
| Installer   | system-backend/hardware-gateway/install/install.sh |

**Fachtest.** Heartbeat und simulierten Grenzwert mit normalisiertem Event nachweisen.

**Ausfall.** Rackdaten fehlen; Funkbetrieb darf davon nicht abhängen.

**Erstdiagnose.** `systemctl status netcore-hardware-gateway.service` und `journalctl -u netcore-hardware-gateway.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt      | Schlüssel im Beispiel                                     |
|----------------|-----------------------------------------------------------|
| [server]       | bind                                                      |
| [security]     | mode                                                      |
| [mqtt]         | host, port, topic_prefix, client_id                       |
| [storage]      | state_file, event_log                                     |
| [monitoring]   | heartbeat_timeout_secs, stale_after_secs, outputs_enabled |
| [[thresholds]] | metric, warning_above, critical_above; 3 Beispieleinträge |

### 14.20 rf-monitor

Sammelt DSP- und optionale kalibrierte Hardware-Messwerte der TBS.

| Eigenschaft           | Wert                                                     |
|-----------------------|----------------------------------------------------------|
| Beispielhost und Port | 10.0.20.29:8260/tcp für Management                       |
| Abhängigkeiten        | iot-gateway                                              |
| Konfigurationsziel    | /etc/netcore/rf-monitor.toml                             |
| Vorlage               | system-backend/rf-monitor/config/rf-monitor.example.toml |
| Unit                  | netcore-rf-monitor.service                               |
| Installer             | system-backend/rf-monitor/install/install.sh             |

**Fachtest.** DSP-Metrik von echter Vor-/Rücklaufprobe unterscheiden und Alarm auslösen.

**Ausfall.** Zentrales RF-Dashboard fehlt; lokale RF-Messung und Funk können weiterlaufen.

**Erstdiagnose.** `systemctl status netcore-rf-monitor.service` und `journalctl -u netcore-rf-monitor.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt    | Schlüssel im Beispiel                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]     | bind                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| [security]   | mode                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| [mqtt]       | enabled, host, port, topic_prefix, client_id                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| [storage]    | state_file, event_log                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| [monitoring] | heartbeat_timeout_secs, event_memory_limit, max_spectrum_bins, max_payload_bytes                                                                                                                                                                                                                                                                                                                                                                                             |
| [thresholds] | vswr_warning, vswr_critical, reflected_ratio_warning_percent, reflected_ratio_critical_percent, pa_temp_warning_c, pa_temp_critical_c, sdr_temp_warning_c, sdr_temp_critical_c, cabinet_temp_warning_c, cabinet_temp_critical_c, evm_warning_pct, evm_critical_pct, papr_warning_db, papr_critical_db, forward_power_warning_below_w, forward_power_critical_below_w, pa_voltage_warning_below_v, pa_voltage_critical_below_v, fan_warning_below_rpm, fan_critical_below_rpm |

### 14.21 alarm-workflow

Dedupliziert Events, eröffnet Alarmakten und eskaliert über SDS.

| Eigenschaft           | Wert                                                             |
|-----------------------|------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.30:8270/tcp für Management                               |
| Abhängigkeiten        | iot-gateway, sds-router, hardware-gateway, rf-monitor            |
| Konfigurationsziel    | /etc/netcore/alarm-workflow.toml                                 |
| Vorlage               | system-backend/alarm-workflow/config/alarm-workflow.example.toml |
| Unit                  | netcore-alarm-workflow.service                                   |
| Installer             | system-backend/alarm-workflow/install/install.sh                 |

**Fachtest.** Gleiches Ereignis zweimal einspeisen, Eskalationsstufe/Ack und Empfänger prüfen.  
**Ausfall.** Zentrale Eskalation fehlt; lokaler Funk/SDS bleibt möglich.

**Erstdiagnose.** `systemctl status netcore-alarm-workflow.service` und `journalctl -u netcore-alarm-workflow.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt               | Schlüssel im Beispiel                                                                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [server]                | bind                                                                                                                                                                 |
| [security]              | mode                                                                                                                                                                 |
| [storage]               | state_file, event_log, audit_log                                                                                                                                     |
| [mqtt]                  | enabled, host, port, qos, topic_prefix, client_id, reconnect_secs, subscribe_topics                                                                                  |
| [sds_router]            | base_url, timeout_secs, poll_interval_secs, process_existing_events, source_issi, protocol_id, default_ttl_secs                                                      |
| [workflow]              | scheduler_interval_secs, event_history_limit, seen_event_limit, stop_escalation_on_ack, stop_escalation_on_assignment, auto_close_on_clear                           |
| [limits]                | max_body_bytes                                                                                                                                                       |
| [[recipients]]          | id, name, enabled, kind, destination, source_issi, protocol_id, priority, ttl_secs, max_text_chars, message_template; 2 Beispieleinträge                             |
| [[escalation_profiles]] | id, name, steps; 2 Beispieleinträge                                                                                                                                  |
| [[rules]]               | id, enabled, action, event_type, alarm_type, title, description, severity, priority, requires_ack, recipients, escalation_profile, dedup_fields; 12 Beispieleinträge |
| [[status_actions]]      | enabled, status_code, action, states, requires_ack_only; 5 Beispieleinträge                                                                                          |

## 14.22 task-workflow

Führt strukturierte Aufgaben über REST/MQTT/SDS/WAP.

| Eigenschaft           | Wert                               |
|-----------------------|------------------------------------|
| Beispielhost und Port | 10.0.20.31:8280/tcp für Management |

| Eigenschaft        | Wert                                                           |
|--------------------|----------------------------------------------------------------|
| Abhängigkeiten     | iot-gateway, sds-router                                        |
| Konfigurationsziel | /etc/netcore/task-workflow.toml                                |
| Vorlage            | system-backend/task-workflow/config/task-workflow.example.toml |
| Unit               | netcore-task-workflow.service                                  |
| Installer          | system-backend/task-workflow/install/install.sh                |

**Fachtest.** Task anlegen, Status rückmelden, WAP-Ansicht und deduplizierten Ack prüfen.

**Ausfall.** Zentrale Aufgaben fehlen; lokal gecachte Formulare können abhängig vom Aufbau bleiben.

**Erstdiagnose.** `systemctl status netcore-task-workflow.service` und `journalctl -u netcore-task-workflow.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt          | Schlüssel im Beispiel                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------|
| [service]          | name, phase, mode                                                                                             |
| [server]           | bind                                                                                                          |
| [security]         | mode                                                                                                          |
| [storage]          | state_file, event_log, audit_log                                                                              |
| [mqtt]             | enabled, host, port, topic_prefix, client_id                                                                  |
| [sds_router]       | enabled, base_url, source_issi, protocol_id, ttl_secs, max_text_length, default_destination, default_is_group |
| [workflow]         | event_history_limit, seen_event_limit, expire_check_interval_secs, notify_on_state_change                     |
| [wap]              | enabled, page_size, xhtml_entry, wml_entry                                                                    |
| [[templates]]      | id, name, description, default_priority, default_severity, requires_ack, fields; 6 Beispieleinträge           |
| [[status_actions]] | status_code, action, label; 5 Beispieleinträge                                                                |

## 14.23 asset-management

Führt physische Geräte, Zuordnung, Firmware und Wartung außerhalb der Funkzulassung.

| Eigenschaft           | Wert                                                                 |
|-----------------------|----------------------------------------------------------------------|
| Beispielhost und Port | 10.0.20.32:8290/tcp für Management                                   |
| Abhängigkeiten        | iot-gateway, subscriber-core, mobility-core, task-workflow           |
| Konfigurationsziel    | /etc/netcore/asset-management.toml                                   |
| Vorlage               | system-backend/asset-management/config/asset-management.example.toml |
| Unit                  | netcore-asset-management.service                                     |
| Installer             | system-backend/asset-management/install/install.sh                   |

**Fachtest.** Gerät einem Subscriber zuordnen und Wartungsauftrag im Task Workflow prüfen.

**Ausfall.** Bestandsverwaltung fehlt; lokale Funkberechtigung ist eigenständig.

**Erstdiagnose.** `systemctl status netcore-asset-management.service` und `journalctl -u netcore-asset-management.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt    | Schlüssel im Beispiel                            |
|--------------|--------------------------------------------------|
| [service]    | name, phase, mode                                |
| [server]     | bind                                             |
| [security]   | mode                                             |
| [storage]    | state_file, event_log, audit_log                 |
| [mqtt]       | enabled, host, port, topic_prefix, client_id     |
| [management] | event_history_limit, upstream_sync_interval_secs |
| [upstreams]  | subscriber_core, mobility_core, task_workflow    |

## 14.24 sip-switch

Routet SIP zwischen PBX und aktueller Serving-TBS mit Edge-Medienpfad.

| Eigenschaft           | Wert                               |
|-----------------------|------------------------------------|
| Beispielhost und Port | 10.0.20.33:8300/tcp für Management |
| Abhängigkeiten        | iot-gateway, mobility-core         |
| Konfigurationsziel    | /etc/netcore/sip-switch.toml       |



| Eigenschaft | Wert                                                     |
|-------------|----------------------------------------------------------|
| Vorlage     | system-backend/sip-switch/config/sip-switch.example.toml |
| Unit        | netcore-sip-switch.service                               |
| Installer   | system-backend/sip-switch/install/install.sh             |

**Fachtest.** Ein- und ausgehende Calls, beide RTP-Richtungen und Phase-11c-Fallback testen.

**Ausfall.** Zentrale SIP-Route fehlt; eingerichteter lokaler Asterisk/PBX-Fallback kann übernehmen.

**Erstdiagnose.** `systemctl status netcore-sip-switch.service` und `journalctl -u netcore-sip-switch.service -b`; danach `GET /health/live`, `GET /health/ready` und den fachlichen Test. Bei `ready=503` die oben genannten Abhängigkeiten prüfen.

#### Konfigurationsfelder

| Abschnitt           | Schlüssel im Beispiel                                                                                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [service]           | name, phase, mode                                                                                                                                                              |
| [server]            | bind                                                                                                                                                                           |
| [security]          | mode                                                                                                                                                                           |
| [storage]           | state_file, event_log, audit_log                                                                                                                                               |
| [mqtt]              | enabled, host, port, topic_prefix, client_id                                                                                                                                   |
| [mobility_core]     | enabled, base_url, timeout_secs                                                                                                                                                |
| [asterisk]          | enabled, binary, config_dir, agi_script, sip_bind, rtp_start, rtp_end                                                                                                          |
| [management]        | route_workers, side_effect_queue_size, call_history_limit, event_history_limit, probe_interval_secs                                                                            |
| [pbx]               | mode, endpoint_id, host, port, transport, username, auth_username, password, from_user, from_domain, contact_user, registration_id, registration_expiration_secs, allow, match |
| [routing]           | tetra_number_prefix, strip_tetra_prefix, pbx_outbound_prefix, strip_pbx_outbound_prefix, accept_stale_routes, require_tbs_contact, dial_timeout_secs                           |
| [[tbs]]             | node_id, endpoint_id, username, password, enabled, max_contacts, aliases                                                                                                       |
| [[number_mappings]] | number, target_type, target, enabled                                                                                                                                           |

## 14.25 Provisioning Core als Zusatzdienst

Der Provisioning Core ist im Cargo-Workspace und in `system-backend/services.toml` enthalten, aber **nicht** im 25er-Inventory. Sein Standardport ist 8125/tcp. Er schreibt auf Subscriber Core und Group Core und besitzt keine direkte TBS-Verbindung. Benutzeroberfläche und Mitgliedschaftsmatrix sind implementierte Verwaltungswege; die Open-Lab-Konfiguration hat keine Anmeldung oder TLS. Installation über `system-backend/provisioning-core/install/install.sh` und Unit `netcore-provisioning-core.service`; Zielkonfiguration `/etc/netcore/provisioning-core.toml`.

| Abschnitt  | Schlüssel im Beispiel                     |
|------------|-------------------------------------------|
| [server]   | bind                                      |
| [upstream] | subscriber_core, group_core, timeout_secs |
| [security] | mode, allow_remote_management             |
| [limits]   | max_body_bytes                            |

## 14.26 TBS Konfigurationsindex

Dieser Index kommt aus `Docs/basisstation.config.sanitized.example.toml`. Verschachtelte oder wiederholte Blöcke sind mit Punktnotation zusammengefasst. Er dokumentiert die vorhandenen **Beispielschlüssel** und ist kein vollständiges Rust-Typschema. Unkommentierte Beispielwerte müssen für den realen Standort überprüft werden.

| Abschnitt        | Aktive Schlüssel im bereinigten Beispiel                                                                                                                                                                                                                                                                                                        |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Wurzel           | config_version, stack_mode, service_name                                                                                                                                                                                                                                                                                                        |
| phy_io           | backend                                                                                                                                                                                                                                                                                                                                         |
| phy_io.soapysdr  | tx_freq, rx_freq, sample_rate,<br>tx_center_freq, rx_center_freq                                                                                                                                                                                                                                                                                |
| net_info         | mcc, mnc                                                                                                                                                                                                                                                                                                                                        |
| cell_info        | freq_band, main_carrier, secondary_carrier,<br>duplex_spacing, freq_offset,<br>reverse_operation, location_area,<br>colour_code, timezone, local_ssi_ranges,<br>system_wide_services, voice_service,<br>registration, deregistration,<br>no_minimum_mode, migration,<br>circuit_mode_data_service, sndcp_service,<br>advanced_link, system_code |
| cell_info.wap_ip | enabled, address, port, response_ttl,<br>dynamic_pool_prefix,<br>dynamic_pool_first_host,<br>dynamic_pool_last_host, allow_static_ipv4,<br>accept_empty_probe, accept_root_path,<br>accept_status_path, accept_status_wml_path,<br>max_request_payload_bytes,<br>assume_pdch_ready_after_data_transmit,                                         |

| Abschnitt                              | Aktive Schlüssel im bereinigten Beispiel                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        | pdu_priority_max, ready_timer_code, standby_timer_code, response_wait_timer_code, mtu_code, network_default_data_priority, max_contexts_per_issi, max_total_contexts, strict_source_address                                                                                                                                                                                                                                                                                                                                           |
| cell_info.packet_data_gateway          | enabled, interface_name, prefix_len, auto_configure, enable_ipv4_forwarding, managed_forwarding, allow_unsolicited_inbound, nat_mode, firewall_backend, dns_servers, channel_capacity, max_pdch_bearers, reserved_voice_slots, prefer_secondary_carrier, downlink_queue_packets_per_context, downlink_queue_bytes_per_context, downlink_queue_ttl_secs, page_retry_secs, fragment_reassembly_timeout_secs, fragment_reassembly_max_datagrams, fragment_reassembly_max_bytes, automatic_filter_ttl_secs, automatic_filter_max_bindings |
| cell_info.sds_command_control          | authorized_issis                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| cell_info.sds_command_control.commands | status_code, action                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| recovery                               | enabled, reactive_enabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| health                                 | enabled, snapshot_interval_secs                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| wx_service                             | enabled, service_issi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| telegram_alerts                        | enabled, bot_token, chat_ids, alert_connect, alert_disconnect, alert_t351, alert_lip, alert_backhaul, alert_critical_logs                                                                                                                                                                                                                                                                                                                                                                                                             |
| dashboard                              | port, bind, username, password                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| media_library                          | enabled, base_url, station_id, publish_recordings, recording_source_base_url, auto_approve_recordings, audio_source_enabled, only_ready, only_approved, retry_seconds, request_timeout_seconds, download_timeout_seconds, max_list_entries                                                                                                                                                                                                                                                                                            |
| recording                              | enabled, active, directory, mode, selected_groups, minimum_free_space_mb, retention_days, max_recording_minutes, idle_finalize_secs, max_list_entries, archive_enabled, archive_directory,                                                                                                                                                                                                                                                                                                                                            |

| Abschnitt                       | Aktive Schlüssel im bereinigten Beispiel                                                                                                                                                                                                                                             |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                 | archive_retry_seconds                                                                                                                                                                                                                                                                |
| audio_player                    | enabled, directory, cache_directory, source_issi, default_priority, max_file_size_mb, max_duration_seconds, lead_in_silence_blocks, tail_silence_blocks, group_release_guard_seconds, individual_answer_timeout_seconds, ffmpeg_path                                                 |
| audio_player.shares             | id, name, path                                                                                                                                                                                                                                                                       |
| netcore_directory               | enabled, base_url, timeout_ms                                                                                                                                                                                                                                                        |
| control_room                    | enabled, host, port, use_tls, endpoint_path, node_id, station_name, site, central_sds_routing                                                                                                                                                                                        |
| edge_fallback                   | enabled, enter_after_secs, recover_after_secs, unknown_service_is_available, service_matrix_lease_secs, policy_cache_path, policy_cache_max_age_secs, keep_last_known_policy, event_spool_path, event_spool_max_entries, event_spool_max_bytes, replay_batch_size, required_services |
| edge_fallback.service_fallbacks | node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, control-room, observability, application-gateway, media-library                                                    |
| brew                            | host, port, tls, username, password, reconnect_delay_secs, feature_rssi_export                                                                                                                                                                                                       |
| asterisk                        | enabled, outbound_prefix, strip_outbound_prefix, inbound_prefix, register, codec, service_numbers, rtp_port_min, rtp_port_max, bind_addr, bind_port, remote_host, remote_port, contact_host, from_domain, local_user, auth_user, password, realm                                     |

### 14.27 Beispielwerte der Backend-Konfigurationen

Die Tabellen zeigen Werte aus den **eingesetzten Beispiel-TOML-Dateien**, keine garantierten Runtime-Defaults und keine Empfehlung, sie unverändert zu übernehmen. Geheime Zugangsdaten, Schlüssel, Token und persönliche Zielkennungen werden nicht

wiedergegeben. Wiederholte TOML-Objektlisten zeigen nur den ersten Eintrag und die Anzahl der Beispiele; die vollständige Datei bleibt für eine konkrete Einrichtung maßgeblich. Damit lassen sich Typ, Schalter, Pfade, Zeitlimits und URL-Beziehungen vor einem Update schnell vergleichen.

#### 14.27.1 node-gateway

Quelle: `system-backend/node-gateway/config/node-gateway.example.toml`. Bei jeder Standortinstallation durch die ausgerollte Datei `/etc/netcore/node-gateway.toml` und das passende Runtime-Schema abgleichen.

| TOML-Pfad                                                 | Beispielwert                                      |
|-----------------------------------------------------------|---------------------------------------------------|
| <code>server.bind</code>                                  | <code>0.0.0.0:8080</code>                         |
| <code>server.node_path</code>                             | <code>/ws/node</code>                             |
| <code>server.backend_path</code>                          | <code>/ws/backend</code>                          |
| <code>server.history_limit</code>                         | <code>1000</code>                                 |
| <code>server.stale_after_secs</code>                      | <code>20</code>                                   |
| <code>server.hello_timeout_secs</code>                    | <code>10</code>                                   |
| <code>server.application_ping_secs</code>                 | <code>15</code>                                   |
| <code>security.mode</code>                                | <code>open_lab</code>                             |
| <code>security.allow_remote_management</code>             | <code>true</code>                                 |
| <code>limits.max_message_bytes</code>                     | <code>1048576</code>                              |
| <code>limits.max_http_body_bytes</code>                   | <code>1048576</code>                              |
| <code>service_monitor.enabled</code>                      | <code>true</code>                                 |
| <code>service_monitor.interval_secs</code>                | <code>5</code>                                    |
| <code>service_monitor.timeout_ms</code>                   | <code>1500</code>                                 |
| <code>service_monitor.failure_threshold</code>            | <code>2</code>                                    |
| <code>service_monitor.recovery_threshold</code>           | <code>2</code>                                    |
| <code>service_monitor.targets</code>                      | 23 Beispielobjekt(e), erster Eintrag unten        |
| <code>service_monitor.targets[0].name</code>              | <code>mobility-core</code>                        |
| <code>service_monitor.targets[0].url</code>               | <code>http://10.0.20.11:8090/health/ready</code>  |
| <code>service_monitor.targets[0].critical_for_edge</code> | <code>true</code>                                 |
| <code>service_monitor.targets[0].fallback_mode</code>     | <code>local_registration_and_location_area</code> |

### 14.27.2 mobility-core

Quelle: system-backend/mobility-core/config/mobility-core.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/mobility-core.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                        | Beispielwert                   |
|----------------------------------|--------------------------------|
| server.bind                      | 0.0.0.0:8090                   |
| server.history_limit             | 2000                           |
| server.transfer_timeout_secs     | 45                             |
| node_gateway.url                 | ws://10.0.1.30:8080/ws/backend |
| node_gateway.reconnect_secs      | 5                              |
| security.mode                    | open_lab                       |
| security.allow_remote_management | true                           |
| limits.max_body_bytes            | 1048576                        |
| limits.max_transfers             | 10000                          |
| limits.max_subscribers           | 100000                         |

### 14.27.3 subscriber-core

Quelle: system-backend/subscriber-core/config/subscriber-core.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/subscriber-core.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                             | Beispielwert                                              |
|---------------------------------------|-----------------------------------------------------------|
| server.bind                           | 0.0.0.0:8100                                              |
| server.history_limit                  | 2000                                                      |
| node_gateway.url                      | ws://127.0.0.1:8080/ws/backend                            |
| node_gateway.reconnect_secs           | 5                                                         |
| storage.database_path                 | /var/lib/netcore-subscriber-core/<br>subscribers.json     |
| storage.backup_path                   | /var/lib/netcore-subscriber-core/<br>subscribers.json.bak |
| access_policy.mode                    | allow_list                                                |
| access_policy.auto_sync               | true                                                      |
| access_policy.disconnect_unauthorized | true                                                      |
| access_policy.sync_timeout_secs       | 30                                                        |

| TOML-Pfad                        | Beispielwert |
|----------------------------------|--------------|
| security.mode                    | open_lab     |
| security.allow_remote_management | true         |
| limits.max_body_bytes            | 2097152      |
| limits.max_subscribers           | 100000       |
| limits.max_groups_per_subscriber | 1024         |

#### 14.27.4 group-core

Quelle: system-backend/group-core/config/group-core.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/group-core.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                        | Beispielwert                                |
|----------------------------------|---------------------------------------------|
| server.bind                      | 0.0.0.0:8110                                |
| server.history_limit             | 2000                                        |
| node_gateway.url                 | ws://10.0.1.XX:8080/ws/backend              |
| node_gateway.reconnect_secs      | 5                                           |
| storage.database_path            | /var/lib/netcore-group-core/groups.json     |
| storage.backup_path              | /var/lib/netcore-group-core/groups.json.bak |
| policy.allow_unlisted_groups     | false                                       |
| policy.enforce_memberships       | true                                        |
| policy.reconcile_registered      | true                                        |
| policy.auto_sync                 | true                                        |
| policy.sync_timeout_secs         | 30                                          |
| policy.dgna_timeout_secs         | 30                                          |
| security.mode                    | open_lab                                    |
| security.allow_remote_management | true                                        |
| limits.max_body_bytes            | 2097152                                     |
| limits.max_groups                | 65536                                       |
| limits.max_memberships           | 1000000                                     |

### 14.27.5 call-control

Quelle: system-backend/call-control/config/call-control.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/call-control.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                              | Beispielwert                                 |
|----------------------------------------|----------------------------------------------|
| server.bind                            | 0.0.0.0:8120                                 |
| server.history_limit                   | 2000                                         |
| node_gateway.url                       | ws://10.0.1.XX:8080/ws/backend               |
| node_gateway.reconnect_secs            | 5                                            |
| mobility_core.enabled                  | true                                         |
| mobility_core.base_url                 | http://10.0.1.XX:8090                        |
| mobility_core.timeout_ms               | 1500                                         |
| mobility_core.allow_local_fallback     | false                                        |
| mobility_core.accept_stale_route       | false                                        |
| storage.database_path                  | /var/lib/netcore-call-control/calls.json     |
| storage.backup_path                    | /var/lib/netcore-call-control/calls.json.bak |
| calls.command_timeout_secs             | 30                                           |
| calls.restore_timeout_secs             | 45                                           |
| calls.reconcile_interval_secs          | 2                                            |
| calls.auto_target_affiliated_nodes     | true                                         |
| calls.release_partial_start_on_failure | false                                        |
| calls.allow_operator_force_floor       | true                                         |
| security.mode                          | open_lab                                     |
| security.allow_remote_management       | true                                         |
| limits.max_body_bytes                  | 2097152                                      |
| limits.max_calls                       | 100000                                       |
| limits.max_legs_per_call               | 1024                                         |
| limits.max_pending_commands            | 20000                                        |



### 14.27.6 media-switch

Quelle: system-backend/media-switch/config/media-switch.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/media-switch.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                                | Beispielwert                                   |
|------------------------------------------|------------------------------------------------|
| server.bind                              | 0.0.0.0:8130                                   |
| server.history_limit                     | 2000                                           |
| node_gateway.url                         | ws://10.0.1.20:8080/ws/backend                 |
| node_gateway.reconnect_secs              | 2                                              |
| call_control.url                         | http://10.0.1.24:8120/api/v1/calls             |
| call_control.events_url                  | ws://10.0.1.24:8120/ws/media                   |
| call_control.route_ready_url             | http://10.0.1.24:8120/api/v1/media/route-ready |
| call_control.reconcile_secs              | 15                                             |
| call_control.reconnect_secs              | 1                                              |
| call_control.request_timeout_secs        | 2                                              |
| media.frame_duration_ms                  | 60                                             |
| media.jitter_buffer_frames               | 2                                              |
| media.min_jitter_buffer_frames           | 1                                              |
| media.max_jitter_buffer_frames           | 12                                             |
| media.adaptive_jitter                    | true                                           |
| media.adaptive_jitter_up_threshold_ms    | 18                                             |
| media.adaptive_jitter_down_stable_frames | 120                                            |
| media.cold_start_buffer_frames           | 5                                              |
| media.cold_start_buffer_max_age_ms       | 600                                            |
| media.session_idle_secs                  | 30                                             |
| media.max_frames_per_tick                | 256                                            |
| media.allow_same_leg_loopback            | false                                          |
| media.tap_history_frames                 | 256                                            |
| media.recorder_tap_history_frames        | 20000                                          |
| security.mode                            | open_lab                                       |

| TOML-Pfad                        | Beispielwert |
|----------------------------------|--------------|
| security.allow_remote_management | true         |
| limits.max_body_bytes            | 1048576      |
| limits.max_sessions              | 10000        |
| limits.max_streams               | 50000        |
| limits.max_pending_frames        | 100000       |

#### 14.27.7 recorder

Quelle: `system-backend/recorder/config/recorder.example.toml`. Bei jeder Standortinstallation durch die ausgerollte Datei `/etc/netcore/recorder.toml` und das passende Runtime-Schema abgleichen.

| TOML-Pfad                         | Beispielwert                               |
|-----------------------------------|--------------------------------------------|
| server.bind                       | 0.0.0.0:8140                               |
| server.history_limit              | 2000                                       |
| media_switch.tap_url              | http://10.0.1.25:8130/api/v1/recorder/taps |
| media_switch.sessions_url         | http://10.0.1.25:8130/api/v1/sessions      |
| media_switch.poll_interval_ms     | 100                                        |
| media_switch.session_reconcile_ms | 1000                                       |
| media_switch.request_timeout_secs | 3                                          |
| media_switch.batch_limit          | 500                                        |
| storage.root                      | /var/lib/netcore-recorder/recordings       |
| storage.export_root               | /var/lib/netcore-recorder/exports          |
| storage.frame_duration_ms         | 60                                         |
| storage.session_absent_grace_secs | 3                                          |
| storage.maximum_idle_secs         | 600                                        |
| storage.default_retention_days    | 30                                         |
| storage.retention_scan_secs       | 60                                         |
| storage.fsync_every_frames        | 50                                         |
| storage.minimum_free_space_mb     | 512                                        |
| security.mode                     | open_lab                                   |
| security.allow_remote_management  | true                                       |

| TOML-Pfad                    | Beispielwert |
|------------------------------|--------------|
| security.allow_delete        | true         |
| limits.max_body_bytes        | 1048576      |
| limits.max_active_recordings | 1000         |
| limits.max_recordings        | 100000       |

#### 14.27.8 sds-router

Quelle: `system-backend/sds-router/config/sds-router.example.toml`. Bei jeder Standortinstallation durch die ausgerollte Datei `/etc/netcore/sds-router.toml` und das passende Runtime-Schema abgleichen.

| TOML-Pfad                        | Beispielwert                                  |
|----------------------------------|-----------------------------------------------|
| server.bind                      | 0.0.0.0:8150                                  |
| server.history_limit             | 4000                                          |
| node_gateway.url                 | ws://10.0.1.20:8080/ws/backend                |
| node_gateway.reconnect_secs      | 5                                             |
| storage.database_path            | /var/lib/netcore-sds-router/messages.json     |
| storage.backup_path              | /var/lib/netcore-sds-router/messages.json.bak |
| routing.default_ttl_secs         | 300                                           |
| routing.max_ttl_secs             | 86400                                         |
| routing.max_attempts             | 5                                             |
| routing.initial_retry_secs       | 2                                             |
| routing.max_retry_secs           | 60                                            |
| routing.dedupe_window_secs       | 30                                            |
| routing.presence_timeout_secs    | 90                                            |
| routing.authoritative_ingress    | true                                          |
| security.mode                    | open_lab                                      |
| security.allow_remote_management | true                                          |
| security.mask_payload_in_list    | false                                         |
| limits.max_body_bytes            | 2097152                                       |
| limits.max_payload_bytes         | 2048                                          |
| limits.max_messages              | 100000                                        |

| TOML-Pfad         | Beispielwert |
|-------------------|--------------|
| limits.max_routes | 4096         |

#### 14.27.9 packet-core

Quelle: system-backend/packet-core/config/packet-core.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/packet-core.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                            | Beispielwert                                |
|--------------------------------------|---------------------------------------------|
| server.bind                          | 0.0.0.0:8160                                |
| server.history_limit                 | 5000                                        |
| node_gateway.url                     | ws://127.0.0.1:8080/ws/backend              |
| node_gateway.reconnect_secs          | 5                                           |
| storage.database_path                | /var/lib/netcore-packet-core/state.json     |
| storage.backup_path                  | /var/lib/netcore-packet-core/state.json.bak |
| packet.mode                          | shadow                                      |
| packet.ready_timer_secs              | 5                                           |
| packet.standby_timer_secs            | 300                                         |
| packet.response_wait_secs            | 10                                          |
| packet.context_ready_secs            | 30                                          |
| packet.default_mtu                   | 1500                                        |
| packet.max_n_pdu_bytes               | 65535                                       |
| packet.max_contexts_per_subscriber   | 14                                          |
| packet.max_total_contexts            | 4096                                        |
| packet.strict_source_address         | true                                        |
| packet.preserve_context_on_node_loss | true                                        |
| address_pool.network_prefix          | [10, 44, 0]                                 |
| address_pool.first_host              | 2                                           |
| address_pool.last_host               | 254                                         |
| address_pool.gateway                 | 10.44.0.1                                   |
| address_pool.allow_static            | true                                        |
| fragmentation.timeout_secs           | 30                                          |

| TOML-Pfad                                  | Beispielwert |
|--------------------------------------------|--------------|
| fragmentation.max_datagrams                | 256          |
| fragmentation.max_total_bytes              | 8388608      |
| fragmentation.max_fragments_per_datagram   | 512          |
| fragmentation.reject_overlaps              | true         |
| flow_control.max_queue_packets_per_context | 64           |
| flow_control.max_queue_bytes_per_context   | 262144       |
| flow_control.queue_ttl_secs                | 30           |
| flow_control.action_retry_secs             | 5            |
| flow_control.action_max_attempts           | 5            |
| security.mode                              | open_lab     |
| security.allow_remote_management           | true         |
| security.expose_payloads                   | true         |
| limits.max_body_bytes                      | 1048576      |
| limits.max_events                          | 10000        |
| limits.max_actions                         | 10000        |
| limits.max_payload_bytes                   | 65535        |

#### 14.27.10 ip-gateway

Quelle: system-backend/ip-gateway/config/ip-gateway.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/ip-gateway.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                      | Beispielwert                               |
|--------------------------------|--------------------------------------------|
| server.bind                    | 0.0.0.0:8170                               |
| packet_core.url                | http://127.0.0.1:8160                      |
| packet_core.poll_interval_ms   | 250                                        |
| packet_core.context_refresh_ms | 1000                                       |
| packet_core.request_timeout_ms | 2000                                       |
| packet_core.outbox_batch       | 250                                        |
| storage.database_path          | /var/lib/netcore-ip-gateway/state.json     |
| storage.backup_path            | /var/lib/netcore-ip-gateway/state.json.bak |

| TOML-Pfad                       | Beispielwert                         |
|---------------------------------|--------------------------------------|
| interface.mode                  | shadow                               |
| interface.name                  | ntc-tun0                             |
| interface.address               | 10.0.0.1/24                          |
| interface.network               | 10.0.0.0/24                          |
| interface.mtu                   | 480                                  |
| interface.owner_user            | netcore                              |
| interface.delete_on_exit        | true                                 |
| routing.enable_ipv4_forwarding  | true                                 |
| routing.reconcile_interval_secs | 5                                    |
| routing.install_connected_route | true                                 |
| nat.enabled                     | true                                 |
| nat.masquerade                  | true                                 |
| nat.egress_interface            | eth0                                 |
| firewall.enabled                | true                                 |
| firewall.default_forward_policy | drop                                 |
| firewall.allow_established      | true                                 |
| firewall.allow_general_internet | true                                 |
| firewall.allow_icmp             | true                                 |
| firewall.log_drops              | false                                |
| dns.enabled                     | true                                 |
| dns.bind                        | 10.0.0.1:53                          |
| dns.upstream                    | 1.1.1.1:53                           |
| dns.local_domain                | netcore.test                         |
| dns.ttl_secs                    | 30                                   |
| dns.query_timeout_ms            | 2000                                 |
| test_server.enabled             | true                                 |
| test_server.bind                | 0.0.0.0:8088                         |
| test_server.udp_echo_bind       | 0.0.0.0:7007                         |
| capture.directory               | /var/lib/netcore-ip-gateway/captures |

| TOML-Pfad                        | Beispielwert |
|----------------------------------|--------------|
| capture.max_captures             | 64           |
| capture.max_file_bytes           | 268435456    |
| capture.snaplen                  | 65535        |
| security.mode                    | open_lab     |
| security.allow_remote_management | true         |
| limits.max_body_bytes            | 2097152      |
| limits.max_events                | 100000       |
| limits.max_flows                 | 100000       |
| limits.max_packet_bytes          | 65535        |

#### 14.27.11 security-core

Quelle: system-backend/security-core/config/security-core.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/security-core.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                         | Beispielwert                                  |
|-----------------------------------|-----------------------------------------------|
| server.bind                       | 0.0.0.0:8180                                  |
| server.history_limit              | 5000                                          |
| node_gateway.url                  | ws://127.0.0.1:8080/ws/backend                |
| node_gateway.reconnect_secs       | 5                                             |
| node_gateway.observe_nodes        | true                                          |
| storage.database_path             | /var/lib/netcore-security-core/state.json     |
| storage.backup_path               | /var/lib/netcore-security-core/state.json.bak |
| storage.lab_seed_path             | /var/lib/netcore-security-core/lab-auth.seed  |
| policy.operating_mode             | shadow                                        |
| policy.default_security_class     | 1                                             |
| policy.minimum_security_class     | 1                                             |
| policy.authentication_required    | true                                          |
| policy.allow_class1_fallback      | true                                          |
| policy.reject_unknown_subscribers | false                                         |

| TOML-Pfad                               | Beispielwert    |
|-----------------------------------------|-----------------|
| policy.disable_after_failures           | false           |
| authentication.provider                 | lab_hmac_sha256 |
| authentication.challenge_bytes          | 16              |
| authentication.response_bytes           | 16              |
| authentication.challenge_ttl_secs       | 30              |
| authentication.max_attempts             | 3               |
| authentication.lockout_secs             | 300             |
| authentication.issue_dck_on_success     | true            |
| dck.key_bytes                           | 16              |
| dck.ttl_secs                            | 3600            |
| dck.rotate_before_secs                  | 300             |
| dck.max_active_per_subscriber           | 2               |
| security.mode                           | open_lab        |
| security.allow_remote_management        | true            |
| security.expose_ephemeral_edge_material | true            |
| limits.max_body_bytes                   | 1048576         |
| limits.max_profiles                     | 100000          |
| limits.max_contexts                     | 20000           |
| limits.max_actions                      | 20000           |
| limits.max_alarms                       | 20000           |
| limits.max_audit                        | 100000          |

#### 14.27.12 kmf

Quelle: system-backend/kmf/config/kmf.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/kmf.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad             | Beispielwert                    |
|-----------------------|---------------------------------|
| server.bind           | 0.0.0.0:8190                    |
| server.history_limit  | 5000                            |
| storage.database_path | /var/lib/netcore-kmf/state.json |
| storage.vault_path    | /var/lib/netcore-kmf/vault.json |



| TOML-Pfad                               | Beispielwert                    |
|-----------------------------------------|---------------------------------|
| storage.master_key_path                 | /var/lib/netcore-kmf/master.key |
| storage.backup_dir                      | /var/lib/netcore-kmf/backups    |
| storage.bootstrap_dir                   | /var/lib/netcore-kmf/bootstrap  |
| policy.operating_mode                   | shadow                          |
| policy.default_key_bytes                | 16                              |
| policy.default_crypto_period_secs       | 86400                           |
| policy.rotation_lead_secs               | 3600                            |
| policy.require_dual_approval            | true                            |
| policy.allow_overlapping_crypto_periods | true                            |
| policy.auto_retire_predecessor          | true                            |
| vault.provider                          | lab_file_vault                  |
| vault.master_key_bytes                  | 32                              |
| vault.fsync                             | true                            |
| otar.action_ttl_secs                    | 600                             |
| otar.max_attempts                       | 5                               |
| otar.retry_backoff_secs                 | 15                              |
| otar.max_claim_batch                    | 100                             |
| security.mode                           | open_lab                        |
| security.allow_remote_management        | true                            |
| security.expose_raw_keys                | false                           |
| limits.max_body_bytes                   | 1048576                         |
| limits.max_keys                         | 100000                          |
| limits.max_nodes                        | 10000                           |
| limits.max_jobs                         | 100000                          |
| limits.max_actions                      | 500000                          |
| limits.max_audit                        | 100000                          |

#### 14.27.13 transit

Quelle: system-backend/transit/config/transit.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/transit.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                         | Beispielwert                                                                                 |
|-----------------------------------|----------------------------------------------------------------------------------------------|
| server.bind                       | 0.0.0.0:8200                                                                                 |
| server.history_limit              | 5000                                                                                         |
| storage.database_path             | /var/lib/netcore-transit/state.json                                                          |
| storage.backup_path               | /var/lib/netcore-transit/state.json.bak                                                      |
| region.region_id                  | region-a                                                                                     |
| region.swmi_id                    | netcore-swmi-a                                                                               |
| region.display_name               | NetCore Region A                                                                             |
| region.advertised_endpoint        | http://10.0.10.12:8200                                                                       |
| region.protocol_version           | netcore-transit-v1                                                                           |
| region.operating_mode             | shadow                                                                                       |
| region.capabilities               | ['mobility', 'individual_call',<br>'group_call', 'sds', 'media',<br>'supplementary_service'] |
| routing.max_hops                  | 8                                                                                            |
| routing.dedupe_ttl_secs           | 900                                                                                          |
| routing.session_idle_ttl_secs     | 3600                                                                                         |
| routing.route_stale_secs          | 120                                                                                          |
| routing.prefer_direct_region_peer | true                                                                                         |
| routing.allow_transitive_routing  | true                                                                                         |
| routing.allow_dynamic_peers       | false                                                                                        |
| routing.fail_closed_on_loop       | true                                                                                         |
| transport.connect_timeout_ms      | 2000                                                                                         |
| transport.io_timeout_ms           | 5000                                                                                         |
| transport.heartbeat_interval_secs | 5                                                                                            |
| transport.peer_timeout_secs       | 20                                                                                           |
| transport.retry_backoff_secs      | 3                                                                                            |
| transport.max_attempts            | 5                                                                                            |
| transport.max_batch               | 100                                                                                          |
| security.mode                     | open_lab                                                                                     |
| security.allow_remote_management  | true                                                                                         |

| TOML-Pfad                   | Beispielwert  |
|-----------------------------|---------------|
| security.tls                | false         |
| security.token_auth         | [ausgelassen] |
| limits.max_body_bytes       | 4194304       |
| limits.max_peers            | 1000          |
| limits.max_routes           | 100000        |
| limits.max_sessions         | 100000        |
| limits.max_envelopes        | 500000        |
| limits.max_local_deliveries | 500000        |
| limits.max_events           | 100000        |

#### 14.27.14 application-gateway

Quelle: system-backend/application-gateway/config/application-gateway.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/application-gateway.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                          | Beispielwert                                            |
|------------------------------------|---------------------------------------------------------|
| server.bind                        | 0.0.0.0:8220                                            |
| server.public_base_url             | http://127.0.0.1:8220                                   |
| server.max_body_bytes              | 4194304                                                 |
| server.history_limit               | 5000                                                    |
| storage.state_path                 | /var/lib/netcore-application-gateway/<br>state.json     |
| storage.state_backup_path          | /var/lib/netcore-application-gateway/<br>state.json.bak |
| storage.secrets_path               | [ausgelassen]                                           |
| storage.spool_dir                  | /var/lib/netcore-application-gateway/spool              |
| storage.backup_dir                 | /var/lib/netcore-application-gateway/<br>backups        |
| security.mode                      | open_lab                                                |
| security.management_token_auth     | [ausgelassen]                                           |
| security.management_tls            | false                                                   |
| security.allow_remote_management   | true                                                    |
| security.connector_secrets_allowed | [ausgelassen]                                           |

| TOML-Pfad                           | Beispielwert                                                                           |
|-------------------------------------|----------------------------------------------------------------------------------------|
| security.warning_banner             | OPEN LAB: no login, no management tokens and no TLS. Isolated management network only. |
| runtime.operating_mode              | shadow                                                                                 |
| runtime.worker_interval_ms          | 1000                                                                                   |
| runtime.probe_interval_secs         | 30                                                                                     |
| runtime.default_ttl_secs            | 300                                                                                    |
| runtime.max_attempts                | 6                                                                                      |
| runtime.base_backoff_secs           | 2                                                                                      |
| runtime.max_backoff_secs            | 120                                                                                    |
| runtime.dedupe_window_secs          | 600                                                                                    |
| runtime.max_response_bytes          | 65536                                                                                  |
| runtime.max_artifact_bytes          | 33554432                                                                               |
| runtime.max_events                  | 20000                                                                                  |
| runtime.max_deliveries              | 50000                                                                                  |
| runtime.max_tts_jobs                | 5000                                                                                   |
| runtime.max_audit_records           | 50000                                                                                  |
| runtime.event_retention_secs        | 604800                                                                                 |
| runtime.delivery_retention_secs     | 1209600                                                                                |
| runtime.audit_retention_secs        | 2592000                                                                                |
| connectors                          | 12 Beispielobjekt(e), erster Eintrag unten                                             |
| connectors[0].connector_id          | sds-router                                                                             |
| connectors[0].display_name          | SDS Router                                                                             |
| connectors[0].kind                  | sds_router                                                                             |
| connectors[0].direction             | outbound                                                                               |
| connectors[0].endpoint              | http://127.0.0.1:8150/api/v1/messages                                                  |
| connectors[0].health_endpoint       | http://127.0.0.1:8150/health/ready                                                     |
| connectors[0].enabled               | true                                                                                   |
| connectors[0].timeout_ms            | 5000                                                                                   |
| connectors[0].rate_limit_per_minute | 600                                                                                    |

| TOML-Pfad                               | Beispielwert                                          |
|-----------------------------------------|-------------------------------------------------------|
| connectors[0].circuit_failure_threshold | 5                                                     |
| connectors[0].circuit_open_secs         | 60                                                    |
| connectors[0].required_secrets          | [ausgelassen]                                         |
| connectors[0].settings.source_issi      | 9999                                                  |
| connectors[0].settings.sds_type         | 4                                                     |
| connectors[0].settings.protocol_id      | 0                                                     |
| connectors[0].settings.priority         | 3                                                     |
| rules                                   | 2 Beispielobjekt(e), erster Eintrag unten             |
| rules[0].rule_id                        | manual-to-sds                                         |
| rules[0].name                           | Manual messages to SDS Router                         |
| rules[0].enabled                        | true                                                  |
| rules[0].priority                       | 100                                                   |
| rules[0].source_connector               | manual                                                |
| rules[0].event_type                     | sds.message                                           |
| rules[0].target_connector               | sds-router                                            |
| rules[0].template_id                    | sds-standard                                          |
| rules[0].stop_processing                | false                                                 |
| templates                               | 3 Beispielobjekt(e), erster Eintrag unten             |
| templates[0].template_id                | sds-standard                                          |
| templates[0].name                       | SDS Standard                                          |
| templates[0].kind                       | text                                                  |
| templates[0].body                       | {{text}}                                              |
| templates[0].content_type               | text/plain; charset=utf-8                             |
| templates[0].enabled                    | true                                                  |
| templates[0].target_connector           | sds-router                                            |
| templates[0].description                | Plain SDS text with destination supplied by the event |

### 14.27.15 media-library

Quelle: system-backend/media-library/config/media-library.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/media-library.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                          | Beispielwert                              |
|------------------------------------|-------------------------------------------|
| server.bind                        | 0.0.0.0:8230                              |
| server.public_base_url             | http://127.0.0.1:8230                     |
| server.max_body_bytes              | 100663296                                 |
| security.mode                      | open_lab                                  |
| security.token_auth                | [ausgelassen]                             |
| security.tls                       | false                                     |
| security.allow_remote_management   | true                                      |
| security.allow_delete              | true                                      |
| security.allow_url_import          | true                                      |
| security.allow_private_import_urls | true                                      |
| storage.root                       | /var/lib/netcore-media-library/assets     |
| storage.state_file                 | /var/lib/netcore-media-library/state.json |
| storage.temp_root                  | /var/lib/netcore-media-library/tmp        |
| storage.backup_root                | /var/lib/netcore-media-library/backups    |
| storage.archive_root               | /mnt/nfs-share/Media-Library              |
| storage.recording_archive_root     | /mnt/nfs-share/Recordings                 |
| storage.tts_archive_root           | /mnt/nfs-share/TTS-Dateien                |
| storage.max_asset_bytes            | 67108864                                  |
| storage.max_total_bytes            | 21474836480                               |
| storage.fsync_imports              | true                                      |
| runtime.operating_mode             | shadow                                    |
| runtime.worker_interval_ms         | 500                                       |
| runtime.probe_interval_secs        | 15                                        |
| runtime.import_timeout_secs        | 120                                       |
| runtime.max_assets                 | 10000                                     |
| runtime.max_jobs                   | 2000                                      |

| TOML-Pfad                       | Beispielwert                                 |
|---------------------------------|----------------------------------------------|
| runtime.max_events              | 5000                                         |
| runtime.max_audit_records       | 10000                                        |
| runtime.max_attempts            | 3                                            |
| runtime.frame_interval_ms       | 60                                           |
| runtime.auto_approve_tts        | false                                        |
| runtime.auto_archive_recordings | true                                         |
| runtime.auto_archive_tts        | true                                         |
| playout.mode                    | basisstation                                 |
| playout.default_station         | srv-m-tbs-01                                 |
| playout.request_timeout_secs    | 15                                           |
| playout.completion_timeout_secs | 900                                          |
| playout.poll_interval_ms        | 500                                          |
| playout.stations                | 1 Beispielobjekt(e), erster Eintrag unten    |
| playout.stations[0].id          | srv-m-tbs-01                                 |
| playout.stations[0].name        | SRV-M-TBS-01                                 |
| playout.stations[0].base_url    | http://10.0.1.22:8080                        |
| playout.stations[0].enabled     | true                                         |
| codec.frame_bytes               | 35                                           |
| codec.ffmpeg_command            | [14 Einträge]                                |
| codec.encoder_command           | [ ]                                          |
| codec.decoder_command           | [ ]                                          |
| tts.enabled                     | true                                         |
| tts.endpoint                    | http://127.0.0.1:5005                        |
| tts.template_directory          | /var/lib/netcore-media-library/tts/templates |
| tts.default_voice               | de-thorsten                                  |
| tts.default_speed               | 0.95                                         |
| tts.max_text_characters         | 2000                                         |
| tts.synthesis_timeout_secs      | 90                                           |
| tts.max_output_file_mb          | 25                                           |

| TOML-Pfad                                 | Beispielwert                              |
|-------------------------------------------|-------------------------------------------|
|                                           |                                           |
| tts.voices                                | 5 Beispielobjekt(e), erster Eintrag unten |
| tts.voices[0].id                          | de-thorsten                               |
| tts.voices[0].name                        | Deutsch – Thorsten (mittel)               |
| tts.voices[0].provider_voice              | de_DE-thorsten-medium                     |
| dependencies.media_switch_base_url        | http://127.0.0.1:8130                     |
| dependencies.recorder_base_url            | http://127.0.0.1:8140                     |
| dependencies.application_gateway_base_url | http://127.0.0.1:8220                     |

#### 14.27.16 control-room

Quelle: system-backend/control-room/config/control-room.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/control-room.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                         | Beispielwert                                       |
|-----------------------------------|----------------------------------------------------|
| server.bind                       | 0.0.0.0:9010                                       |
| server.node_path                  | /node                                              |
| server.ui_path                    | /ui                                                |
| server.history_limit              | 2000                                               |
| persistence.enabled               | true                                               |
| persistence.database_path         | /var/lib/netcore-control-room/control-room.sqlite3 |
| persistence.persist_events        | true                                               |
| persistence.persist_noisy_events  | false                                              |
| persistence.load_recent_limit     | 2000                                               |
| auth.enabled                      | false                                              |
| auth.allow_health_unauthenticated | true                                               |
| auth.node_token_env               | [ausgelassen]                                      |
| auth.bootstrap_username_env       | (leer)                                             |
| auth.bootstrap_password_env       | [ausgelassen]                                      |
| auth.bootstrap_role               | admin                                              |
| federation.enabled                | true                                               |



| TOML-Pfad                                    | Beispielwert                                          |
|----------------------------------------------|-------------------------------------------------------|
| federation.poll_interval_secs                | 5                                                     |
| federation.request_timeout_ms                | 1200                                                  |
| federation.failure_threshold                 | 3                                                     |
| federation.fetch_summaries                   | true                                                  |
| operations.state_path                        | /var/lib/netcore-control-room/<br>operations.json     |
| operations.backup_path                       | /var/lib/netcore-control-room/<br>operations.json.bak |
| operations.auto_service_incidents            | true                                                  |
| operations.incident_limit                    | 5000                                                  |
| operations.shift_log_limit                   | 10000                                                 |
| services                                     | 20 Beispielobjekt(e), erster Eintrag unten            |
| services[0].name                             | node-gateway                                          |
| services[0].display_name                     | Node Gateway                                          |
| services[0].kind                             | edge                                                  |
| services[0].base_url                         | http://10.0.20.10:8080                                |
| services[0].health_live                      | /health/live                                          |
| services[0].health_ready                     | /health/ready                                         |
| services[0].summary_path                     | /api/v1/status                                        |
| services[0].webui_path                       | /                                                     |
| services[0].critical                         | true                                                  |
| services[0].enabled                          | true                                                  |
| directory.hide_infrastructure                | true                                                  |
| service_monitor.targets                      | 6 Beispielobjekt(e), erster Eintrag unten             |
| service_monitor.targets[0].name              | hardware-gateway                                      |
| service_monitor.targets[0].url               | http://10.0.20.28:8250/health/ready                   |
| service_monitor.targets[0].critical_for_edge | false                                                 |
| service_monitor.targets[0].fallback_mode     | rack_telemetry_unavailable_local_radio_continues      |
| service_monitor.targets[0].depends_on        | ['iot-gateway']                                       |

**14.27.17 observability**

Quelle: system-backend/observability/config/observability.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/observability.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                        | Beispielwert                                                                   |
|----------------------------------|--------------------------------------------------------------------------------|
| server.bind                      | 0.0.0.0:8210                                                                   |
| server.history_limit             | 5000                                                                           |
| server.max_body_bytes            | 4194304                                                                        |
| storage.state_path               | /var/lib/netcore-observability/state.json                                      |
| storage.backup_path              | /var/lib/netcore-observability/<br>state.json.bak                              |
| storage.diagnostic_dir           | /var/lib/netcore-observability/diagnostics                                     |
| security.mode                    | open_lab                                                                       |
| security.token_auth              | [ausgelassen]                                                                  |
| security.tls                     | false                                                                          |
| security.allow_remote_management | true                                                                           |
| security.warning_banner          | OPEN LAB: no login, no tokens and no TLS.<br>Isolated management network only. |
| collection.scrape_interval_secs  | 15                                                                             |
| collection.request_timeout_ms    | 2000                                                                           |
| collection.max_response_bytes    | 2097152                                                                        |
| collection.scrape_on_start       | true                                                                           |
| collection.ingest_logs           | true                                                                           |
| collection.ingest_traces         | true                                                                           |
| retention.metric_retention_secs  | 86400                                                                          |
| retention.log_retention_secs     | 604800                                                                         |
| retention.trace_retention_secs   | 86400                                                                          |
| retention.audit_retention_secs   | 2592000                                                                        |
| retention.max_series             | 10000                                                                          |
| retention.max_samples_per_series | 5760                                                                           |
| retention.max_logs               | 100000                                                                         |
| retention.max_spans              | 50000                                                                          |

| TOML-Pfad                     | Beispielwert                                             |
|-------------------------------|----------------------------------------------------------|
| retention.max_alerts          | 10000                                                    |
| retention.max_audit_records   | 50000                                                    |
| stack.prometheus_url          | http://127.0.0.1:9090                                    |
| stack.grafana_url             | http://127.0.0.1:3000                                    |
| stack.loki_url                | http://127.0.0.1:3100                                    |
| stack.alertmanager_url        | http://127.0.0.1:9093                                    |
| stack.prometheus_ready_path   | /-/ready                                                 |
| stack.grafana_ready_path      | /api/health                                              |
| stack.loki_ready_path         | /ready                                                   |
| stack.alertmanager_ready_path | /-/ready                                                 |
| targets                       | 21 Beispielobjekt(e), erster Eintrag unten               |
| targets[0].target_id          | node-gateway                                             |
| targets[0].display_name       | Node Gateway                                             |
| targets[0].service            | node-gateway                                             |
| targets[0].base_url           | http://127.0.0.1:8080                                    |
| targets[0].metrics_path       | /metrics                                                 |
| targets[0].live_path          | /health/live                                             |
| targets[0].ready_path         | /health/ready                                            |
| targets[0].enabled            | true                                                     |
| targets[0].labels.environment | open-lab                                                 |
| targets[0].labels.component   | node-gateway                                             |
| alert_rules                   | 3 Beispielobjekt(e), erster Eintrag unten                |
| alert_rules[0].rule_id        | target-down                                              |
| alert_rules[0].name           | Target down                                              |
| alert_rules[0].description    | A monitored target does not answer its liveness endpoint |
| alert_rules[0].metric         | netcore_observability_target_up                          |
| alert_rules[0].comparator     | <                                                        |
| alert_rules[0].threshold      | 1.0                                                      |
| alert_rules[0].for_secs       | 30                                                       |

| TOML-Pfad                          | Beispielwert                                         |
|------------------------------------|------------------------------------------------------|
| alert_rules[0].severity            | critical                                             |
| alert_rules[0].enabled             | true                                                 |
| alert_rules[0].labels.source       | netcore-observability                                |
| alert_rules[0].annotations.runbook | Check service process, network path and /health/live |

#### 14.27.18 iot-gateway

Quelle: system-backend/iot-gateway/config/iot-gateway.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/iot-gateway.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                        | Beispielwert        |
|----------------------------------|---------------------|
| server.bind                      | 0.0.0.0:8240        |
| server.history_limit             | 1000                |
| server.max_body_bytes            | 1048576             |
| security.mode                    | open_lab            |
| security.allow_remote_management | true                |
| mqtt.host                        | 127.0.0.1           |
| mqtt.port                        | 1883                |
| mqtt.client_id                   | netcore-iot-gateway |
| mqtt.topic_prefix                | netcore/v1          |
| mqtt.keep_alive_secs             | 30                  |
| mqtt.clean_session               | false               |
| mqtt.reconnect_secs              | 3                   |
| mqtt.publish_timeout_secs        | 8                   |
| mqtt.qos                         | 1                   |
| mqtt.event_retain                | false               |
| mqtt.state_retain                | true                |
| mqtt.observe_commands            | true                |
| mqtt.execute_commands            | false               |
| home_assistant.enabled           | true                |

| TOML-Pfad                             | Beispielwert         |
|---------------------------------------|----------------------|
| home_assistant.discovery_enabled      | true                 |
| home_assistant.discovery_prefix       | homeassistant        |
| home_assistant.status_topic           | homeassistant/status |
| home_assistant.node_id                | netcore_tetra        |
| home_assistant.discovery_qos          | 1                    |
| home_assistant.discovery_retain       | true                 |
| home_assistant.expose_gateway         | true                 |
| home_assistant.expose_sources         | true                 |
| home_assistant.expose_virtual_devices | true                 |
| home_assistant.accept_state_ingress   | true                 |
| home_assistant.state_ingress_topic    | (leer)               |
| home_assistant.allow_command_egress   | false                |
| home_assistant.command_egress_topic   | (leer)               |
| homematic.enabled                     | false                |
| homematic.mode                        | home_assistant_mqtt  |
| homematic.ccu_host                    | 127.0.0.1            |
| homematic.ccu_port                    | 2010                 |
| homematic.poll_interval_ms            | 2000                 |
| homematic.request_timeout_ms          | 2500                 |
| homematic.allow_writes                | false                |
| commands.enabled                      | true                 |
| commands.mode                         | open_lab_sandbox     |
| commands.default_deny                 | true                 |
| commands.allow_retained               | false                |
| commands.default_ttl_secs             | 30                   |
| commands.max_ttl_secs                 | 300                  |
| commands.max_future_skew_secs         | 30                   |
| commands.publish_lifecycle_acks       | true                 |
| commands.ack_qos                      | 1                    |

| TOML-Pfad                           | Beispielwert                                   |
|-------------------------------------|------------------------------------------------|
| commands.ack_retain                 | false                                          |
| storage.state_dir                   | /var/lib/netcore-iot-gateway                   |
| storage.outbox_dir                  | outbox                                         |
| storage.dedup_file                  | dedup.json                                     |
| storage.command_inbox_file          | command-inbox.ndjson                           |
| storage.command_ledger_file         | command-ledger.json                            |
| storage.command_audit_file          | command-audit.ndjson                           |
| storage.virtual_state_file          | virtual-device-state.json                      |
| storage.external_state_file         | external-entity-state.json                     |
| storage.homematic_state_file        | homematic-datapoint-state.json                 |
| storage.dedup_limit                 | 50000                                          |
| storage.outbox_limit                | 20000                                          |
| storage.command_ledger_limit        | 50000                                          |
| polling.interval_ms                 | 2000                                           |
| polling.batch_limit                 | 500                                            |
| polling.request_timeout_ms          | 2500                                           |
| command_policies                    | 5 Beispielobjekt(e), erster Eintrag unten      |
| command_policies[0].id              | allow-openlab-virtual-relays                   |
| command_policies[0].enabled         | true                                           |
| command_policies[0].effect          | allow                                          |
| command_policies[0].command_types   | ['virtual.relay.set']                          |
| command_policies[0].target_types    | ['virtual_relay']                              |
| command_policies[0].target_prefixes | ['lab-']                                       |
| command_policies[0].max_ttl_secs    | 120                                            |
| command_policies[0].allow_dry_run   | true                                           |
| sources                             | 4 Beispielobjekt(e), erster Eintrag unten      |
| sources[0].id                       | node-gateway                                   |
| sources[0].url                      | http://node-gateway:8080/api/v1/events/netcore |
| sources[0].enabled                  | true                                           |

| TOML-Pfad | Beispielwert |
|-----------|--------------|
|           |              |

#### 14.27.19 hardware-gateway

Quelle: system-backend/hardware-gateway/config/hardware-gateway.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/hardware-gateway.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                         | Beispielwert                                        |
|-----------------------------------|-----------------------------------------------------|
| server.bind                       | 0.0.0.0:8250                                        |
| security.mode                     | open_lab                                            |
| mqtt.host                         | 127.0.0.1                                           |
| mqtt.port                         | 1883                                                |
| mqtt.topic_prefix                 | netcore/v1                                          |
| mqtt.client_id                    | netcore-hardware-gateway                            |
| storage.state_file                | /var/lib/netcore-hardware-gateway/<br>state.json    |
| storage.event_log                 | /var/lib/netcore-hardware-gateway/<br>events.ndjson |
| monitoring.heartbeat_timeout_secs | 30                                                  |
| monitoring.stale_after_secs       | 20                                                  |
| monitoring.outputs_enabled        | false                                               |
| thresholds                        | 3 Beispielobjekt(e), erster Eintrag unten           |
| thresholds[0].metric              | temperature_c                                       |
| thresholds[0].warning_above       | 40.0                                                |
| thresholds[0].critical_above      | 55.0                                                |

#### 14.27.20 rf-monitor

Quelle: system-backend/rf-monitor/config/rf-monitor.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/rf-monitor.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad     | Beispielwert |
|---------------|--------------|
| server.bind   | 0.0.0.0:8260 |
| security.mode | open_lab     |

| TOML-Pfad                                   | Beispielwert                              |
|---------------------------------------------|-------------------------------------------|
| mqtt.enabled                                | true                                      |
| mqtt.host                                   | 127.0.0.1                                 |
| mqtt.port                                   | 1883                                      |
| mqtt.topic_prefix                           | netcore/v1                                |
| mqtt.client_id                              | netcore-rf-monitor                        |
| storage.state_file                          | /var/lib/netcore-rf-monitor/state.json    |
| storage.event_log                           | /var/lib/netcore-rf-monitor/events.ndjson |
| monitoring.heartbeat_timeout_secs           | 20                                        |
| monitoring.event_memory_limit               | 1000                                      |
| monitoring.max_spectrum_bins                | 512                                       |
| monitoring.max_payload_bytes                | 524288                                    |
| thresholds.vswr_warning                     | 1.8                                       |
| thresholds.vswr_critical                    | 2.5                                       |
| thresholds.reflected_ratio_warning_percent  | 10.0                                      |
| thresholds.reflected_ratio_critical_percent | 20.0                                      |
| thresholds.pa_temp_warning_c                | 70.0                                      |
| thresholds.pa_temp_critical_c               | 85.0                                      |
| thresholds.sdr_temp_warning_c               | 65.0                                      |
| thresholds.sdr_temp_critical_c              | 80.0                                      |
| thresholds.cabinet_temp_warning_c           | 45.0                                      |
| thresholds.cabinet_temp_critical_c          | 60.0                                      |
| thresholds.evm_warning_pct                  | 6.0                                       |
| thresholds.evm_critical_pct                 | 10.0                                      |
| thresholds.papr_warning_db                  | 7.0                                       |
| thresholds.papr_critical_db                 | 9.0                                       |
| thresholds.forward_power_warning_below_w    | 0.0                                       |
| thresholds.forward_power_critical_below_w   | 0.0                                       |
| thresholds.pa_voltage_warning_below_v       | 0.0                                       |
| thresholds.pa_voltage_critical_below_v      | 0.0                                       |



| TOML-Pfad                         | Beispielwert |
|-----------------------------------|--------------|
| thresholds.fan_warning_below_rpm  | 0            |
| thresholds.fan_critical_below_rpm | 0            |

#### 14.27.21 alarm-workflow

Quelle: system-backend/alarm-workflow/config/alarm-workflow.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/alarm-workflow.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad                          | Beispielwert                                  |
|------------------------------------|-----------------------------------------------|
| server.bind                        | 0.0.0.0:8270                                  |
| security.mode                      | open_lab                                      |
| storage.state_file                 | /var/lib/netcore-alarm-workflow/state.json    |
| storage.event_log                  | /var/lib/netcore-alarm-workflow/events.ndjson |
| storage.audit_log                  | /var/lib/netcore-alarm-workflow/audit.ndjson  |
| mqtt.enabled                       | true                                          |
| mqtt.host                          | 10.0.20.27                                    |
| mqtt.port                          | 1883                                          |
| mqtt.qos                           | 1                                             |
| mqtt.topic_prefix                  | netcore/v1                                    |
| mqtt.client_id                     | netcore-alarm-workflow                        |
| mqtt.reconnect_secs                | 3                                             |
| mqtt.subscribe_topics              | ['netcore/v1/events/#']                       |
| sds_router.base_url                | http://10.0.20.17:8150                        |
| sds_router.timeout_secs            | 3                                             |
| sds_router.poll_interval_secs      | 2                                             |
| sds_router.process_existing_events | false                                         |
| sds_router.source_issi             | 9999                                          |
| sds_router.protocol_id             | 130                                           |
| sds_router.default_ttl_secs        | 300                                           |

| TOML-Pfad                                  | Beispielwert                                  |
|--------------------------------------------|-----------------------------------------------|
| workflow.scheduler_interval_secs           | 2                                             |
| workflow.event_history_limit               | 2000                                          |
| workflow.seen_event_limit                  | 10000                                         |
| workflow.stop_escalation_on_ack            | true                                          |
| workflow.stop_escalation_on_assignment     | true                                          |
| workflow.auto_close_on_clear               | false                                         |
| limits.max_body_bytes                      | 2097152                                       |
| recipients                                 | 2 Beispielobjekt(e), erster Eintrag unten     |
| recipients[0].id                           | technik-gruppe                                |
| recipients[0].name                         | Technikgruppe                                 |
| recipients[0].enabled                      | true                                          |
| recipients[0].kind                         | group_sds                                     |
| recipients[0].destination                  | 15201                                         |
| recipients[0].source_issi                  | 9999                                          |
| recipients[0].protocol_id                  | 130                                           |
| recipients[0].priority                     | 7                                             |
| recipients[0].ttl_secs                     | 300                                           |
| recipients[0].max_text_chars               | 180                                           |
| recipients[0].message_template             | ALARM {severity} {token} {title}. ACK {token} |
| escalation_profiles                        | 2 Beispielobjekt(e), erster Eintrag unten     |
| escalation_profiles[0].id                  | technical-default                             |
| escalation_profiles[0].name                | Technische Standardescalation                 |
| escalation_profiles[0].steps               | 3 Beispielobjekt(e), erster Eintrag unten     |
| escalation_profiles[0].steps[0].after_secs | 0                                             |
| escalation_profiles[0].steps[0].recipients | [ 'technik-gruppe' ]                          |
| rules                                      | 12 Beispielobjekt(e), erster Eintrag unten    |
| rules[0].id                                | rf-alarm-raised                               |
| rules[0].enabled                           | true                                          |
| rules[0].action                            | raise                                         |

| TOML-Pfad                           | Beispielwert                                       |
|-------------------------------------|----------------------------------------------------|
|                                     |                                                    |
| rules[0].event_type                 | rf.alarm_raised                                    |
| rules[0].alarm_type                 | rf_fault                                           |
| rules[0].title                      | RF {subject_id}: {payload_alarm_key}               |
| rules[0].description                | HF-Alarm {payload_alarm_key}; Wert {payload_value} |
| rules[0].severity                   | inherit                                            |
| rules[0].priority                   | 8                                                  |
| rules[0].requires_ack               | true                                               |
| rules[0].recipients                 | ['technik-gruppe']                                 |
| rules[0].escalation_profile         | technical-default                                  |
| rules[0].dedup_fields               | ['subject.id', 'payload.alarm_key']                |
| status_actions                      | 5 Beispielobjekt(e), erster Eintrag unten          |
| status_actions[0].enabled           | true                                               |
| status_actions[0].status_code       | 5201                                               |
| status_actions[0].action            | ack                                                |
| status_actions[0].states            | ['open']                                           |
| status_actions[0].requires_ack_only | true                                               |

#### 14.27.22 task-workflow

Quelle: system-backend/task-workflow/config/task-workflow.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/task-workflow.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad          | Beispielwert                              |
|--------------------|-------------------------------------------|
| service.name       | netcore-task-workflow                     |
| service.phase      | 9                                         |
| service.mode       | open_lab                                  |
| server.bind        | 0.0.0.0:8280                              |
| security.mode      | open_lab                                  |
| storage.state_file | /var/lib/netcore-task-workflow/state.json |
| storage.event_log  | /var/lib/netcore-task-workflow/           |

| TOML-Pfad                           | Beispielwert                                    |
|-------------------------------------|-------------------------------------------------|
|                                     | events.ndjson                                   |
| storage.audit_log                   | /var/lib/netcore-task-workflow/<br>audit.ndjson |
| mqtt.enabled                        | true                                            |
| mqtt.host                           | 127.0.0.1                                       |
| mqtt.port                           | 1883                                            |
| mqtt.topic_prefix                   | netcore/v1                                      |
| mqtt.client_id                      | netcore-task-workflow                           |
| sds_router.enabled                  | true                                            |
| sds_router.base_url                 | http://127.0.0.1:8150                           |
| sds_router.source_issi              | 9999                                            |
| sds_router.protocol_id              | 130                                             |
| sds_router.ttl_secs                 | 600                                             |
| sds_router.max_text_length          | 160                                             |
| sds_router.default_destination      | 15201                                           |
| sds_router.default_is_group         | true                                            |
| workflow.event_history_limit        | 2000                                            |
| workflow.seen_event_limit           | 10000                                           |
| workflow.expire_check_interval_secs | 5                                               |
| workflow.notify_on_state_change     | true                                            |
| wap.enabled                         | true                                            |
| wap.page_size                       | 6                                               |
| wap.xhtml_entry                     | /x                                              |
| wap.wml_entry                       | /w                                              |
| templates                           | 6 Beispielobjekt(e), erster Eintrag unten       |
| templates[0].id                     | technical_fault                                 |
| templates[0].name                   | Technische Stoerung                             |
| templates[0].description            | Technische Stoerung aufnehmen und<br>bearbeiten |
| templates[0].default_priority       | 7                                               |

| TOML-Pfad                       | Beispielwert                              |
|---------------------------------|-------------------------------------------|
| templates[0].default_severity   | warning                                   |
| templates[0].requires_ack       | true                                      |
| templates[0].fields             | 3 Beispielobjekt(e), erster Eintrag unten |
| templates[0].fields[0].id       | asset                                     |
| templates[0].fields[0].label    | Anlage/Geraet                             |
| templates[0].fields[0].type     | text                                      |
| templates[0].fields[0].required | true                                      |
| status_actions                  | 5 Beispielobjekt(e), erster Eintrag unten |
| status_actions[0].status_code   | 5301                                      |
| status_actions[0].action        | accept                                    |
| status_actions[0].label         | Auftrag annehmen                          |

#### 14.27.23 asset-management

Quelle: system-backend/asset-management/config/asset-management.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/asset-management.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad          | Beispielwert                                        |
|--------------------|-----------------------------------------------------|
| service.name       | netcore-asset-management                            |
| service.phase      | 10                                                  |
| service.mode       | open_lab                                            |
| server.bind        | 0.0.0.0:8290                                        |
| security.mode      | open_lab                                            |
| storage.state_file | /var/lib/netcore-asset-management/<br>state.json    |
| storage.event_log  | /var/lib/netcore-asset-management/<br>events.ndjson |
| storage.audit_log  | /var/lib/netcore-asset-management/<br>audit.ndjson  |
| mqtt.enabled       | true                                                |
| mqtt.host          | 127.0.0.1                                           |
| mqtt.port          | 1883                                                |
| mqtt.topic_prefix  | netcore/v1                                          |

| TOML-Pfad                              | Beispielwert             |
|----------------------------------------|--------------------------|
| mqtt.client_id                         | netcore-asset-management |
| management.event_history_limit         | 3000                     |
| management.upstream_sync_interval_secs | 60                       |
| upstreams.subscriber_core.enabled      | true                     |
| upstreams.subscriber_core.base_url     | http://127.0.0.1:8100    |
| upstreams.mobility_core.enabled        | true                     |
| upstreams.mobility_core.base_url       | http://127.0.0.1:8090    |
| upstreams.task_workflow.enabled        | true                     |
| upstreams.task_workflow.base_url       | http://127.0.0.1:8280    |
| upstreams.task_workflow.default_gssi   | 15201                    |

#### 14.27.24 sip-switch

Quelle: system-backend/sip-switch/config/sip-switch.example.toml. Bei jeder Standortinstallation durch die ausgerollte Datei /etc/netcore/sip-switch.toml und das passende Runtime-Schema abgleichen.

| TOML-Pfad             | Beispielwert                              |
|-----------------------|-------------------------------------------|
| service.name          | netcore-sip-switch                        |
| service.phase         | 11                                        |
| service.mode          | open_lab                                  |
| server.bind           | 0.0.0.0:8300                              |
| security.mode         | open_lab                                  |
| storage.state_file    | /var/lib/netcore-sip-switch/state.json    |
| storage.event_log     | /var/lib/netcore-sip-switch/events.ndjson |
| storage.audit_log     | /var/lib/netcore-sip-switch/audit.ndjson  |
| mqtt.enabled          | true                                      |
| mqtt.host             | 127.0.0.1                                 |
| mqtt.port             | 1883                                      |
| mqtt.topic_prefix     | netcore/v1                                |
| mqtt.client_id        | netcore-sip-switch                        |
| mobility_core.enabled | true                                      |

| TOML-Pfad                         | Beispielwert                                   |
|-----------------------------------|------------------------------------------------|
| mobility_core.base_url            | http://127.0.0.1:8090                          |
| mobility_core.timeout_secs        | 2                                              |
| asterisk.enabled                  | true                                           |
| asterisk.binary                   | /usr/sbin/asterisk                             |
| asterisk.config_dir               | /etc/asterisk                                  |
| asterisk.agi_script               | /var/lib/asterisk/agi-bin/netcore-sip-route.py |
| asterisk.sip_bind                 | 0.0.0.0:5060                                   |
| asterisk.rtp_start                | 10000                                          |
| asterisk.rtp_end                  | 20000                                          |
| management.route_workers          | 8                                              |
| management.side_effect_queue_size | 512                                            |
| management.call_history_limit     | 2000                                           |
| management.event_history_limit    | 3000                                           |
| management.probe_interval_secs    | 10                                             |
| pbx.mode                          | registration                                   |
| pbx.endpoint_id                   | netcore-pbx                                    |
| pbx.host                          | 127.0.0.1                                      |
| pbx.port                          | 5060                                           |
| pbx.transport                     | udp                                            |
| pbx.username                      | (leer)                                         |
| pbx.auth_username                 | (leer)                                         |
| pbx.password                      | [ausgelassen]                                  |
| pbx.from_user                     | netcore-tetra                                  |
| pbx.from_domain                   | (leer)                                         |
| pbx.contact_user                  | netcore-tetra                                  |
| pbx.registration_id               | netcore-pbx-registration                       |
| pbx.registration_expiration_secs  | 30                                             |
| pbx.allow                         | ulaw                                           |
| pbx.match                         | [ ]                                            |

| TOML-Pfad                         | Beispielwert                              |
|-----------------------------------|-------------------------------------------|
|                                   |                                           |
| routing.tetra_number_prefix       | (leer)                                    |
| routing.strip_tetra_prefix        | false                                     |
| routing.pbx_outbound_prefix       | (leer)                                    |
| routing.strip_pbx_outbound_prefix | false                                     |
| routing.accept_stale_routes       | false                                     |
| routing.require_tbs_contact       | true                                      |
| routing.dial_timeout_secs         | 60                                        |
| tbs                               | 1 Beispielobjekt(e), erster Eintrag unten |
| tbs[0].node_id                    | SRV-M-TBS-01                              |
| tbs[0].endpoint_id                | tbs-srv-m-tbs-01                          |
| tbs[0].username                   | tbs-srv-m-tbs-01                          |
| tbs[0].password                   | [ausgelassen]                             |
| tbs[0].enabled                    | false                                     |
| tbs[0].max_contacts               | 1                                         |
| tbs[0].aliases                    | [ 'TBS-01' ]                              |
| number_mappings                   | 1 Beispielobjekt(e), erster Eintrag unten |
| number_mappings[0].number         | 4010001                                   |
| number_mappings[0].target_type    | issi                                      |
| number_mappings[0].target         | 4010001                                   |
| number_mappings[0].enabled        | false                                     |

#### 14.27.25 provisioning-core

Quelle: system-backend/provisioning-core/config/provisioning-core.example.toml. Dieser Dienst wird separat vom 25er-Inventory installiert.

| TOML-Pfad                | Beispielwert           |
|--------------------------|------------------------|
| server.bind              | 0.0.0.0:8125           |
| upstream.subscriber_core | http://10.0.1.181:8100 |
| upstream.group_core      | http://10.0.1.182:8110 |
| upstream.timeout_secs    | 5                      |



| TOML-Pfad                        | Beispielwert |
|----------------------------------|--------------|
| security.mode                    | open_lab     |
| security.allow_remote_management | true         |
| limits.max_body_bytes            | 2097152      |

**Abgleich nach Provisioning-Änderungen.** Eine Mitgliedschaft ist erst fachlich abgeschlossen, wenn Subscriber Core, Group Core und die angebundene TBS denselben Testfall widerspruchsfrei zeigen. Vor einer Änderung beide Autoritätsdienste exportieren, dann ein eindeutig markiertes Testobjekt über Provisioning anlegen. Nach dem Schreiben Profile und Gruppenzuordnung direkt an beiden Cores lesen, Synchronisationsrevision auf der TBS prüfen und eine positive sowie eine negative Funkprobe durchführen. Bei Teilerfolg keinen zweiten unkontrollierten Schreibversuch starten: zuerst die tatsächlichen Zustände beider Cores und die Auditspur vergleichen. Bei Löschung außerdem abhängige Mitgliedschaften und gecachte TBS-Policy prüfen. Das alte Installationsdokument mit Patch-Banches ist für diesen bereits vorhandenen Workspace-Dienst nicht der maßgebliche Stand.

## 15 Protokolle Pfade Quellen und Checklisten

### 15.1 Managementportkatalog

Die Portwerte in dieser Tabelle sind **Inventory-Beispielwerte** für TCP-Management/API/WebUI, keine global reservierten Standardports. Die 24 bisherigen Einträge und die Warnzentrale auf 10.0.20.34:8310 sind im gleichen Open-Lab-Subnetz geplant.

| Dienst              | Beispiel-IP | TCP-Port | Unit                                |
|---------------------|-------------|----------|-------------------------------------|
| node-gateway        | 10.0.20.10  | 8080     | netcore-node-gateway.service        |
| mobility-core       | 10.0.20.11  | 8090     | netcore-mobility-core.service       |
| subscriber-core     | 10.0.20.12  | 8100     | netcore-subscriber-core.service     |
| group-core          | 10.0.20.13  | 8110     | netcore-group-core.service          |
| call-control        | 10.0.20.14  | 8120     | netcore-call-control.service        |
| media-switch        | 10.0.20.15  | 8130     | netcore-media-switch.service        |
| recorder            | 10.0.20.16  | 8140     | netcore-recorder.service            |
| sds-router          | 10.0.20.17  | 8150     | netcore-sds-router.service          |
| packet-core         | 10.0.20.18  | 8160     | netcore-packet-core.service         |
| ip-gateway          | 10.0.20.19  | 8170     | netcore-ip-gateway.service          |
| security-core       | 10.0.20.20  | 8180     | netcore-security-core.service       |
| kmf                 | 10.0.20.21  | 8190     | netcore-kmf.service                 |
| transit             | 10.0.20.22  | 8200     | netcore-transit.service             |
| application-gateway | 10.0.20.23  | 8220     | netcore-application-gateway.service |
| media-library       | 10.0.20.24  | 8230     | netcore-media-library.service       |

| Dienst           | Beispiel-IP | TCP-Port | Unit                             |
|------------------|-------------|----------|----------------------------------|
| control-room     | 10.0.20.25  | 9010     | netcore-control-room.service     |
| observability    | 10.0.20.26  | 8210     | netcore-observability.service    |
| iot-gateway      | 10.0.20.27  | 8240     | netcore-iot-gateway.service      |
| hardware-gateway | 10.0.20.28  | 8250     | netcore-hardware-gateway.service |
| rf-monitor       | 10.0.20.29  | 8260     | netcore-rf-monitor.service       |
| alarm-workflow   | 10.0.20.30  | 8270     | netcore-alarm-workflow.service   |
| task-workflow    | 10.0.20.31  | 8280     | netcore-task-workflow.service    |
| asset-management | 10.0.20.32  | 8290     | netcore-asset-management.service |
| sip-switch       | 10.0.20.33  | 8300     | netcore-sip-switch.service       |

## 15.2 Weitere Netzports und Pfade

| Komponente         | Beispiel         | Transport        | Hinweis                                       |
|--------------------|------------------|------------------|-----------------------------------------------|
| TBS-Dashboard      | 8080             | HTTP/TCP         | Host ist die Basisstation, nicht Node Gateway |
| TBS zu Gateway     | 8080 /ws/node    | WebSocket/TCP    | Health-Matrix und Control                     |
| Backend zu Gateway | 8080 /ws/backend | WebSocket/TCP    | Verbindung je Dienst                          |
| Provisioning Core  | 8125             | HTTP/TCP         | Zusatzdienst außerhalb Inventory              |
| Directory          | 8095             | HTTP/TCP         | Namen, Gruppen, Statuslabels                  |
| Piper              | 5005             | HTTP/TCP         | GET /voices, Synthese-API                     |
| MQTT Broker        | 1883             | MQTT/TCP         | im Open Lab ohne behauptetes TLS              |
| SIP Switch         | 5060             | SIP/UDP oder TCP | konkret nach TOML/Firewall                    |

| Komponente            | Beispiel             | Transport               | Hinweis                                                           |
|-----------------------|----------------------|-------------------------|-------------------------------------------------------------------|
| SIP Switch RTP        | 10000–20000          | RTP/UDP                 | konfigurierbarer Bereich                                          |
| native TBS SIP        | 5062                 | SIP/UDP                 | Beispiel aus TBS-Konfiguration                                    |
| native TBS RTP        | 30000–30100          | RTP/UDP                 | Beispiel aus TBS-Konfiguration                                    |
| Brew TBS-Peer         | 8081 im TBS-Beispiel | WebSocket/TCP           | unabhängiger Zielserver                                           |
| separater Brew-Server | 9000/9001/9002/9003  | WebSocket/HTTP über TCP | Core, Telemetrie, Control, Dashboard; TLS/Ports laut eigener TOML |

### 15.3 Gesundheits- und API-Endpunkte

Die Backends bieten in ihrer Open-Lab-Familie `/health/live`, `/health/ready`, `/metrics`, `/openapi.json` und `/api/v1/...` an. Ein älterer Dienst kann einen anderen OpenAPI-Pfad verwenden. Der Node Gateway liefert `/api/v1/core-services`; die TBS `/api/edge-fallback` und `/api/rf-monitor`. Media Library nutzt beispielsweise `POST /api/v1/assets/import-url`, `GET /api/v1/assets` und `Asset-Preview`; die TBS liefert `Recording-Audio/Metadaten` unter `/api/media-library/recordings/<id>/...`. Provisioning Core besitzt `/api/v1/dashboard`. Directory liefert `/api/devices`, `/api/basestations`, `/api/groups` und `/api/status`. Für vollständige Methoden und Schemas immer die OpenAPI-Ausgabe des **laufenden** Dienstes öffnen.

### 15.4 Dateisystem und Zustandsorte

| Ort                                                              | Zweck                       | Betriebshinweis                    |
|------------------------------------------------------------------|-----------------------------|------------------------------------|
| <code>/etc/netcore/config.toml</code> und <code>.fallback</code> | TBS-Konfiguration           | Fallback unabhängig sichern        |
| <code>/etc/netcore/&lt;dienst&gt;.toml</code>                    | Backend-Konfiguration       | Installer-/Inventory-Ziel beachten |
| <code>/var/lib/flowstation/edge-policy-cache.json</code>         | letzte TBS-Policy           | Eigentümer/Alter prüfen            |
| <code>/var/lib/flowstation/edge-event-spool.jsonl</code>         | Replay-Ereignisse           | Limits und Disk prüfen             |
| <code>/var/lib/netcore/recordings</code>                         | lokale WAV + JSON           | vor NFS-Import primäre Quelle      |
| <code>/var/lib/netcore/audio</code>                              | lokale Audioquelle          | Dateigröße und Rechte              |
| <code>/var/cache/netcore/audio</code>                            | vorbereiteter Playout-Cache | nach Codec-/Asset-Update erneuern  |
| <code>/var/cache/netcore/tts</code>                              | TTS-Cache                   | Retention beachten                 |
| <code>/mnt/nfs-share</code>                                      | optionales Archiv/Medien    | Mount und Schreibrechte prüfen     |
| <code>tests/e2e/artifacts/&lt;run-</code>                        | Testreports                 | zu Abnahme archivieren             |

| Ort  | Zweck | Betriebshinweis |
|------|-------|-----------------|
| id>/ |       |                 |

## 15.5 Protokollbegriffe

| Begriff                 | Bedeutung im Projekt                                               |
|-------------------------|--------------------------------------------------------------------|
| TBS                     | TETRA-Basisstation mit lokaler Funkautorität                       |
| MS                      | Funkgerät bzw. mobile Station                                      |
| SwMI                    | vermittelnde Netzinfrastruktur                                     |
| PHY/MAC/LLC/MLE/MM/CMCE | Funk-, Zugriffs-, Link-, Mobilitäts-, Management- und Rufschichten |
| SDS                     | Kurznachrichten und Status über die TETRA-Luftschnittstelle        |
| SNDCP/PDP/NSAPI         | Paketdaten-Kontext und Dienstzugriff                               |
| MCC/MNC/LA/CC           | Netz- und Zellidentitäten                                          |
| ISSI/GSSI               | individuelle und Gruppenkennung                                    |
| DGNA                    | dynamische Gruppenadressenzuordnung                                |
| PEI/ISI                 | Terminal- bzw. Inter-System-Schnittstelle                          |
| TACELP/ACELP            | TETRA-Sprachcodierung und Medienrahmung                            |
| RTP/SIP                 | Medienpakete und Telefoniesignalisierung                           |
| LIP                     | TETRA-Positionsmeldung                                             |
| Shadow                  | prüfender Dienstmodus ohne volle Autorität                         |
| Authoritative           | Dienst übernimmt explizit festgelegte Autorität                    |

## 15.6 Quellen und Editionskontrolle

Primäre Implementierungsquelle: Repository <https://github.com/JanHG98/netcore-tetra> bei Commit 086a81fa8820ef579c475a65a38e3d23644c52f0; besonders `deploy/open-lab/inventory.example.toml`, `Docs/NetCore-Tetra-Komplettguide.md`, `Docs/ETSI_SOURCE_REGISTER.md`, `Docs/ETSI_CONFORMANCE_MATRIX.md`, `Docs/IMPLEMENTATION_GAPS.md`, `Docs/EDGE_FALLBACK.md` sowie die jeweiligen `system-backend/*/config/*.example.toml`. Die beigefügte Datei `ETSI.pdf` ist ein umfangreiches Sammel-PDF; für Versionsbezüge werden die einzelnen Dateien herangezogen.

| ETSI-Dokument            | Verwendung hier   | Beigefügte Datei           |
|--------------------------|-------------------|----------------------------|
| ETSI EN 300 392-1 V1.6.1 | Netzdesign        | 15-en_30039201v010601p.pdf |
| ETSI EN 300 392-2 V3.8.1 | Luftschnittstelle | 24-en_30039202v030801p.pdf |

| ETSI-Dokument                      | Verwendung hier                    | Beigefügte Datei             |
|------------------------------------|------------------------------------|------------------------------|
| ETSI EN 300 392-3-3 V1.3.1         | ISI Gruppenruf                     | 21-en_3003920303v010301p.pdf |
| ETSI EN 300 392-3-4 V1.3.1         | ISI SDS                            | 05-en_3003920304v010301p.pdf |
| ETSI EN 300 392-3-8 V1.4.1         | ISI Sprachformat                   | 01-en_3003920308v010401p.pdf |
| ETSI EN 300 392-3-13 V1.2.1        | ISI weitere Verfahren              | 19-en_3003920313v010201p.pdf |
| Draft ETSI EN 300 392-3-15 V1.5.0  | ISI Mobilität, Entwurf 2026        | 23-en_3003920315v010500a.pdf |
| ETSI EN 300 392-5 V2.7.1           | PEI                                | 22-en_30039205v020701p.pdf   |
| ETSI EN 300 392-7 V3.5.1           | Security                           | 17-en_30039207v030501p.pdf   |
| ETSI EN 300 392-9 V1.7.1           | Zusatzdienste allgemein            | 02-en_30039209v010701p.pdf   |
| ETSI EN 300 392-10-6 V1.4.1        | Zusatzdienst Stage 1               | 12-en_3003921006v010401p.pdf |
| ETSI EN 300 392-10-18 V1.3.1       | Zusatzdienst Stage 1               | 13-en_3003921018v010301p.pdf |
| ETSI EN 300 392-11-1 V1.2.1        | Zusatzdienst Stage 2               | 11-en_3003921101v010201p.pdf |
| ETSI EN 300 392-11-14 V1.1.1       | Zusatzdienst Stage 2               | 07-en_3003921114v010101p.pdf |
| ETSI EN 300 392-11-17 V1.1.2       | Zusatzdienst Stage 2               | 06-en_3003921117v010102p.pdf |
| ETSI EN 300 392-12-1 V1.2.2        | Zusatzdienst Stage 3               | 04-en_3003921201v010202p.pdf |
| Draft ETSI EN 300 392-12-16 V1.4.0 | Zusatzdienst Stage 3, Entwurf 2026 | 14-en_3003921216v010400a.pdf |
| ETSI EN 300 394-1 V3.3.1           | Radio-Konformitätstests            | 18-en_30039401v030301p.pdf   |
| ETSI EN 300 395-2 V1.3.3           | TETRA-Sprachcodec                  | 20-en_30039502v010303p.pdf   |
| ETSI EN 300 812 V2.1.1             | TSIM-Sicherheit                    | 10-en_300812v020101p.pdf     |
| ETSI ES 200 812-1 V2.2.5           | TSIM-Schnittstelle                 | 09-es_20081201v020205p.pdf   |
| ETSI ES 200 812-2 V2.4.1           | TSIM-Schnittstelle                 | 08-es_20081202v020401m.pdf   |
| ETSI TS 100 812-1 V2.2.5           | TSIM-Schnittstelle                 | 03-ts_10081201v020205p.pdf   |
| pr ETS 300 392-14                  | historisches PICS-Raster           | 16-ets_30039214e01v.pdf      |

Die Kapitel dieses Handbuchs paraphrasieren Normen und Projektquellen. Für eine rechtliche oder technische Konformitätserklärung sind die Originalnorm, der konkrete Softwarestand und Mess-/Testevidenz erforderlich.

## 15.7 Inbetriebnahmecheckliste

### Anlage

- ☐ Berechtigung, Frequenzen und Standort dokumentiert
- ☐ Versorgung, Kühlung, Dummyload, Duplexer und Antenne gemessen

- ☐ SDR-Treiber, Clock, TX/RX-Kanal und Gains geprüft
- ☐ Software-Commit, Feature-Build und Backup festgehalten

#### Netz

- ☐ Inventory mit realen IPs konsistent
- ☐ TBS zeigt auf Node Gateway und richtige /ws/node-Route
- ☐ 25 Dienste live/ready oder begründet degradiert
- ☐ Open-Lab-Netz isoliert, Firewall und Zeitquelle geprüft

#### Funk und Anwendungen

- ☐ Zwei Geräte registrieren und re-registrieren
- ☐ Gruppen-PTT, Floor, Release und Einzel-SDS geprüft
- ☐ SIP beide Richtungen, RTP, DTMF und Failover geprüft
- ☐ Paketdaten/WAP, TTS, Aufnahme und Archiv einzeln geprüft
- ☐ MQTT-Topic/Ack und Home-Assistant-Zustand verifiziert

#### Ausfall und Evidenz

- ☐ Gateway-/Fachkern-Ausfall und Recovery getestet
- ☐ Policy-Cache und Replay-Spool geprüft
- ☐ On-Air-Template mit Hersteller/Firmware, Log und UTC gefüllt
- ☐ Rollback und Datenrestore praktisch getestet

### 15.8 Änderungsblatt für die nächste Ausgabe

Vor einer Überarbeitung Commit und Tag erfassen, `git diff` der drei Inventarquellen, 25er-Matrix und Zusatzdienste vergleichen, jeden betroffenen TOML-Index neu erzeugen, `validate` und Full-System-Audit ausführen, echte E2E-/On-Air-Protokolle beilegen und alle bekannten Grenzen neu bewerten. Versionsname und Datum des Handbuchs erst danach erhöhen.

## 16 Schnittstellenatlas und API-Verträge

Diese Referenz wurde aus den OpenAPI-paths-Objekten der Quelltexte extrahiert. Methoden und Pfade gelten für den festgehaltenen Commit, nicht automatisch für ein anders gebautes oder laufendes System. Die Kurzfunktion ist eine operative Einordnung; maßgeblich für Requestkörper, Statuscodes und Autorisierung ist `/openapi.json` des tatsächlich gestarteten Dienstes. Bei Diensten ohne eingebetteten OpenAPI-Pfadkatalog werden keine Routen geraten. In Open Lab sind Managementendpunkte nur für das isolierte Testnetz vorgesehen.

### 16.1 node-gateway: HTTP- und WebSocket-Vertrag

Beispielziel `10.0.20.10:8080`; Unit `netcore-node-gateway.service`. Vor einem Fachtest zuerst `GET /health/live`, dann `GET /health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/node-gateway/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                          | Zweck / Prüffrage                                   | Quelle                                               |
|---------|-----------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| GET     | <code>/api/v1/core-services</code>            | Backend health matrix delivered to TBS nodes        | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/events</code>                   | Legacy gateway events with embedded canonical event | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/events/netcore</code>           | Canonical netcore-event-v1 records                  | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/nodes</code>                    | Known TBS nodes                                     | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/nodes/{node_id}</code>          | Node detail                                         | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/status</code>                   | Gateway status                                      | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/health/live</code>                     | Liveness                                            | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/health/ready</code>                    | Readiness                                           | <code>system-backend/node-gateway/src/http.rs</code> |
| GET     | <code>/metrics</code>                         | Prometheus metrics                                  | <code>system-backend/node-gateway/src/http.rs</code> |
| POST    | <code>/api/v1/nodes/{node_id}/commands</code> | Queue ControlCommand                                | <code>system-backend/node-gateway/src/http.rs</code> |
| POST    | <code>/api/v1/nodes/</code>                   | Disconnect node                                     | <code>system-backend/node-</code>                    |



| Methode | Pfad                         | Zweck / Prüffrage      | Quelle                                  |
|---------|------------------------------|------------------------|-----------------------------------------|
|         | {node_id}/disconnect         |                        | gateway/src/http.rs                     |
| POST    | /api/v1/nodes/{node_id}/ping | Queue application ping | system-backend/node-gateway/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.2 mobility-core: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.11:8090; Unit `netcore-mobility-core.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/mobility-core/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                             | Zweck / Prüffrage                                                                | Quelle                                   |
|---------|----------------------------------|----------------------------------------------------------------------------------|------------------------------------------|
| GET     | /api/v1/events                   | Legacy event records with embedded canonical event                               | system-backend/mobility-core/src/http.rs |
| GET     | /api/v1/events/netcore           | Canonical netcore-event-v1 records                                               | system-backend/mobility-core/src/http.rs |
| GET     | /api/v1/nodes                    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/mobility-core/src/http.rs |
| GET     | /api/v1/status                   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/mobility-core/src/http.rs |
| GET     | /api/v1/subscribers              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/mobility-core/src/http.rs |
| GET     | /api/v1/subscribers/{issi}/route | Canonical serving-TBS route for one ISSI                                         | system-backend/mobility-core/src/http.rs |

| Methode | Pfad                          | Zweck / Prüffrage                                                                    | Quelle                                           |
|---------|-------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------|
| GET     | /api/v1/transfers             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>mobility-core/src/<br>http.rs |
| GET     | /health/live                  | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>mobility-core/src/<br>http.rs |
| GET     | /health/ready                 | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>mobility-core/src/<br>http.rs |
| GET     | /metrics                      | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>mobility-core/src/<br>http.rs |
| POST    | /api/v1/transfers             | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>mobility-core/src/<br>http.rs |
| POST    | /api/v1/transfers/{id}/cancel | Aktion cancel auslösen; Ergebnis, Idempotenz und Audit prüfen.                       | system-backend/<br>mobility-core/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.3 subscriber-core: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.12:8100; Unit `netcore-subscriber-core.service`. Vor einem Fachtest zuerst `GET /health/live`, dann `GET /health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/subscriber-core/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                       | Zweck / Prüffrage                                                                | Quelle                                             |
|---------|----------------------------|----------------------------------------------------------------------------------|----------------------------------------------------|
| DELETE  | /api/v1/subscribers/{issi} | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/export.csv         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/<br>subscriber-core/<br>src/http.rs |

| Methode | Pfad                       | Zweck / Prüffrage                                                                    | Quelle                                             |
|---------|----------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------|
| GET     | /api/v1/export.json        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/nodes              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/observed           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/status             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/subscribers        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/subscribers/{issi} | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /api/v1/syncs              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /health/live               | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /health/ready              | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>subscriber-core/<br>src/http.rs |
| GET     | /metrics                   | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>subscriber-core/<br>src/http.rs |
| POST    | /api/v1/import             | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>subscriber-core/<br>src/http.rs |
| POST    | /api/v1/subscribers        | Ressource oder Aktion anlegen;                                                       | system-backend/<br>subscriber-core/                |

| Methode | Pfad                       | Zweck / Prüffrage                                            | Quelle                                             |
|---------|----------------------------|--------------------------------------------------------------|----------------------------------------------------|
|         |                            | Requestschema und Antwortcode im Live-OpenAPI prüfen.        | src/http.rs                                        |
| POST    | /api/v1/sync               | Aktion sync auslösen; Ergebnis, Idempotenz und Audit prüfen. | system-backend/<br>subscriber-core/<br>src/http.rs |
| PUT     | /api/v1/subscribers/{issi} | Eintrag ändern; Revision und Folgesynchronisation prüfen.    | system-backend/<br>subscriber-core/<br>src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

#### 16.4 group-core: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.13:8110; Unit `netcore-group-core.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/group-core/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                              | Zweck / Prüffrage                                                                | Quelle                                        |
|---------|-----------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------|
| DELETE  | /api/v1/groups/{gssi}             | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/<br>group-core/src/<br>http.rs |
| DELETE  | /api/v1/memberships/{issi}/{gssi} | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/affiliations              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/dgna                      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/export.json               | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/<br>group-core/src/<br>http.rs |

| Methode | Pfad                  | Zweck / Prüffrage                                                                    | Quelle                                        |
|---------|-----------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|
| GET     | /api/v1/groups        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/groups/{gssi} | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/memberships   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /api/v1/status        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /health/live          | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /health/ready         | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>group-core/src/<br>http.rs |
| GET     | /metrics              | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>group-core/src/<br>http.rs |
| POST    | /api/v1/dgna          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>group-core/src/<br>http.rs |
| POST    | /api/v1/groups        | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>group-core/src/<br>http.rs |
| POST    | /api/v1/memberships   | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>group-core/src/<br>http.rs |
| POST    | /api/v1/sync          | Aktion sync auslösen; Ergebnis, Idempotenz und Audit prüfen.                         | system-backend/<br>group-core/src/<br>http.rs |
| PUT     | /api/v1/groups/       | Eintrag ändern; Revision                                                             | system-backend/                               |

| Methode | Pfad   | Zweck / Prüffrage                      | Quelle                     |
|---------|--------|----------------------------------------|----------------------------|
|         | {gssi} | und<br>Folgesynchronisation<br>prüfen. | group-core/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.5 call-control: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.14:8120; Unit `netcore-call-control.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/call-control/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                         | Zweck / Prüffrage                                                                | Quelle                                               |
|---------|----------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------|
| GET     | <code>/api/v1/calls</code>                   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/calls/{logical_call_id}</code> | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/events</code>                  | Legacy event records with embedded canonical event                               | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/events/netcore</code>          | Canonical netcore-event-v1 records                                               | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/nodes</code>                   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/participants</code>            | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/call-control/src/http.rs</code> |
| GET     | <code>/api/v1/restores</code>                | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/call-control/src/http.rs</code> |

| Methode | Pfad                                          | Zweck / Prüffrage                                                                    | Quelle                             |
|---------|-----------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------|
| GET     | /api/v1/status                                | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/control/src/http.rs |
| GET     | /health/live                                  | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/control/src/http.rs |
| GET     | /health/ready                                 | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/control/src/http.rs |
| GET     | /metrics                                      | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/control/src/http.rs |
| GET     | /ws/media                                     | Call/media topology event WebSocket                                                  | system-backend/control/src/http.rs |
| POST    | /api/v1/calls/group                           | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/control/src/http.rs |
| POST    | /api/v1/calls/individual                      | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/control/src/http.rs |
| POST    | /api/v1/calls/{logical_call_id}/floor         | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/control/src/http.rs |
| POST    | /api/v1/calls/{logical_call_id}/floor/release | Aktion release auslösen; Ergebnis, Idempotenz und Audit prüfen.                      | system-backend/control/src/http.rs |
| POST    | /api/v1/calls/{logical_call_id}/release       | Aktion release auslösen; Ergebnis, Idempotenz und Audit prüfen.                      | system-backend/control/src/http.rs |
| POST    | /api/v1/media/route-ready                     | Media Switch RouteReady acknowledgement                                              | system-backend/control/src/http.rs |
| POST    | /api/v1/restores                              | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-                | system-backend/control/src/http.rs |

| Methode | Pfad                                 | Zweck / Prüffrage                                              | Quelle                             |
|---------|--------------------------------------|----------------------------------------------------------------|------------------------------------|
|         |                                      | OpenAPI prüfen.                                                |                                    |
| POST    | /api/v1/restores/{restore_id}/cancel | Aktion cancel auslösen; Ergebnis, Idempotenz und Audit prüfen. | system-backend/control/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.6 media-switch: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.15:8130; Unit `netcore-media-switch.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/media-switch/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                  | Zweck / Prüffrage                                                                | Quelle                                  |
|---------|-----------------------|----------------------------------------------------------------------------------|-----------------------------------------|
| GET     | /api/v1/buffers       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |
| GET     | /api/v1/config        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |
| GET     | /api/v1/events        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |
| GET     | /api/v1/nodes         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |
| GET     | /api/v1/recorder/taps | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |
| GET     | /api/v1/sessions      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-switch/src/http.rs |



| Methode | Pfad                                 | Zweck / Prüffrage                                                                    | Quelle                                          |
|---------|--------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------|
| GET     | /api/v1/sessions/{session_id}        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /api/v1/status                       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /api/v1/streams                      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /api/v1/taps                         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /health/live                         | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /health/ready                        | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>media-switch/src/<br>http.rs |
| GET     | /metrics                             | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>media-switch/src/<br>http.rs |
| POST    | /api/v1/gateway/ping                 | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>media-switch/src/<br>http.rs |
| POST    | /api/v1/sessions/{session_id}/flush  | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>media-switch/src/<br>http.rs |
| POST    | /api/v1/sessions/{session_id}/inject | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>media-switch/src/<br>http.rs |
| POST    | /api/v1/sessions/{session_id}/mute   | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-                | system-backend/<br>media-switch/src/<br>http.rs |

| Methode | Pfad | Zweck / Prüffrage | Quelle |
|---------|------|-------------------|--------|
|         |      | OpenAPI prüfen.   |        |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.7 recorder: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.16:8140; Unit `netcore-recorder.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/recorder/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                              | Zweck / Prüffrage                                                                | Quelle                                           |
|---------|---------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------|
| GET     | <code>/api/v1/active</code>                       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/config</code>                       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/events</code>                       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/recordings</code>                   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/recordings/{id}</code>              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/recordings/{id}/audio.tacelp</code> | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/recorder/src/http.rs</code> |
| GET     | <code>/api/v1/recordings/{id}/export</code>       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung                        | <code>system-backend/recorder/src/http.rs</code> |

| Methode | Pfad                              | Zweck / Prüffrage                                                                    | Quelle                              |
|---------|-----------------------------------|--------------------------------------------------------------------------------------|-------------------------------------|
|         |                                   | am Live-Dienst prüfen.                                                               |                                     |
| GET     | /api/v1/status                    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/recorder/src/http.rs |
| GET     | /health/live                      | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/recorder/src/http.rs |
| GET     | /health/ready                     | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/recorder/src/http.rs |
| GET     | /metrics                          | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/recorder/src/http.rs |
| POST    | /api/v1/recordings/{id}/delete    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/recorder/src/http.rs |
| POST    | /api/v1/recordings/{id}/finalize  | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/recorder/src/http.rs |
| POST    | /api/v1/recordings/{id}/hold      | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/recorder/src/http.rs |
| POST    | /api/v1/recordings/{id}/retention | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/recorder/src/http.rs |
| POST    | /api/v1/recordings/{id}/verify    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/recorder/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.8 sds-router: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.17:8150; Unit netcore-sds-router.service. Vor einem Fachtest zuerst GET /health/live, dann GET /health/ready und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: system-backend/sds-router/src/http.rs. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                       | Zweck / Prüffrage                                                                | Quelle                                |
|---------|----------------------------|----------------------------------------------------------------------------------|---------------------------------------|
| DELETE  | /api/v1/messages/{id}      | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/sds-router/src/http.rs |
| DELETE  | /api/v1/routes/{id}        | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/application-outbox | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/events             | Legacy event records with embedded canonical event                               | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/events/netcore     | Canonical netcore-event-v1 records                                               | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/groups             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/messages           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/messages/{id}      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/nodes              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/routes             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung                        | system-backend/sds-router/src/http.rs |

| Methode | Pfad                                              | Zweck / Prüffrage                                                                    | Quelle                                |
|---------|---------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------|
|         |                                                   | am Live-Dienst prüfen.                                                               |                                       |
| GET     | /api/v1/status                                    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/sds-router/src/http.rs |
| GET     | /api/v1/subscribers                               | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/sds-router/src/http.rs |
| GET     | /health/live                                      | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/sds-router/src/http.rs |
| GET     | /health/ready                                     | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/sds-router/src/http.rs |
| GET     | /metrics                                          | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/application-outbox/{application}/{id}/ack | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/messages                                  | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/messages/{id}/cancel                      | Aktion cancel auslösen; Ergebnis, Idempotenz und Audit prüfen.                       | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/messages/{id}/requeue                     | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/messages/{id}/retry                       | Aktion retry auslösen; Ergebnis, Idempotenz und Audit prüfen.                        | system-backend/sds-router/src/http.rs |
| POST    | /api/v1/routes                                    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sds-router/src/http.rs |

| Methode | Pfad                | Zweck / Prüffrage                                         | Quelle                                |
|---------|---------------------|-----------------------------------------------------------|---------------------------------------|
| PUT     | /api/v1/routes/{id} | Eintrag ändern; Revision und Folgesynchronisation prüfen. | system-backend/sds-router/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.9 packet-core: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.18:8160; Unit `netcore-packet-core.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/packet-core/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                     | Zweck / Prüffrage                                                                | Quelle                                 |
|---------|--------------------------|----------------------------------------------------------------------------------|----------------------------------------|
| DELETE  | /api/v1/npdu-outbox/{id} | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/actions          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/bearers          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/contexts         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/contexts/{id}    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/events           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/packet-core/src/http.rs |
| GET     | /api/v1/nodes            | Eintrag oder Zustandsliste lesen;                                                | system-backend/packet-core/src/        |

| Methode | Pfad                           | Zweck / Prüffrage                                                                    | Quelle                                         |
|---------|--------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------|
|         |                                | Filter und Berechtigung am Live-Dienst prüfen.                                       | http.rs                                        |
| GET     | /api/v1/npdu-outbox            | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>packet-core/src/<br>http.rs |
| GET     | /api/v1/reassemblies           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>packet-core/src/<br>http.rs |
| GET     | /api/v1/status                 | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>packet-core/src/<br>http.rs |
| GET     | /health/live                   | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>packet-core/src/<br>http.rs |
| GET     | /health/ready                  | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>packet-core/src/<br>http.rs |
| GET     | /metrics                       | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>packet-core/src/<br>http.rs |
| POST    | /api/v1/actions/{id}/ack       | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/<br>packet-core/src/<br>http.rs |
| POST    | /api/v1/contexts/{id}/{action} | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>packet-core/src/<br>http.rs |
| POST    | /api/v1/downlink               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>packet-core/src/<br>http.rs |
| POST    | /api/v1/edge/events            | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>packet-core/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.10 ip-gateway: HTTP- und WebSocket-Vertrag

Beispielziel `10.0.20.19:8170`; Unit `netcore-ip-gateway.service`. Vor einem Fachtest zuerst `GET /health/live`, dann `GET /health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/ip-gateway/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                        | Zweck / Prüffrage                                                                | Quelle                                             |
|---------|---------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------|
| DELETE  | <code>/api/v1/blocked/{address}</code>      | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | <code>system-backend/ip-gateway/src/http.rs</code> |
| DELETE  | <code>/api/v1/dns/{id}</code>               | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | <code>system-backend/ip-gateway/src/http.rs</code> |
| DELETE  | <code>/api/v1/firewall/{id}</code>          | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | <code>system-backend/ip-gateway/src/http.rs</code> |
| DELETE  | <code>/api/v1/nat/{id}</code>               | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | <code>system-backend/ip-gateway/src/http.rs</code> |
| DELETE  | <code>/api/v1/routes/{id}</code>            | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | <code>system-backend/ip-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/blocked</code>                | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/ip-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/captures</code>               | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/ip-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/captures/{id}/download</code> | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/ip-gateway/src/http.rs</code> |
| GET     | <code>/api/v1/contexts</code>               | Eintrag oder Zustandsliste lesen; Filter und Berechtigung                        | <code>system-backend/ip-gateway/src/http.rs</code> |



| Methode | Pfad                | Zweck / Prüffrage                                                                    | Quelle                                |
|---------|---------------------|--------------------------------------------------------------------------------------|---------------------------------------|
|         |                     | am Live-Dienst prüfen.                                                               |                                       |
| GET     | /api/v1/dns         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/firewall    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/flows       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/kernel/plan | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/nat         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/routes      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /api/v1/status      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/ip-gateway/src/http.rs |
| GET     | /health/live        | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/ip-gateway/src/http.rs |
| GET     | /health/ready       | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/ip-gateway/src/http.rs |
| GET     | /metrics            | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/blocked     | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |

| Methode | Pfad                       | Zweck / Prüffrage                                                                       | Quelle                                |
|---------|----------------------------|-----------------------------------------------------------------------------------------|---------------------------------------|
| POST    | /api/v1/captures           | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/captures/{id}/stop | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/dns                | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/firewall           | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/kernel/reconcile   | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/nat                | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| POST    | /api/v1/routes             | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/ip-gateway/src/http.rs |
| PUT     | /api/v1/dns/{id}           | Eintrag ändern; Revision und Folgesynchronisation prüfen.                               | system-backend/ip-gateway/src/http.rs |
| PUT     | /api/v1/firewall/{id}      | Eintrag ändern; Revision und Folgesynchronisation prüfen.                               | system-backend/ip-gateway/src/http.rs |
| PUT     | /api/v1/nat/{id}           | Eintrag ändern; Revision und                                                            | system-backend/ip-gateway/src/http.rs |

| Methode | Pfad                | Zweck / Prüffrage                                         | Quelle                                |
|---------|---------------------|-----------------------------------------------------------|---------------------------------------|
|         |                     | Folgesynchronisation prüfen.                              |                                       |
| PUT     | /api/v1/routes/{id} | Eintrag ändern; Revision und Folgesynchronisation prüfen. | system-backend/ip-gateway/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.11 security-core: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.20:8180; Unit `netcore-security-core.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/security-core/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                    | Zweck / Prüffrage                                                                | Quelle                                   |
|---------|-------------------------|----------------------------------------------------------------------------------|------------------------------------------|
| DELETE  | /api/v1/profiles/{issi} | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/security-core/src/http.rs |
| GET     | /api/v1/actions         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/alarms          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/audit           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/auth-contexts   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |

| Methode | Pfad                       | Zweck / Prüffrage                                                                | Quelle                                   |
|---------|----------------------------|----------------------------------------------------------------------------------|------------------------------------------|
| GET     | /api/v1/auth-contexts/{id} | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/config             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/dck-contexts       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/nodes              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/policy             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/profiles           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/profiles/{issi}    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/status             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /api/v1/subscribers        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/security-core/src/http.rs |
| GET     | /health/live               | Prozess antwortet; sagt nichts über Gegenstellen.                                | system-backend/security-core/src/http.rs |
| GET     | /health/ready              | Start- und Abhängigkeitsbereitschaft prüfen.                                     | system-backend/security-core/src/http.rs |
| GET     | /metrics                   | Prometheus-Textformat; Zähler und Scrapezeit                                     | system-backend/security-core/src/        |

| Methode | Pfad                                | Zweck / Prüffrage                                                                    | Quelle                                           |
|---------|-------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------|
|         |                                     | beobachten.                                                                          | http.rs                                          |
| POST    | /api/v1/alarms/{id}/ack             | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/auth-contexts/{id}/response | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/auth-contexts/{id}/revoke   | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/auth/start                  | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/dck-contexts/{id}/revoke    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/edge/actions/claim          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/edge/actions/{id}/ack       | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/maintenance/backup          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/maintenance/expire          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/policy                      | Ressource oder Aktion anlegen;                                                       | system-backend/<br>security-core/src/            |

| Methode | Pfad                            | Zweck / Prüffrage                                                                       | Quelle                                           |
|---------|---------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------|
|         |                                 | Requestschema und Antwortcode im Live-OpenAPI prüfen.                                   | http.rs                                          |
| POST    | /api/v1/profiles                | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/profiles/{issi}/disable | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |
| POST    | /api/v1/profiles/{issi}/enable  | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>security-core/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.12 kmf: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.21:8190; Unit `netcore-kmf.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/kmf/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad            | Zweck / Prüffrage                                                                   | Quelle                             |
|---------|-----------------|-------------------------------------------------------------------------------------|------------------------------------|
| GET     | /api/v1/audit   | Eintrag oder Zustandsliste lesen;<br>Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/<br>src/http.rs |
| GET     | /api/v1/backups | Eintrag oder Zustandsliste lesen;<br>Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/<br>src/http.rs |
| GET     | /api/v1/config  | Eintrag oder Zustandsliste lesen;<br>Filter und Berechtigung                        | system-backend/kmf/<br>src/http.rs |

| Methode | Pfad                   | Zweck / Prüffrage                                                                | Quelle                         |
|---------|------------------------|----------------------------------------------------------------------------------|--------------------------------|
|         |                        | am Live-Dienst prüfen.                                                           |                                |
| GET     | /api/v1/export.json    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/keys           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/keys/{id}      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/nodes          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/otar/actions   | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/otar/jobs      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/otar/jobs/{id} | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/policy         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /api/v1/status         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/kmf/src/http.rs |
| GET     | /health/live           | Prozess antwortet; sagt nichts über Gegenstellen.                                | system-backend/kmf/src/http.rs |
| GET     | /health/ready          | Start- und Abhängigkeitsbereitschaft prüfen.                                     | system-backend/kmf/src/http.rs |

| Methode | Pfad                          | Zweck / Prüffrage                                                                    | Quelle                         |
|---------|-------------------------------|--------------------------------------------------------------------------------------|--------------------------------|
| GET     | /metrics                      | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/kmf/src/http.rs |
| POST    | /api/v1/backups               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/edge/actions/claim    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/edge/actions/{id}/ack | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys                  | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys/{id}/activate    | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys/{id}/destroy     | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys/{id}/retire      | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys/{id}/revoke      | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/keys/{id}/rotate      | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-                | system-backend/kmf/src/http.rs |



| Methode | Pfad                            | Zweck / Prüffrage                                                                       | Quelle                         |
|---------|---------------------------------|-----------------------------------------------------------------------------------------|--------------------------------|
|         |                                 | OpenAPI prüfen.                                                                         |                                |
| POST    | /api/v1/keys/{id}/stage         | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/maintenance/tick        | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/nodes                   | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/nodes/{node_id}/disable | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/nodes/{node_id}/enable  | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/otar/jobs               | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/otar/jobs/{id}/approve  | Aktion approve auslösen; Ergebnis, Idempotenz und Audit prüfen.                         | system-backend/kmf/src/http.rs |
| POST    | /api/v1/otar/jobs/{id}/cancel   | Aktion cancel auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/kmf/src/http.rs |
| POST    | /api/v1/otar/jobs/{id}/queue    | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/kmf/src/http.rs |
| POST    | /api/v1/policy                  | Ressource oder Aktion                                                                   | system-backend/kmf/            |

| Methode | Pfad | Zweck / Prüffrage                                                        | Quelle      |
|---------|------|--------------------------------------------------------------------------|-------------|
|         |      | anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.13 transit: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.22:8200; Unit `netcore-transit.service`. Vor einem Fachttest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/transit/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                       | Zweck / Prüffrage                                                                         | Quelle                                          |
|---------|--------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------|
| DELETE  | <code>/api/v1/routes/{route_id}</code>     | Eintrag entfernen;<br>vorher Referenzen und<br>Backup prüfen.                             | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/config</code>                | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/events</code>                | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/local-deliveries</code>      | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/locations/groups</code>      | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/locations/subscribers</code> | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | <code>system-backend/transit/src/http.rs</code> |
| GET     | <code>/api/v1/outbound</code>              | Eintrag oder<br>Zustandsliste lesen;                                                      | <code>system-backend/</code>                    |

| Methode | Pfad                                       | Zweck / Prüffrage                                                                    | Quelle                                 |
|---------|--------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------|
|         |                                            | Filter und Berechtigung am Live-Dienst prüfen.                                       | transit/src/http.rs                    |
| GET     | /api/v1/peers                              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>transit/src/http.rs |
| GET     | /api/v1/routes                             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>transit/src/http.rs |
| GET     | /api/v1/sessions                           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>transit/src/http.rs |
| GET     | /api/v1/status                             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>transit/src/http.rs |
| GET     | /health/live                               | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>transit/src/http.rs |
| GET     | /health/ready                              | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>transit/src/http.rs |
| GET     | /metrics                                   | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>transit/src/http.rs |
| POST    | /api/v1/local-deliveries/{delivery_id}/ack | Aktion ack auslösen; Ergebnis, Idempotenz und Audit prüfen.                          | system-backend/<br>transit/src/http.rs |
| POST    | /api/v1/locations/groups                   | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>transit/src/http.rs |
| POST    | /api/v1/locations/subscribers              | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>transit/src/http.rs |
| POST    | /api/v1/maintenance/backup                 | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-                | system-backend/<br>transit/src/http.rs |

| Methode | Pfad                                   | Zweck / Prüffrage                                                                       | Quelle                             |
|---------|----------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------|
|         |                                        | OpenAPI prüfen.                                                                         |                                    |
| POST    | /api/v1/maintenance/tick               | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/peer/envelopes                 | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/peer/heartbeat                 | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/peers                          | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/peers/{peer_id}/{action}       | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/route/resolve                  | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/routes                         | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/routes/{route_id}/{action}     | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/transit/src/http.rs |
| POST    | /api/v1/sessions/{session_id}/{action} | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-                | system-backend/transit/src/http.rs |

| Methode | Pfad                       | Zweck / Prüffrage                                                                                 | Quelle                                 |
|---------|----------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------|
|         |                            | OpenAPI prüfen.                                                                                   |                                        |
| POST    | /api/v1/transit/<br>submit | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>transit/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.14 application-gateway: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.23:8220; Unit `netcore-application-gateway.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/application-gateway/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                                     | Zweck / Prüffrage                                                                         | Quelle                                                 |
|---------|----------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------|
| DELETE  | /api/v1/connectors/<br>{connector_id}                    | Eintrag entfernen;<br>vorher Referenzen und<br>Backup prüfen.                             | system-backend/<br>application-<br>gateway/src/http.rs |
| DELETE  | /api/v1/connectors/<br>{connector_id}/<br>secrets/{name} | Eintrag entfernen;<br>vorher Referenzen und<br>Backup prüfen.                             | system-backend/<br>application-<br>gateway/src/http.rs |
| DELETE  | /api/v1/rules/<br>{rule_id}                              | Eintrag entfernen;<br>vorher Referenzen und<br>Backup prüfen.                             | system-backend/<br>application-<br>gateway/src/http.rs |
| DELETE  | /api/v1/templates/<br>{template_id}                      | Eintrag entfernen;<br>vorher Referenzen und<br>Backup prüfen.                             | system-backend/<br>application-<br>gateway/src/http.rs |
| GET     | /api/v1/audit                                            | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/<br>application-<br>gateway/src/http.rs |
| GET     | /api/v1/backups                                          | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/<br>application-<br>gateway/src/http.rs |
| GET     | /api/v1/connectors                                       | Eintrag oder<br>Zustandsliste lesen;                                                      | system-backend/<br>application-                        |

| Methode | Pfad                                      | Zweck / Prüffrage                                                                | Quelle                                         |
|---------|-------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------|
|         |                                           | Filter und Berechtigung am Live-Dienst prüfen.                                   | gateway/src/http.rs                            |
| GET     | /api/v1/connectors/{connector_id}         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/connectors/{connector_id}/secrets | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/deliveries                        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/events                            | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/rules                             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/status                            | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/templates                         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/tts/jobs                          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /api/v1/tts/jobs/{job_id}/artifact        | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/application-gateway/src/http.rs |
| GET     | /health/live                              | Prozess antwortet; sagt nichts über Gegenstellen.                                | system-backend/application-gateway/src/http.rs |

| Methode | Pfad                                            | Zweck / Prüffrage                                                                    | Quelle                                             |
|---------|-------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------|
| GET     | /health/ready                                   | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>application-gateway/src/http.rs |
| GET     | /metrics                                        | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/backups                                 | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors                              | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors/{connector_id}/disable       | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors/{connector_id}/enable        | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors/{connector_id}/reset-circuit | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors/{connector_id}/secrets       | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/connectors/{connector_id}/test          | Aktion test auslösen; Ergebnis, Idempotenz und Audit prüfen.                         | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/deliveries/{delivery_id}/{action}       | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |

| Methode | Pfad                                   | Zweck / Prüffrage                                                                       | Quelle                                             |
|---------|----------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------|
| POST    | /api/v1/events                         | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/maintenance/process-now        | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/maintenance/tick               | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/rules                          | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/templates                      | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/templates/{template_id}/render | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/tts/jobs                       | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/tts/jobs/{job_id}/publish      | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |
| POST    | /api/v1/webhooks/{connector_id}        | Ressource oder Aktion anlegen;<br>Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>application-gateway/src/http.rs |



| Methode | Pfad                              | Zweck / Prüffrage                                         | Quelle                                         |
|---------|-----------------------------------|-----------------------------------------------------------|------------------------------------------------|
| PUT     | /api/v1/connectors/{connector_id} | Eintrag ändern; Revision und Folgesynchronisation prüfen. | system-backend/application-gateway/src/http.rs |
| PUT     | /api/v1/rules/{rule_id}           | Eintrag ändern; Revision und Folgesynchronisation prüfen. | system-backend/application-gateway/src/http.rs |
| PUT     | /api/v1/templates/{template_id}   | Eintrag ändern; Revision und Folgesynchronisation prüfen. | system-backend/application-gateway/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.15 media-library: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.24:8230; Unit `netcore-media-library.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/media-library/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                                   | Zweck / Prüffrage                                                                | Quelle                                   |
|---------|----------------------------------------|----------------------------------------------------------------------------------|------------------------------------------|
| DELETE  | /api/v1/assets/{asset_id}              | Eintrag entfernen; vorher Referenzen und Backup prüfen.                          | system-backend/media-library/src/http.rs |
| GET     | /api/v1/assets                         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-library/src/http.rs |
| GET     | /api/v1/assets/{asset_id}              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-library/src/http.rs |
| GET     | /api/v1/assets/{asset_id}/audio.tacelp | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/media-library/src/http.rs |
| GET     | /api/v1/assets/{asset_id}/preview      | Eintrag oder Zustandsliste lesen;                                                | system-backend/media-library/src/        |

| Methode | Pfad                               | Zweck / Prüffrage                                                                    | Quelle                                           |
|---------|------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------|
|         |                                    | Filter und Berechtigung am Live-Dienst prüfen.                                       | http.rs                                          |
| GET     | /api/v1/assets/{asset_id}/waveform | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /api/v1/jobs                       | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /api/v1/status                     | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /api/v1/tts/status                 | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /api/v1/tts/templates              | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /api/v1/tts/voices                 | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /health/live                       | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /health/ready                      | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /metrics                           | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/<br>media-library/src/<br>http.rs |
| GET     | /openapi.json                      | Laufenden API-Vertrag dieses Builds abrufen.                                         | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/assets/import-url          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |

| Methode | Pfad                                  | Zweck / Prüffrage                                                                                 | Quelle                                           |
|---------|---------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------|
| POST    | /api/v1/assets/<br>upload-json        | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/assets/<br>{asset_id}/approve | Aktion approve<br>auslösen; Ergebnis,<br>Idempotenz und Audit<br>prüfen.                          | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/assets/<br>{asset_id}/archive | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/assets/<br>{asset_id}/process | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/assets/<br>{asset_id}/reject  | Aktion reject auslösen;<br>Ergebnis, Idempotenz<br>und Audit prüfen.                              | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/dispatch                      | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/jobs/<br>{job_id}/cancel      | Aktion cancel auslösen;<br>Ergebnis, Idempotenz<br>und Audit prüfen.                              | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/jobs/<br>{job_id}/retry       | Aktion retry auslösen;<br>Ergebnis, Idempotenz<br>und Audit prüfen.                               | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/recorder/<br>import           | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/tts/generate                  | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| POST    | /api/v1/tts/                          | Ressource oder Aktion                                                                             | system-backend/                                  |

| Methode | Pfad                           | Zweck / Prüffrage                                                                                 | Quelle                                           |
|---------|--------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------|
|         | templates/delete               | anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen.                          | media-library/src/<br>http.rs                    |
| POST    | /api/v1/tts/<br>templates/save | Ressource oder Aktion<br>anlegen;<br>Requestschema und<br>Antwortcode im Live-<br>OpenAPI prüfen. | system-backend/<br>media-library/src/<br>http.rs |
| PUT     | /api/v1/assets/<br>{asset_id}  | Eintrag ändern; Revision<br>und<br>Folgesynchronisation<br>prüfen.                                | system-backend/<br>media-library/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.16 control-room: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.25:9010; Unit `netcore-control-room.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Im geprüften Dienstquelltext wurde kein auswertbarer OpenAPI-paths-Katalog gefunden. API-Namen hier nicht als garantiert annehmen; README und laufenden Handler beziehungsweise `/openapi.json` prüfen.

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.17 observability: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.26:8210; Unit `netcore-observability.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/observability/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad           | Zweck / Prüffrage    | Quelle                                           |
|---------|----------------|----------------------|--------------------------------------------------|
| GET     | /api/v1/alerts | List alert instances | system-backend/<br>observability/src/<br>http.rs |

| Methode | Pfad                                   | Zweck / Prüffrage                 | Quelle                                           |
|---------|----------------------------------------|-----------------------------------|--------------------------------------------------|
| GET     | /api/v1/diagnostics                    | List diagnostic bundles           | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/logs                           | Search logs                       | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/metrics/<br>catalog            | Metric catalog                    | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/metrics/<br>series             | Query bounded time<br>series      | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/rules                          | List alert rules                  | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/silences                       | List silences                     | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/status                         | NMS status                        | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/targets                        | List scrape targets               | system-backend/<br>observability/src/<br>http.rs |
| GET     | /api/v1/traces                         | Search trace spans                | system-backend/<br>observability/src/<br>http.rs |
| GET     | /metrics                               | Prometheus metrics for<br>the NMS | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/diagnostics                    | List diagnostic bundles           | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/logs/ingest                    | Ingest NetCore JSON<br>logs       | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/<br>maintenance/scrape-<br>now | Run collection<br>immediately     | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/rules                          | List alert rules                  | system-backend/<br>observability/src/<br>http.rs |

| Methode | Pfad                                 | Zweck / Prüffrage             | Quelle                                           |
|---------|--------------------------------------|-------------------------------|--------------------------------------------------|
| POST    | /api/v1/silences                     | List silences                 | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/targets                      | List scrape targets           | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/targets/<br>{target_id}/test | Test and ingest one<br>target | system-backend/<br>observability/src/<br>http.rs |
| POST    | /api/v1/traces/<br>ingest            | Ingest NetCore JSON<br>spans  | system-backend/<br>observability/src/<br>http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.18 iot-gateway: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.27:8240; Unit `netcore-iot-gateway.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/iot-gateway/src/http.rs`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                              | Zweck / Prüffrage                                                                         | Quelle                                     |
|---------|-----------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------|
| GET     | /api/v1/commands                  | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/iot-<br>gateway/src/http.rs |
| GET     | /api/v1/commands/<br>{command_id} | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/iot-<br>gateway/src/http.rs |
| GET     | /api/v1/events                    | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/iot-<br>gateway/src/http.rs |
| GET     | /api/v1/home-<br>assistant        | Eintrag oder<br>Zustandsliste lesen;<br>Filter und Berechtigung<br>am Live-Dienst prüfen. | system-backend/iot-<br>gateway/src/http.rs |

| Methode | Pfad                                     | Zweck / Prüffrage                                                                | Quelle                                 |
|---------|------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------|
| GET     | /api/v1/home-assistant/entities          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/homematic/datapoints             | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/outbox                           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/policies                         | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/sources                          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/status                           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/topics                           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /api/v1/virtual-devices                  | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/iot-gateway/src/http.rs |
| GET     | /health/live                             | Prozess antwortet; sagt nichts über Gegenstellen.                                | system-backend/iot-gateway/src/http.rs |
| GET     | /health/ready                            | Start- und Abhängigkeitsbereitschaft prüfen.                                     | system-backend/iot-gateway/src/http.rs |
| GET     | /metrics                                 | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                         | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/actions/home-assistant-discovery | Ressource oder Aktion anlegen; Requestschema und                                 | system-backend/iot-gateway/src/http.rs |

| Methode | Pfad                               | Zweck / Prüffrage                                                                    | Quelle                                 |
|---------|------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------|
|         |                                    | Antwortcode im Live-OpenAPI prüfen.                                                  |                                        |
| POST    | /api/v1/actions/homematic-poll-now | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/actions/poll-now           | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/actions/reconnect          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/test/command               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/test/homeassistant-state   | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |
| POST    | /api/v1/test/publish               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/iot-gateway/src/http.rs |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.19 hardware-gateway: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.28:8250; Unit `netcore-hardware-gateway.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Im geprüften Dienstquelltext wurde kein auswertbarer OpenAPI-paths-Katalog gefunden. API-Namen hier nicht als garantiert annehmen; README und laufenden Handler beziehungsweise `/openapi.json` prüfen.



**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.20 rf-monitor: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.29:8260; Unit `netcore-rf-monitor.service`. Vor einem Fachtest zuerst `GET /health/live`, dann `GET /health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/rf-monitor/src/netcore_rf_monitor.py`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                           | Zweck / Prüffrage                                                                | Quelle                                                               |
|---------|--------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| GET     | <code>/api/v1/alarms</code>    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:578</code> |
| GET     | <code>/api/v1/events</code>    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:579</code> |
| GET     | <code>/api/v1/stations</code>  | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:577</code> |
| GET     | <code>/api/v1/status</code>    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:576</code> |
| GET     | <code>/health/live</code>      | Prozess antwortet; sagt nichts über Gegenstellen.                                | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:581</code> |
| GET     | <code>/health/ready</code>     | Start- und Abhängigkeitsbereitschaft prüfen.                                     | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:582</code> |
| GET     | <code>/metrics</code>          | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                         | <code>system-backend/rf-monitor/src/netcore_rf_monitor.py:583</code> |
| POST    | <code>/api/v1/telemetry</code> | Ressource oder Aktion anlegen; Requestschema und                                 | <code>system-backend/rf-monitor/src/netcore_rf_monitor.p</code>      |

| Methode | Pfad | Zweck / Prüffrage                   | Quelle |
|---------|------|-------------------------------------|--------|
|         |      | Antwortcode im Live-OpenAPI prüfen. | y:580  |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.21 alarm-workflow: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.30:8270; Unit `netcore-alarm-workflow.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/alarm-workflow/src/netcore_alarm_workflow.py`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                             | Zweck / Prüffrage                                                                | Quelle                                                                        |
|---------|----------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| GET     | <code>/api/v1/alarms</code>      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1137</code> |
| GET     | <code>/api/v1/alarms/{id}</code> | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1138</code> |
| GET     | <code>/api/v1/events</code>      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1142</code> |
| GET     | <code>/api/v1/status</code>      | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1136</code> |
| GET     | <code>/health/live</code>        | Prozess antwortet; sagt nichts über Gegenstellen.                                | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1143</code> |
| GET     | <code>/health/ready</code>       | Start- und Abhängigkeitsbereitschaft prüfen.                                     | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1144</code> |
| GET     | <code>/metrics</code>            | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                         | <code>system-backend/alarm-workflow/src/netcore_alarm_workflow.py:1144</code> |

| Methode | Pfad                         | Zweck / Prüffrage                                                                    | Quelle                                                                       |
|---------|------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
|         |                              |                                                                                      | ow.py:1145                                                                   |
| POST    | /api/v1/alarms               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>alarm-workflow/src/<br>netcore_alarm_workfl<br>ow.py:1137 |
| POST    | /api/v1/alarms/{id}/{action} | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>alarm-workflow/src/<br>netcore_alarm_workfl<br>ow.py:1139 |
| POST    | /api/v1/ingest-event         | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>alarm-workflow/src/<br>netcore_alarm_workfl<br>ow.py:1140 |
| POST    | /api/v1/ingest-status        | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/<br>alarm-workflow/src/<br>netcore_alarm_workfl<br>ow.py:1141 |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 16.22 task-workflow: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.31:8280; Unit `netcore-task-workflow.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/task-workflow/src/netcore_task_workflow.py`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad           | Zweck / Prüffrage                                                                | Quelle                                                                |
|---------|----------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| GET     | /api/v1/status | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/task-workflow/src/<br>netcore_task_workflo<br>w.py:865 |
| GET     | /api/v1/tasks  | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | system-backend/task-workflow/src/<br>netcore_task_workflo<br>w.py:867 |

| Methode | Pfad                        | Zweck / Prüffrage                                                                    | Quelle                                                        |
|---------|-----------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------|
| GET     | /api/v1/tasks/{id}          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/task-workflow/src/netcore_task_workflow.py:868 |
| GET     | /api/v1/templates           | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/task-workflow/src/netcore_task_workflow.py:866 |
| GET     | /health/live                | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/task-workflow/src/netcore_task_workflow.py:873 |
| GET     | /health/ready               | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/task-workflow/src/netcore_task_workflow.py:874 |
| GET     | /metrics                    | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/task-workflow/src/netcore_task_workflow.py:875 |
| GET     | /w                          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/task-workflow/src/netcore_task_workflow.py:872 |
| GET     | /x                          | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/task-workflow/src/netcore_task_workflow.py:871 |
| POST    | /api/v1/ingest-sds          | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/task-workflow/src/netcore_task_workflow.py:870 |
| POST    | /api/v1/tasks               | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/task-workflow/src/netcore_task_workflow.py:867 |
| POST    | /api/v1/tasks/{id}/{action} | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/task-workflow/src/netcore_task_workflow.py:869 |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.23 asset-management: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.32:8290; Unit `netcore-asset-management.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Im geprüften Dienst Quelltext wurde kein auswertbarer OpenAPI-paths-Katalog gefunden. API-Namen hier nicht als garantiert annehmen; README und laufenden Handler beziehungsweise `/openapi.json` prüfen.

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.24 sip-switch: HTTP- und WebSocket-Vertrag

Beispielziel 10.0.20.33:8300; Unit `netcore-sip-switch.service`. Vor einem Fachtest zuerst GET `/health/live`, dann GET `/health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Quellpfad: `system-backend/sip-switch/src/netcore_sip_switch.py`. Die folgenden Pfade stammen aus dem im Quellcode deklarierten OpenAPI-Objekt; dynamische interne oder ältere Routen können zusätzlich existieren.

| Methode | Pfad                           | Zweck / Prüffrage                                                                | Quelle                                                                |
|---------|--------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| GET     | <code>/api/v1/calls</code>     | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/sip-switch/src/netcore_sip_switch.py:1029</code> |
| GET     | <code>/api/v1/decisions</code> | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/sip-switch/src/netcore_sip_switch.py:1030</code> |
| GET     | <code>/api/v1/events</code>    | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/sip-switch/src/netcore_sip_switch.py:1031</code> |
| GET     | <code>/api/v1/mappings</code>  | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen. | <code>system-backend/sip-switch/src/netcore_sip_switch.py:1028</code> |
| GET     | <code>/api/v1/resolve</code>   | Eintrag oder Zustandsliste lesen;                                                | <code>system-backend/sip-switch/src/</code>                           |

| Methode | Pfad                            | Zweck / Prüffrage                                                                    | Quelle                                                   |
|---------|---------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------|
|         |                                 | Filter und Berechtigung am Live-Dienst prüfen.                                       | netcore_sip_switch.py:1032                               |
| GET     | /api/v1/status                  | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/sip-switch/src/netcore_sip_switch.py:1026 |
| GET     | /api/v1/tbs                     | Eintrag oder Zustandsliste lesen; Filter und Berechtigung am Live-Dienst prüfen.     | system-backend/sip-switch/src/netcore_sip_switch.py:1027 |
| GET     | /health/live                    | Prozess antwortet; sagt nichts über Gegenstellen.                                    | system-backend/sip-switch/src/netcore_sip_switch.py:1024 |
| GET     | /health/ready                   | Start- und Abhängigkeitsbereitschaft prüfen.                                         | system-backend/sip-switch/src/netcore_sip_switch.py:1025 |
| GET     | /metrics                        | Prometheus-Textformat; Zähler und Scrapezeit beobachten.                             | system-backend/sip-switch/src/netcore_sip_switch.py:1036 |
| POST    | /api/v1/actions/reload-asterisk | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sip-switch/src/netcore_sip_switch.py:1035 |
| POST    | /api/v1/actions/render-asterisk | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sip-switch/src/netcore_sip_switch.py:1034 |
| POST    | /api/v1/calls/{token}/state     | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sip-switch/src/netcore_sip_switch.py:1033 |
| POST    | /api/v1/resolve                 | Ressource oder Aktion anlegen; Requestschema und Antwortcode im Live-OpenAPI prüfen. | system-backend/sip-switch/src/netcore_sip_switch.py:1032 |

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

### 16.25 provisioning-core: HTTP- und WebSocket-Vertrag

Beispielziel separat:8125; Unit `netcore-provisioning-core.service`. Vor einem Fachtest zuerst `GET /health/live`, dann `GET /health/ready` und anschließend die fachliche Route ausführen. Lesende Abfragen sind für eine Erstdiagnose geeignet; schreibende Requests nur im dafür vorgesehenen Testsystem mit dokumentierter Testkennung auslösen.

Im geprüften Dienstquelltext wurde kein auswertbarer OpenAPI-paths-Katalog gefunden. API-Namen hier nicht als garantiert annehmen; README und laufenden Handler beziehungsweise `/openapi.json` prüfen.

**Lesende Probe.** `curl -fsS http://<DIENST-IP>:<PORT>/health/ready` und danach eine passende GET-Route. `live=200` bei `ready=503` bedeutet: Prozess läuft, Fachbereitschaft oder Abhängigkeit fehlt. Ein 404 verlangt den Abgleich des laufenden Buildstands mit der obigen Quelle.

## 17 Backend-Konfiguration im Detail

Die nachfolgenden Werte sind aus den eingetragenen Vorlagen geparkt. Die genaue Bedeutung eines Feldes ist zusätzlich im `config`-Modul des jeweiligen Dienstes zu prüfen. Wiederholte Objekte werden anhand ihres ersten Beispieleintrags beschrieben; weitere Einträge gehören weiterhin zur Vorlage. `[]` bezeichnet einen Listenwert. Secrets werden absichtlich ausgelassen. Der Prüfhinweis beschreibt einen sinnvollen Betriebsschritt und ist keine Behauptung über implementierte Validierungsregeln.

### 17.1 node-gateway: Felder und Prüfpunkte

Vorlage `system-backend/node-gateway/config/node-gateway.example.toml`; Ziel `/etc/netcore/node-gateway.toml`; Unit `netcore-node-gateway.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.1.1 server

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                            | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|--------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>             | <code>0.0.0.0:8080</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.node_path</code>        | <code>/ws/node</code>     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.                 |
| <code>server.backend_path</code>     | <code>/ws/backend</code>  | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.                 |
| <code>server.history_limit</code>    | <code>1000</code>         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |
| <code>server.stale_after_secs</code> | <code>20</code>           | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung                     |



| Schlüssel                    | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                              |          |                                                                      | nachstellen.                                                                                      |
| server.hello_timeout_secs    | 10       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| server.application_ping_secs | 15       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.1.2 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

### 17.1.3 limits

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel | Bedeutung im Betrieb                                             | Konkrete Prüfung                                                                       |
|----------------------------|----------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_message_bytes   | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer. | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_http_body_bytes | 1048576  | Größenlimit für Nachricht, Frame,                                | Grenzwerttest und RAM-/Disk-Budget                                                     |

| Schlüssel | Beispiel | Bedeutung im Betrieb           | Konkrete Prüfung                                    |
|-----------|----------|--------------------------------|-----------------------------------------------------|
|           |          | Speicher, Queue oder Transfer. | ausführen; zu große Payloads kontrolliert ablehnen. |

#### 17.1.4 service\_monitor

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|------------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| service_monitor.enabled            | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| service_monitor.interval_secs      | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| service_monitor.timeout_ms         | 1500     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| service_monitor.failure_threshold  | 2        | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                            | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.        |
| service_monitor.recovery_threshold | 2        | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                            | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.        |

#### 17.1.5 service\_monitor.targets[0]

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                    | Beispiel                             | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| service_monitor.targets[0].name              | mobility-core                        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| service_monitor.targets[0].url               | http://10.0.20.11:8090/health/ready  | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| service_monitor.targets[0].critical_for_edge | true                                 | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.    |
| service_monitor.targets[0].fallback_mode     | local_registration_and_location_area | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |

**Nach der Änderung.** Für TBS-Registrierung, Core-Health-Matrix, Command-Ack den Fachtest ausführen: /api/v1/nodes und /api/v1/core-services mit TBS-Dashboard vergleichen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Verbindung und letzte Ack-Revision vor einer erneuten Aktion prüfen.

## 17.2 mobility-core: Felder und Prüfpunkte

Vorlage system-backend/mobility-core/config/mobility-core.example.toml; Ziel /etc/netcore/mobility-core.toml; Unit netcore-mobility-core.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.2.1 server

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|----------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8090 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge,                                                            | Zähler unter Last                                                                           |

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                               |          | Verbindungen, Retries oder Aufbewahrung.                             | beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                                   |
| server.transfer_time_out_secs | 45       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.2.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel                               | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|-----------------------------|----------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| node_gateway.url            | ws://<br>10.0.1.30:8080/ws/<br>backend | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| node_gateway.reconnect_secs | 5                                      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.2.3 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

### 17.2.4 limits

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel              | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                      |
|------------------------|----------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| limits.max_body_bytes  | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_transfers   | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_subscribers | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |

**Nach der Änderung.** Für Serving-Node, Übergabe und MM-Kontext den Fachtest ausführen: zwei TBS und dieselbe Test-ISSI mit Zeitstempel verfolgen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Kollisionen und veraltete Serving-Lage vor Restore prüfen.

### 17.3 subscriber-core: Felder und Prüfpunkte

Vorlage `system-backend/subscriber-core/config/subscriber-core.example.toml`; Ziel `/etc/netcore/subscriber-core.toml`; Unit `netcore-subscriber-core.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.3.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|----------------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8100 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und                                              |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung      |
|-----------|----------|----------------------|-----------------------|
|           |          |                      | nach Freigabe prüfen. |

### 17.3.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel                               | Bedeutung im Betrieb                                                                                 | Konkrete Prüfung                                                                                                 |
|-----------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| node_gateway.url            | ws://<br>127.0.0.1:8080/ws/<br>backend | Zieladresse einer<br>Gegenstelle; Schema,<br>Host, Port und Pfad<br>bilden gemeinsam den<br>Vertrag. | DNS/Route und Ziel-<br>Health prüfen; bei ws://<br>den WebSocket-Pfad<br>und Ack separat testen.                 |
| node_gateway.reconnect_secs | 5                                      | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                           | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |

### 17.3.3 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                                      | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                     |
|-----------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-<br>subscriber-core/<br>subscribers.json     | Datei- oder<br>Verzeichnisort für<br>Zustand, Konfiguration,<br>Cache oder Archiv. | Existenz, Eigentümer,<br>Schreibrecht und<br>Backup-/Restore-Grenze<br>am Zielhost<br>kontrollieren. |
| storage.backup_path   | /var/lib/netcore-<br>subscriber-core/<br>subscribers.json.bak | Datei- oder<br>Verzeichnisort für<br>Zustand, Konfiguration,<br>Cache oder Archiv. | Existenz, Eigentümer,<br>Schreibrecht und<br>Backup-/Restore-Grenze<br>am Zielhost<br>kontrollieren. |

### 17.3.4 access\_policy

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                             | Beispiel   | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                  |
|---------------------------------------|------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| access_policy.mode                    | allow_list | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| access_policy.auto_sync               | true       | Boolesche Steuerung des betreffenden Funktionsblocks.                              | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| access_policy.disconnect_unauthorized | true       | Boolesche Steuerung des betreffenden Funktionsblocks.                              | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| access_policy.sync_timeout_secs       | 30         | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.3.5 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

### 17.3.6 limits

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung  |
|----------------------|----------|----------------------|-------------------|
| limits.max_body_byte | 2097152  | Größenlimit für      | Grenzwerttest und |

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                  |
|----------------------------------|----------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| s                                |          | Nachricht, Frame, Speicher, Queue oder Transfer.                  | RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.              |
| limits.max_subscribers           | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |
| limits.max_groups_per_subscriber | 1024     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |

**Nach der Änderung.** Für Zulassung, Profile, Sperren, Sync den Fachtest ausführen: Test-ISSI zulassen, sperren, Synchronisationsstand lesen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; bei Policy-Lücke die letzte bekannte TBS-Policy nicht still öffnen.

## 17.4 group-core: Felder und Prüfpunkte

Vorlage system-backend/group-core/config/group-core.example.toml; Ziel /etc/netcore/group-core.toml; Unit netcore-group-core.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.4.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|----------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8110 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.           |

### 17.4.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.



| Schlüssel                   | Beispiel                               | Bedeutung im Betrieb                                                                                 | Konkrete Prüfung                                                                                                 |
|-----------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| node_gateway.url            | ws://<br>10.0.1.XX:8080/ws/<br>backend | Zieladresse einer<br>Gegenstelle; Schema,<br>Host, Port und Pfad<br>bilden gemeinsam den<br>Vertrag. | DNS/Route und Ziel-<br>Health prüfen; bei ws://<br>den WebSocket-Pfad<br>und Ack separat testen.                 |
| node_gateway.reconnect_secs | 5                                      | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                           | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |

#### 17.4.3 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                            | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                     |
|-----------------------|-----------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-<br>group-core/<br>groups.json     | Datei- oder<br>Verzeichnisort für<br>Zustand, Konfiguration,<br>Cache oder Archiv. | Existenz, Eigentümer,<br>Schreibrecht und<br>Backup-/Restore-Grenze<br>am Zielhost<br>kontrollieren. |
| storage.backup_path   | /var/lib/netcore-<br>group-core/<br>groups.json.bak | Datei- oder<br>Verzeichnisort für<br>Zustand, Konfiguration,<br>Cache oder Archiv. | Existenz, Eigentümer,<br>Schreibrecht und<br>Backup-/Restore-Grenze<br>am Zielhost<br>kontrollieren. |

#### 17.4.4 policy

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel | Bedeutung im Betrieb                                                                                | Konkrete Prüfung                                                                              |
|------------------------------|----------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| policy.allow_unlisted_groups | false    | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern. | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren. |
| policy.enforce_memberships   | true     | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                         | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.   |

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| policy.reconcile_registered | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| policy.auto_sync            | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| policy.sync_timeout_secs    | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| policy.dgna_timeout_secs    | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.4.5 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

#### 17.4.6 limits

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel              | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|------------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes  | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_groups      | 65536    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_memberships | 1000000  | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |

**Nach der Änderung.** Für GSSI, Mitgliedschaft, Affiliation, DGNA den Fachtest ausführen: Testgruppe und Mitgliedschaft auf beiden TBS abgleichen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; fehlende Affiliation von gesperrter Mitgliedschaft unterscheiden.

### 17.5 call-control: Felder und Prüfpunkte

Vorlage `system-backend/call-control/config/call-control.example.toml`; Ziel `/etc/netcore/call-control.toml`; Unit `netcore-call-control.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.5.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|----------------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8120 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

#### 17.5.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel                               | Bedeutung im Betrieb                                                                                 | Konkrete Prüfung                                                                                                 |
|-----------------------------|----------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| node_gateway.url            | ws://<br>10.0.1.XX:8080/ws/<br>backend | Zieladresse einer<br>Gegenstelle; Schema,<br>Host, Port und Pfad<br>bilden gemeinsam den<br>Vertrag. | DNS/Route und Ziel-<br>Health prüfen; bei ws://<br>den WebSocket-Pfad<br>und Ack separat testen.                 |
| node_gateway.reconnect_secs | 5                                      | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                           | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |

### 17.5.3 mobility\_core

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel                  | Bedeutung im Betrieb                                                                                 | Konkrete Prüfung                                                                                                 |
|------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| mobility_core.enabled              | true                      | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern.  | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren.                    |
| mobility_core.base_url             | http://<br>10.0.1.XX:8090 | Zieladresse einer<br>Gegenstelle; Schema,<br>Host, Port und Pfad<br>bilden gemeinsam den<br>Vertrag. | DNS/Route und Ziel-<br>Health prüfen; bei ws://<br>den WebSocket-Pfad<br>und Ack separat testen.                 |
| mobility_core.timeout_ms           | 1500                      | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                           | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |
| mobility_core.allow_local_fallback | false                     | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern.  | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren.                    |
| mobility_core.accept_stale_route   | false                     | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                          | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                      |

### 17.5.4 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                     | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|----------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-call-control/calls.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path   | /var/lib/netcore-call-control/calls.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.5.5 calls

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| calls.command_timeout_secs         | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| calls.restore_timeout_secs         | 45       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| calls.reconcile_interval_secs      | 2        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| calls.auto_target_affiliated_nodes | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| calls.release_partial              | false    | Zeitgrenze für Retry,                                                | Zeitquelle und                                                                                    |

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|----------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| l_start_on_failure               |          | Lease, Ablauf, Pufferung oder Zustandswechsel.                                         | beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.   |
| calls.allow_operator_force_floor | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |

#### 17.5.6 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

#### 17.5.7 limits

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|-----------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_calls      | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_legs_per_  | 1024     | Obergrenze für Einträge,                                          | Zähler unter Last                                                                      |

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                  |
|-----------------------------|----------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| call                        |          | Verbindungen, Retries oder Aufbewahrung.                          | beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                   |
| limits.max_pending_commands | 20000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |

**Nach der Änderung.** Für logische Rufe, Floor, Call Legs, Restore den Fachtest ausführen: Aufbau, Sprecherwechsel und Release korreliert verfolgen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; hängende Call Legs erst nach Session-/Medienprüfung lösen.

## 17.6 media-switch: Felder und Prüfpunkte

Vorlage system-backend/media-switch/config/media-switch.example.toml; Ziel /etc/netcore/media-switch.toml; Unit netcore-media-switch.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.6.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|----------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8130 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.           |

### 17.6.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel        | Beispiel                       | Bedeutung im Betrieb                                       | Konkrete Prüfung                                               |
|------------------|--------------------------------|------------------------------------------------------------|----------------------------------------------------------------|
| node_gateway.url | ws://10.0.1.20:8080/ws/backend | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad |

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                             |          | bilden gemeinsam den Vertrag.                                        | und Ack separat testen.                                                                           |
| node_gateway.reconnect_secs | 2        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.6.3 call\_control

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                                       | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|-----------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| call_control.url                  | http://10.0.1.24:8120/api/v1/calls             | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| call_control.events_url           | ws://10.0.1.24:8120/ws/media                   | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| call_control.route_ready_url      | http://10.0.1.24:8120/api/v1/media/route-ready | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| call_control.reconcile_secs       | 15                                             | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| call_control.reconnect_secs       | 1                                              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| call_control.request_timeout_secs | 2                                              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den                                                  |



| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung                                 |
|-----------|----------|----------------------|--------------------------------------------------|
|           |          |                      | Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.6.4 media

Hier werden 14 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                | Beispiel | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                           |
|------------------------------------------|----------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| media.frame_duration_ms                  | 60       | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.               |
| media.jitter_buffer_frames               | 2        | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.             | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.     |
| media.min_jitter_buffer_frames           | 1        | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.             | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.     |
| media.max_jitter_buffer_frames           | 12       | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.             | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.     |
| media.adaptive_jitter                    | true     | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.               |
| media.adaptive_jitter_up_threshold_ms    | 18       | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                  | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| media.adaptive_jitter_down_stable_frames | 120      | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.               |
| media.cold_start_buffer_frames           | 5        | Größenlimit für Nachricht, Frame,                                            | Grenzwerttest und RAM-/Disk-Budget                                                         |

| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|------------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                    |          | Speicher, Queue oder Transfer.                                                         | ausführen; zu große Payloads kontrolliert ablehnen.                                               |
| media.cold_start_buffer_max_age_ms | 600      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                       | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.             |
| media.session_idle_secs            | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| media.max_frames_per_tick          | 256      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                      | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| media.allow_same_leg_loopback      | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| media.tap_history_frames           | 256      | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.           | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| media.recorder_tap_history_frames  | 20000    | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.           | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |

### 17.6.5 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode        | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remot | true     | Funktions- oder Policy-                                                            | Wirksamen Modus nach                                                                        |

| Schlüssel    | Beispiel | Bedeutung im Betrieb                                            | Konkrete Prüfung                                                |
|--------------|----------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| e_management |          | Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Neustart ablesen und positive wie negative Probe dokumentieren. |

### 17.6.6 limits

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|---------------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes     | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_sessions       | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_streams        | 50000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_pending_frames | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |

**Nach der Änderung.** Für Frame-Routing, Jitter, Drop, Codec den Fachtest ausführen: beide Richtungen mit hörbarem Testton und Zählern messen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; bei Stille zuerst RouteReady und Frameeingang je Leg trennen.

## 17.7 recorder: Felder und Prüfpunkte

Vorlage system-backend/recorder/config/recorder.example.toml; Ziel /etc/netcore/recorder.toml; Unit netcore-recorder.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.7.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|----------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8140 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 2000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.           |

### 17.7.2 media\_switch

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                                   | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|-----------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| media_switch.tap_url              | http://10.0.1.25:8130/api/v1/recorder/taps | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| media_switch.sessions_url         | http://10.0.1.25:8130/api/v1/sessions      | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| media_switch.poll_interval_ms     | 100                                        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| media_switch.session_reconcile_ms | 1000                                       | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Playout oder Cache.             | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| media_switch.request_timeout_secs | 3                                          | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| media_switch.batch_limit          | 500                                        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                        | Zähler unter Last beobachten; Verhalten direkt am Limit und                                       |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung      |
|-----------|----------|----------------------|-----------------------|
|           |          |                      | nach Freigabe prüfen. |

### 17.7.3 storage

Hier werden 9 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                             | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|-----------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| storage.root                      | /var/lib/netcore-recorder/recordings | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| storage.export_root               | /var/lib/netcore-recorder/exports    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| storage.frame_duration_ms         | 60                                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| storage.session_absent_grace_secs | 3                                    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| storage.maximum_idle_secs         | 600                                  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| storage.default_retention_days    | 30                                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| storage.retention_scan_secs       | 60                                   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                      |
|-------------------------------|----------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| storage.fsync_every_frames    | 50       | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen. |
| storage.minimum_free_space_mb | 512      | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen. |

#### 17.7.4 security

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| security.allow_delete            | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

#### 17.7.5 limits

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel | Bedeutung im Betrieb                                             | Konkrete Prüfung                                                                       |
|------------------------------|----------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes        | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer. | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_active_recordings | 1000     | Obergrenze für Einträge, Verbindungen, Retries                   | Zähler unter Last beobachten; Verhalten                                                |

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                  |
|-----------------------|----------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|                       |          | oder Aufbewahrung.                                                | direkt am Limit und nach Freigabe prüfen.                                         |
| limits.max_recordings | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |

**Nach der Änderung.** Für Tap-Aufnahme, Asset, Export, Retention den Fachtest ausführen: kurzen Testcall aufnehmen und Datei/Metadaten abspielen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; unvollständige Aufnahmen vor Archivrotation sichern.

## 17.8 sds-router: Felder und Prüfpunkte

Vorlage `system-backend/sds-router/config/sds-router.example.toml`; Ziel `/etc/netcore/sds-router.toml`; Unit `netcore-sds-router.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.8.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|----------------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8150 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 4000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

### 17.8.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel        | Beispiel                                    | Bedeutung im Betrieb                                                            | Konkrete Prüfung                                                                                    |
|------------------|---------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| node_gateway.url | <code>ws://10.0.1.20:8080/ws/backend</code> | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den | DNS/Route und Ziel-Health prüfen; bei <code>ws://</code> den WebSocket-Pfad und Ack separat testen. |

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                             |          | Vertrag.                                                             |                                                                                                   |
| node_gateway.reconnect_secs | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.8.3 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                      | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|-----------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-sds-router/messages.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path   | /var/lib/netcore-sds-router/messages.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.8.4 routing

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|--------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| routing.default_ttl_secs | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.max_ttl_secs     | 86400    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.max_attempts     | 5        | Obergrenze für Einträge,                                             | Zähler unter Last                                                                                 |



| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                               |          | Verbindungen, Retries oder Aufbewahrung.                             | beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                                   |
| routing.initial_retry_secs    | 2        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.max_retry_secs        | 60       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.dedupe_window_secs    | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.presence_timeout_secs | 90       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.authoritative_ingress | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.8.5 security

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen                                        | Wirksamen Modus nach Neustart ablesen und                                                   |

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                  | Konkrete Prüfung                                                                   |
|-------------------------------|----------|-------------------------------------------------------|------------------------------------------------------------------------------------|
|                               |          | Datenweg oder eine Schutzbedingung ändern.            | positive wie negative Probe dokumentieren.                                         |
| security.mask_payload_in_list | false    | Boolesche Steuerung des betreffenden Funktionsblocks. | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen. |

### 17.8.6 limits

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                      |
|--------------------------|----------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| limits.max_body_bytes    | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_payload_bytes | 2048     | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_messages      | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_routes        | 4096     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |

**Nach der Änderung.** Für Einzel-/Gruppen-SDS, Offline-Spool, Replay den Fachtest ausführen: Delivery-Ack und deduplizierten Replay mit Test-ISSI prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; alte Spool-Einträge mit Ziel- und Ablaufzeit abgleichen.

### 17.9 packet-core: Felder und Prüfpunkte

Vorlage system-backend/packet-core/config/packet-core.example.toml; Ziel /etc/netcore/packet-core.toml; Unit netcore-packet-core.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.9.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|----------------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind          | 0.0.0.0:8160 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit | 5000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

### 17.9.2 node\_gateway

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel                       | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|-----------------------------|--------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| node_gateway.url            | ws://127.0.0.1:8080/ws/backend | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| node_gateway.reconnect_secs | 5                              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.9.3 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|-----------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-packet-core/state.json | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

| Schlüssel           | Beispiel                                    | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|---------------------|---------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.backup_path | /var/lib/netcore-packet-core/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

#### 17.9.4 packet

Hier werden 11 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                  |
|---------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| packet.mode               | shadow   | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| packet.ready_timer_secs   | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| packet.standby_timer_secs | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| packet.response_wait_secs | 10       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| packet.context_ready_secs | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| packet.default_mtu        | 1500     | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                   | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |

| Schlüssel                            | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                       |
|--------------------------------------|----------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| packet.max_n_pdu_bytes               | 65535    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                       | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| packet.max_contexts_per_subscriber   | 14       | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                      | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| packet.max_total_contexts            | 4096     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                      | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| packet.strict_source_address         | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.   |
| packet.preserve_context_on_node_loss | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.   |

### 17.9.5 address\_pool

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel    | Bedeutung im Betrieb                                                     | Konkrete Prüfung                                                                     |
|-----------------------------|-------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| address_pool.network_prefix | [10, 44, 0] | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen. | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen. |
| address_pool.first_host     | 2           | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen. | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen. |
| address_pool.last_host      | 254         | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen. | Mit ip route, Poolgrenzen und Gegenstellen                                           |

| Schlüssel                 | Beispiel  | Bedeutung im Betrieb                                                     | Konkrete Prüfung                                                                        |
|---------------------------|-----------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
|                           |           | Netz passen.                                                             | vergleichen;<br>Doppelbelegung ausschließen.                                            |
| address_pool.gateway      | 10.44.0.1 | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen. | Mit ip route, Poolgrenzen und Gegenstellen vergleichen;<br>Doppelbelegung ausschließen. |
| address_pool.allow_static | true      | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen. | Mit ip route, Poolgrenzen und Gegenstellen vergleichen;<br>Doppelbelegung ausschließen. |

### 17.9.6 fragmentation

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|------------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| fragmentation.timeout_secs               | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| fragmentation.max_datagrams              | 256      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| fragmentation.max_total_bytes            | 8388608  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| fragmentation.max_fragments_per_datagram | 512      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| fragmentation.reject_overlaps            | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.9.7 flow\_control

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                  | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                                                  |
|--------------------------------------------|----------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| flow_control.max_queue_packets_per_context | 64       | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                       | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| flow_control.max_queue_bytes_per_context   | 262144   | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                        | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| flow_control.queue_timeout_secs            | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                    | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| flow_control.action_retry_secs             | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                    | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| flow_control.action_max_attempts           | 5        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |

### 17.9.8 security

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine                     | Wirksamen Modus nach Neustart ablesen und positive wie negative                             |

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                  | Konkrete Prüfung                                                                   |
|--------------------------|----------|-------------------------------------------------------|------------------------------------------------------------------------------------|
|                          |          | Schutzbedingung ändern.                               | Probe dokumentieren.                                                               |
| security.expose_payloads | true     | Boolesche Steuerung des betreffenden Funktionsblocks. | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen. |

### 17.9.9 limits

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                      |
|--------------------------|----------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| limits.max_body_bytes    | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_events        | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_actions       | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_payload_bytes | 65535    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |

**Nach der Änderung.** Für PDP/NSAPI, Adresspool, Fragmentierung den Fachtest ausführen: Kontextaufbau, IP-Zuordnung und Release beobachten. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; IP-Konflikt und verwaisten Kontext getrennt beheben.

### 17.10 ip-gateway: Felder und Prüfpunkte

Vorlage system-backend/ip-gateway/config/ip-gateway.example.toml; Ziel /etc/netcore/ip-gateway.toml; Unit netcore-ip-gateway.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.



### 17.10.1 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|-------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind | 0.0.0.0:8170 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.10.2 packet\_core

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                      | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                    |
|--------------------------------|-----------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| packet_core.url                | http://127.0.0.1:8160 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei <code>ws://</code> den WebSocket-Pfad und Ack separat testen. |
| packet_core.poll_interval_ms   | 250                   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.   |
| packet_core.context_refresh_ms | 1000                  | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                               |
| packet_core.request_timeout_ms | 2000                  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.   |
| packet_core.outbox_batch       | 250                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                               |

### 17.10.3 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                   | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|--------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-ip-gateway/state.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path   | /var/lib/netcore-ip-gateway/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.10.4 interface

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel         | Beispiel    | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------|-------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| interface.mode    | shadow      | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| interface.name    | ntc-tun0    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| interface.address | 10.0.0.1/24 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| interface.network | 10.0.0.0/24 | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen.                         | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen.          |
| interface.mtu     | 480         | Größenlimit für Nachricht, Frame,                                                                | Grenzwerttest und RAM-/Disk-Budget                                                            |

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                          |          | Speicher, Queue oder Transfer.                                                                   | ausführen; zu große Payloads kontrolliert ablehnen.                                           |
| interface.owner_user     | netcore  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| interface.delete_on_exit | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

#### 17.10.5 routing

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|---------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| routing.enable_ipv4_forwarding  | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| routing.reconcile_interval_secs | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.install_connected_route | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

#### 17.10.6 nat

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel | Bedeutung im Betrieb                                           | Konkrete Prüfung                                                |
|-------------|----------|----------------------------------------------------------------|-----------------------------------------------------------------|
| nat.enabled | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine | Wirksamen Modus nach Neustart ablesen und positive wie negative |

| Schlüssel            | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                      |          | Schutzbedingung ändern.                                                                          | Probe dokumentieren.                                                                          |
| nat.masquerade       | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| nat.egress_interface | eth0     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.10.7 firewall

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| firewall.enabled                | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| firewall.default_forward_policy | drop     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| firewall.allow_established      | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| firewall.allow_general_internet | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| firewall.allow_icmp             | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| firewall.log_drops              | false    | Boolesche Steuerung                                                                              | Effektiven Zustand und                                                                        |

| Schlüssel | Beispiel | Bedeutung im Betrieb              | Konkrete Prüfung                                            |
|-----------|----------|-----------------------------------|-------------------------------------------------------------|
|           |          | des betreffenden Funktionsblocks. | einen Test mit aktivierter/deaktivierter Option abgleichen. |

#### 17.10.8 dns

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|----------------------|--------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| dns.enabled          | true         | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                     |
| dns.bind             | 10.0.0.1:53  | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.              | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| dns.upstream         | 1.1.1.1:53   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| dns.local_domain     | netcore.test | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| dns.ttl_secs         | 30           | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |
| dns.query_timeout_ms | 2000         | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |

### 17.10.9 test\_server

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel     | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|---------------------------|--------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| test_server.enabled       | true         | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| test_server.bind          | 0.0.0.0:8088 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.    | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| test_server.udp_echo_bind | 0.0.0.0:7007 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.    | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.10.10 capture

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel              | Beispiel                             | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| capture.directory      | /var/lib/netcore-ip-gateway/captures | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| capture.max_captures   | 64                                   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.             |
| capture.max_file_bytes | 268435456                            | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| capture.snaplen        | 65535                                | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |

**17.10.11 security**

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

**17.10.12 limits**

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel               | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|-------------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes   | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_events       | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_flows        | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_packet_bytes | 65535    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |

**Nach der Änderung.** Für TUN, Route, DNS, NAT, Firewall den Fachtest ausführen: TUN-Interface, Route und einen echten IP-Rückweg prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; bei einseitigem Verkehr SNAT und Reverse Route auseinanderhalten.

## 17.11 security-core: Felder und Prüfpunkte

Vorlage `system-backend/security-core/config/security-core.example.toml`; Ziel `/etc/netcore/security-core.toml`; Unit `netcore-security-core.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.11.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|-----------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>          | <code>0.0.0.0:8180</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.history_limit</code> | <code>5000</code>         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

### 17.11.2 node\_gateway

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                | Beispiel                                    | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                    |
|------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>node_gateway.url</code>            | <code>ws://127.0.0.1:8080/ws/backend</code> | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei <code>ws://</code> den WebSocket-Pfad und Ack separat testen. |
| <code>node_gateway.reconnect_secs</code> | <code>5</code>                              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.   |
| <code>node_gateway.observe_nodes</code>  | <code>true</code>                           | Boolesche Steuerung des betreffenden Funktionsblocks.                                    | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                  |



### 17.11.3 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                      | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|-----------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-security-core/state.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path   | /var/lib/netcore-security-core/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.lab_seed_path | /var/lib/netcore-security-core/lab-auth.seed  | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.11.4 policy

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                      | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                                            |
|--------------------------------|----------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| policy.operating_mode          | shadow   | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.      | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| policy.default_security_class  | 1        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                       |
| policy.minimum_security_class  | 1        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                       |
| policy.authentication_required | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.          |

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|-----------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| policy.allow_classl_fallback      | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| policy.reject_unknown_subscribers | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| policy.disable_after_failures     | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |

#### 17.11.5 authentication

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel        | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|-----------------------------------|-----------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| authentication.provider           | lab_hmac_sha256 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| authentication.challenge_bytes    | 16              | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| authentication.response_bytes     | 16              | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| authentication.challenge_ttl_secs | 30              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| authentication.max_attempts       | 3               | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

| Schlüssel                           | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| authentication.lockout_secs         | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| authentication.issue_dck_on_success | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

#### 17.11.6 dck

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| dck.key_bytes                 | 16       | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| dck.ttl_secs                  | 3600     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| dck.rotate_before_secs        | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| dck.max_active_per_subscriber | 2        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

#### 17.11.7 security

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                               | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|-----------------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                           | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management        | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| security.expose_ephemeral_edge_material | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.          |

### 17.11.8 limits

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|-----------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_profiles   | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_contexts   | 20000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_actions    | 20000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_alarms     | 20000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_audit      | 100000   | Obergrenze für Einträge, Verbindungen, Retries                    | Zähler unter Last beobachten; Verhalten                                                |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung                          |
|-----------|----------|----------------------|-------------------------------------------|
|           |          | oder Aufbewahrung.   | direkt am Limit und nach Freigabe prüfen. |

**Nach der Änderung.** Für Admission und Security-Policy den Fachtest ausführen: erlaubte und gesperrte Testkennung plus Audit prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; keine Sicherheitsklasse zur Fehlerumgehung absenken.

## 17.12 kmf: Felder und Prüfpunkte

Vorlage `system-backend/kmf/config/kmf.example.toml`; Ziel `/etc/netcore/kmf.toml`; Unit `netcore-kmf.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.12.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|-----------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>          | <code>0.0.0.0:8190</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.history_limit</code> | <code>5000</code>         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

### 17.12.2 storage

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel                                     | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|------------------------------------|----------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>storage.database_path</code> | <code>/var/lib/netcore-kmf/state.json</code> | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| <code>storage.vault_path</code>    | <code>/var/lib/netcore-kmf/vault.json</code> | Datei- oder Verzeichnisort für Zustand, Konfiguration,                    | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze                            |

| Schlüssel               | Beispiel                        | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-------------------------|---------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
|                         |                                 | Cache oder Archiv.                                                        | am Zielhost kontrollieren.                                                               |
| storage.master_key_path | /var/lib/netcore-kmf/master.key | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_dir      | /var/lib/netcore-kmf/backups    | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.bootstrap_dir   | /var/lib/netcore-kmf/bootstrap  | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.12.3 policy

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                  |
|-----------------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| policy.operating_mode             | shadow   | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| policy.default_key_bytes          | 16       | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                   | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| policy.default_crypto_period_secs | 86400    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| policy.rotation_lead_secs         | 3600     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung              |

| Schlüssel                               | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|-----------------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|                                         |          |                                                                                        | nachstellen.                                                                         |
| policy.require_dual_approval            | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| policy.allow_overlapping_crypto_periods | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| policy.auto_retire_predecessor          | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |

#### 17.12.4 vault

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel              | Beispiel       | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------|----------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| vault.provider         | lab_file_vault | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| vault.master_key_bytes | 32             | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| vault.fsync            | true           | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

#### 17.12.5 otar

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel | Bedeutung im Betrieb  | Konkrete Prüfung |
|----------------------|----------|-----------------------|------------------|
| otar.action_ttl_secs | 600      | Zeitgrenze für Retry, | Zeitquelle und   |

| Schlüssel               | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                         |          | Lease, Ablauf, Pufferung oder Zustandswechsel.                       | beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.                |
| otar.max_attempts       | 5        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| otar.retry_backoff_secs | 15       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| otar.max_claim_batch    | 100      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

#### 17.12.6 security

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| security.expose_raw_keys         | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.          |

#### 17.12.7 limits

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.



| Schlüssel             | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                      |
|-----------------------|----------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| limits.max_body_bytes | 1048576  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_keys       | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_nodes      | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_jobs       | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_actions    | 500000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |
| limits.max_audit      | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.     |

**Nach der Änderung.** Für Schlüssel-Metadaten, OTAR-Job, Revision den Fachtest ausführen: nur autorisierten Laborschlüssel-Job und Audit verfolgen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; vor Rotation alten Schlüsselszustand und Rückfall dokumentieren.

### 17.13 transit: Felder und Prüfpunkte

Vorlage system-backend/transit/config/transit.example.toml; Ziel /etc/netcore/transit.toml; Unit netcore-transit.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.13.1 server

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb    | Konkrete Prüfung       |
|-------------|--------------|-------------------------|------------------------|
| server.bind | 0.0.0.0:8200 | Lokale Bindeadresse des | Mit ss -ltnp und einer |

| Schlüssel            | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                  |
|----------------------|----------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|                      |          | Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.       | Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen.              |
| server.history_limit | 5000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |

### 17.13.2 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                                | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------|-----------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.database_path | /var/lib/netcore-transit/state.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path   | /var/lib/netcore-transit/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.13.3 region

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel        | Beispiel       | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                           |
|------------------|----------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| region.region_id | region-a       | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen. |
| region.swmi_id   | netcore-swmi-a | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen. |

| Schlüssel                  | Beispiel                                                          | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| region.display_name        | NetCore Region A                                                  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| region.advertised_endpoint | http://10.0.10.12:8200                                            | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| region.protocol_version    | netcore-transit-v1                                                | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| region.operating_mode      | shadow                                                            | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| region.capabilities        | ['mobility', 'individual_call', 'group_call', 'sds', 'media'] ... | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.13.4 routing

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| routing.max_hops              | 8        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| routing.dedupe_ttl_secs       | 900      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.session_idle_ttl_secs | 3600     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den                                                  |

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|-----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                   |          |                                                                                        | Wert prüfen; Grenzfall vor Änderung nachstellen.                                                  |
| routing.route_stale_secs          | 120      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| routing.prefer_direct_region_peer | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| routing.allow_transitive_routing  | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| routing.allow_dynamic_peers       | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| routing.fail_closed_on_loop       | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.13.5 transport

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| transport.connect_timeout_ms | 2000     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| transport.io_timeout_ms      | 5000     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung              |

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                   |          |                                                                      | nachstellen.                                                                                      |
| transport.heartbeat_interval_secs | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| transport.peer_timeout_secs       | 20       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| transport.retry_backoff_secs      | 3        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| transport.max_attempts            | 5        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| transport.max_batch               | 100      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

### 17.13.6 security

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|----------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                    | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |

| Schlüssel           | Beispiel      | Bedeutung im Betrieb                                                                           | Konkrete Prüfung                                                                   |
|---------------------|---------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| security.tls        | false         | Boolesche Steuerung des betreffenden Funktionsblocks.                                          | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen. |
| security.token_auth | [ausgelassen] | Zugangsdaten oder Schlüsselmateriale; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.    |

### 17.13.7 limits

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|-----------------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes       | 4194304  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| limits.max_peers            | 1000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_routes           | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_sessions         | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_envelopes        | 500000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_local_deliveries | 500000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| limits.max_events           | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und                            |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung      |
|-----------|----------|----------------------|-----------------------|
|           |          |                      | nach Freigabe prüfen. |

**Nach der Änderung.** Für Peer, Route, Session, Schleifenschutz den Fachtest ausführen: Peer-Ausfall und Wiederkehr mit Routenrevision testen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Route erst nach Loop-/Session-Abgleich aktivieren.

### 17.14 application-gateway: Felder und Prüfpunkte

Vorlage `system-backend/application-gateway/config/application-gateway.example.toml`; Ziel `/etc/netcore/application-gateway.toml`; Unit `netcore-application-gateway.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.14.1 server

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                           | Beispiel                           | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|-------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>            | <code>0.0.0.0:8220</code>          | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.public_base_url</code> | <code>http://127.0.0.1:8220</code> | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei <code>ws://</code> den WebSocket-Pfad und Ack separat testen.      |
| <code>server.max_body_bytes</code>  | <code>4194304</code>               | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.                   |
| <code>server.history_limit</code>   | <code>5000</code>                  | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

#### 17.14.2 storage

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel                                            | Bedeutung im Betrieb                                                                           | Konkrete Prüfung                                                                         |
|---------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.state_path        | /var/lib/netcore-application-gateway/state.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                      | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.state_backup_path | /var/lib/netcore-application-gateway/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                      | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.secrets_path      | [ausgelassen]                                       | Zugangsdaten oder Schlüsselmateriale; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.          |
| storage.spool_dir         | /var/lib/netcore-application-gateway/spool          | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                      | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_dir        | /var/lib/netcore-application-gateway/backups        | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                      | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

### 17.14.3 security

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                      | Beispiel      | Bedeutung im Betrieb                                                                           | Konkrete Prüfung                                                                            |
|--------------------------------|---------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode                  | open_lab      | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.             | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.management_token_auth | [ausgelassen] | Zugangsdaten oder Schlüsselmateriale; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.             |
| security.management_tls        | false         | Boolesche Steuerung des betreffenden Funktionsblocks.                                          | Effektiven Zustand und einen Test mit aktivierter/deaktivierter                             |



| Schlüssel                          | Beispiel                                                                               | Bedeutung im Betrieb                                                                           | Konkrete Prüfung                                                                            |
|------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|                                    |                                                                                        |                                                                                                | Option abgleichen.                                                                          |
| security.allow_remote_management   | true                                                                                   | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.         | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| security.connector_secrets_allowed | [ausgelassen]                                                                          | Zugangsdaten oder Schlüsselmateriale; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.             |
| security.warning_banнер            | OPEN LAB: no login, no management tokens and no TLS. Isolated management network only. | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                    | Ereignis knapp unter und über der Schwelle einspeisen und Hysteresis/Deduplizierung prüfen. |

#### 17.14.4 runtime

Hier werden 17 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| runtime.operating_mode      | shadow   | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| runtime.worker_interval_ms  | 1000     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.probe_interval_secs | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.default_ttl_secs    | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung              |

| Schlüssel                  | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|----------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                            |          |                                                                      | nachstellen.                                                                                      |
| runtime.max_attempts       | 6        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.base_backoff_secs  | 2        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.max_backoff_secs   | 120      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.dedupe_window_secs | 600      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.max_response_bytes | 65536    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| runtime.max_artifact_bytes | 33554432 | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| runtime.max_events         | 20000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_deliveries     | 50000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_tts_jobs       | 5000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|---------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| runtime.max_audit_records       | 50000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.event_retention_secs    | 604800   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.delivery_retention_secs | 1209600  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.audit_retention_secs    | 2592000  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.14.5 connectors[0]

Hier werden 12 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel   | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------|------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| connectors[0].connector_id | sds-router | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| connectors[0].display_name | SDS Router | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| connectors[0].kind         | sds_router | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel                               | Beispiel                              | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|-----------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| connectors[0].direction                 | outbound                              | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| connectors[0].endpoint                  | http://127.0.0.1:8150/api/v1/messages | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| connectors[0].health_endpoint           | http://127.0.0.1:8150/health/ready    | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| connectors[0].enabled                   | true                                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| connectors[0].timeout_ms                | 5000                                  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| connectors[0].rate_limit_per_minute     | 600                                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| connectors[0].circuit_failure_threshold | 5                                     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.        |
| connectors[0].circuit_open_secs         | 60                                    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| connectors[0].require_secrets           | [ausgelassen]                         | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen.    | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.                   |

**17.14.6 connectors[0].settings**

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| connectors[0].settings.source_iss  | 9999     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| connectors[0].settings.sds_type    | 4        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| connectors[0].settings.protocol_id | 0        | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| connectors[0].settings.priority    | 3        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**17.14.7 rules[0]**

Hier werden 9 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel        | Beispiel                      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------|-------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| rules[0].rule_id | manual-to-sds                 | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| rules[0].name    | Manual messages to SDS Router | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].enabled | true                          | Funktions- oder Policy-                                                                          | Wirksamen Modus nach                                                                          |

| Schlüssel                 | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------------|--------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                           |              | Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.                                  | Neustart ablesen und positive wie negative Probe dokumentieren.                               |
| rules[0].priority         | 100          | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| rules[0].source_connector | manual       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].event_type       | sds.message  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].target_connector | sds-router   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].template_id      | sds-standard | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| rules[0].stop_processing  | false        | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

#### 17.14.8 templates[0]

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel     | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                             |
|--------------------------|--------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| templates[0].template_id | sds-standard | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe |

| Schlüssel                     | Beispiel                                              | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                               |                                                       |                                                                                                  | ausschließen.                                                                                 |
| templates[0].name             | SDS Standard                                          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].kind             | text                                                  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].body             | {{text}}                                              | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].content_type     | text/plain;<br>charset=utf-8                          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].enabled          | true                                                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| templates[0].target_connector | sds-router                                            | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].description      | Plain SDS text with destination supplied by the event | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**Nach der Änderung.** Für Connector, Webhook, Regeln, TTS den Fachtest ausführen: Event mit Correlation ID bis zum Ziel nachverfolgen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Retry/Deduplizierung vor erneutem Webhook trennen.

### 17.15 media-library: Felder und Prüfpunkte

Vorlage system-backend/media-library/config/media-library.example.toml; Ziel /etc/netcore/media-library.toml; Unit netcore-media-library.service. Vor einer Änderung die aktuelle Datei sichern,

nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.15.1 server

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel              | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                            |
|------------------------|-----------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind            | 0.0.0.0:8230          | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.      | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.public_base_url | http://127.0.0.1:8230 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.      |
| server.max_body_bytes  | 100663296             | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                         | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.       |

#### 17.15.2 security

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel      | Bedeutung im Betrieb                                                                          | Konkrete Prüfung                                                                            |
|----------------------|---------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode        | open_lab      | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.            | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| security.token_auth  | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.             |
| security.tls         | false         | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.          |
| security.allow_remot | true          | Funktions- oder Policy-                                                                       | Wirksamen Modus nach                                                                        |



| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|------------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| e_management                       |          | Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.                        | Neustart ablesen und positive wie negative Probe dokumentieren.                      |
| security.allow_delete              | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| security.allow_url_import          | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| security.allow_private_import_urls | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |

### 17.15.3 storage

Hier werden 10 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel           | Beispiel                                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.root        | /var/lib/netcore-media-library/assets     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.state_file  | /var/lib/netcore-media-library/state.json | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.temp_root   | /var/lib/netcore-media-library/tmp        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.backup_root | /var/lib/netcore-media-library/backups    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-         | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und                      |

| Schlüssel                      | Beispiel                         | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                |                                  | Default.                                                                                         | Fachtest nachweisen.                                                                          |
| storage.archive_root           | /mnt/nfs-share/<br>Media-Library | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.recording_archive_root | /mnt/nfs-share/<br>Recordings    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.tts_archive_root       | /mnt/nfs-share/TTS-<br>Dateien   | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.                      | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                  |
| storage.max_asset_bytes        | 67108864                         | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| storage.max_total_bytes        | 21474836480                      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| storage.fsync_imports          | true                             | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

#### 17.15.4 runtime

Hier werden 13 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| runtime.operating_mode     | shadow   | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| runtime.worker_interval_ms | 500      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.               | Zeitquelle und beobachtete Verzögerung gegen den                                            |

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                             |          |                                                                              | Wert prüfen; Grenzfall vor Änderung nachstellen.                                                  |
| runtime.probe_interval_secs | 15       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.         | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.import_timeout_secs | 120      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.         | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.max_assets          | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_jobs            | 2000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_events          | 5000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_audit_records   | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.max_attempts        | 3        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| runtime.frame_interval_ms   | 60       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.         | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| runtime.auto_approve_tts    | false    | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                   |
|---------------------------------|----------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| runtime.auto_archive_recordings | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                        | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen. |
| runtime.auto_archive_tts        | true     | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.       |

#### 17.15.5 playout

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                       | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|---------------------------------|--------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| playout.mode                    | basisstation | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| playout.default_station         | srv-m-tbs-01 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| playout.request_timeout_secs    | 15           | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| playout.completion_timeout_secs | 900          | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| playout.poll_interval_ms        | 500          | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.15.6 `playout.stations[0]`

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                 | Beispiel                           | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                    |
|-------------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>playout.stations[0].id</code>       | <code>srv-m-tbs-01</code>          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.       |
| <code>playout.stations[0].name</code>     | <code>SRV-M-TBS-01</code>          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.       |
| <code>playout.stations[0].base_url</code> | <code>http://10.0.1.22:8080</code> | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei <code>ws://</code> den WebSocket-Pfad und Ack separat testen. |
| <code>playout.stations[0].enabled</code>  | <code>true</code>                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                |

### 17.15.7 `codec`

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel                                                                         | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>codec.frame_bytes</code>     | <code>35</code>                                                                  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| <code>codec.ffmpeg_command</code>  | <code>['/usr/bin/ffmpeg', '-hide_banner', '-loglevel', 'error', '-y'] ...</code> | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| <code>codec.encoder_command</code> | <code>[]</code>                                                                  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein                                | Mit Schema und ausgerollter Datei vergleichen; Wirkung                                        |

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                       |          | zugesicherter Runtime-Default.                                                                   | anhand Status und Fachtest nachweisen.                                                        |
| codec.decoder_command | []       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

### 17.15.8 tts

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel                                     | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                         |
|----------------------------|----------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| tts.enabled                | true                                         | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.     |
| tts.endpoint               | http://127.0.0.1:5005                        | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.            | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |
| tts.template_directory     | /var/lib/netcore-media-library/tts/templates | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.              | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| tts.default_voice          | de-thorsten                                  | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.            | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |
| tts.default_speed          | 0.95                                         | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.            | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |
| tts.max_text_characters    | 2000                                         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                      | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.        |
| tts.synthesis_timeout_secs | 90                                           | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall                  |

| Schlüssel              | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                  |
|------------------------|----------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|                        |          |                                                                   | vor Änderung nachstellen.                                                         |
| tts.max_output_file_mb | 25       | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen. |

#### 17.15.9 tts.voices[0]

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel                    | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                             |
|------------------------------|-----------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| tts.voices[0].id             | de-thorsten                 | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen. |
| tts.voices[0].name           | Deutsch – Thorsten (mittel) | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen. |
| tts.voices[0].provider_voice | de_DE-thorsten-medium       | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen. |

#### 17.15.10 dependencies

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                 | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                       |
|-------------------------------------------|-----------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| dependencies.media_switch_base_url        | http://127.0.0.1:8130 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| dependencies.recorder_base_url            | http://127.0.0.1:8140 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| dependencies.application_gateway_base_url | http://               | Zieladresse einer Gegenstelle; Schema,                                                   | DNS/Route und Ziel-Health prüfen; bei ws://                                            |

| Schlüssel | Beispiel       | Bedeutung im Betrieb                              | Konkrete Prüfung                           |
|-----------|----------------|---------------------------------------------------|--------------------------------------------|
| l         | 127.0.0.1:8220 | Host, Port und Pfad bilden gemeinsam den Vertrag. | den WebSocket-Pfad und Ack separat testen. |

**Nach der Änderung.** Für Import, Preview, Freigabe, Cache, Payout den Fachtest ausführen: Asset importieren, genehmigen, previewen und an TBS abspielen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Format-/Cachefehler vor erneuter Freigabe isolieren.

### 17.16 control-room: Felder und Prüfpunkte

Vorlage `system-backend/control-room/config/control-room.example.toml`; Ziel `/etc/netcore/control-room.toml`; Unit `netcore-control-room.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.16.1 server

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|-----------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>          | <code>0.0.0.0:9010</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.node_path</code>     | <code>/node</code>        | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.                 |
| <code>server.ui_path</code>       | <code>/ui</code>          | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.                 |
| <code>server.history_limit</code> | <code>2000</code>         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |

#### 17.16.2 persistence

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.



| Schlüssel                        | Beispiel                                           | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                         |
|----------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| persistence.enabled              | true                                               | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.     |
| persistence.database_path        | /var/lib/netcore-control-room/control-room.sqlite3 | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.              | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| persistence.persist_events       | true                                               | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.       |
| persistence.persist_noisy_events | false                                              | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.       |
| persistence.load_rec_ent_limit   | 2000                                               | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                      | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.        |

### 17.16.3 auth

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel      | Bedeutung im Betrieb                                                                          | Konkrete Prüfung                                                                     |
|-----------------------------------|---------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| auth.enabled                      | false         | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.        | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| auth.allow_health_unauthenticated | true          | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.        | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| auth.node_token_env               | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.      |

| Schlüssel                   | Beispiel      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------------|---------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| auth.bootstrap_username_env | (leer)        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| auth.bootstrap_password_env | [ausgelassen] | Zugangsdaten oder Schlüsselmateriale; nur den benötigten Dienstberechtigten zugänglich machen.   | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.               |
| auth.bootstrap_role         | admin         | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.16.4 federation

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| federation.enabled            | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| federation.poll_interval_secs | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| federation.request_timeout_ms | 1200     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| federation.failure_threshold  | 3        | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                            | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.        |
| federation.fetch_summary      | true     | Boolesche Steuerung                                                                    | Effektiven Zustand und                                                                            |

| Schlüssel | Beispiel | Bedeutung im Betrieb              | Konkrete Prüfung                                            |
|-----------|----------|-----------------------------------|-------------------------------------------------------------|
| maries    |          | des betreffenden Funktionsblocks. | einen Test mit aktivierter/deaktivierter Option abgleichen. |

#### 17.16.5 operations

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                                          | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|-----------------------------------|---------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| operations.state_path             | /var/lib/netcore-control-room/operations.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| operations.backup_path            | /var/lib/netcore-control-room/operations.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| operations.auto_service_incidents | true                                              | Boolesche Steuerung des betreffenden Funktionsblocks.                     | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.       |
| operations.incident_limit         | 5000                                              | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.         | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.        |
| operations.shift_log_limit        | 10000                                             | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.         | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.        |

#### 17.16.6 services[0]

Hier werden 10 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel        | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------|--------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| services[0].name | node-gateway | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel                | Beispiel               | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------------|------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| services[0].display_name | Node Gateway           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| services[0].kind         | edge                   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| services[0].base_url     | http://10.0.20.10:8080 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| services[0].health_live  | /health/live           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| services[0].health_ready | /health/ready          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| services[0].summary_path | /api/v1/status         | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| services[0].webui_path   | /                      | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| services[0].critical     | true                   | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.    |
| services[0].enabled      | true                   | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |

**17.16.7 directory**

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                  | Konkrete Prüfung                                                                   |
|-------------------------------|----------|-------------------------------------------------------|------------------------------------------------------------------------------------|
| directory.hide_infrastructure | true     | Boolesche Steuerung des betreffenden Funktionsblocks. | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen. |

**17.16.8 service\_monitor.targets[0]**

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                    | Beispiel                                         | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| service_monitor.targets[0].name              | hardware-gateway                                 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| service_monitor.targets[0].url               | http://10.0.20.28:8250/health/ready              | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| service_monitor.targets[0].critical_for_edge | false                                            | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.    |
| service_monitor.targets[0].fallback_mode     | rack_telemetry_unavailable_local_radio_continues | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| service_monitor.targets[0].depends_on        | ['iot-gateway']                                  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**Nach der Änderung.** Für Operatorrolle, Lage, Incident, Schichtbuch den Fachtest ausführen: Fachstatus und eine protokollierte Operatoraktion prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; UI-Cache von tatsächlichem Autoritätsdienst unterscheiden.

## 17.17 observability: Felder und Prüfpunkte

Vorlage system-backend/observability/config/observability.example.toml; Ziel /etc/netcore/observability.toml; Unit netcore-observability.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.17.1 server

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|-----------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind           | 0.0.0.0:8210 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| server.history_limit  | 5000         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                   | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.           |
| server.max_body_bytes | 4194304      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                    | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.      |

### 17.17.2 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel                                      | Bedeutung im Betrieb                                                      | Konkrete Prüfung                                                                         |
|----------------------|-----------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| storage.state_path   | /var/lib/netcore-observability/state.json     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.backup_path  | /var/lib/netcore-observability/state.json.bak | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv. | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| storage.diagnostic_d | /var/lib/netcore-                             | Datei- oder                                                               | Existenz, Eigentümer,                                                                    |

| Schlüssel | Beispiel                      | Bedeutung im Betrieb                                                | Konkrete Prüfung                                                            |
|-----------|-------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| ir        | observability/<br>diagnostics | Verzeichnisort für<br>Zustand, Konfiguration,<br>Cache oder Archiv. | Schreibrecht und<br>Backup-/Restore-Grenze<br>am Zielhost<br>kontrollieren. |

### 17.17.3 security

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel                                                                                | Bedeutung im Betrieb                                                                                      | Konkrete Prüfung                                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| security.mode                    | open_lab                                                                                | Betriebsmodus oder<br>Autoritätsentscheidung;<br>bestimmt den aktiven<br>Verarbeitungspfad.               | Zulässige Varianten im<br>zugehörigen Schema<br>prüfen und einen<br>Ausfall-/Rückfalltest<br>ausführen. |
| security.token_auth              | [ausgelassen]                                                                           | Zugangsdaten oder<br>Schlüsselmaterial; nur<br>den benötigten<br>Dienstberechtigten<br>zugänglich machen. | Dateirechte und<br>Rotation prüfen; Wert<br>weder loggen noch in<br>ein Ticket kopieren.                |
| security.tls                     | false                                                                                   | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                               | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.             |
| security.allow_remote_management | true                                                                                    | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern.       | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren.           |
| security.warning_ban<br>ner      | OPEN LAB: no login,<br>no tokens and no<br>TLS. Isolated<br>management network<br>only. | Schwellwert für<br>Entscheidung, Alarm<br>oder<br>Schutzmechanismus.                                      | Ereignis knapp unter<br>und über der Schwelle<br>einspeisen und<br>Hysteresis/Deduplizierung<br>prüfen. |

### 17.17.4 collection

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                           | Beispiel | Bedeutung im Betrieb                              | Konkrete Prüfung              |
|-------------------------------------|----------|---------------------------------------------------|-------------------------------|
| collection.scrape_in<br>terval_secs | 15       | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung | Zeitquelle und<br>beobachtete |

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                               |          | oder Zustandswechsel.                                                | Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.                            |
| collection.request_timeout_ms | 2000     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| collection.max_response_bytes | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.             |
| collection.scrape_on_start    | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| collection.ingest_logs        | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| collection.ingest_traces      | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.17.5 retention

Hier werden 10 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|---------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| retention.metric_retention_secs | 86400    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| retention.log_retention_secs    | 604800   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung              |



| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|----------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                  |          |                                                                      | nachstellen.                                                                                      |
| retention.trace_retention_secs   | 86400    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| retention.audit_retention_secs   | 2592000  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| retention.max_series             | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| retention.max_samples_per_series | 5760     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| retention.max_logs               | 100000   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| retention.max_spans              | 50000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| retention.max_alerts             | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| retention.max_audit_records      | 50000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

#### 17.17.6 stack

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                         |
|-------------------------------|-----------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| stack.prometheus_url          | http://127.0.0.1:9090 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.   |
| stack.grafana_url             | http://127.0.0.1:3000 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.   |
| stack.loki_url                | http://127.0.0.1:3100 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.   |
| stack.alertmanager_url        | http://127.0.0.1:9093 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.   |
| stack.prometheus_ready_path   | /-/ready              | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| stack.grafana_ready_path      | /api/health           | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| stack.loki_ready_path         | /ready                | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| stack.alertmanager_ready_path | /-/ready              | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |

#### 17.17.7 targets[0]

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel               | Beispiel              | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------------|-----------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| targets[0].target_id    | node-gateway          | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| targets[0].display_name | Node Gateway          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| targets[0].service      | node-gateway          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| targets[0].base_url     | http://127.0.0.1:8080 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| targets[0].metrics_path | /metrics              | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| targets[0].live_path    | /health/live          | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| targets[0].ready_path   | /health/ready         | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| targets[0].enabled      | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |

#### 17.17.8 targets[0].labels

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------------------|--------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| targets[0].labels.environment | open-lab     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| targets[0].labels.component   | node-gateway | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.17.9 alert\_rules[0]

Hier werden 9 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel                                                 | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| alert_rules[0].rule_id     | target-down                                              | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| alert_rules[0].name        | Target down                                              | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| alert_rules[0].description | A monitored target does not answer its liveness endpoint | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| alert_rules[0].metric      | netcore_observability_target_up                          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| alert_rules[0].comparator  | <                                                        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| alert_rules[0].threshold   | 1.0                                                      | Schwellwert für Entscheidung, Alarm                                                              | Ereignis knapp unter und über der Schwelle                                                    |

| Schlüssel               | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|-------------------------|----------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                         |          | oder Schutzmechanismus.                                                                          | einspeisen und Hysterese/Deduplizierung prüfen.                                                   |
| alert_rules[0].for_secs | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| alert_rules[0].severity | critical | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| alert_rules[0].enabled  | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |

#### 17.17.10 alert\_rules[0].labels

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel              | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| alert_rules[0].labels.source | netcore-observability | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.17.11 alert\_rules[0].annotations

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel                                             | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| alert_rules[0].annotations.runbook | Check service process, network path and /health/live | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**Nach der Änderung.** Für Scrape, Log, Trace, Alert, Silence den Fachtest ausführen: ein markiertes Event bis Alert/Trace und Retention verfolgen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Scrape-Lücke nicht als Gesundheitszustand interpretieren.

### 17.18 iot-gateway: Felder und Prüfpunkte

Vorlage `system-backend/iot-gateway/config/iot-gateway.example.toml`; Ziel `/etc/netcore/iot-gateway.toml`; Unit `netcore-iot-gateway.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.18.1 server

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code>           | <code>0.0.0.0:8240</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| <code>server.history_limit</code>  | <code>1000</code>         | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                        |
| <code>server.max_body_bytes</code> | <code>1048576</code>      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.                    |

#### 17.18.2 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                     | Beispiel              | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|-----------------------------------------------|-----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>security.mode</code>                    | <code>open_lab</code> | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |
| <code>security.allow_remote_management</code> | <code>true</code>     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine                     | Wirksamen Modus nach Neustart ablesen und positive wie negative                             |

| Schlüssel | Beispiel | Bedeutung im Betrieb    | Konkrete Prüfung     |
|-----------|----------|-------------------------|----------------------|
|           |          | Schutzbedingung ändern. | Probe dokumentieren. |

### 17.18.3 mqtt

Hier werden 13 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel            | Beispiel            | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                                         |
|----------------------|---------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| mqtt.host            | 127.0.0.1           | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| mqtt.port            | 1883                | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.client_id       | netcore-iot-gateway | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.               |
| mqtt.topic_prefix    | netcore/v1          | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                          | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.keep_alive_secs | 30                  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                        | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |
| mqtt.clean_session   | false               | Boolesche Steuerung des betreffenden Funktionsblocks.                                       | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                       |
| mqtt.reconnect_secs  | 3                   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                        | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung                     |

| Schlüssel                 | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                                                  |
|---------------------------|----------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                           |          |                                                                                         | nachstellen.                                                                                      |
| mqtt.publish_timeout_secs | 8        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                    | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| mqtt.qos                  | 1        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| mqtt.event_retain         | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| mqtt.state_retain         | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| mqtt.observe_commands     | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| mqtt.execute_commands     | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

#### 17.18.4 home\_assistant

Hier werden 14 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                       | Beispiel      | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|---------------------------------|---------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| home_assistant.enabled          | true          | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| home_assistant.discover_enabled | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| home_assistant.discover         | homeassistant | Konfigurationswert des                                                                 | Mit Schema und                                                                       |



| Schlüssel                                 | Beispiel             | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                           |
|-------------------------------------------|----------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| very_prefix                               |                      | betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default.              | ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen.                    |
| home_assistant.statu<br>s_topic           | homeassistant/status | MQTT-Verbindung oder<br>Topic-Zuordnung für<br>Ereignisse und<br>Kommandos.                      | Publish/Subscribe und<br>retained State mit<br>Testkennung prüfen;<br>Loop- und Ack-Pfad<br>beobachten.    |
| home_assistant.node_<br>id                | netcore_tetra        | Identität oder<br>Zuordnung; muss über<br>Geräte, TBS und<br>Backend konsistent<br>sein.         | Eindeutigkeit und<br>berechtigte<br>Testkennung prüfen;<br>Fehlzuordnung vor<br>Funkprobe<br>ausschließen. |
| home_assistant.disco<br>very_qos          | 1                    | Numerischer Parameter<br>des betreffenden Blocks;<br>Einheit aus Namen und<br>Quelltyp ableiten. | Grenze im Runtime-<br>Schema und<br>Mess-/Zählerwirkung bei<br>Testlast prüfen.                            |
| home_assistant.disco<br>very_retain       | true                 | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                      | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                |
| home_assistant.expos<br>e_gateway         | true                 | Adressierung oder Next<br>Hop; muss zum<br>gerouteten IP- und LXC-<br>Netz passen.               | Mit ip route,<br>Poolgrenzen und<br>Gegenstellen<br>vergleichen;<br>Doppelbelegung<br>ausschließen.        |
| home_assistant.expos<br>e_sources         | true                 | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                      | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                |
| home_assistant.expos<br>e_virtual_devices | true                 | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                      | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                |
| home_assistant.accep<br>t_state_ingress   | true                 | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                      | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                |
| home_assistant.state<br>_ingress_topic    | (leer)               | MQTT-Verbindung oder<br>Topic-Zuordnung für<br>Ereignisse und<br>Kommandos.                      | Publish/Subscribe und<br>retained State mit<br>Testkennung prüfen;<br>Loop- und Ack-Pfad                   |

| Schlüssel                           | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                            |
|-------------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|                                     |          |                                                                                        | beobachten.                                                                                 |
| home_assistant.allow_command_egress | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.        |
| home_assistant.command_egress_topic | (leer)   | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                     | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten. |

#### 17.18.5 homematic

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel            | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                                         |
|----------------------------|---------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| homematic.enabled          | false               | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.      | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                     |
| homematic.mode             | home_assistant_mqtt | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.          | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.              |
| homematic.ccu_host         | 127.0.0.1           | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| homematic.ccu_port         | 2010                | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| homematic.poll_interval_ms | 2000                | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                        | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |

| Schlüssel                    | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| homematic.request_timeout_ms | 2500     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| homematic.allow_writes       | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |

#### 17.18.6 commands

Hier werden 10 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel         | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                                  |
|---------------------------|------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| commands.enabled          | true             | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| commands.mode             | open_lab_sandbox | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.     | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.       |
| commands.default_deny     | true             | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| commands.allow_retained   | false            | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| commands.default_ttl_secs | 30               | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| commands.max_ttl_secs     | 300              | Zeitgrenze für Retry, Lease, Ablauf, Pufferung                                         | Zeitquelle und beobachtete                                                                        |

| Schlüssel                       | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                                                  |
|---------------------------------|----------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                 |          | oder Zustandswechsel.                                                                   | Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.                            |
| commands.max_future_skew_secs   | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                    | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| commands.publish_lifecycle_acks | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| commands.ack_qos                | 1        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| commands.ack_retain             | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.18.7 storage

Hier werden 12 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_dir  | /var/lib/netcore-iot-gateway | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| storage.outbox_dir | outbox                       | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| storage.dedup_file | dedup.json                   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel                    | Beispiel                       | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.command_inbox_file   | command-inbox.ndjson           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.command_ledger_file  | command-ledger.json            | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.command_audit_file   | command-audit.ndjson           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.virtual_state_file   | virtual-device-state.json      | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.external_state_file  | external-entity-state.json     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.homematic_state_file | homematic-datapoint-state.json | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.dedup_limit          | 50000                          | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.             |
| storage.outbox_limit         | 20000                          | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.             |
| storage.command_ledger_limit | 50000                          | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.             |

**17.18.8 polling**

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| polling.interval_ms         | 2000     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| polling.batch_limit         | 500      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| polling.request_time_out_ms | 2500     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

**17.18.9 command\_policies[0]**

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------------------|------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| command_policies[0].id            | allow-openlab-virtual-relays | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| command_policies[0].enabled       | true                         | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| command_policies[0].effect        | allow                        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| command_policies[0].command_types | ['virtual.relay.set']        | Konfigurationswert des betreffenden Blocks;                                                      | Mit Schema und ausgerollter Datei                                                             |

| Schlüssel                           | Beispiel          | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|-------------------------------------|-------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                     |                   | Beispielwert ist kein zugesicherter Runtime-Default.                                             | vergleichen; Wirkung anhand Status und Fachtest nachweisen.                                       |
| command_policies[0].target_types    | ['virtual_relay'] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| command_policies[0].target_prefixes | ['lab-']          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| command_policies[0].max_ttl_secs    | 120               | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| command_policies[0].allow_dry_run   | true              | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |

#### 17.18.10 sources[0]

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                       | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| sources[0].id      | node-gateway                                   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| sources[0].url     | http://node-gateway:8080/api/v1/events/netcore | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag.         | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.        |
| sources[0].enabled | true                                           | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung                   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung |
|-----------|----------|----------------------|------------------|
|           |          | ändern.              |                  |

**Nach der Änderung.** Für MQTT Bridge, HA Discovery, Command/Ack den Fachtest ausführen: Registrierung, retained State und Test-Command komplett prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Topic-Loop und verwaisten retained State bereinigen.

### 17.19 hardware-gateway: Felder und Prüfpunkte

Vorlage `system-backend/hardware-gateway/config/hardware-gateway.example.toml`; Ziel `/etc/netcore/hardware-gateway.toml`; Unit `netcore-hardware-gateway.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.19.1 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|--------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code> | <code>0.0.0.0:8250</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

#### 17.19.2 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel              | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------------|-----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>security.mode</code> | <code>open_lab</code> | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

#### 17.19.3 mqtt

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.



| Schlüssel         | Beispiel                 | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                                         |
|-------------------|--------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| mqtt.host         | 127.0.0.1                | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| mqtt.port         | 1883                     | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.topic_prefix | netcore/v1               | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                          | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.client_id    | netcore-hardware-gateway | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.               |

#### 17.19.4 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                        | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-hardware-gateway/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-hardware-gateway/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.19.5 monitoring

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| monitoring.heartbeat_timeout_secs | 30       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| monitoring.stale_after_secs       | 20       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| monitoring.outputs_enabled        | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

#### 17.19.6 thresholds[0]

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                    | Beispiel      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|------------------------------|---------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| thresholds[0].metric         | temperature_c | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| thresholds[0].warning_above  | 40.0          | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.    |
| thresholds[0].critical_above | 55.0          | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus.                                      | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.    |

**Nach der Änderung.** Für Edge-Telemetrie, Schwellwert, Rackzustand den Fachtest ausführen: Heartbeat und kontrollierten Grenzwert auslösen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Sensorfehler von Funk-/Core-Ausfall getrennt melden.

## 17.20 rf-monitor: Felder und Prüfpunkte

Vorlage `system-backend/rf-monitor/config/rf-monitor.example.toml`; Ziel `/etc/netcore/rf-monitor.toml`; Unit `netcore-rf-monitor.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.20.1 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|--------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code> | <code>0.0.0.0:8260</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.20.2 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel              | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|----------------------------|-----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>security.mode</code> | <code>open_lab</code> | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

### 17.20.3 mqtt

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel               | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|---------------------------|------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>mqtt.enabled</code> | <code>true</code>      | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| <code>mqtt.host</code>    | <code>127.0.0.1</code> | Hostname oder IP-Adresse der Gegenstelle; <code>127.0.0.1</code> bezeichnet immer den  | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host  |

| Schlüssel         | Beispiel           | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                                         |
|-------------------|--------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
|                   |                    | eigenen Namespace.                                                           | ableiten.                                                                                                |
| mqtt.port         | 1883               | Transportport oder Bereich für Listener, Medien oder Paketpfad.              | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.topic_prefix | netcore/v1         | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.           | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.client_id    | netcore-rf-monitor | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzugehörigkeit vor Funkprobe ausschließen.           |

#### 17.20.4 storage

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-rf-monitor/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-rf-monitor/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.20.5 monitoring

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                 |
|-----------------------------------|----------|----------------------------------------------------------------------|--------------------------------------------------|
| monitoring.heartbeat_timeout_secs | 20       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den |

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                              | Konkrete Prüfung                                                                       |
|-------------------------------|----------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|                               |          |                                                                   | Wert prüfen; Grenzfall vor Änderung nachstellen.                                       |
| monitoring.event_memory_limit | 1000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| monitoring.max_spectrum_bins  | 512      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung. | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| monitoring.max_payload_bytes  | 524288   | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.  | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |

#### 17.20.6 thresholds

Hier werden 20 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                   | Beispiel | Bedeutung im Betrieb                                        | Konkrete Prüfung                                                                           |
|---------------------------------------------|----------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| thresholds.vswr_warning                     | 1.8      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.vswr_critical                    | 2.5      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.reflected_ratio_warning_percent  | 10.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.reflected_ratio_critical_percent | 20.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.pa_temp_w                        | 70.0     | Schwellwert für                                             | Ereignis knapp unter                                                                       |

| Schlüssel                          | Beispiel | Bedeutung im Betrieb                                        | Konkrete Prüfung                                                                           |
|------------------------------------|----------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| arning_c                           |          | Entscheidung, Alarm oder Schutzmechanismus.                 | und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.                      |
| thresholds.pa_temp_critical_c      | 85.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.sdr_temp_warning_c      | 65.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.sdr_temp_critical_c     | 80.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.cabinet_temp_warning_c  | 45.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.cabinet_temp_critical_c | 60.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.evm_warning_pct         | 6.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.evm_critical_pct        | 10.0     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.papr_warning_db         | 7.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.papr_crit               | 9.0      | Schwellwert für                                             | Ereignis knapp unter                                                                       |

| Schlüssel                                 | Beispiel | Bedeutung im Betrieb                                        | Konkrete Prüfung                                                                           |
|-------------------------------------------|----------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| ical_db                                   |          | Entscheidung, Alarm oder Schutzmechanismus.                 | und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen.                      |
| thresholds.forward_power_warning_below_w  | 0.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.forward_power_critical_below_w | 0.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.pa_voltage_warning_below_v     | 0.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.pa_voltage_critical_below_v    | 0.0      | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.fan_warning_below_rpm          | 0        | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |
| thresholds.fan_critical_below_rpm         | 0        | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |

**Nach der Änderung.** Für DSP-Werte, RF-Agent, Kalibrierung den Fachtest ausführen: Messquelle und Alarm bei definierter Teständerung prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; DSP-Schätzwert nicht als kalibrierte Vor-/Rücklaufmessung ausgeben.

### 17.21 alarm-workflow: Felder und Prüfpunkte

Vorlage system-backend/alarm-workflow/config/alarm-workflow.example.toml; Ziel /etc/netcore/alarm-workflow.toml; Unit netcore-alarm-workflow.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.21.1 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|-------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind | 0.0.0.0:8270 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.21.2 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|---------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

### 17.21.3 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-alarm-workflow/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-alarm-workflow/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.audit_log  | /var/lib/netcore-alarm-workflow/audit.ndjson  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |



**17.21.4 mqtt**

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel                  | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                                         |
|-----------------------|---------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| mqtt.enabled          | true                      | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.      | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                     |
| mqtt.host             | 10.0.20.27                | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| mqtt.port             | 1883                      | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.qos              | 1                         | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.     | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                                    |
| mqtt.topic_prefix     | netcore/v1                | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                          | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.client_id        | netcore-alarm-workflow    | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.               |
| mqtt.reconnect_secs   | 3                         | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                        | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |
| mqtt.subscribe_topics | [ 'netcore/v1/events/#' ] | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                          | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |

### 17.21.5 sds\_router

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel               | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|------------------------------------|------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| sds_router.base_url                | http://10.0.20.17:8150 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| sds_router.timeout_secs            | 3                      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| sds_router.poll_interval_secs      | 2                      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| sds_router.process_existing_events | false                  | Boolesche Steuerung des betreffenden Funktionsblocks.                                    | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| sds_router.source_is_si            | 9999                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| sds_router.protocol_id             | 130                    | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.             | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.        |
| sds_router.default_ttl_secs        | 300                    | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

**17.21.6 workflow**

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                              | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|----------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| workflow.scheduler_interval_secs       | 2        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| workflow.event_history_limit           | 2000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| workflow.seen_event_limit              | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| workflow.stop_escalation_on_ack        | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| workflow.stop_escalation_on_assignment | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| workflow.auto_close_on_clear           | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

**17.21.7 limits**

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                             | Konkrete Prüfung                                                                       |
|-----------------------|----------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer. | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |

**17.21.8 recipients[0]**

Hier werden 11 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel       | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------------|----------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| recipients[0].id          | technik-gruppe | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| recipients[0].name        | Technikgruppe  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| recipients[0].enabled     | true           | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| recipients[0].kind        | group_sds      | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| recipients[0].destination | 15201          | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| recipients[0].source_issi | 9999           | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| recipients[0].protocol_id | 130            | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.    |
| recipients[0].priority    | 7              | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| recipients[0].ttl_secs    | 300            | Zeitgrenze für Retry, Lease, Ablauf, Pufferung                                                   | Zeitquelle und beobachtete                                                                    |

| Schlüssel                      | Beispiel                                      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                |                                               | oder Zustandswechsel.                                                                            | Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.                        |
| recipients[0].max_text_chars   | 180                                           | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.             |
| recipients[0].message_template | ALARM {severity} {token} {title}. ACK {token} | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.21.9 escalation\_profiles[0]

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                   | Beispiel                      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------------|-------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| escalation_profiles[0].id   | technical-default             | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| escalation_profiles[0].name | Technische Standardescalation | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.21.10 escalation\_profiles[0].steps[0]

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                                  | Beispiel           | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|--------------------------------------------|--------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| escalation_profiles[0].steps[0].after_secs | 0                  | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| escalation_profiles[0].steps[0].group      | ['technik-gruppe'] | Konfigurationswert des                                               | Mit Schema und                                                                                    |

| Schlüssel                           | Beispiel | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                        |
|-------------------------------------|----------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>0].steps[0].recipients</code> |          | betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |

#### 17.21.11 rules[0]

Hier werden 13 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel                                                                | Bedeutung im Betrieb                                                                                          | Konkrete Prüfung                                                                                          |
|-----------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>rules[0].id</code>          | <code>rf-alarm-raised</code>                                            | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |
| <code>rules[0].enabled</code>     | <code>true</code>                                                       | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern.           | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren.             |
| <code>rules[0].action</code>      | <code>raise</code>                                                      | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |
| <code>rules[0].event_type</code>  | <code>rf.alarm_raised</code>                                            | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |
| <code>rules[0].alarm_type</code>  | <code>rf_fault</code>                                                   | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |
| <code>rules[0].title</code>       | <code>RF {subject_id}:<br/>{payload_alarm_key}</code>                   | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen. |
| <code>rules[0].description</code> | <code>HF-Alarm<br/>{payload_alarm_key};<br/>Wert {payload_value}</code> | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-             | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und                         |

| Schlüssel                   | Beispiel                            | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                             |                                     | Default.                                                                                         | Fachtest nachweisen.                                                                          |
| rules[0].severity           | inherit                             | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].priority           | 8                                   | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| rules[0].requires_ack       | true                                | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| rules[0].recipients         | ['technik-gruppe']                  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].escalation_profile | technical-default                   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| rules[0].dedup_fields       | ['subject.id', 'payload.alarm_key'] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.21.12 status\_actions[0]

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|-------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| status_actions[0].enabled     | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| status_actions[0].status_code | 5201     | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und                   | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei                                 |

| Schlüssel                           | Beispiel   | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------------------------|------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                     |            | Quellentyp ableiten.                                                                             | Testlast prüfen.                                                                              |
| status_actions[0].action            | ack        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| status_actions[0].states            | [ 'open' ] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| status_actions[0].requires_ack_only | true       | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

**Nach der Änderung.** Für Deduplizierung, Eskalation, Ack den Fachtest ausführen: dasselbe Ereignis zweimal und Ack einmal senden. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Eskalationsstufe mit Zustellstatus und Zuständigkeit prüfen.

## 17.22 task-workflow: Felder und Prüfpunkte

Vorlage `system-backend/task-workflow/config/task-workflow.example.toml`; Ziel `/etc/netcore/task-workflow.toml`; Unit `netcore-task-workflow.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.22.1 service

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel              | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------|-----------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| service.name  | netcore-task-workflow | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| service.phase | 9                     | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| service.mode  | open_lab              | Betriebsmodus oder Autoritätsentscheidung;                                                       | Zulässige Varianten im zugehörigen Schema                                                     |



| Schlüssel | Beispiel | Bedeutung im Betrieb                    | Konkrete Prüfung                                  |
|-----------|----------|-----------------------------------------|---------------------------------------------------|
|           |          | bestimmt den aktiven Verarbeitungspfad. | prüfen und einen Ausfall-/Rückfalltest ausführen. |

#### 17.22.2 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|-------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind | 0.0.0.0:8280 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

#### 17.22.3 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|---------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

#### 17.22.4 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-task-workflow/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-task-workflow/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel         | Beispiel                                    | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.audit_log | /var/lib/netcore-task-workflow/audit.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

### 17.22.5 mqtt

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel         | Beispiel              | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                                         |
|-------------------|-----------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| mqtt.enabled      | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.      | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                     |
| mqtt.host         | 127.0.0.1             | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| mqtt.port         | 1883                  | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.topic_prefix | netcore/v1            | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.                          | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.client_id    | netcore-task-workflow | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzusammenhang vor Funkprobe ausschließen.            |

### 17.22.6 sds\_router

Hier werden 8 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                      | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|--------------------------------|-----------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| sds_router.enabled             | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| sds_router.base_url            | http://127.0.0.1:8150 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| sds_router.source_is_si        | 9999                  | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| sds_router.protocol_id         | 130                   | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.             | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.        |
| sds_router.ttl_secs            | 600                   | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| sds_router.max_text_length     | 160                   | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                        | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| sds_router.default_destination | 15201                 | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| sds_router.default_is_group    | true                  | Boolesche Steuerung des betreffenden Funktionsblocks.                                    | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

### 17.22.7 workflow

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                           | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| workflow.event_history_limit        | 2000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| workflow.seen_event_limit           | 10000    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| workflow.expire_check_interval_secs | 5        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| workflow.notify_on_state_change     | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |

#### 17.22.8 wap

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel       | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| wap.enabled     | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| wap.page_size   | 6        | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.        |
| wap.xhtml_entry | /x       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| wap.wml_entry   | /w       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**17.22.9 templates[0]**

Hier werden 6 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel                                    | Bedeutung im Betrieb                                                                                | Konkrete Prüfung                                                                              |
|-------------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| templates[0].id               | technical_fault                             | Konfigurationswert des betreffenden Blocks;<br>Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].name             | Technische Störung                          | Konfigurationswert des betreffenden Blocks;<br>Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].description      | Technische Störung aufnehmen und bearbeiten | Konfigurationswert des betreffenden Blocks;<br>Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].default_priority | 7                                           | Numerischer Parameter des betreffenden Blocks;<br>Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| templates[0].default_severity | warning                                     | Konfigurationswert des betreffenden Blocks;<br>Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].requires_ack     | true                                        | Boolesche Steuerung des betreffenden Funktionsblocks.                                               | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

**17.22.10 templates[0].fields[0]**

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                 | Beispiel | Bedeutung im Betrieb                                                                                | Konkrete Prüfung                                                                              |
|---------------------------|----------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| templates[0].fields[0].id | asset    | Konfigurationswert des betreffenden Blocks;<br>Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel                       | Beispiel      | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------------------|---------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| templates[0].fields[0].label    | Anlage/Geraet | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].fields[0].type     | text          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| templates[0].fields[0].required | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |

#### 17.22.11 status\_actions[0]

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                     | Beispiel         | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-------------------------------|------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| status_actions[0].status_code | 5301             | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| status_actions[0].action      | accept           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| status_actions[0].label       | Auftrag annehmen | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

**Nach der Änderung.** Für Taskstatus, WAP, SDS, MQTT den Fachtest ausführen: Task anlegen, annehmen, abschließen und doppelte Antwort senden. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; blocked/reopen im Audit mit derselben Task-ID verfolgen.

#### 17.23 asset-management: Felder und Prüfpunkte

Vorlage system-backend/asset-management/config/asset-management.example.toml; Ziel /etc/netcore/asset-management.toml; Unit netcore-asset-management.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie

den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

#### 17.23.1 service

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel                 | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------|--------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| service.name  | netcore-asset-management | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| service.phase | 10                       | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| service.mode  | open_lab                 | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |

#### 17.23.2 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                            |
|-------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| server.bind | 0.0.0.0:8290 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

#### 17.23.3 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                 |
|---------------|----------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| security.mode | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest |

| Schlüssel | Beispiel | Bedeutung im Betrieb | Konkrete Prüfung |
|-----------|----------|----------------------|------------------|
|           |          |                      | ausführen.       |

#### 17.23.4 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                        | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-asset-management/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-asset-management/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.audit_log  | /var/lib/netcore-asset-management/audit.ndjson  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.23.5 mqtt

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel    | Beispiel  | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                              |
|--------------|-----------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| mqtt.enabled | true      | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.      | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| mqtt.host    | 127.0.0.1 | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten. |
| mqtt.port    | 1883      | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten          |



| Schlüssel         | Beispiel                 | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                              |
|-------------------|--------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                   |                          |                                                                              | UDP-Bereich testen.                                                                           |
| mqtt.topic_prefix | netcore/v1               | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.           | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.   |
| mqtt.client_id    | netcore-asset-management | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzusammenhang vor Funkprobe ausschließen. |

### 17.23.6 management

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                              | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|----------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| management.event_history_limit         | 3000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| management.upstream_sync_interval_secs | 60       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.23.7 upstreams.subscriber\_core

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                          | Beispiel              | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|------------------------------------|-----------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| upstreams.subscriber_core.enabled  | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| upstreams.subscriber_core.base_url | http://127.0.0.1:8100 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad                             | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad                       |

| Schlüssel | Beispiel | Bedeutung im Betrieb          | Konkrete Prüfung        |
|-----------|----------|-------------------------------|-------------------------|
|           |          | bilden gemeinsam den Vertrag. | und Ack separat testen. |

#### 17.23.8 upstreams.mobility\_core

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                        | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                       |
|----------------------------------|-----------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| upstreams.mobility_core.enabled  | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.   |
| upstreams.mobility_core.base_url | http://127.0.0.1:8090 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |

#### 17.23.9 upstreams.task\_workflow

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                            | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                       |
|--------------------------------------|-----------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| upstreams.task_workflow.enabled      | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.   |
| upstreams.task_workflow.base_url     | http://127.0.0.1:8280 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| upstreams.task_workflow.default_gssi | 15201                 | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.  | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                  |

**Nach der Änderung.** Für Asset, Gerätebindung, Firmware, Wartung den Fachtest ausführen: Asset mit Test-ISSI verbinden und Wartungsfall öffnen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; physische Zuordnung nicht mit Teilnehmerzulassung verwechseln.

## 17.24 sip-switch: Felder und Prüfpunkte

Vorlage `system-backend/sip-switch/config/sip-switch.example.toml`; Ziel `/etc/netcore/sip-switch.toml`; Unit `netcore-sip-switch.service`. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und `ready` sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.24.1 service

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel                        | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------------------|---------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>service.name</code>  | <code>netcore-sip-switch</code> | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| <code>service.phase</code> | <code>11</code>                 | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| <code>service.mode</code>  | <code>open_lab</code>           | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |

### 17.24.2 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                         |
|--------------------------|---------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>server.bind</code> | <code>0.0.0.0:8300</code> | Lokale Bindeadresse des Listeners; <code>0.0.0.0</code> erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.24.3 security

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|---------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

#### 17.24.4 storage

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel          | Beispiel                                  | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| storage.state_file | /var/lib/netcore-sip-switch/state.json    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.event_log  | /var/lib/netcore-sip-switch/events.ndjson | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| storage.audit_log  | /var/lib/netcore-sip-switch/audit.ndjson  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.24.5 mqtt

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel    | Beispiel  | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                              |
|--------------|-----------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| mqtt.enabled | true      | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.      | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| mqtt.host    | 127.0.0.1 | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten. |

| Schlüssel         | Beispiel           | Bedeutung im Betrieb                                                         | Konkrete Prüfung                                                                                         |
|-------------------|--------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| mqtt.port         | 1883               | Transportport oder Bereich für Listener, Medien oder Paketpfad.              | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| mqtt.topic_prefix | netcore/v1         | MQTT-Verbindung oder Topic-Zuordnung für Ereignisse und Kommandos.           | Publish/Subscribe und retained State mit Testkennung prüfen; Loop- und Ack-Pfad beobachten.              |
| mqtt.client_id    | netcore-sip-switch | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.               |

#### 17.24.6 mobility\_core

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                  | Beispiel              | Bedeutung im Betrieb                                                                     | Konkrete Prüfung                                                                                  |
|----------------------------|-----------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| mobility_core.enabled      | true                  | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| mobility_core.base_url     | http://127.0.0.1:8090 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen.            |
| mobility_core.timeout_secs | 2                     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.24.7 asterisk

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel           | Beispiel                                       | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|---------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| asterisk.enabled    | true                                           | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| asterisk.binary     | /usr/sbin/asterisk                             | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| asterisk.config_dir | /etc/asterisk                                  | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.      |
| asterisk.agi_script | /var/lib/asterisk/agi-bin/netcore-sip-route.py | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| asterisk.sip_bind   | 0.0.0.0:5060                                   | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.              | Mit ss -ltnp und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen.   |
| asterisk.rtp_start  | 10000                                          | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| asterisk.rtp_end    | 20000                                          | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |

#### 17.24.8 management

Hier werden 5 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel | Bedeutung im Betrieb                                                                    | Konkrete Prüfung                                                      |
|--------------------------|----------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| management.route_workers | 8        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen. |

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|-----------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| management.side_effect_queue_size | 512      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.     | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.             |
| management.call_history_limit     | 2000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| management.event_history_limit    | 3000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| management.probe_interval_secs    | 10       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.24.9 pbx

Hier werden 15 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel       | Beispiel     | Bedeutung im Betrieb                                                                        | Konkrete Prüfung                                                                              |
|-----------------|--------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| pbx.mode        | registration | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.          | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| pbx.endpoint_id | netcore-pbx  | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.   | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.        |
| pbx.host        | 127.0.0.1    | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten. |
| pbx.port        | 5060         | Transportport oder Bereich für Listener, Medien oder Paketpfad.                             | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten          |

| Schlüssel                        | Beispiel                 | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                      |
|----------------------------------|--------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
|                                  |                          |                                                                                                  | UDP-Bereich testen.                                                                                   |
| pbx.transport                    | udp                      | Transportauswahl der Gegenstelle, zum Beispiel UDP oder TCP.                                     | Beide Endpunkte und Firewall auf dasselbe Transportprotokoll prüfen; SIP und RTP getrennt betrachten. |
| pbx.username                     | (leer)                   | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.        | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.                |
| pbx.auth_username                | (leer)                   | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.        | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.                |
| pbx.password                     | [ausgelassen]            | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen.    | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.                       |
| pbx.from_user                    | netcore-tetra            | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.         |
| pbx.from_domain                  | (leer)                   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.         |
| pbx.contact_user                 | netcore-tetra            | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.         |
| pbx.registration_id              | netcore-pbx-registration | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.            |
| pbx.registration_expiration_secs | 30                       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den                                                      |



| Schlüssel | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|           |          |                                                                                                  | Wert prüfen; Grenzfall vor Änderung nachstellen.                                              |
| pbx.allow | ulaw     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| pbx.match | []       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.24.10 routing

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                         | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|-----------------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| routing.tetra_number_prefix       | (leer)   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| routing.strip_tetra_prefix        | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| routing.pbx_outbound_prefix       | (leer)   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| routing.strip_pbx_outbound_prefix | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| routing.accept_stale_routes       | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| routing.require_tbs_contact       | true     | Funktions- oder Policy-Schalter; kann einen                                                      | Wirksamen Modus nach Neustart ablesen und                                                     |

| Schlüssel                 | Beispiel | Bedeutung im Betrieb                                                 | Konkrete Prüfung                                                                                  |
|---------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                           |          | Datenweg oder eine Schutzbedingung ändern.                           | positive wie negative Probe dokumentieren.                                                        |
| routing.dial_timeout_secs | 60       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

#### 17.24.11 tbs[0]

Hier werden 7 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel           | Beispiel         | Bedeutung im Betrieb                                                                          | Konkrete Prüfung                                                                           |
|---------------------|------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| tbs[0].node_id      | SRV-M-TBS-01     | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                  | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen. |
| tbs[0].endpoint_id  | tbs-srv-m-tbs-01 | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.     | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.     |
| tbs[0].username     | tbs-srv-m-tbs-01 | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.     | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.     |
| tbs[0].password     | [ausgelassen]    | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.            |
| tbs[0].enabled      | false            | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.        | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.       |
| tbs[0].max_contacts | 1                | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                             | Zähler unter Last beobachten; Verhalten direkt am Limit und                                |

| Schlüssel      | Beispiel     | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|----------------|--------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                |              |                                                                                                  | nach Freigabe prüfen.                                                                         |
| tbs[0].aliases | [ 'TBS-01' ] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

#### 17.24.12 number\_mappings[0]

Hier werden 4 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                      | Beispiel | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                              |
|--------------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| number_mappings[0].number      | 4010001  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| number_mappings[0].target_type | issi     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| number_mappings[0].target      | 4010001  | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| number_mappings[0].enabled     | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |

**Nach der Änderung.** Für SIP Route, TBS-Kontakt, RTP, Edge-Fallback den Fachtest ausführen: ein- und ausgehend Sprachweg samt PAI/DTMF prüfen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; bei Ausfall externe Registrierung und State Machine prüfen.

#### 17.25 provisioning-core: Felder und Prüfpunkte

Vorlage system-backend/provisioning-core/config/provisioning-core.example.toml; Ziel /etc/netcore/provisioning-core.toml; Unit netcore-provisioning-core.service. Vor einer Änderung die aktuelle Datei sichern, nur die betroffene Gruppe ändern, Dienst neu starten und ready sowie den Fachtest nachweisen. Tabelle und Zielkonfiguration nach einem Update gegeneinander diffen.

### 17.25.1 server

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel   | Beispiel     | Bedeutung im Betrieb                                                                | Konkrete Prüfung                                                                                         |
|-------------|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| server.bind | 0.0.0.0:8125 | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers. | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |

### 17.25.2 upstream

Hier werden 3 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel                | Beispiel               | Bedeutung im Betrieb                                                                             | Konkrete Prüfung                                                                                  |
|--------------------------|------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| upstream.subscriber_core | http://10.0.1.181:8100 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| upstream.group_core      | http://10.0.1.182:8110 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| upstream.timeout_seconds | 5                      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 17.25.3 security

Hier werden 2 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel     | Beispiel | Bedeutung im Betrieb                                                               | Konkrete Prüfung                                                                            |
|---------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| security.mode | open_lab | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

| Schlüssel                        | Beispiel | Bedeutung im Betrieb                                                                   | Konkrete Prüfung                                                                     |
|----------------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| security.allow_remote_management | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |

#### 17.25.4 limits

Hier werden 1 Beispielwerte gemeinsam wirksam. Besonders Hostnamen, Pfade und Grenzwerte müssen mit den tatsächlich ausgerollten Gegenstellen und Ressourcen zusammenpassen.

| Schlüssel             | Beispiel | Bedeutung im Betrieb                                             | Konkrete Prüfung                                                                       |
|-----------------------|----------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| limits.max_body_bytes | 2097152  | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer. | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |

**Nach der Änderung.** Für Geräte-/Gruppenverwaltung und gemeinsame Synchronisation den Fachtest ausführen: Test-ISSI/GSSI mit Mitgliedschaft anlegen und rückgängig machen. Bei Abweichung zuerst die aktive Datei, anschließend Journal und Gegenstelle vergleichen; Teilerfolg in Subscriber/Group Core vor erneutem Sync abgleichen.

## 18 TBS-Konfiguration und RF-Parameter

Basis: Docs/basisstation.config.sanitized.example.toml. Die bereinigte Datei enthält 208 aktive Beispielschlüssel. Auskommentierte Optionen sind gesondert zu prüfen, bevor man sie übernimmt. Die Angaben sind keine Freigabe für einen realen Sendebetrieb: Frequenzplan, Endgerät und Hardware müssen am autorisierten Standort zusammenpassen. Bei jedem Konfigurationswechsel config.toml und die unabhängig gepflegte .fallback sichern.

### 18.1 config\_version: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel      | Beispiel | Auswirkung                                                                                       | Prüfung                                                                                       |
|----------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| config_version | 0.6      | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

### 18.2 stack\_mode: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel  | Beispiel | Auswirkung                                                                         | Prüfung                                                                                     |
|------------|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| stack_mode | Bs       | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad. | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen. |

### 18.3 service\_name: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel    | Beispiel | Auswirkung                                                                                       | Prüfung                                                                                       |
|--------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| service_name | tetra    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

## 18.4 phy\_io: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                      | Beispiel  | Auswirkung                                                                                | Prüfung                                                                                                    |
|--------------------------------|-----------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| phy_io.backend                 | SoapySdr  | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.        | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.                |
| phy_io.soapysdr.tx_freq        | 418000000 | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| phy_io.soapysdr.rx_freq        | 408000000 | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| phy_io.soapysdr.sample_rate    | 600000    | Geräteabhängige SDR-Auswahl oder Verstärkung; Einfluss auf Pegel, Empfang und Stabilität. | SoapySDRUtil --probe mit Gerät vergleichen, niedrige Pegel wählen und Messwerte protokollieren.            |
| phy_io.soapysdr.tx_center_freq | 418012500 | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| phy_io.soapysdr.rx_center_freq | 408012500 | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |

**RF-Abgleich.** SDR-Parameter, ausgestrahlten Träger, Broadcast-Zellinformation und programmierte Gerätekennung zusammen prüfen. Eine WebUI-Anzeige oder ein Parsererfolg allein belegt weder Pegel noch On-Air-Verhalten.

### 18.5 net\_info: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel    | Beispiel | Auswirkung                                                                   | Prüfung                                                                                    |
|--------------|----------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| net_info.mcc | 1        | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen. |
| net_info.mnc | 333      | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein. | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen. |

**RF-Abgleich.** SDR-Parameter, ausgestrahlten Träger, Broadcast-Zellinformation und programmierte Gerätekennung zusammen prüfen. Eine WebUI-Anzeige oder ein Parsererfolg allein belegt weder Pegel noch On-Air-Verhalten.

### 18.6 cell\_info: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                   | Beispiel | Auswirkung                                                                                | Prüfung                                                                                                    |
|-----------------------------|----------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| cell_info.freq_band         | 4        | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| cell_info.main_carrier      | 720      | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| cell_info.secondary_carrier | 721      | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen. | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |



| Schlüssel                      | Beispiel      | Auswirkung                                                                                       | Prüfung                                                                                                    |
|--------------------------------|---------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| cell_info.duplex_spacing       | 0             | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen.        | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| cell_info.freq_offset          | 0             | RF- oder Zellfrequenzparameter; PHY, Zellinformation und Funkgerät müssen zusammenpassen.        | Vor TX mit Frequenzplan und SDR-Probe abgleichen; Spektrum und Registrierung im autorisierten Test messen. |
| cell_info.reverse_operation    | false         | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                         |
| cell_info.location_area        | 1             | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                                      |
| cell_info.colour_code          | 1             | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                                      |
| cell_info.timezone             | Europe/Berlin | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.              |
| cell_info.local_ssi_ranges     | [[0, 90]]     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.              |
| cell_info.system_wide_services | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                         |
| cell_info.voice_service        | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                         |
| cell_info.registration         | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter                                            |

| Schlüssel                           | Beispiel | Auswirkung                                                                                       | Prüfung                                                                                       |
|-------------------------------------|----------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                     |          |                                                                                                  | Option abgleichen.                                                                            |
| cell_info.deregistration            | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.no_minimum_mode           | true     | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| cell_info.migration                 | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.circuit_mode_data_service | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.sndcp_service             | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.advanced_link             | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.system_code               | 1        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| cell_info.wap_ip.enabled            | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| cell_info.wap_ip.address            | 10.0.0.1 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| cell_info.wap_ip.port               | 9200     | Transportport oder Bereich für Listener, Medien oder Paketpfad.                                  | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten          |

| Schlüssel                                | Beispiel | Auswirkung                                                                                       | Prüfung                                                                                           |
|------------------------------------------|----------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                          |          |                                                                                                  | UDP-Bereich testen.                                                                               |
| cell_info.wap_ip.response_ttl            | 32       | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| cell_info.wap_ip.dynamic_pool_prefix     | 10.0.0   | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| cell_info.wap_ip.dynamic_pool_first_host | 2        | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace.      | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.     |
| cell_info.wap_ip.dynamic_pool_last_host  | 254      | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace.      | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.     |
| cell_info.wap_ip.allow_static_ipv4       | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| cell_info.wap_ip.accept_empty_probe      | true     | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| cell_info.wap_ip.accept_root_path        | true     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.          |
| cell_info.wap_ip.accept_status_path      | true     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.          |
| cell_info.wap_ip.accept_status_wml_path  | true     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost                         |

| Schlüssel                                              | Beispiel | Auswirkung                                                                              | Prüfung                                                                                |
|--------------------------------------------------------|----------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|                                                        |          |                                                                                         | kontrollieren.                                                                         |
| cell_info.wap_ip.max_request_payload_bytes             | 1024     | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                        | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| cell_info.wap_ip.assume_pdch_ready_after_data_transmit | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.     |
| cell_info.wap_ip.pdu_priority_max                      | 4        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                  |
| cell_info.wap_ip.ready_timer_code                      | 8        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                  |
| cell_info.wap_ip.standby_timer_code                    | 4        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                  |
| cell_info.wap_ip.response_wait_timer_code              | 7        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                  |
| cell_info.wap_ip.mtu_code                              | 2        | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                        | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen. |
| cell_info.wap_ip.network_default_data_priority         | 4        | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen.                | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen.   |
| cell_info.wap_ip.max_contexts_per_issi                 | 4        | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                       | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.      |
| cell_info.wap_ip.max_total_contexts                    | 64       | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                       | Zähler unter Last beobachten; Verhalten direkt am Limit und                            |

| Schlüssel                                               | Beispiel   | Auswirkung                                                                                       | Prüfung                                                                                       |
|---------------------------------------------------------|------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                         |            |                                                                                                  | nach Freigabe prüfen.                                                                         |
| cell_info.wap_ip.strict_source_address                  | true       | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| cell_info.packet_data_gateway.enabled                   | true       | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| cell_info.packet_data_gateway.interface_name            | ntetra0    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| cell_info.packet_data_gateway.prefix_len                | 24         | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                         |
| cell_info.packet_data_gateway.auto_configuration        | true       | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.packet_data_gateway.enable_ipv4_forwarding    | true       | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| cell_info.packet_data_gateway.managed_forwarding        | true       | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.            |
| cell_info.packet_data_gateway.allow_unsolicited_inbound | false      | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| cell_info.packet_data_gateway.nat_mode                  | masquerade | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| cell_info.packet_data_gateway                           | auto       | Konfigurationswert des                                                                           | Mit Schema und                                                                                |

| Schlüssel                                                        | Beispiel               | Auswirkung                                                                                       | Prüfung                                                                                           |
|------------------------------------------------------------------|------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| a_gateway.firewall_blockend                                      |                        | betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default.                        | ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.                    |
| cell_info.packet_data_gateway.dns_servers                        | ['1.1.1.1', '9.9.9.9'] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| cell_info.packet_data_gateway.channel_capacity                   | 256                    | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| cell_info.packet_data_gateway.max_pdchbearers                    | 0                      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                                | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| cell_info.packet_data_gateway.reserved_voice_slots               | 1                      | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| cell_info.packet_data_gateway.prefer_secondary_carrier           | true                   | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| cell_info.packet_data_gateway.downlink_queue_packets_per_context | 64                     | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| cell_info.packet_data_gateway.downlink_queue_bytes_per_context   | 262144                 | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| cell_info.packet_data_gateway.downlink_queue_ttl_secs            | 30                     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| cell_info.packet_data_gateway.page_retry_secs                    | 5                      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung              |

| Schlüssel                                                       | Beispiel                                   | Auswirkung                                                                                       | Prüfung                                                                                           |
|-----------------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                                                 |                                            |                                                                                                  | nachstellen.                                                                                      |
| cell_info.packet_data_gateway.fragment_reassembly_timeout_secs  | 30                                         | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| cell_info.packet_data_gateway.fragment_reassembly_max_datagrams | 128                                        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| cell_info.packet_data_gateway.fragment_reassembly_max_bytes     | 4194304                                    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| cell_info.packet_data_gateway.automatic_filter_ttl_secs         | 300                                        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| cell_info.packet_data_gateway.automatic_filter_max_bindings     | 4096                                       | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| cell_info.sds_command_control.authorized_issis                  | [2010001, 2010002, 2020001, 2020002, 5102] | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| cell_info.sds_command_control.commands[0].status_code           | 33001                                      | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| cell_info.sds_command_control.commands[0].action                | restart                                    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |

**RF-Abgleich.** SDR-Parameter, ausgestrahlten Träger, Broadcast-Zellinformation und programmierte Geräteerkennung zusammen prüfen. Eine WebUI-Anzeige oder ein Parsererfolg allein belegt weder Pegel noch On-Air-Verhalten.

### 18.7 recovery: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                 | Beispiel | Auswirkung                                                                             | Prüfung                                                                              |
|---------------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| recovery.enabled          | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| recovery.reactive_enabled | false    | Boolesche Steuerung des betreffenden Funktionsblocks.                                  | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |

### 18.8 health: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                     | Beispiel | Auswirkung                                                                             | Prüfung                                                                                           |
|-------------------------------|----------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| health.enabled                | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.              |
| health.snapshot_interval_secs | 300      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                   | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |

### 18.9 wx\_service: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel          | Beispiel | Auswirkung                                                                             | Prüfung                                                                              |
|--------------------|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| wx_service.enabled | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern. | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |



| Schlüssel               | Beispiel | Auswirkung                                                                              | Prüfung                                                               |
|-------------------------|----------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| wx_service.service_issi | 4010001  | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen. |

### 18.10 telegram\_alerts: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                        | Beispiel      | Auswirkung                                                                                    | Prüfung                                                                              |
|----------------------------------|---------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| telegram_alerts.enabled          | true          | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.        | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| telegram_alerts.bot_token        | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.      |
| telegram_alerts.chat_ids         | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.      |
| telegram_alerts.alert_connect    | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| telegram_alerts.alert_disconnect | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| telegram_alerts.alert_t351       | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| telegram_alerts.alert_lip        | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.   |
| telegram_alerts.alert_backhaul   | true          | Boolesche Steuerung des betreffenden                                                          | Effektiven Zustand und einen Test mit                                                |

| Schlüssel                           | Beispiel | Auswirkung                                                  | Prüfung                                                                                    |
|-------------------------------------|----------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|
|                                     |          | Funktionsblocks.                                            | aktivierter/deaktivierter Option abgleichen.                                               |
| telegram_alerts.alert_critical_logs | true     | Schwellwert für Entscheidung, Alarm oder Schutzmechanismus. | Ereignis knapp unter und über der Schwelle einspeisen und Hysterese/Deduplizierung prüfen. |

### 18.11 dashboard: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel          | Beispiel      | Auswirkung                                                                                    | Prüfung                                                                                                  |
|--------------------|---------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| dashboard.port     | 8080          | Transportport oder Bereich für Listener, Medien oder Paketpfad.                               | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| dashboard.bind     | 0.0.0.0       | Lokale Bindeadresse des Listeners; 0.0.0.0 erreicht alle Interfaces des Containers.           | Mit <code>ss -ltnp</code> und einer Anfrage aus dem vorgesehenen VLAN prüfen; Firewall-Grenze mitprüfen. |
| dashboard.username | admin         | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.     | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.                   |
| dashboard.password | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.                          |

### 18.12 media\_library: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel             | Beispiel | Auswirkung                                                     | Prüfung                                                         |
|-----------------------|----------|----------------------------------------------------------------|-----------------------------------------------------------------|
| media_library.enabled | false    | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine | Wirksamen Modus nach Neustart ablesen und positive wie negative |

| Schlüssel                               | Beispiel               | Auswirkung                                                                               | Prüfung                                                                                |
|-----------------------------------------|------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|                                         |                        | Schutzbedingung ändern.                                                                  | Probe dokumentieren.                                                                   |
| media_library.base_url                  | http://10.0.1.154:8230 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| media_library.station_id                | SRV-M-TBS-01           | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.publish_recordings        | true                   | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.recording_source_base_url | http://10.0.1.163:8080 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| media_library.auto_approve_recordings   | false                  | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.audio_source_enabled      | true                   | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.only_ready                | true                   | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.only_approved             | true                   | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.retry_seconds             | 60                     | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.              | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.           |
| media_library.request_timeout_seconds   | 15                     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall                |

| Schlüssel                              | Beispiel | Auswirkung                                                           | Prüfung                                                                                           |
|----------------------------------------|----------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                        |          |                                                                      | vor Änderung nachstellen.                                                                         |
| media_library.download_timeout_seconds | 120      | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel. | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| media_library.max_list_entries         | 1000     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.    | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |

### 18.13 recording: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                 | Beispiel                    | Auswirkung                                                                                       | Prüfung                                                                                       |
|---------------------------|-----------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| recording.enabled         | true                        | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| recording.active          | true                        | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.          |
| recording.directory       | /var/lib/netcore/recordings | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| recording.mode            | all                         | Betriebsmodus oder Autoritätsentscheidung; bestimmt den aktiven Verarbeitungspfad.               | Zulässige Varianten im zugehörigen Schema prüfen und einen Ausfall-/Rückfalltest ausführen.   |
| recording.selected_groups | []                          | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |

| Schlüssel                       | Beispiel                  | Auswirkung                                                                              | Prüfung                                                                                           |
|---------------------------------|---------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| recording.minimum_free_space_mb | 2048                      | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| recording.retention_days        | 30                        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| recording.max_recording_minutes | 480                       | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                       | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| recording.idle_finalize_secs    | 15                        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                    | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| recording.max_list_entries      | 2000                      | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.                       | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| recording.archive_enabled       | false                     | Boolesche Steuerung des betreffenden Funktionsblocks.                                   | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| recording.archive_directory     | /mnt/nfs-share/Recordings | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.               | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.          |
| recording.archive_retry_seconds | 60                        | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten. | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |

### 18.14 audio\_player: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel            | Beispiel | Auswirkung                                  | Prüfung                                   |
|----------------------|----------|---------------------------------------------|-------------------------------------------|
| audio_player.enabled | true     | Funktions- oder Policy-Schalter; kann einen | Wirksamen Modus nach Neustart ablesen und |

| Schlüssel                                | Beispiel                 | Auswirkung                                                                   | Prüfung                                                                                           |
|------------------------------------------|--------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                          |                          | Datenweg oder eine Schutzbedingung ändern.                                   | positive wie negative Probe dokumentieren.                                                        |
| audio_player.directory                   | /var/lib/netcore/audio   | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| audio_player.cache_directory             | /var/cache/netcore/audio | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.    | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.          |
| audio_player.source_issi                 | 4010001                  | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| audio_player.default_priority            | 5                        | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| audio_player.max_file_size_mb            | 100                      | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.             | Grenzwerttest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.            |
| audio_player.max_duration_seconds        | 1800                     | Obergrenze für Einträge, Verbindungen, Retries oder Aufbewahrung.            | Zähler unter Last beobachten; Verhalten direkt am Limit und nach Freigabe prüfen.                 |
| audio_player.lead_in_silence_blocks      | 12                       | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| audio_player.tail_silence_blocks         | 3                        | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                      |
| audio_player.group_release_guard_seconds | 6                        | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.         | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| audio_player.individ                     | 30                       | Zeitgrenze für Retry,                                                        | Zeitquelle und                                                                                    |

| Schlüssel                   | Beispiel       | Auswirkung                                                                  | Prüfung                                                                                  |
|-----------------------------|----------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| ual_answer_timeout_seconds  |                | Lease, Ablauf, Pufferung oder Zustandswechsel.                              | beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.       |
| audio_player.ffmpeg_path    | ffmpeg         | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.   | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren. |
| audio_player.shares[0].id   | server         | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |
| audio_player.shares[0].name | NFS-Server     | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |
| audio_player.shares[0].path | /mnt/nfs-share | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache. | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.             |

### 18.15 netcore\_directory: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                    | Beispiel                 | Auswirkung                                                                               | Prüfung                                                                                |
|------------------------------|--------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| netcore_directory.enabled    | true                     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.   |
| netcore_directory.base_url   | http://DIRECTORY-IP:8095 | Zieladresse einer Gegenstelle; Schema, Host, Port und Pfad bilden gemeinsam den Vertrag. | DNS/Route und Ziel-Health prüfen; bei ws:// den WebSocket-Pfad und Ack separat testen. |
| netcore_directory.timeout_ms | 2000                     | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                     | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung   |

| Schlüssel | Beispiel | Auswirkung | Prüfung      |
|-----------|----------|------------|--------------|
|           |          |            | nachstellen. |

### 18.16 control\_room: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                  | Beispiel     | Auswirkung                                                                                       | Prüfung                                                                                                  |
|----------------------------|--------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| control_room.enabled       | false        | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.           | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren.                     |
| control_room.host          | 10.0.20.10   | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace.      | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| control_room.port          | 8080         | Transportport oder Bereich für Listener, Medien oder Paketpfad.                                  | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| control_room.use_tls       | false        | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                       |
| control_room.endpoint_path | /ws/node     | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.                 |
| control_room.node_id       | SRV-M_TBS-01 | Identität oder Zuordnung; muss über Geräte, TBS und Backend konsistent sein.                     | Eindeutigkeit und berechnete Testkennung prüfen; Fehlzuordnung vor Funkprobe ausschließen.               |
| control_room.station_name  | SRV-M_TBS-01 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| control_room.site          | Main         | Konfigurationswert des                                                                           | Mit Schema und                                                                                           |



| Schlüssel                            | Beispiel | Auswirkung                                                                          | Prüfung                                                                                     |
|--------------------------------------|----------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|                                      |          | betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen.     |
| control_room.central<br>_sds_routing | false    | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                         | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen. |

### 18.17 edge\_fallback: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                                      | Beispiel                                            | Auswirkung                                                                                          | Prüfung                                                                                                          |
|------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| edge_fallback.enabled                          | true                                                | Funktions- oder Policy-<br>Schalter; kann einen<br>Datenweg oder eine<br>Schutzbedingung<br>ändern. | Wirksamen Modus nach<br>Neustart ablesen und<br>positive wie negative<br>Probe dokumentieren.                    |
| edge_fallback.enter_<br>after_secs             | 15                                                  | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                          | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |
| edge_fallback.recover_<br>after_secs           | 20                                                  | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                          | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |
| edge_fallback.unknown_<br>service_is_available | false                                               | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                         | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                      |
| edge_fallback.service_<br>matrix_lease_secs    | 60                                                  | Zeitgrenze für Retry,<br>Lease, Ablauf, Pufferung<br>oder Zustandswechsel.                          | Zeitquelle und<br>beobachtete<br>Verzögerung gegen den<br>Wert prüfen; Grenzfall<br>vor Änderung<br>nachstellen. |
| edge_fallback.policy_<br>cache_path            | /var/lib/<br>flowstation/edge-<br>policy-cache.json | Datei- oder<br>Verzeichnisort für<br>Zustand, Konfiguration,                                        | Existenz, Eigentümer,<br>Schreibrecht und<br>Backup-/Restore-Grenze                                              |

| Schlüssel                                       | Beispiel                                                                               | Auswirkung                                                                                       | Prüfung                                                                                           |
|-------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                                 |                                                                                        | Cache oder Archiv.                                                                               | am Zielhost kontrollieren.                                                                        |
| edge_fallback.policy_cache_max_age_secs         | 604800                                                                                 | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                             | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen. |
| edge_fallback.keep_last_known_policy            | true                                                                                   | Boolesche Steuerung des betreffenden Funktionsblocks.                                            | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                |
| edge_fallback.event_spool_path                  | /var/lib/flowstation/edge-event-spool.jsonl                                            | Datei- oder Verzeichnisort für Zustand, Konfiguration, Cache oder Archiv.                        | Existenz, Eigentümer, Schreibrecht und Backup-/Restore-Grenze am Zielhost kontrollieren.          |
| edge_fallback.event_spool_max_entries           | 10000                                                                                  | Numerischer Parameter des betreffenden Blocks; Einheit aus Namen und Quelltyp ableiten.          | Grenze im Runtime-Schema und Mess-/Zählerwirkung bei Testlast prüfen.                             |
| edge_fallback.event_spool_max_bytes             | 16777216                                                                               | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.             |
| edge_fallback.replay_batch_size                 | 128                                                                                    | Größenlimit für Nachricht, Frame, Speicher, Queue oder Transfer.                                 | Grenzwertest und RAM-/Disk-Budget ausführen; zu große Payloads kontrolliert ablehnen.             |
| edge_fallback.required_services                 | ['subscriber-core', 'group-core', 'mobility-core', 'call-control', 'media-switch'] ... | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.     |
| edge_fallback.service_fallbacks.node_gateway    | local_edge_autonomy                                                                    | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen.                         | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen.              |
| edge_fallback.service_fallbacks.subscriber-core | cached_policy_then_static_config                                                       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein                                | Mit Schema und ausgerollter Datei vergleichen; Wirkung                                            |

| Schlüssel                                     | Beispiel                                 | Auswirkung                                                                                       | Prüfung                                                                                       |
|-----------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                               |                                          | zugesicherter Runtime-Default.                                                                   | anhand Status und Fachtest nachweisen.                                                        |
| edge_fallback.service_fallbacks.group-core    | cached_policy_then_local_affiliations    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.mobility-core | local_registration_and_location_area     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.call-control  | local_cell_calls_only                    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.media-switch  | local_air_interface_media_only           | Medien-/Audioeinstellung; betrifft Format, Verarbeitung, Payout oder Cache.                      | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                  |
| edge_fallback.service_fallbacks.recorder      | local_recorder_continues                 | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.sds-router    | local_delivery_and_durable_store_forward | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.packet-core   | local_sndcp_contexts                     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.ip-gateway    | local_tun_gateway_when_configured        | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen.                         | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen.          |
| edge_fallback.service_fallbacks.security-core | last_known_security_policy_no_downgrade  | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein                                | Mit Schema und ausgerollter Datei vergleichen; Wirkung                                        |

| Schlüssel                                           | Beispiel                       | Auswirkung                                                                                       | Prüfung                                                                                       |
|-----------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                     |                                | zugesicherter Runtime-Default.                                                                   | anhand Status und Fachtest nachweisen.                                                        |
| edge_fallback.service_fallbacks.kmf                 | installed_keys_only_no_otar    | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.transit             | no_inter_region_routing        | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.control-room        | local_dashboard_and_audit      | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.observability       | local_logs_and_health_continue | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen. |
| edge_fallback.service_fallbacks.application-gateway | local_integrations_only        | Adressierung oder Next Hop; muss zum gerouteten IP- und LXC-Netz passen.                         | Mit ip route, Poolgrenzen und Gegenstellen vergleichen; Doppelbelegung ausschließen.          |
| edge_fallback.service_fallbacks.media-library       | local_media_cache_and_playout  | Medien-/ Audioeinstellung; betrifft Format, Verarbeitung, Playout oder Cache.                    | Mit einem echten Testasset Format, Pegel, Dauer und hörbaren Ausgang prüfen.                  |

### 18.18 brew: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel | Beispiel | Auswirkung                                                                                  | Prüfung                                                                                       |
|-----------|----------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| brew.host | BREW-IP  | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace. | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten. |

| Schlüssel                 | Beispiel      | Auswirkung                                                                                    | Prüfung                                                                                                  |
|---------------------------|---------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| brew.port                 | 8081          | Transportport oder Bereich für Listener, Medien oder Paketpfad.                               | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| brew.tls                  | false         | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                       |
| brew.username             | 0             | Kennung für Gegenstelle, Konto oder Endpoint; zur passenden Vertrauensbeziehung zuordnen.     | Kennung auf beiden Seiten abgleichen, ohne zugehörige Passwörter in Logs offenzulegen.                   |
| brew.password             | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen. | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.                          |
| brew.reconnect_delay_secs | 5             | Zeitgrenze für Retry, Lease, Ablauf, Pufferung oder Zustandswechsel.                          | Zeitquelle und beobachtete Verzögerung gegen den Wert prüfen; Grenzfall vor Änderung nachstellen.        |
| brew.feature_rssi_export  | true          | Boolesche Steuerung des betreffenden Funktionsblocks.                                         | Effektiven Zustand und einen Test mit aktivierter/deaktivierter Option abgleichen.                       |

### 18.19 asterisk: aktive Schlüssel

Die Tabelle dokumentiert die tatsächlich geparsten Werte der bereinigten Projektvorlage. Änderungen an diesem Block nur mit einer separaten, bekannten Startkonfiguration und einer messbaren Funktionsprobe vornehmen.

| Schlüssel                | Beispiel | Auswirkung                                                                               | Prüfung                                                                              |
|--------------------------|----------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| asterisk.enabled         | true     | Funktions- oder Policy-Schalter; kann einen Datenweg oder eine Schutzbedingung ändern.   | Wirksamen Modus nach Neustart ablesen und positive wie negative Probe dokumentieren. |
| asterisk.outbound_prefix | 91       | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime- | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und             |

| Schlüssel                          | Beispiel    | Auswirkung                                                                                                    | Prüfung                                                                                                              |
|------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
|                                    |             | Default.                                                                                                      | Fachtest nachweisen.                                                                                                 |
| asterisk.strip_outbo<br>und_prefix | true        | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                                   | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                          |
| asterisk.inbound_pre<br>fix        | T           | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen.            |
| asterisk.register                  | true        | Boolesche Steuerung<br>des betreffenden<br>Funktionsblocks.                                                   | Effektiven Zustand und<br>einen Test mit<br>aktivierter/deaktivierter<br>Option abgleichen.                          |
| asterisk.codec                     | PCMU        | Medien-/<br>Audioeinstellung; betrifft<br>Format, Verarbeitung,<br>Playout oder Cache.                        | Mit einem echten<br>Testasset Format, Pegel,<br>Dauer und hörbaren<br>Ausgang prüfen.                                |
| asterisk.service_num<br>bers       | ['*']       | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen.            |
| asterisk.rtp_port_mi<br>n          | 30000       | Numerischer Parameter<br>des betreffenden Blocks;<br>Einheit aus Namen und<br>Quelltyp ableiten.              | Grenze im Runtime-<br>Schema und<br>Mess-/Zählerwirkung bei<br>Testlast prüfen.                                      |
| asterisk.rtp_port_ma<br>x          | 30100       | Numerischer Parameter<br>des betreffenden Blocks;<br>Einheit aus Namen und<br>Quelltyp ableiten.              | Grenze im Runtime-<br>Schema und<br>Mess-/Zählerwirkung bei<br>Testlast prüfen.                                      |
| asterisk.bind_addr                 | 0.0.0.0     | Konfigurationswert des<br>betreffenden Blocks;<br>Beispielwert ist kein<br>zugesicherter Runtime-<br>Default. | Mit Schema und<br>ausgerollter Datei<br>vergleichen; Wirkung<br>anhand Status und<br>Fachtest nachweisen.            |
| asterisk.bind_port                 | 5062        | Transportport oder<br>Bereich für Listener,<br>Medien oder Paketpfad.                                         | Bindung und Freigabe in<br>beide Richtungen<br>prüfen; bei RTP den<br>gesamten konfigurierten<br>UDP-Bereich testen. |
| asterisk.remote_host               | ASTERISK-IP | Hostname oder IP-<br>Adresse der<br>Gegenstelle; 127.0.0.1                                                    | Namensauflösung und<br>Route vom betroffenen<br>LXC aus prüfen; nicht                                                |

| Schlüssel             | Beispiel      | Auswirkung                                                                                       | Prüfung                                                                                                  |
|-----------------------|---------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
|                       |               | bezeichnet immer den eigenen Namespace.                                                          | vom Deployment-Host ableiten.                                                                            |
| asterisk.remote_port  | 5060          | Transportport oder Bereich für Listener, Medien oder Paketpfad.                                  | Bindung und Freigabe in beide Richtungen prüfen; bei RTP den gesamten konfigurierten UDP-Bereich testen. |
| asterisk.contact_host | TBS-IP        | Hostname oder IP-Adresse der Gegenstelle; 127.0.0.1 bezeichnet immer den eigenen Namespace.      | Namensauflösung und Route vom betroffenen LXC aus prüfen; nicht vom Deployment-Host ableiten.            |
| asterisk.from_domain  | 10.0.1.21     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| asterisk.local_user   | 101           | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| asterisk.auth_user    | CHANGE-ME     | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |
| asterisk.password     | [ausgelassen] | Zugangsdaten oder Schlüsselmaterial; nur den benötigten Dienstberechtigten zugänglich machen.    | Dateirechte und Rotation prüfen; Wert weder loggen noch in ein Ticket kopieren.                          |
| asterisk.realm        | asterisk      | Konfigurationswert des betreffenden Blocks; Beispielwert ist kein zugesicherter Runtime-Default. | Mit Schema und ausgerollter Datei vergleichen; Wirkung anhand Status und Fachtest nachweisen.            |

### 18.99 Kommentierte Optionen und sichere Aktivierung

Die Vorlage enthält außerdem kommentierte Beispielooptionen. Sie sind nicht Teil des geparsten aktiven Beispiels. Im Folgenden stehen nur Zeilen, die wie konkrete TOML-Zuweisungen aussehen; erläuternde Kommentare sind keine Konfigurationsschlüssel. Vor Aktivierung im Parser/Schema des aktuellen Builds prüfen, ob Name und Typ akzeptiert werden.

Für jede aktivierte Option: Wert in einer Kopie eintragen; Parser und Start im Test prüfen; effektive Konfiguration und Logs kontrollieren; betroffenen Funk- oder Dienstpfad testen; bei Fehler auf die bekannte `.fallback` zurückgehen und Ursache festhalten.

| Block           | Option                               | Kommentiertes Beispiel                 | Zeile |
|-----------------|--------------------------------------|----------------------------------------|-------|
| Wurzel          | debug_log                            | "./verbose_log.txt"                    | 14    |
| phy_io          | dl_tx_file                           | "./dl_output.bin"                      | 48    |
| phy_io          | ul_rx_file                           | "./ul_output.bin"                      | 49    |
| phy_io          | ul_input_file                        | "./ul_input.bin"                       | 53    |
| phy_io          | dl_input_file                        | "./dl_input.bin"                       | 54    |
| phy_io.soapysdr | ppm_err                              | 0.0                                    | 63    |
| phy_io.soapysdr | rx_channel                           | 0                                      | 78    |
| phy_io.soapysdr | tx_channel                           | 0                                      | 79    |
| phy_io.soapysdr | device                               | "driver=plutosdr,uri=ip:192.168.42.42" | 89    |
| phy_io.soapysdr | device                               | "driver=lime,serial=123456789"         | 90    |
| phy_io.soapysdr | rx_antenna                           | "LNAW"                                 | 93    |
| phy_io.soapysdr | tx_antenna                           | "BAND2"                                | 94    |
| phy_io.soapysdr | rx_gain_lna                          | 30.0                                   | 109   |
| phy_io.soapysdr | rx_gain_tia                          | 9.0                                    | 110   |
| phy_io.soapysdr | rx_gain_pga                          | 12.0                                   | 111   |
| phy_io.soapysdr | tx_gain_pad                          | 50.0                                   | 114   |
| phy_io.soapysdr | tx_gain_iamp                         | 0.0                                    | 115   |
| cell_info       | custom_duplex_spacing                | 7600000                                | 136   |
| cell_info       | hangtime_secs                        | 5                                      | 168   |
| cell_info       | release_group_on_same_speaker_retake | false                                  | 176   |
| cell_info       | call_timeout_secs                    | 120                                    | 182   |
| cell_info       | ul_inactivity_secs                   | 3                                      | 188   |
| cell_info       | periodic_registration_secs           | 3600                                   | 201   |
| cell_info       | priority_cell                        | false                                  | 208   |



| Block     | Option                           | Kommentiertes Beispiel | Zeile |
|-----------|----------------------------------|------------------------|-------|
| cell_info | aie_service                      | false                  | 212   |
| cell_info | neighbor_cell_broadcast          | 0                      | 219   |
| cell_info | late_entry_supported             | false                  | 220   |
| cell_info | sharing_mode                     | 0                      | 222   |
| cell_info | ts_reserved_frames               | 0                      | 223   |
| cell_info | u_plane_dtx                      | false                  | 224   |
| cell_info | frame_18_ext                     | false                  | 225   |
| cell_info | ms_txpwr_max_cell                | 4                      | 226   |
| cell_info | subscriber_class                 | 0xFFFF                 | 227   |
| cell_info | cell_identifier_ca               | 1                      | 243   |
| cell_info | cell_reselection_types_supported | 1                      | 244   |

| Block     | Option                        | Kommentiertes Beispiel | Zeile |
|-----------|-------------------------------|------------------------|-------|
| cell_info | neighbor_cell_synchronized    | false                  | 245   |
| cell_info | cell_load_ca                  | 0                      | 246   |
| cell_info | main_carrier_number           | 1525                   | 247   |
| cell_info | main_carrier_number_extension | 0                      | 250   |
| cell_info | mcc                           | 204                    | 251   |
| cell_info | mnc                           | 1337                   | 252   |
| cell_info | location_area                 | 3                      | 253   |
| cell_info | maximum_ms_transmit_power     | 7                      | 254   |
| cell_info | minimum_rx_access_level       | 0                      | 255   |
| cell_info | subscriber_class              | 0xFFFF                 | 256   |
| cell_info | timeshare_cell_info           | 0                      | 257   |

| Block     | Option                           | Kommentiertes Beispiel | Zeile |
|-----------|----------------------------------|------------------------|-------|
|           | mation_or_security_parameters    |                        |       |
| cell_info | tdma_frame_offset                | 0                      | 258   |
| cell_info | registration                     | true                   | 263   |
| cell_info | deregistration                   | true                   | 264   |
| cell_info | priority_cell                    | false                  | 265   |
| cell_info | no_minimum_mode                  | false                  | 266   |
| cell_info | migration                        | false                  | 267   |
| cell_info | system_wide_services             | false                  | 268   |
| cell_info | voice_service                    | true                   | 269   |
| cell_info | circuit_mode_data_service        | false                  | 270   |
| cell_info | sndcp_service                    | false                  | 271   |
| cell_info | aie_service                      | false                  | 272   |
| cell_info | advanced_link                    | false                  | 273   |
| cell_info | cell_identifier_ca               | 1                      | 277   |
| cell_info | cell_reselection_types_supported | 1                      | 278   |
| cell_info | neighbor_cell_synchronized       | false                  | 279   |
| cell_info | cell_load_ca                     | 0                      | 280   |
| cell_info | main_carrier_number              | 1525                   | 281   |
| cell_info | cell_identifier_ca               | 2                      | 284   |
| cell_info | cell_reselection_types_supported | 1                      | 285   |
| cell_info | neighbor_cell_synchronized       | false                  | 286   |
| cell_info | cell_load_ca                     | 0                      | 287   |
| cell_info | main_carrier_number              | 1529                   | 288   |
| cell_info | source_issi                      | 16777215               | 299   |
| cell_info | interval_multiframes             | 96                     | 300   |

| Block                         | Option                      | Kommentiertes Beispiel      | Zeile |
|-------------------------------|-----------------------------|-----------------------------|-------|
| cell_info                     | protocol_id                 | 220                         | 301   |
| cell_info                     | text_coding_scheme          | "LATIN"                     | 302   |
| cell_info                     | text                        | "FlowStation"               | 303   |
| cell_info                     | source_issi                 | 16777215                    | 315   |
| cell_info                     | interval_multiframes        | 96                          | 316   |
| cell_info                     | protocol_id                 | 130                         | 317   |
| cell_info                     | text_coding_scheme          | "LATIN"                     | 318   |
| cell_info                     | text                        | "Y06RZV"                    | 319   |
| cell_info.packet_data_gateway | mtu                         | 576                         | 360   |
| cell_info.packet_data_gateway | external_interface          | "eth0"                      | 368   |
| cell_info.sds_command_control | issi_whitelist              | [2260571, 2260572, 2260575] | 445   |
| recovery                      | issi_allowlist              | []                          | 469   |
| recovery                      | cache_path                  | ""                          | 470   |
| recovery                      | max_replay_attempts         | 150                         | 471   |
| recovery                      | replay_per_frame            | 1                           | 472   |
| recovery                      | debounce_secs               | 5                           | 473   |
| recovery                      | max_cached_issis            | 1024                        | 474   |
| recovery                      | reactive_cooldown_secs      | 10                          | 476   |
| health                        | restart_on_core_stall       | false                       | 495   |
| health                        | core_stall_secs             | 10                          | 496   |
| health                        | restart_after_critical_secs | 30                          | 497   |
| health                        | restart_cooldown_secs       | 600                         | 498   |
| health                        | radios_silent_secs          | 900                         | 499   |
| health                        | dl_queue_degraded           | 64                          | 502   |
| health                        | dl_queue_critical           | 192                         | 503   |

| Block           | Option                 | Kommentiertes Beispiel | Zeile |
|-----------------|------------------------|------------------------|-------|
| health          | sds_queue_degraded     | 32                     | 504   |
| health          | sds_queue_critical     | 128                    | 505   |
| wx_service      | periodic_enabled       | false                  | 524   |
| wx_service      | periodic_issi          | 1001                   | 525   |
| wx_service      | periodic_is_group      | false                  | 526   |
| wx_service      | periodic_icao          | "LR0P"                 | 527   |
| wx_service      | periodic_interval_secs | 1800                   | 528   |
| telegram_alerts | forward_to_brew        | false                  | 575   |
| telegram_alerts | telegram_alert         | true                   | 576   |
| telegram_alerts | clear_timeout_secs     | 30                     | 577   |

| Block             | Option                        | Kommentiertes Beispiel                    | Zeile |
|-------------------|-------------------------------|-------------------------------------------|-------|
| dashboard         | public_overview               | true                                      | 605   |
| dashboard         | source_dir                    | "/opt/tetra-bluestation"                  | 613   |
| netcore_directory | host                          | "telemetry.example.com"                   | 712   |
| netcore_directory | port                          | 443                                       | 713   |
| netcore_directory | use_tls                       | true                                      | 714   |
| netcore_directory | ca_cert                       | "/etc/tetra-bluestation/telemetry-ca.der" | 715   |
| netcore_directory | username                      | "bts"                                     | 718   |
| netcore_directory | password                      | "changeme"                                | 719   |
| brew              | jitter_initial_latency_frames | 0                                         | 818   |
| brew              | feature_sds_enabled           | true                                      | 821   |
| brew              | whitelisted_issis             | [91]                                      | 835   |
| brew              | pbx_gateway_issis             | [16777184, 16777186]                      | 839   |
| brew              | enabled                       | false                                     | 856   |

| Block | Option                | Kommentiertes Beispiel          | Zeile |
|-------|-----------------------|---------------------------------|-------|
| brew  | api_url               | "https://hampager.de/api/calls" | 857   |
| brew  | username              | " "                             | 858   |
| brew  | password              | " "                             | 859   |
| brew  | poll_interval_secs    | 30                              | 860   |
| brew  | forward_sds           | false                           | 862   |
| brew  | forward_callout       | false                           | 863   |
| brew  | forward_telegram      | false                           | 864   |
| brew  | sds_source_issi       | 4010001                         | 866   |
| brew  | sds_dest_issi         | 0                               | 867   |
| brew  | sds_dest_is_group     | false                           | 868   |
| brew  | ric_issi_routes       | { "0632585" = 2632585 }         | 869   |
| brew  | ric_gssi_routes       | { "0004520" = 80 }              | 870   |
| brew  | sds_allowed_rics      | []                              | 871   |
| brew  | callout_allowed_rics  | []                              | 872   |
| brew  | telegram_allowed_rics | []                              | 873   |
| brew  | callout_source_issi   | 4010001                         | 875   |
| brew  | callout_dest_issi     | 0                               | 876   |
| brew  | callout_incident_base | 2                               | 877   |
| brew  | callout_text_prefix   | "DAPNET"                        | 878   |
| brew  | telegram_prefix       | "DAPNET"                        | 880   |
| brew  | rwth_core_enabled     | true                            | 882   |
| brew  | rwth_core_host        | "dapnet.afu.rwth-aachen.de"     | 883   |

| Block | Option         | Kommentiertes Beispiel | Zeile |
|-------|----------------|------------------------|-------|
| brew  | rwth_core_port | 43434                  | 884   |

| Block | Option              | Kommentiertes Beispiel    | Zeile |
|-------|---------------------|---------------------------|-------|
| brew  | rwth_core_device    | "FlowStation"             | 885   |
| brew  | rwth_core_version   | "1.0"                     | 886   |
| brew  | rwth_core_callsign  | " "                       | 887   |
| brew  | rwth_core_authkey   | " "                       | 888   |
| brew  | rwth_messages_limit | 100                       | 889   |
| brew  | enabled             | false                     | 909   |
| brew  | token               | " "                       | 910   |
| brew  | source_issi         | 4010001                   | 911   |
| brew  | dest_issi           | 0                         | 912   |
| brew  | incident_base       | 1                         | 913   |
| brew  | default_text        | "ALARM"                   | 914   |
| brew  | max_text_chars      | 80                        | 915   |
| brew  | enabled             | false                     | 936   |
| brew  | ami_host            | "127.0.0.1"               | 937   |
| brew  | ami_port            | 5038                      | 938   |
| brew  | ami_username        | "flowstation"             | 939   |
| brew  | ami_password        | " "                       | 940   |
| brew  | endpoints           | ["385"]                   | 941   |
| brew  | notify_sds          | true                      | 942   |
| brew  | notify_dapnet       | true                      | 943   |
| brew  | notify_telegram     | true                      | 944   |
| brew  | sds_directions      | ["rx", "net", "tx"]       | 945   |
| brew  | dapnet_allowed_rics | []                        | 946   |
| brew  | sds_allowed_issis   | []                        | 947   |
| brew  | title_prefix        | "FlowStation"             | 948   |
| brew  | notify_event        | "xml"                     | 949   |
| brew  | content_type        | "application/<br>snomxml" | 950   |
| brew  | subscription_state  | "active;expires=3000"     | 951   |

| Block | Option               | Kommentiertes Beispiel | Zeile |
|-------|----------------------|------------------------|-------|
|       |                      | 0"                     |       |
| brew  | max_text_chars       | 240                    | 952   |
| brew  | connect_timeout_secs | 3                      | 953   |
| brew  | enabled              | false                  | 969   |
| brew  | flowstation_lat      | 0.0                    | 970   |
| brew  | flowstation_lon      | 0.0                    | 971   |
| brew  | radius_m             | 500.0                  | 972   |

| Block | Option                   | Kommentiertes Beispiel | Zeile |
|-------|--------------------------|------------------------|-------|
| brew  | cooldown_secs            | 300                    | 973   |
| brew  | trigger_tetra            | true                   | 975   |
| brew  | trigger_meshcom          | true                   | 976   |
| brew  | forward_tpg2200          | false                  | 978   |
| brew  | forward_sds              | false                  | 979   |
| brew  | forward_sip              | false                  | 980   |
| brew  | forward_telegram         | false                  | 981   |
| brew  | tetra_issi_whitelist     | []                     | 983   |
| brew  | tetra_issi_blacklist     | []                     | 984   |
| brew  | meshcom_source_whitelist | []                     | 985   |
| brew  | meshcom_source_blacklist | []                     | 986   |
| brew  | sds_source_issi          | 4010001                | 988   |
| brew  | sds_dest_issi            | 0                      | 989   |
| brew  | sds_dest_is_group        | false                  | 990   |
| brew  | tpg2200_source_issi      | 4010001                | 992   |
| brew  | tpg2200_dest_issi        | 0                      | 993   |
| brew  | tpg2200_incident_base    | 1                      | 994   |
| brew  | tpg2200_text_prefix      | "GeoAlarm"             | 995   |

| Block | Option                 | Kommentiertes Beispiel | Zeile |
|-------|------------------------|------------------------|-------|
|       |                        |                        |       |
| brew  | tpg2200_max_text_chars | 80                     | 996   |
| brew  | sip_title_prefix       | "GeoAlarm"             | 998   |
| brew  | telegram_prefix        | "GeoAlarm"             | 999   |

**Aktivierungsprobe in der Praxis.** Kommentierte Zeilen sind Dokumentation, nicht automatisch ein vollständiges Schema. Die erste Frage lautet, ob der Parser des tatsächlich installierten Builds den Schlüssel überhaupt annimmt. Eine Kopie der aktiven Datei in einem isolierten Testverzeichnis erhält genau eine zusätzliche Option. Danach folgen Parser-/Startprobe, Vergleich der effektiven Konfiguration und ein gezielter Fachtest. Bei RF-Parametern gehören Spektrum und Funkgerät hinzu; bei Netz-URLs eine Verbindung vom TBS-Host zur Gegenstelle; bei Speicherpfaden ein Eigentümer- und Schreibtest. Erst wenn diese Kette funktioniert, wird die Option in der Standortkonfiguration aktiviert.

**Rückweg.** Die primäre TBS-Datei und `.fallback` haben unterschiedliche Aufgaben. Die `.fallback` wird als bekannte funktionierende Version vorgehalten und nicht automatisch mit einem neuen, ungetesteten Wert überschrieben. Bei einem Parserfehler zunächst den Fehler mit Zeilennummer sichern, dann die geänderte Primärdatei reparieren. Für einen falsch gewählten RF- oder Netzwerkparameter reicht ein Parsererfolg nicht: den vorherigen Wert wiederherstellen und erneut an Gerät sowie Gegenstelle prüfen. Ein Änderungsblatt enthält Schlüssel, alten/neuen Wert, Quellcommit, Messung, Tester und UTC.



## 19 Dienstbetrieb: 25 konkrete Arbeitsblätter

Dieses Kapitel macht aus Inventory, Konfigurationsvorlage und fachlichem Zweck einen wiederholbaren Arbeitsablauf. Die ersten 25 Dienste gehören zum Open-Lab-Inventory; Provisioning Core ist separat. Befehle mit `<...>` verlangen lokale Werte. Installationsskripte können Konfigurationen verändern: vorher deren Inhalt und den Istzustand prüfen. Jede Änderung wird mit Commit, Host, Zeitpunkt, alter/neuer Konfiguration, Test und Rückweg protokolliert. Die Fachtests sind Szenarien; sie sind erst nach einem realen Lauf als bestanden zu markieren.

### 19.1 node-gateway - Betrieb und Wiederanlauf

**Auftrag.** TBS-Registrierung, Core-Health-Matrix, Command-Ack. Diensthost im Beispiel 10.0.20.10:8080, Systemd-Unit `netcore-node-gateway.service`, Konfigurationsziel `/etc/netcore/node-gateway.toml`. Die Inventory-Abhängigkeiten sind keine. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                    | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/node-gateway/config/node-gateway.example.toml</code> mit <code>/etc/netcore/node-gateway.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-node-gateway.service</code> ; <code>systemctl status netcore-node-gateway.service</code>                                                      | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8080</code>                                                                                                                  | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8080/health/live</code> und <code>/health/ready</code>                                                                                        | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-node-gateway.service -b --no-pager</code>                                                                                                           | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/core-services`, `GET /api/v1/events`, `GET /api/v1/events/netcore`, `GET /api/v1/nodes`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/nodes/{node_id}/commands`, `POST /api/v1/nodes/{node_id}/disconnect`, `POST /api/v1/nodes/{node_id}/ping` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte

Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** `/api/v1/nodes` und `/api/v1/core-services` mit TBS-Dashboard vergleichen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Verbindung und letzte Ack-Revision vor einer erneuten Aktion prüfen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: Listener, Storage und lokale Konfiguration. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-node-gateway.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/node-gateway/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.2 mobility-core – Betrieb und Wiederanlauf

**Auftrag.** Serving-Node, Übergabe und MM-Kontext. Diensthost im Beispiel `10.0.20.11:8090`, Systemd-Unit `netcore-mobility-core.service`, Konfigurationsziel `/etc/netcore/mobility-core.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                       | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/mobility-core/config/mobility-core.example.toml</code> mit <code>/etc/netcore/mobility-core.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-mobility-core.service</code> ; <code>systemctl status netcore-mobility-core.service</code>                                                       | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8090</code>                                                                                                                     | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8090/health/live</code> und <code>/health/ready</code>                                                                                           | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-</code>                                                                                                                                                | Kein anhaltender Fehler,                        |

| Prüfpunkt | Durchführung                                     | Erwartung             |
|-----------|--------------------------------------------------|-----------------------|
|           | <code>mobility-core.service -b --no-pager</code> | Zeitstempel plausibel |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/events`, `GET /api/v1/events/netcore`, `GET /api/v1/nodes`, `GET /api/v1/status`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/transfers`, `POST /api/v1/transfers/{id}/cancel` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** zwei TBS und dieselbe Test-ISSI mit Zeitstempel verfolgen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Kollisionen und veraltete Serving-Lage vor Restore prüfen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `node-gateway`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-mobility-core.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/mobility-core/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.3 subscriber-core – Betrieb und Wiederanlauf

**Auftrag.** Zulassung, Profile, Sperren, Sync. Diensthost im Beispiel `10.0.20.12:8100`, `Systemd-Unit netcore-subscriber-core.service`, Konfigurationsziel `/etc/netcore/subscriber-core.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                                                                        | Erwartung                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Sollstand | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/subscriber-core/config/subscriber-core.example.toml</code> mit <code>/etc/netcore/subscriber-</code> | Abweichungen sind genehmigt und datiert |

| Prüfpunkt    | Durchführung                                                                                          | Erwartung                                       |
|--------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------|
|              | core.toml vergleichen                                                                                 |                                                 |
| Prozess      | systemctl is-active netcore-subscriber-core.service; systemctl status netcore-subscriber-core.service | Unit ist active, Neustartschleifen fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:8100                                                                         | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | curl -i http://<HOST>:8100/health/live und /health/ready                                              | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | journalctl -u netcore-subscriber-core.service -b --no-pager                                           | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/export.csv, GET /api/v1/export.json, GET /api/v1/nodes, GET /api/v1/observed. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/import, POST /api/v1/subscribers, POST /api/v1/sync deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Test-ISSI zulassen, sperren, Synchronisationsstand lesen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. bei Policy-Lücke die letzte bekannte TBS-Policy nicht still öffnen. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: node-gateway. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. systemctl cat netcore-subscriber-core.service und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer system-backend/subscriber-core/install/install.sh gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.4 group-core – Betrieb und Wiederanlauf

**Auftrag.** GSSI, Mitgliedschaft, Affiliation, DGNA. Diensthost im Beispiel 10.0.20.13:8110, Systemd-Unit netcore-group-core.service, Konfigurationsziel /etc/netcore/group-core.toml. Die Inventory-Abhängigkeiten sind node-gateway, subscriber-core. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                      | Erwartung                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | git rev-parse HEAD; Vorlage system-backend/group-core/config/group-core.example.toml mit /etc/netcore/group-core.toml vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | systemctl is-active netcore-group-core.service; systemctl status netcore-group-core.service                                       | Unit ist active, Neustartschleifen fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:8110                                                                                                     | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | curl -i http://<HOST>:8110/health/live und /health/ready                                                                          | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | journalctl -u netcore-group-core.service -b --no-pager                                                                            | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/affiliations, GET /api/v1/dgna, GET /api/v1/export.json, GET /api/v1/groups. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/dgna, POST /api/v1/groups, POST /api/v1/memberships deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Testgruppe und Mitgliedschaft auf beiden TBS abgleichen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. fehlende Affiliation von gesperrter Mitgliedschaft unterscheiden. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: node-gateway, subscriber-core. Eine bestehende TBS-Session nicht

ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-group-core.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/group-core/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.5 call-control – Betrieb und Wiederanlauf

**Auftrag.** logische Rufe, Floor, Call Legs, Restore. Diensthof im Beispiel 10.0.20.14:8120, Systemd-Unit `netcore-call-control.service`, Konfigurationsziel `/etc/netcore/call-control.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`, `subscriber-core`, `group-core`, `mobility-core`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                    | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/call-control/config/call-control.example.toml</code> mit <code>/etc/netcore/call-control.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-call-control.service</code> ; <code>systemctl status netcore-call-control.service</code>                                                      | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8120</code>                                                                                                                  | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8120/health/live</code> und <code>/health/ready</code>                                                                                        | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-call-control.service -b --no-pager</code>                                                                                                           | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/calls`, `GET /api/v1/calls/{logical_call_id}`, `GET /api/v1/events`, `GET /api/v1/events/netcore`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/calls/group`, `POST /api/v1/calls/individual`, `POST /api/v1/calls/{logical_call_id}/floor` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen



Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Aufbau, Sprecherwechsel und Release korreliert verfolgen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. hängende Call Legs erst nach Session-/Medienprüfung lösen. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: node-gateway, subscriber-core, group-core, mobility-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-call-control.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/call-control/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.6 media-switch – Betrieb und Wiederanlauf

**Auftrag.** Frame-Routing, Jitter, Drop, Codec. Diensthof im Beispiel 10.0.20.15:8130, Systemd-Unit `netcore-media-switch.service`, Konfigurationsziel `/etc/netcore/media-switch.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`, `call-control`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                    | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/media-switch/config/media-switch.example.toml</code> mit <code>/etc/netcore/media-switch.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-media-switch.service</code> ; <code>systemctl status netcore-media-switch.service</code>                                                      | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8130</code>                                                                                                                  | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8130/health/live</code> und <code>/health/ready</code>                                                                                        | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-</code>                                                                                                                                             | Kein anhaltender Fehler,                        |

| Prüfpunkt | Durchführung                                    | Erwartung             |
|-----------|-------------------------------------------------|-----------------------|
|           | <code>media-switch.service -b --no-pager</code> | Zeitstempel plausibel |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/buffers`, `GET /api/v1/config`, `GET /api/v1/events`, `GET /api/v1/nodes`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/gateway/ping`, `POST /api/v1/sessions/{session_id}/flush`, `POST /api/v1/sessions/{session_id}/inject` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** beide Richtungen mit hörbarem Testton und Zählern messen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. bei Stille zuerst `RouteReady` und `Frameeingang` je Leg trennen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `node-gateway`, `call-control`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-media-switch.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/media-switch/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.7 recorder – Betrieb und Wiederanlauf

**Auftrag.** Tap-Aufnahme, Asset, Export, Retention. Diensthost im Beispiel `10.0.20.16:8140`, Systemd-Unit `netcore-recorder.service`, Konfigurationsziel `/etc/netcore/recorder.toml`. Die Inventory-Abhängigkeiten sind `media-switch`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                    | Erwartung                               |
|-----------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Sollstand | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/recorder/config/recorder.example.toml</code> mit | Abweichungen sind genehmigt und datiert |



| Prüfpunkt    | Durchführung                                                                                      | Erwartung                                          |
|--------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------|
|              | /etc/netcore/recorder.toml<br>vergleichen                                                         |                                                    |
| Prozess      | systemctl is-active<br>netcore-recorder.service;<br>systemctl status netcore-<br>recorder.service | Unit ist active, Neustartschleifen<br>fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:8140                                                                     | Bindung stimmt mit<br>VLAN/Firewall überein        |
| Bereitschaft | curl -i<br>http://<HOST>:8140/health/l<br>ive und /health/ready                                   | Liveness und Fachbereitschaft<br>getrennt bewertet |
| Zustand      | journalctl -u netcore-<br>recorder.service -b --no-<br>pager                                      | Kein anhaltender Fehler,<br>Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/active, GET /api/v1/config, GET /api/v1/events, GET /api/v1/recordings. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/recordings/{id}/delete, POST /api/v1/recordings/{id}/finalize, POST /api/v1/recordings/{id}/hold deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** kurzen Testcall aufnehmen und Datei/Metadaten abspielen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. unvollständige Aufnahmen vor Archivrotation sichern. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: media-switch. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. systemctl cat netcore-recorder.service und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer system-backend/recorder/install/install.sh gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.8 sds-router – Betrieb und Wiederanlauf

**Auftrag.** Einzel-/Gruppen-SDS, Offline-Spool, Replay. Diensthof im Beispiel 10.0.20.17:8150, Systemd-Unit `netcore-sds-router.service`, Konfigurationsziel `/etc/netcore/sds-router.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`, `subscriber-core`, `group-core`, `mobility-core`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                              | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/sds-router/config/sds-router.example.toml</code> mit <code>/etc/netcore/sds-router.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-sds-router.service</code> ; <code>systemctl status netcore-sds-router.service</code>                                                    | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8150</code>                                                                                                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8150/health/live</code> und <code>/health/ready</code>                                                                                  | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-sds-router.service -b --no-pager</code>                                                                                                       | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/application-outbox`, `GET /api/v1/events`, `GET /api/v1/events/netcore`, `GET /api/v1/groups`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/application-outbox/{application}/{id}/ack`, `POST /api/v1/messages`, `POST /api/v1/messages/{id}/cancel` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Delivery-Ack und deduplizierten Replay mit Test-SSI prüfen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. alte Spool-Einträge mit Ziel- und Ablaufzeit abgleichen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle

Vorgänger prüfen: node-gateway, subscriber-core, group-core, mobility-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-sds-router.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/sds-router/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.9 packet-core – Betrieb und Wiederanlauf

**Auftrag.** PDP/NSAPI, Adresspool, Fragmentierung. Diensthost im Beispiel 10.0.20.18:8160, Systemd-Unit `netcore-packet-core.service`, Konfigurationsziel `/etc/netcore/packet-core.toml`. Die Inventory-Abhängigkeiten sind node-gateway, subscriber-core, mobility-core. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                 | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/packet-core/config/packet-core.example.toml</code> mit <code>/etc/netcore/packet-core.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-packet-core.service</code> ;<br><code>systemctl status netcore-packet-core.service</code>                                                  | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8160</code>                                                                                                               | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8160/health/live</code> und <code>/health/ready</code>                                                                                     | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-packet-core.service -b --no-pager</code>                                                                                                         | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/actions`, `GET /api/v1/bearers`, `GET /api/v1/contexts`, `GET /api/v1/contexts/{id}`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/actions/{id}/ack`, `POST /api/v1/contexts/{id}/{action}`, `POST /api/v1/downlink` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf

abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Kontextaufbau, IP-Zuordnung und Release beobachten. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. IP-Konflikt und verwaisten Kontext getrennt beheben. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `node-gateway`, `subscriber-core`, `mobility-core`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-packet-core.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/packet-core/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.10 ip-gateway – Betrieb und Wiederanlauf

**Auftrag.** TUN, Route, DNS, NAT, Firewall. Diensthost im Beispiel `10.0.20.19:8170`, Systemd-Unit `netcore-ip-gateway.service`, Konfigurationsziel `/etc/netcore/ip-gateway.toml`. Die Inventory-Abhängigkeiten sind `packet-core`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                              | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/ip-gateway/config/ip-gateway.example.toml</code> mit <code>/etc/netcore/ip-gateway.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-ip-gateway.service</code> ;<br><code>systemctl status netcore-ip-gateway.service</code>                                                 | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8170</code>                                                                                                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8170/health/live</code> und <code>/health/ready</code>                                                                                  | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-ip-gateway.service -b --no-</code>                                                                                                            | Kein anhaltender Fehler,                        |

| Prüfpunkt | Durchführung | Erwartung             |
|-----------|--------------|-----------------------|
|           | pager        | Zeitstempel plausibel |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/blocked, GET /api/v1/captures, GET /api/v1/captures/{id}/download, GET /api/v1/contexts. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/blocked, POST /api/v1/captures, POST /api/v1/captures/{id}/stop deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** TUN-Interface, Route und einen echten IP-Rückweg prüfen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. bei einseitigem Verkehr SNAT und Reverse Route auseinanderhalten. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: packet-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. systemctl cat netcore-ip-gateway.service und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer system-backend/ip-gateway/install/install.sh gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.11 security-core – Betrieb und Wiederanlauf

**Auftrag.** Admission und Security-Policy. Diensthost im Beispiel 10.0.20.20:8180, Systemd-Unit netcore-security-core.service, Konfigurationsziel /etc/netcore/security-core.toml. Die Inventory-Abhängigkeiten sind node-gateway, subscriber-core. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                                               | Erwartung                               |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Sollstand | git rev-parse HEAD; Vorlage system-backend/security-core/config/security-core.example.toml mit /etc/netcore/security-core.toml vergleichen | Abweichungen sind genehmigt und datiert |

| Prüfpunkt    | Durchführung                                                                                                   | Erwartung                                       |
|--------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Prozess      | <code>systemctl is-active netcore-security-core.service; systemctl status netcore-security-core.service</code> | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp; HTTP an &lt;HOST&gt;:8180</code>                                                               | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8180/health/live und /health/ready</code>                                    | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-security-core.service -b --no-pager</code>                                         | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/actions, GET /api/v1/alarms, GET /api/v1/audit, GET /api/v1/auth-contexts. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/alarms/{id}/ack, POST /api/v1/auth-contexts/{id}/response, POST /api/v1/auth-contexts/{id}/revoke deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** erlaubte und gesperrte Testkennung plus Audit prüfen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. keine Sicherheitsklasse zur Fehlerumgehung absenken. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: node-gateway, subscriber-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. systemctl cat netcore-security-core.service und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer system-backend/security-core/install/install.sh gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.



## 19.12 kmf – Betrieb und Wiederanlauf

**Auftrag.** Schlüssel-Metadaten, OTAR-Job, Revision. Diensthost im Beispiel 10.0.20.21:8190, Systemd-Unit netcore-kmf.service, Konfigurationsziel /etc/netcore/kmf.toml. Die Inventory-Abhängigkeiten sind security-core. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                 | Erwartung                                       |
|--------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | git rev-parse HEAD; Vorlage system-backend/kmf/config/kmf.example.toml mit /etc/netcore/kmf.toml vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | systemctl is-active netcore-kmf.service;<br>systemctl status netcore-kmf.service                             | Unit ist active, Neustartschleifen fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:8190                                                                                | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | curl -i http://<HOST>:8190/health/live und /health/ready                                                     | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | journalctl -u netcore-kmf.service -b --no-pager                                                              | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem GET /api/v1/audit, GET /api/v1/backups, GET /api/v1/config, GET /api/v1/export.json. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise POST /api/v1/backups, POST /api/v1/edge/actions/claim, POST /api/v1/edge/actions/{id}/ack deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** nur autorisierten Laborschlüssel-Job und Audit verfolgen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. vor Rotation alten Schlüsselzustand und Rückfall dokumentieren. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: security-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-kmf.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/kmf/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.13 transit – Betrieb und Wiederanlauf

**Auftrag.** Peer, Route, Session, Schleifenschutz. Diensthost im Beispiel 10.0.20.22:8200, Systemd-Unit `netcore-transit.service`, Konfigurationsziel `/etc/netcore/transit.toml`. Die Inventory-Abhängigkeiten sind `mobility-core`, `call-control`, `media-switch`, `sds-router`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                     | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/transit/config/transit.example.toml</code> mit <code>/etc/netcore/transit.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-transit.service</code> ;<br><code>systemctl status netcore-transit.service</code>                                              | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8200</code>                                                                                                   | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8200/health/live</code> und <code>/health/ready</code>                                                                         | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-transit.service -b --no-pager</code>                                                                                                 | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/config`, `GET /api/v1/events`, `GET /api/v1/local-deliveries`, `GET /api/v1/locations/groups`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/local-deliveries/{delivery_id}/ack`, `POST /api/v1/locations/groups`, `POST /api/v1/locations/subscribers` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.



**Fachlicher Nachweis.** Peer-Ausfall und Wiederkehr mit Routenrevision testen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Route erst nach Loop-/Session-Abgleich aktivieren. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: mobility-core, call-control, media-switch, sds-router. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-transit.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/transit/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. live, ready, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

#### 19.14 application-gateway – Betrieb und Wiederanlauf

**Auftrag.** Connector, Webhook, Regeln, TTS. Diensthost im Beispiel 10.0.20.23:8220, Systemd-Unit `netcore-application-gateway.service`, Konfigurationsziel `/etc/netcore/application-gateway.toml`. Die Inventory-Abhängigkeiten sind `sds-router`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                                         | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/application-gateway/config/application-gateway.example.toml</code> mit <code>/etc/netcore/application-gateway.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-application-gateway.service</code> ; <code>systemctl status netcore-application-gateway.service</code>                                                             | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8220</code>                                                                                                                                       | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8220/health/live</code> und <code>/health/ready</code>                                                                                                             | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-application-gateway.service -b --no-pager</code>                                                                                                                         | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/audit`, `GET /api/v1/backups`, `GET /api/v1/connectors`, `GET /api/v1/connectors/{connector_id}`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/backups`, `POST /api/v1/connectors`, `POST /api/v1/connectors/{connector_id}/disable` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Event mit Correlation ID bis zum Ziel nachverfolgen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpoint abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Retry/Deduplizierung vor erneutem Webhook trennen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `sds-router`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-application-gateway.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/application-gateway/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.15 media-library – Betrieb und Wiederanlauf

**Auftrag.** Import, Preview, Freigabe, Cache, Payout. Diensthost im Beispiel `10.0.20.24:8230`, Systemd-Unit `netcore-media-library.service`, Konfigurationsziel `/etc/netcore/media-library.toml`. Die Inventory-Abhängigkeiten sind `media-switch`, `recorder`, `application-gateway`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                                                                                       | Erwartung                                 |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Sollstand | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/media-library/config/media-library.example.toml</code> mit <code>/etc/netcore/media-library.toml</code> vergleichen | Abweichungen sind genehmigt und datiert   |
| Prozess   | <code>systemctl is-active netcore-media-library.service</code> ; <code>systemctl</code>                                                                                            | Unit ist active, Neustartschleifen fehlen |

| Prüfpunkt    | Durchführung                                                                | Erwartung                                       |
|--------------|-----------------------------------------------------------------------------|-------------------------------------------------|
|              | <code>status netcore-media-library.service</code>                           |                                                 |
| Listener     | <code>ss -ltnp; HTTP an &lt;HOST&gt;:8230</code>                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8230/health/live und /health/ready</code> | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-media-library.service -b --no-pager</code>      | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/assets`, `GET /api/v1/assets/{asset_id}`, `GET /api/v1/assets/{asset_id}/audio.tacelp`, `GET /api/v1/assets/{asset_id}/preview`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/assets/import-url`, `POST /api/v1/assets/upload-json`, `POST /api/v1/assets/{asset_id}/approve` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Asset importieren, genehmigen, previewen und an TBS abspielen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Format-/Cachefehler vor erneuter Freigabe isolieren. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `media-switch`, `recorder`, `application-gateway`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-media-library.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/media-library/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.16 control-room – Betrieb und Wiederanlauf

**Auftrag.** Operatorrolle, Lage, Incident, Schichtbuch. Diensthost im Beispiel 10.0.20.25:9010, Systemd-Unit netcore-control-room.service, Konfigurationsziel /etc/netcore/control-room.toml. Die Inventory-Abhängigkeiten sind node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, iot-gateway, hardware-gateway, rf-monitor, alarm-workflow, task-workflow, asset-management, sip-switch. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                            | Erwartung                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | git rev-parse HEAD; Vorlage system-backend/control-room/config/control-room.example.toml mit /etc/netcore/control-room.toml vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | systemctl is-active netcore-control-room.service; systemctl status netcore-control-room.service                                         | Unit ist active, Neustartschleifen fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:9010                                                                                                           | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | curl -i http://<HOST>:9010/health/live und /health/ready                                                                                | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | journalctl -u netcore-control-room.service -b --no-pager                                                                                | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Für diesen Dienst keinen nicht belegten API-Pfad voraussetzen. Den Handler und die laufende /openapi.json-Ausgabe, soweit vorhanden, zusammen mit der WebUI prüfen.

**Fachlicher Nachweis.** Fachstatus und eine protokollierte Operatoraktion prüfen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. UI-Cache von tatsächlichem Autoritätsdienst unterscheiden. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: node-gateway, subscriber-core, group-core, mobility-core, call-control, media-switch, recorder, sds-router, packet-core, ip-gateway, security-core, kmf, transit, application-gateway, media-library, iot-gateway, hardware-gateway, rf-monitor, alarm-workflow, task-workflow, asset-management, sip-switch. Eine bestehende TBS-Session nicht

ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-control-room.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/control-room/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.17 observability - Betrieb und Wiederanlauf

**Auftrag.** Scrape, Log, Trace, Alert, Silence. Diensthost im Beispiel 10.0.20.26:8210, Systemd-Unit `netcore-observability.service`, Konfigurationsziel `/etc/netcore/observability.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`, `subscriber-core`, `group-core`, `mobility-core`, `call-control`, `media-switch`, `recorder`, `sds-router`, `packet-core`, `ip-gateway`, `security-core`, `kmf`, `transit`, `application-gateway`, `media-library`, `iot-gateway`, `hardware-gateway`, `rf-monitor`, `alarm-workflow`, `task-workflow`, `asset-management`, `sip-switch`, `control-room`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                       | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/observability/config/observability.example.toml</code> mit <code>/etc/netcore/observability.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-observability.service</code> ;<br><code>systemctl status netcore-observability.service</code>                                                    | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8210</code>                                                                                                                     | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8210/health/live</code> und <code>/health/ready</code>                                                                                           | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-observability.service -b --no-pager</code>                                                                                                             | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/alerts`, `GET /api/v1/diagnostics`, `GET /api/v1/logs`, `GET /api/v1/metrics/catalog`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/diagnostics`, `POST /api/v1/logs/ingest`, `POST /api/v1/maintenance/scrape-now` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** ein markiertes Event bis Alert/Trace und Retention verfolgen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Scrape-Lücke nicht als Gesundheitszustand interpretieren. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `node-gateway`, `subscriber-core`, `group-core`, `mobility-core`, `call-control`, `media-switch`, `recorder`, `sds-router`, `packet-core`, `ip-gateway`, `security-core`, `kmf`, `transit`, `application-gateway`, `media-library`, `iot-gateway`, `hardware-gateway`, `rf-monitor`, `alarm-workflow`, `task-workflow`, `asset-management`, `sip-switch`, `control-room`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-observability.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/observability/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.18 iot-gateway – Betrieb und Wiederanlauf

**Auftrag.** MQTT Bridge, HA Discovery, Command/Ack. Diensthost im Beispiel `10.0.20.27:8240`, Systemd-Unit `netcore-iot-gateway.service`, Konfigurationsziel `/etc/netcore/iot-gateway.toml`. Die Inventory-Abhängigkeiten sind `node-gateway`, `mobility-core`, `call-control`, `sds-router`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                                                                                 | Erwartung                                 |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Sollstand | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/iot-gateway/config/iot-gateway.example.toml</code> mit <code>/etc/netcore/iot-gateway.toml</code> vergleichen | Abweichungen sind genehmigt und datiert   |
| Prozess   | <code>systemctl is-active netcore-iot-gateway.service</code> ;<br><code>systemctl status netcore-iot-gateway.service</code>                                                  | Unit ist active, Neustartschleifen fehlen |



| Prüfpunkt    | Durchführung                                                                | Erwartung                                       |
|--------------|-----------------------------------------------------------------------------|-------------------------------------------------|
| Listener     | <code>ss -ltnp; HTTP an &lt;HOST&gt;:8240</code>                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8240/health/live und /health/ready</code> | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-iot-gateway.service -b --no-pager</code>        | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/commands`, `GET /api/v1/commands/{command_id}`, `GET /api/v1/events`, `GET /api/v1/home-assistant`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/actions/home-assistant-discovery`, `POST /api/v1/actions/homematic-poll-now`, `POST /api/v1/actions/poll-now` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Registrierung, retained State und Test-Command komplett prüfen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Topic-Loop und verwaisten retained State bereinigen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `node-gateway`, `mobility-core`, `call-control`, `sds-router`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-iot-gateway.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/iot-gateway/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.19 hardware-gateway – Betrieb und Wiederanlauf

**Auftrag.** Edge-Telemetrie, Schwellwert, Rackzustand. Diensthost im Beispiel `10.0.20.28:8250`, Systemd-Unit `netcore-hardware-gateway.service`, Konfigurationsziel `/etc/netcore/hardware-gateway.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                                | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/hardware-gateway/config/hardware-gateway.example.toml</code> mit <code>/etc/netcore/hardware-gateway.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-hardware-gateway.service</code> ; <code>systemctl status netcore-hardware-gateway.service</code>                                                          | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8250</code>                                                                                                                              | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8250/health/live</code> und <code>/health/ready</code>                                                                                                    | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-hardware-gateway.service -b --no-pager</code>                                                                                                                   | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Für diesen Dienst keinen nicht belegten API-Pfad voraussetzen. Den Handler und die laufende `/openapi.json`-Ausgabe, soweit vorhanden, zusammen mit der WebUI prüfen.

**Fachlicher Nachweis.** Heartbeat und kontrollierten Grenzwert auslösen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem `200 OK` am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Sensorfehler von Funk-/Core-Ausfall getrennt melden. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-hardware-gateway.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/hardware-gateway/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.



## 19.20 rf-monitor – Betrieb und Wiederanlauf

**Auftrag.** DSP-Werte, RF-Agent, Kalibrierung. Diensthost im Beispiel 10.0.20.29:8260, Systemd-Unit `netcore-rf-monitor.service`, Konfigurationsziel `/etc/netcore/rf-monitor.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                              | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/rf-monitor/config/rf-monitor.example.toml</code> mit <code>/etc/netcore/rf-monitor.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-rf-monitor.service</code> ;<br><code>systemctl status netcore-rf-monitor.service</code>                                                 | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8260</code>                                                                                                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8260/health/live</code> und <code>/health/ready</code>                                                                                  | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-rf-monitor.service -b --no-pager</code>                                                                                                       | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/alarms`, `GET /api/v1/events`, `GET /api/v1/stations`, `GET /api/v1/status`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/telemetry` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Messquelle und Alarm bei definierter Teständerung prüfen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. DSP-Schätzwert nicht als kalibrierte Vor-/Rücklaufmessung ausgeben. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`. Eine bestehende TBS-Session nicht ohne Not

unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-rf-monitor.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/rf-monitor/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.21 alarm-workflow – Betrieb und Wiederanlauf

**Auftrag.** Deduplizierung, Eskalation, Ack. Diensthost im Beispiel `10.0.20.30:8270`, Systemd-Unit `netcore-alarm-workflow.service`, Konfigurationsziel `/etc/netcore/alarm-workflow.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`, `sds-router`, `hardware-gateway`, `rf-monitor`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                          | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/alarm-workflow/config/alarm-workflow.example.toml</code> mit <code>/etc/netcore/alarm-workflow.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-alarm-workflow.service</code> ; <code>systemctl status netcore-alarm-workflow.service</code>                                                        | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8270</code>                                                                                                                        | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8270/health/live</code> und <code>/health/ready</code>                                                                                              | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-alarm-workflow.service -b --no-pager</code>                                                                                                               | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/alarms`, `GET /api/v1/alarms/{id}`, `GET /api/v1/events`, `GET /api/v1/status`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/alarms`, `POST /api/v1/alarms/{id}/{action}`, `POST /api/v1/ingest-event` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten

kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** dasselbe Ereignis zweimal und Ack einmal senden. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Eskalationsstufe mit Zustellstatus und Zuständigkeit prüfen. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`, `sds-router`, `hardware-gateway`, `rf-monitor`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-alarm-workflow.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/alarm-workflow/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.22 task-workflow – Betrieb und Wiederanlauf

**Auftrag.** Taskstatus, WAP, SDS, MQTT. Diensthost im Beispiel `10.0.20.31:8280`, Systemd-Unit `netcore-task-workflow.service`, Konfigurationsziel `/etc/netcore/task-workflow.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`, `sds-router`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                                       | Erwartung                                       |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/task-workflow/config/task-workflow.example.toml</code> mit <code>/etc/netcore/task-workflow.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-task-workflow.service</code> ; <code>systemctl status netcore-task-workflow.service</code>                                                       | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8280</code>                                                                                                                     | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8280/health/live</code> und <code>/health/ready</code>                                                                                           | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-task-</code>                                                                                                                                           | Kein anhaltender Fehler,                        |

| Prüfpunkt | Durchführung                                | Erwartung             |
|-----------|---------------------------------------------|-----------------------|
|           | <code>workflow.service -b --no-pager</code> | Zeitstempel plausibel |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/status`, `GET /api/v1/tasks`, `GET /api/v1/tasks/{id}`, `GET /api/v1/templates`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/ingest-sds`, `POST /api/v1/tasks`, `POST /api/v1/tasks/{id}/{action}` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** Task anlegen, annehmen, abschließen und doppelte Antwort senden. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. `blocked/reopen` im Audit mit derselben Task-ID verfolgen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`, `sds-router`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-task-workflow.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/task-workflow/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.23 asset-management – Betrieb und Wiederanlauf

**Auftrag.** Asset, Gerätebindung, Firmware, Wartung. Diensthost im Beispiel `10.0.20.32:8290`, Systemd-Unit `netcore-asset-management.service`, Konfigurationsziel `/etc/netcore/asset-management.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`, `subscriber-core`, `mobility-core`, `task-workflow`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt | Durchführung                                                                                                                    | Erwartung                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Sollstand | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/asset-management/config/asset-management.example.toml</code> mit | Abweichungen sind genehmigt und datiert |

| Prüfpunkt    | Durchführung                                                                                                                         | Erwartung                                       |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
|              | <code>/etc/netcore/asset-management.toml</code> vergleichen                                                                          |                                                 |
| Prozess      | <code>systemctl is-active netcore-asset-management.service;</code><br><code>systemctl status netcore-asset-management.service</code> | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp; HTTP an &lt;HOST&gt;:8290</code>                                                                                     | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8290/health/live</code> und <code>/health/ready</code>                                             | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-asset-management.service -b --no-pager</code>                                                            | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Für diesen Dienst keinen nicht belegten API-Pfad voraussetzen. Den Handler und die laufende `/openapi.json`-Ausgabe, soweit vorhanden, zusammen mit der WebUI prüfen.

**Fachlicher Nachweis.** Asset mit Test-ISSI verbinden und Wartungsfall öffnen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. physische Zuordnung nicht mit Teilnehmerzulassung verwechseln. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`, `subscriber-core`, `mobility-core`, `task-workflow`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-asset-management.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/asset-management/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblem State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

## 19.24 sip-switch – Betrieb und Wiederanlauf

**Auftrag.** SIP Route, TBS-Kontakt, RTP, Edge-Fallback. Diensthof im Beispiel `10.0.20.33:8300`, Systemd-Unit `netcore-sip-switch.service`, Konfigurationsziel `/etc/netcore/sip-switch.toml`. Die Inventory-Abhängigkeiten sind `iot-gateway`, `mobility-core`. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                                              | Erwartung                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | <code>git rev-parse HEAD</code> ; Vorlage <code>system-backend/sip-switch/config/sip-switch.example.toml</code> mit <code>/etc/netcore/sip-switch.toml</code> vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | <code>systemctl is-active netcore-sip-switch.service</code> ;<br><code>systemctl status netcore-sip-switch.service</code>                                                 | Unit ist active, Neustartschleifen fehlen       |
| Listener     | <code>ss -ltnp</code> ; HTTP an <code>&lt;HOST&gt;:8300</code>                                                                                                            | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | <code>curl -i http://&lt;HOST&gt;:8300/health/live</code> und <code>/health/ready</code>                                                                                  | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | <code>journalctl -u netcore-sip-switch.service -b --no-pager</code>                                                                                                       | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Der Quellstand deklariert unter anderem `GET /api/v1/calls`, `GET /api/v1/decisions`, `GET /api/v1/events`, `GET /api/v1/mappings`. Zuerst den Status lesen, danach die betroffene Ressource mit einer bekannten Testkennung suchen. Bei 404 den laufenden Build und die deklarierte Route vergleichen; bei 503 zuerst die Abhängigkeiten, bei 500 das lokale Journal und eine Correlation ID.

**Schreibende Probe im Testnetz.** Im Quellstand sind beispielsweise `POST /api/v1/actions/reload-asterisk`, `POST /api/v1/actions/render-asterisk`, `POST /api/v1/calls/{token}/state` deklariert. Vorher ein Testobjekt eindeutig kennzeichnen, alte Daten exportieren und Folgeeffekte auf abhängigen Diensten kennen. Nachher Antwort, Audit, Replikation, Löschung des Testobjekts und Stabilität bei einer Wiederholung kontrollieren.

**Fachlicher Nachweis.** ein- und ausgehend Sprachweg samt PAI/DTMF prüfen. Die Reihenfolge lautet: Eingangereignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. bei Ausfall externe Registrierung und State Machine prüfen. Bei `ready=503` trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: `iot-gateway`, `mobility-core`. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** 1. `systemctl cat netcore-sip-switch.service` und Commit notieren. 2. Ziel-TOML und persistente Daten an einem zweiten Ort sichern. 3. Installer `system-backend/sip-switch/install/install.sh` gegen den vorherigen Stand lesen. 4. Im Wartungsfenster ausrollen. 5. `live`, `ready`, Fachprobe und abhängige Dienste prüfen. 6. Bei Regression vorige



Binärdatei/Konfiguration wiederherstellen und Zustand konsistent zurückspielen; keine neue Version mit altem inkompatiblen State erzwingen. 7. Abschluss mit UTC-Zeit, Testergebnis und Rollbackbeleg dokumentieren.

### 19.25 provisioning-core – Betrieb und Wiederanlauf

**Auftrag.** Geräte-/Gruppenverwaltung und gemeinsame Synchronisation. Diensthost im Beispiel separat:8125, Systemd-Unit netcore-provisioning-core.service, Konfigurationsziel /etc/netcore/provisioning-core.toml. Die Inventory-Abhängigkeiten sind subscriber-core, group-core. Bei einer Störung die funktionale Zuständigkeit dieses Dienstes von seiner technischen Erreichbarkeit trennen.

| Prüfpunkt    | Durchführung                                                                                                                                           | Erwartung                                       |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Sollstand    | git rev-parse HEAD; Vorlage system-backend/provisioning-core/config/provisioning-core.example.toml mit /etc/netcore/provisioning-core.toml vergleichen | Abweichungen sind genehmigt und datiert         |
| Prozess      | systemctl is-active netcore-provisioning-core.service; systemctl status netcore-provisioning-core.service                                              | Unit ist active, Neustartschleifen fehlen       |
| Listener     | ss -ltnp; HTTP an <HOST>:8125                                                                                                                          | Bindung stimmt mit VLAN/Firewall überein        |
| Bereitschaft | curl -i http://<HOST>:8125/health/live und /health/ready                                                                                               | Liveness und Fachbereitschaft getrennt bewertet |
| Zustand      | journalctl -u netcore-provisioning-core.service -b --no-pager                                                                                          | Kein anhaltender Fehler, Zeitstempel plausibel  |

**Lesende Fachprobe.** Für diesen Dienst keinen nicht belegten API-Pfad voraussetzen. Den Handler und die laufende /openapi.json-Ausgabe, soweit vorhanden, zusammen mit der WebUI prüfen.

**Fachlicher Nachweis.** Test-ISSI/GSSI mit Mitgliedschaft anlegen und rückgängig machen. Die Reihenfolge lautet: Eingangseignis auslösen; an der Dienstgrenze ankommen sehen; internen Zustand lesen; Ausgang/Ack an der nächsten Grenze nachweisen; am Endgerät oder Zielsystem die Wirkung prüfen. Die letzten beiden Punkte dürfen nicht aus einem 200 OK am ersten HTTP-Endpunkt abgeleitet werden.

**Wenn der Dienst ausfällt.** Zuerst den letzten erfolgreichen Fachzeitpunkt, den betroffenen Funktionsumfang und aktive Rückfallwege festhalten. Teilerfolg in Subscriber/Group Core vor erneutem Sync abgleichen. Bei ready=503 trotz laufendem Prozess in Inventory-Reihenfolge alle Vorgänger prüfen: subscriber-core, group-core. Eine bestehende TBS-Session nicht ohne Not unterbrechen. Einen Neustart nur ausführen, wenn Zustand, Backup und Impact klar sind; dann denselben Fachtest erneut fahren.

**Update und Rückweg.** Unit, Commit und Installer

system-backend/provisioning-core/install/install.sh prüfen. Vor dem Ausrollen TOML sowie Zustand beider autoritativer Cores sichern. Nachher eine Test-Mitgliedschaft auf beiden Seiten und an der TBS prüfen. Bei Teilerfolg erst die beiden Cores abgleichen, dann konsistent zurückrollen; UTC und Ergebnis festhalten.

**Spezielle Wiederanlaufprobe.** Das Dashboard ist nur die gemeinsame Bedienoberfläche. Nach einem Neustart werden dieselbe ISSI im Subscriber Core und dieselbe GSSI/Mitgliedschaft im Group Core direkt gelesen. Anschließend eine neue Testkennung über die UI anlegen, den Sync zur TBS abwarten und den tatsächlichen Attach beziehungsweise Gruppenruf prüfen. Scheitert nur ein Schreibteil, die beiden Exportstände sichern, den betroffenen Datensatz identifizieren und erst dann eine gezielte Korrektur vornehmen. Der Rückweg enthält auch die Entfernung der Testdaten und eine erneute Prüfung der letzten bekannten TBS-Policy. Ein grüner Provisioning-Healthstatus allein belegt keine konsistente Transaktion über beide Fachkerne.



## 20 Fehlersuche und Reparatur nach Symptomen

Jede Diagnose beginnt am **ersten fehlschlagenden Übergang**. Ein grünes Dashboard ist nur ein Indiz; eine echte Ende-zu-Ende-Probe umfasst Endgerät, Luftschnittstelle, TBS, Gateway, Fachkern und Ziel. Zuerst nichtinvasiv lesen, dann eine kontrollierte Testaktion ausführen und erst anschließend einen Neustart oder eine Wiederherstellung erwägen. Die folgenden Abläufe sind Laborverfahren; Datenbankdateien und Spools nie blind löschen.

### 20.1 TBS startet nach Konfigurationsänderung nicht

**Eingrenzung.** `journalctl -u tetra -b`; Parsefehler und aktiven Config-Pfad lesen; `config.toml` und `.fallback` vergleichen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** TOML-Syntax, Schlüsselnamen, SDR-Gain-Typen, Dateirechte. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Bekannte Fallback-Datei aktiv lassen, fehlerhafte Zeile in Primärdatei korrigieren und Parser/Start erneut prüfen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Dienststart, Health, Empfang und autorisierte Funkprobe. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.2 SDR sichtbar, aber kein sauberer Downlink

**Eingrenzung.** `SoapySDRUtil --probe`; SDR-Treiber, Clock und TX-Ausgabe prüfen; Spektrum am Messplatz beobachten. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Frequenz/Träger, Center Frequency, Sample Rate, Gain, Duplexer, Versorgung. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Hardwarepfad und Konfiguration einzeln korrigieren; zuerst am Dummyload mit kleinen Pegeln verifizieren. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** gemessener Träger, Broadcastdaten und stabile Registrierung. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.3 Funkgerät sieht Zelle, registriert sich nicht

**Eingrenzung.** TBS-MM/MAC-Log zu genau einer ISSI mit Zeitfenster; Subscriber-Policy und MCC/MNC/LAC vergleichen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** UL-Empfang, Colour Code, Geräteprogrammierung, Sperre, Cachezustand. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Ursache je Schicht beheben; erst nach erfolgreichem UL/Policy-Nachweis erneut anmelden. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Location Update akzeptiert, ISSI sichtbar, Re-Registration nach Neustart. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.4 Registrierung klappt nur lokal, nicht netzweit

**Eingrenzung.** Gateway /api/v1/nodes, Mobility Serving-Lage und TBS /api/edge-fallback korrelieren. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** WebSocket /ws/node, Lease-Frische, Node-ID, alte Revision. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Routing/Node-Konfiguration korrigieren, auf neue Matrix warten und Serving-Node-Lage prüfen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Handover/zweite Zelle und derselbe Teilnehmerzustand. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.5 PTT erhält keinen Floor

**Eingrenzung.** Call Control Call-ID, Group Core GSSI/Affiliation und TBS-CMCE-Log vergleichen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Gruppenzulassung, konkurrierender Floor, belegte Timeslots. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Mitgliedschaft oder verwaisten Callzustand gezielt reparieren; keinen pauschalen Cache-Reset. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Aufbau, Sprecherwechsel, Release und erneuter Ruf. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.6 Gruppenruf aufgebaut, aber nur eine Richtung hörbar

**Eingrenzung.** Media Switch RouteReady, Framezähler pro Leg und TBS RX/TX-Audio auseinanderhalten. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Codec, Route, lokale Playout-Gates, Pegel, Frame-Drops. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Fehlendes Leg/Format korrigieren, Testton beidseitig einspeisen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** hörbarer Inhalt plus Zähler in beiden Richtungen. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.7 SIP klingelt, Audio fehlt

**Eingrenzung.** SIP-Dialog, SDP-Adressen/Ports, RTP-Pakete und TBS-Bridge getrennt prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** NAT, Firewall, RTP-Bereich, Codec, einseitige Route. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** SIP-/RTP-Konfiguration konsistent herstellen; kein blindes Registrierungs-Umschalten. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** beide Audiokanäle, DTMF und sauberer Release. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.8 SIP-Fallback schaltet zu früh oder nicht zurück

**Eingrenzung.** Lokale State Machine, Health-Proben, Fehlversuchszähler und stabile Erholungszeit lesen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung

festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

#### **Häufige Ursachen im Projekt.**

CENTRAL\_ACTIVE/FAILOVER\_PENDING/PBX\_DIRECT\_ACTIVE/RECOVERY\_PENDING, Registrierungsobjekte. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Grenzen und Erreichbarkeit nach Quellstand prüfen; doppelte aktive Registrierung vermeiden. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** neuer Ruf zentral, Ausfallruf direkt, Rückkehr nach Hysterese. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### **20.9 PBX-Ziel T5102 wird abgewiesen**

**Eingrenzung.** Asterisk-Dialplan `netcore-from-pbx` und AGI-Skriptpfad prüfen; Resolve-API des Switches testen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** T-Marker-Akzeptanz vor numerischer Zuordnung, Endpoint-Alias. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Dialplan/Fallback-Renderer gemäß SIP-README korrigieren und laden. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** 5102, T5102, t5102 und ungültiges Ziel getrennt testen. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### **20.10 SDS zugestellt, aber doppelt nach Rückkehr**

**Eingrenzung.** TBS-Spool, Router-Legs, Delivery-IDs und `air_fallback_local_delivered` prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Replay-Ack, lokale Vorzustellung, Deduplizierung. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Fehlerhafte Replay-Zuordnung reparieren; Eintrag erst nach gesicherter Zustellung quittieren. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** einmalige lokale und entfernte Zustellung nach Isolationslauf. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.11 PDP aktiv, aber keine IP-Daten

**Eingrenzung.** TBS-SNDP-Kontext, Packet Core NSAPI/Lease und IP Gateway TUN/Route nacheinander prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Adresspool, Fragmentierung, MTU, NAT, Rückroute. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Ersten defekten Übergang beheben und Testpaket mit Capture auf beiden Seiten verfolgen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Uplink, Downlink, DNS und Kontextfreigabe. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.12 WAP lädt lokal, aber keine externe Ressource

**Eingrenzung.** WAP-Server/Portal getrennt von IP-Gateway und DNS prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** PDP und geroutete oder NAT-Topologie, MIME/WML/XHTML. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Host-Route oder DNS reparieren, ohne die TBS-Zellkonfiguration zu verändern. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** gleiche Ressource via Funkgerät und Testhost. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.13 MQTT-Registrierungsstatus fehlt in Home Assistant

**Eingrenzung.** Brokerverbindung, Topic, Discovery-Payload und retained State der einen ISSI verfolgen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** IoT-Gateway-Mapping, Broker-ACL, Prefix, Entity-ID, Payloadformat. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder

Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Mapping/Topic anpassen, verwaisten retained State gezielt entfernen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Neuregistrierung, Deregistrierung und Ack in HA. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.14 MQTT-Command erscheint, aber Funkaktion fehlt

**Eingrenzung.** Command-Topic, Bridge-Log, Node Gateway Ack und TBS-Lokalausführung mit Correlation ID prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Berechtigung, Offline-Spool, Loop-Schutz, falsche GSSI/ISSI. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Nur fehlenden Übergang reparieren; deduplizierten Retry testen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** ein Befehl, eine Funkwirkung, ein Ack. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.15 TTS wird erzeugt, aber nicht ausgesendet

**Eingrenzung.** Piper-Ergebnis, Media-Library-Assetstatus, Format, Cache und TBS-Playout-Freigabe prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** TACELP-Vorbereitung, SCCH/Timeslot, konkurrierende PTT, Dateirecht. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Asset/Format reparieren und mit kurzer Testansage neu freigeben. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** vollständige hörbare Ansage ohne unterbrochene Rufkontrolle. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.



## 20.16 Aufnahme existiert lokal, fehlt aber im Archiv

**Eingrenzung.** Lokale WAV+JSON, Recorder-Tap, Importjob, NFS-Mount und Freigabestatus prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Speicherplatz, Mount, Besitzer, Retention, Asset-ID. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Original sichern, Import erneut idempotent ausführen; keine lokale Datei vor Archivbeleg löschen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Preview/Metadaten/Prüfsumme und Restore. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.17 RF-Monitor meldet Fehlalarm

**Eingrenzung.** Messquelle im RF-Agent und DSP-Zähler identifizieren; Schwellwert/Einheit kontrollieren. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** kalibrierte Probe vs Schätzwert, Zeitstempel, Sensor-Ausfall. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Sensor/Mapping kalibrieren; Alarm mit bekannter Teständerung nachstellen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** physikalische Messung und Monitoralarm stimmen überein. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.18 Control Room ist grün, Fachfunktion fällt aus

**Eingrenzung.** Zeitstempel des letzten Polls und Origin-Health je Kachel ansehen; direkten Fachtest ausführen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** stale Cache, Pollziel, Operatorrechte, Fachdienst-Ready. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** Poll/Mapping korrigieren und direkt am autoritativen Dienst gegenprüfen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne

beobachtete Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** aktuelle Lage plus erfolgreiche Fachaktion. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.19 Observability zeigt keine neuen Daten

**Eingrenzung.** Scrape-Target, Uhrzeit, `/metrics`, Log-Ingest und Retention getrennt prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** Firewall, Endpoint, Zeitdrift, Parser und Speichergrenze. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** betroffenen Ingest-Pfad reparieren, markiertes Testevent einspeisen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne beobachtete Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Metrikzeitpunkt, Log-/Trace-ID und Alert vorhanden. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.20 Nach Update fehlt eine API-Route

**Eingrenzung.** laufenden Commit/Binary, `/openapi.json`, Reverse-Proxy-Pfad und Quell-OpenAPI vergleichen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** alter Prozess, falsches Ziel, noch nicht ausgerollter Handler. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** korrektes Bundle und Konfiguration konsistent ausrollen; ABI/State-Verträglichkeit prüfen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne beobachtete Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** Route und Fachtest auf derselben Version. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

### 20.21 Policy wird nach Core-Ausfall offen statt konservativ

**Eingrenzung.** TBS `/api/edge-fallback`, Cache-Datei, Matrix-Lease und Security-Policy prüfen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.



**Häufige Ursachen im Projekt.** Cachealter, keep\_last\_known\_policy, unbekannter Dienstzustand. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** letzte explizite Policy wiederherstellen; Ursache der Degradation beheben. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** gesperrte Testkennung bleibt gesperrt, erlaubte arbeitet. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 20.22 Fault-Test endet mit gestopptem Dienst

**Eingrenzung.** Report/Run-ID lesen, Opfer-Unit und abhängige Dienste im Inventory bestimmen. Dabei Uhrzeit, Softwarestand und eine eindeutige Testkennung festhalten. Nicht mehrere Stellschrauben zugleich verändern; der erste abweichende Übergang bestimmt die nächste Maßnahme.

**Häufige Ursachen im Projekt.** abgebrochener Testlauf, unvollständiger Restart, State-Konflikt. Ein TCP-Port oder HTTP-200 allein beweist den nachgelagerten Funk- oder Medienpfad nicht. Die Gegenstelle mit derselben Kennung und demselben Zeitraum abgleichen.

**Behebung.** betroffene Units kontrolliert starten und Fachzustand prüfen. Vor zustandsändernden Schritten aktuelle Konfiguration und Daten sichern. Ein Neustart ohne behobene Ursache ist nur ein vorübergehender Recovery-Schritt und wird als solcher protokolliert.

**Abschlussprobe.** alle Opfer active/ready, keine Fixtures/Spoolfehler. Zusätzlich einmal den Rückweg beziehungsweise die Wiederholung durchführen und den Zustand nach einem Dienstneustart prüfen, wenn die Reparatur Persistenz oder Retry verändert hat.

## 21 Protokollinventur, Zustände und Konformitätsgrenzen

Die Tabellen dieses Kapitels sind eine **statische Quellcode-Inventur** des festgehaltenen Commits. vorhanden heißt weder normkonform noch interoperabel. Statische Treffer beweisen weder Laufzeitpfad noch Verhalten eines Funkgeräts. Normative Details stehen in der betreffenden ETSI-Ausgabe; hier werden keine Normtexte wiedergegeben. Die Projektmatrizen Docs/ETSI\_CONFORMANCE\_MATRIX.md, Docs/SAP\_PRIMITIVE\_MATRIX.md und Docs/STATE\_MACHINE\_INVENTORY.md sind generierte Prüflisten und müssen nach Codeänderungen neu erzeugt werden.

### 21.1 PDU-Katalog nach Schicht

Für jede PDU nennt die Quellmatrix Richtung, Parser, Encoder, Runtime-Hinweis, Test- und Golden-Vector-Treffer sowie den Quellpfad. Ein offener Hinweis ist ein Anlass für Code-Review und On-Air-Probe, keine automatische Laufzeitstörung. Besonders teilweiser Parser/Encoder und unimplemented!-Pfad dürfen nicht aus einem erfolgreichen Build als funktionsfähig abgeleitet werden.

CMCE: 30 erfasste PDUs

| PDU                      | Richtung            | Parser / Encoder      | Runtime         | Test / Vector                  | Quelldatei                                                     |
|--------------------------|---------------------|-----------------------|-----------------|--------------------------------|----------------------------------------------------------------|
| CmceFunctionNotSupported | nicht klassifiziert | teilweise / teilweise | blockiert       | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/cmce_function_not_supported.rs |
| DAlert                   | Downlink            | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/d_alert.rs                     |
| DCallProceeding          | Downlink            | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/d_call_proceeding.rs           |
| DCallRestore             | Downlink            | vorhanden / vorhanden | offene Hinweise | vorhanden / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_call_restore.rs              |
| DConnect                 | Downlink            | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/d_connect.rs                   |
| DConnectAckno            | Downlink            | vorhanden /           | offene Hinweise | vorhanden /                    | crates/tetra-                                                  |

| PDU         | Richtung | Parser / Encoder      | Runtime         | Test / Vector              | Quelldatei                                       |
|-------------|----------|-----------------------|-----------------|----------------------------|--------------------------------------------------|
| wledge      |          | vorhanden             |                 | vorhanden                  | pdus/src/cmce/pdus/d_connect_acknowledge.rs      |
| DDisconnect | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_disconnect.rs  |
| DFacility   | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_facility.rs    |
| DInfo       | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_info.rs        |
| DRelease    | Downlink | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden      | crates/tetra-pdus/src/cmce/pdus/d_release.rs     |
| DSdsData    | Downlink | teilweise / vorhanden | blockiert       | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_sds_data.rs    |
| DSetup      | Downlink | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden      | crates/tetra-pdus/src/cmce/pdus/d_setup.rs       |
| DStatus     | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_status.rs      |
| DTxCeased   | Downlink | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden      | crates/tetra-pdus/src/cmce/pdus/d_tx_ceased.rs   |
| DTxContinue | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/d_tx_continue.rs |

| PDU          | Richtung | Parser / Encoder      | Runtime         | Test / Vector                  | Quelldatei                                        |
|--------------|----------|-----------------------|-----------------|--------------------------------|---------------------------------------------------|
| DTxGranted   | Downlink | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/d_tx_granted.rs   |
| DTxInterrupt | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/d_tx_interrupt.rs |
| DTxWait      | Downlink | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/d_tx_wait.rs      |
| UAlert       | Uplink   | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/u_alert.rs        |
| UCallRestore | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / nicht nachgewiesen | crates/tetra-pdus/src/cmce/pdus/u_call_restore.rs |
| UConnect     | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/u_connect.rs      |
| UDisconnect  | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/u_disconnect.rs   |
| UFacility    | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden          | crates/tetra-pdus/src/cmce/pdus/u_facility.rs     |
| UInfo        | Uplink   | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/u_info.rs         |
| URelease     | Uplink   | vorhanden / vorhanden | offene Hinweise | fehlt / nicht nachgewiesen     | crates/tetra-pdus/src/cmce/pdus/u_release.rs      |

| PDU       | Richtung | Parser / Encoder      | Runtime         | Test / Vector         | Quelldatei                                     |
|-----------|----------|-----------------------|-----------------|-----------------------|------------------------------------------------|
| USdsData  | Uplink   | teilweise / vorhanden | blockiert       | vorhanden / vorhanden | crates/tetra-pdus/src/cmce/pdus/u_sds_data.rs  |
| USetup    | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden | crates/tetra-pdus/src/cmce/pdus/u_setup.rs     |
| UStatus   | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden | crates/tetra-pdus/src/cmce/pdus/u_status.rs    |
| UTxCeased | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden | crates/tetra-pdus/src/cmce/pdus/u_tx_ceased.rs |
| UTxDemand | Uplink   | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden | crates/tetra-pdus/src/cmce/pdus/u_tx_demand.rs |

**Review.** Für auffällige Einträge den Decoder und Encoder mit gültigem, verkürztem und ungültigem PDU-Puffer prüfen; danach den realen Runtime-Aufrufpfad und mindestens einen Endgerätetest nachweisen. Zeitstempel, Hersteller/Firmware, Rohpuffer und Logs im Testfall referenzieren.

#### LLC: 4 erfasste PDUs

| PDU     | Richtung      | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                 |
|---------|---------------|-----------------------|-------------------------------|----------------------------|--------------------------------------------|
| BLack   | bidirektional | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/llc/pdus/bl_ack.rs   |
| BLadata | bidirektional | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/llc/pdus/bl_adata.rs |
| BLData  | bidirektional | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/llc/pdus/bl_data.rs  |
| BLUdata | bidirektional | vorhanden /           | ohne                          | fehlt / nicht              | crates/tetra-                              |

| PDU | Richtung | Parser / Encoder | Runtime                  | Test / Vector | Quelldatei                    |
|-----|----------|------------------|--------------------------|---------------|-------------------------------|
|     |          | vorhanden        | offensichtlichen Blocker | nachgewiesen  | pdus/src/llc/pdus/bl_udata.rs |

**Review.** Für auffällige Einträge den Decoder und Encoder mit gültigem, verkürztem und ungültigem PDU-Puffer prüfen; danach den realen Runtime-Aufrufpfad und mindestens einen Endgerätetest nachweisen. Zeitstempel, Hersteller/Firmware, Rohpuffer und Logs im Testfall referenzieren.

#### MLE: 13 erfasste PDUs

| PDU                  | Richtung | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                                |
|----------------------|----------|-----------------------|-------------------------------|----------------------------|-----------------------------------------------------------|
| DChannelResponse     | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/d_channel_response.rs      |
| DMleSync             | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mle/pdus/d_mle_sync.rs              |
| DMleSysinfo          | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mle/pdus/d_mle_sysinfo.rs           |
| DNewCell             | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/d_new_cell.rs              |
| DNwrkBroadcast       | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mle/pdus/d_nwrk_broadcast.rs        |
| DNwrkBroadcastRemove | Downlink | teilweise / teilweise | blockiert                     | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mle/pdus/d_nwrk_broadcast_remove.rs |
| DPrepareFail         | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/d_prepare_fail.rs          |

| PDU                  | Richtung | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                               |
|----------------------|----------|-----------------------|-------------------------------|----------------------------|----------------------------------------------------------|
|                      |          |                       |                               |                            | l.rs                                                     |
| DRestoreAck          | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/d_restore_ack.rs          |
| DRestoreFail         | Downlink | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/d_restore_fail.rs         |
| UChannelClass Advice | Uplink   | teilweise / teilweise | blockiert                     | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mle/pdus/u_channel_class_advice.rs |
| UChannelRequest      | Uplink   | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/u_channel_request.rs      |
| UPprepare            | Uplink   | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/u_prepare.rs              |
| URestore             | Uplink   | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mle/pdus/u_restore.rs              |

**Review.** Für auffällige Einträge den Decoder und Encoder mit gültigem, verkürztem und ungültigem PDU-Puffer prüfen; danach den realen Runtime-Aufrufpfad und mindestens einen Endgerätetest nachweisen. Zeitstempel, Hersteller/Firmware, Rohpuffer und Logs im Testfall referenzieren.

#### MM: 14 erfasste PDUs

| PDU                         | Richtung | Parser / Encoder      | Runtime         | Test / Vector         | Quelldatei                                                      |
|-----------------------------|----------|-----------------------|-----------------|-----------------------|-----------------------------------------------------------------|
| DAttachDetach GroupIdentity | Downlink | vorhanden / vorhanden | offene Hinweise | vorhanden / vorhanden | crates/tetra-pdus/src/mm/pdus/d_attach_detach_group_identity.rs |

| PDU                                       | Richtung            | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                                                      |
|-------------------------------------------|---------------------|-----------------------|-------------------------------|----------------------------|---------------------------------------------------------------------------------|
| DAttachDetachGroupIdentityAcknowledgement | Downlink            | vorhanden / vorhanden | offene Hinweise               | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_attach_detach_group_identity_acknowledgement.rs |
| DLocationUpdateAccept                     | Downlink            | vorhanden / vorhanden | offene Hinweise               | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_location_update_accept.rs                       |
| DLocationUpdateCommand                    | Downlink            | teilweise / vorhanden | blockiert                     | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_location_update_command.rs                      |
| DLocationUpdateProceeding                 | Downlink            | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_location_update_proceeding.rs                   |
| DLocationUpdateReject                     | Downlink            | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_location_update_reject.rs                       |
| DMmStatus                                 | Downlink            | teilweise / teilweise | blockiert                     | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/d_mm_status.rs                                    |
| MmPduFunctionNotSupported                 | nicht klassifiziert | teilweise / teilweise | blockiert                     | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/mm_pdu_function_not_supported.rs                  |
| UAttachDetachGroupIdentity                | Uplink              | vorhanden / vorhanden | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/u_attach_detach_group_identity.rs                 |



| PDU                                       | Richtung | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                                                      |
|-------------------------------------------|----------|-----------------------|-------------------------------|----------------------------|---------------------------------------------------------------------------------|
|                                           |          |                       |                               |                            | ch_group_identity.rs                                                            |
| UAttachDetachGroupIdentityAcknowledgement | Uplink   | vorhanden / vorhanden | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/u_attach_detach_group_identity_acknowledgement.rs |
| UItsiDetach                               | Uplink   | vorhanden / vorhanden | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/u_itsi_detach.rs                                  |
| ULocationUpdateDemand                     | Uplink   | vorhanden / vorhanden | offene Hinweise               | vorhanden / vorhanden      | crates/tetra-pdus/src/mm/pdus/u_location_update_demand.rs                       |
| UMmStatus                                 | Uplink   | vorhanden / vorhanden | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/u_mm_status.rs                                    |
| UTeiProvide                               | Uplink   | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/mm/pdus/u_tei_provide.rs                                  |

**Review.** Für auffällige Einträge den Decoder und Encoder mit gültigem, verkürztem und ungültigem PDU-Puffer prüfen; danach den realen Runtime-Aufrufpfad und mindestens einen Endgerätetest nachweisen. Zeitstempel, Hersteller/Firmware, Rohpuffer und Logs im Testfall referenzieren.

#### UMAC: 16 erfasste PDUs

| PDU         | Richtung     | Parser / Encoder      | Runtime   | Test / Vector              | Quelldatei                                       |
|-------------|--------------|-----------------------|-----------|----------------------------|--------------------------------------------------|
| AccessField | MAC/abhängig | teilweise / vorhanden | blockiert | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/access_assign.rs |

| PDU              | Richtung     | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                            |
|------------------|--------------|-----------------------|-------------------------------|----------------------------|-------------------------------------------------------|
| AccessAssignFr18 | MAC/abhängig | teilweise / teilweise | blockiert                     | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/access_assign_fr18.rs |
| AccessDefine     | MAC/abhängig | teilweise / teilweise | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/access_define.rs      |
| MacAccess        | MAC/abhängig | teilweise / teilweise | blockiert                     | vorhanden / vorhanden      | crates/tetra-pdus/src/umac/pdus/mac_access.rs         |
| MacDBlck         | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_d_blk.rs          |
| MacData          | MAC/abhängig | teilweise / teilweise | blockiert                     | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_data.rs           |
| MacEndDL         | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_end_dl.rs         |
| MacEndHu         | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_end_hu.rs         |
| MacEndUL         | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_end_ul.rs         |
| MacFragDL        | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_frag_dl.rs        |
| MacFragUL        | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_frag_ul.rs        |

| PDU         | Richtung     | Parser / Encoder      | Runtime                       | Test / Vector              | Quelldatei                                      |
|-------------|--------------|-----------------------|-------------------------------|----------------------------|-------------------------------------------------|
| MacResource | MAC/abhängig | vorhanden / teilweise | blockiert                     | vorhanden / vorhanden      | crates/tetra-pdus/src/umac/pdus/mac_resource.rs |
| MacSync     | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_sync.rs     |
| MacSysinfo  | MAC/abhängig | teilweise / vorhanden | offene Hinweise               | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_sysinfo.rs  |
| MacUBlck    | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | fehlt / nicht nachgewiesen | crates/tetra-pdus/src/umac/pdus/mac_u_blk.rs    |
| MacUSignal  | MAC/abhängig | vorhanden / vorhanden | ohne offensichtlichen Blocker | vorhanden / vorhanden      | crates/tetra-pdus/src/umac/pdus/mac_u_signal.rs |

**Review.** Für auffällige Einträge den Decoder und Encoder mit gültigem, verkürztem und ungültigem PDU-Puffer prüfen; danach den realen Runtime-Aufrufpfad und mindestens einen Endgerätetest nachweisen. Zeitstempel, Hersteller/Firmware, Rohpuffer und Logs im Testfall referenzieren.

## 21.2 SAP-Primitiven und Verdrahtung

SapMsgInner-Mitgliedschaft und statische Erzeuger-/Handler-Treffer sind Wegweiser im Code. nicht verdrahtet bedeutet, dass keine direkte Variante im untersuchten Router erscheint; das ist kein Beweis, dass eine Anforderung generell fehlt. Jeder fachlich benötigte Pfad braucht zusätzlich einen Test für Erzeugung, Übergabe, Antwort und Fehlerbehandlung.

### CONTROL: 2 Primitive

| Primitive              | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status      | Definition                           |
|------------------------|-------------|---------------------|-------|------------------------|--------------------------------------|
| CmceSdsData            | ja          | 4 / 3               | 16    | bidirektional sichtbar | crates/tetra-saps/src/control/sds.rs |
| MmSubscriberU<br>pdate | ja          | 16 / 5              | 12    | bidirektional sichtbar | crates/tetra-saps/src/               |

| Primitive | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status | Definition          |
|-----------|-------------|---------------------|-------|-------------------|---------------------|
|           |             |                     |       |                   | control/<br>brew.rs |

## LCMC: 21 Primitive

| Primitive            | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                | Definition                        |
|----------------------|-------------|---------------------|-------|----------------------------------|-----------------------------------|
| LcmcMleActivityReq   | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleBreakInd      | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleBusyInd       | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleCancelReq     | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleCloseInd      | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleConfigureInd  | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleConfigureReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleDisableInd    | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleEnableInd     | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleIdentitiesReq | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleIdleInd       | nein        | 0 / 0               | 0     | nicht in SapMsgInnner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleInfoInd       | nein        | 0 / 0               | 0     | nicht in SapMsgInnner            | crates/tetra-saps/src/            |

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                        |
|--------------------|-------------|---------------------|-------|---------------------------------|-----------------------------------|
|                    |             |                     |       | verdrahtet                      | lcmc/mod.rs                       |
| LcmcMleOpenInd     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleReopenInd   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleReportInd   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleRestoreConf | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleRestoreInd  | ja          | 1 / 2               | 1     | bidirektional sichtbar          | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleRestoreReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleResumeInd   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleUnitdataInd | ja          | 3 / 27              | 26    | bidirektional sichtbar          | crates/tetra-saps/src/lcmc/mod.rs |
| LcmcMleUnitdataReq | ja          | 18 / 6              | 22    | bidirektional sichtbar          | crates/tetra-saps/src/lcmc/mod.rs |

**LMM: 25 Primitive**

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                       |
|--------------------|-------------|---------------------|-------|---------------------------------|----------------------------------|
| LmmMleActivateConf | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleActivateInd  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleActivateReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInner            | crates/tetra-saps/src/           |

| Primitive           | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                       |
|---------------------|-------------|---------------------|-------|---------------------------------|----------------------------------|
|                     |             |                     |       | verdrahtet                      | lmm/mod.rs                       |
| LmmMleActivityReq   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleBusyReq       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleCancelReq     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleCloseReq      | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleConfigureInd  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleConfigureReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleDeactivateReq | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleDisableReq    | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleEnableReq     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleIdentitiesReq | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleIdleReq       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleInfoInd       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleInfoReq       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |

| Primitive             | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                       |
|-----------------------|-------------|---------------------|-------|---------------------------------|----------------------------------|
| LmmMleLinkInd         | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleLinkReq         | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMlePrepare Confirm | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMlePrepare Ind     | ja          | 1 / 1               | 3     | bidirektional sichtbar          | crates/tetra-saps/src/lmm/mod.rs |
| LmmMlePrepare Req     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleReportInd       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleUnitdataInd     | ja          | 3 / 10              | 9     | bidirektional sichtbar          | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleUnitdataReq     | ja          | 10 / 4              | 11    | bidirektional sichtbar          | crates/tetra-saps/src/lmm/mod.rs |
| LmmMleUpdateReq       | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/lmm/mod.rs |

## LTPD: 27 Primitive

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status            | Definition                        |
|--------------------|-------------|---------------------|-------|------------------------------|-----------------------------------|
| LtpdMleActivityReq | ja          | 0 / 1               | 4     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleBreakInd    | ja          | 0 / 2               | 1     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleBusyInd     | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleCancel      | ja          | 0 / 3               | 3     | nur                          | crates/tetra-                     |

| Primitive             | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                      | Definition                            |
|-----------------------|-------------|---------------------|-------|----------------------------------------|---------------------------------------|
| Req                   |             |                     |       | Empfang/Handler sichtbar               | saps/src/<br>ltpd/mod.rs              |
| LtpdMleCloseInd       | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConfigureInd   | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConfigureReq   | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConnectConfirm | ja          | 0 / 3               | 1     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConnectInd     | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConnectReq     | ja          | 0 / 1               | 3     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleConnectResp    | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleDisableInd     | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleDisconnectInd  | ja          | 0 / 2               | 1     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleDisconnectReq  | ja          | 0 / 1               | 6     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleEnableInd      | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleIdleInd        | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/<br>ltpd/mod.rs |
| LtpdMleInfoInd        | ja          | 0 / 2               | 2     | nur Empfang/Handler                    | crates/tetra-saps/src/                |



| Primitive               | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                      | Definition                        |
|-------------------------|-------------|---------------------|-------|----------------------------------------|-----------------------------------|
|                         |             |                     |       | er sichtbar                            | ltpd/mod.rs                       |
| LtpdMleOpenInd          | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReceiveInd       | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReconnectConfirm | ja          | 0 / 3               | 2     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReconnectInd     | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReconnectReq     | ja          | 0 / 1               | 5     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReleaseReq       | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleReportInd        | ja          | 0 / 7               | 6     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleResumeInd        | ja          | 0 / 2               | 1     | nur Empfang/Handler sichtbar           | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleUnitdataInd      | ja          | 4 / 2               | 1     | bidirektional sichtbar                 | crates/tetra-saps/src/ltpd/mod.rs |
| LtpdMleUnitdataReq      | ja          | 1 / 7               | 3     | bidirektional sichtbar                 | crates/tetra-saps/src/ltpd/mod.rs |

## TLA: 14 Primitive

| Primitive     | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                       |
|---------------|-------------|---------------------|-------|---------------------------------|----------------------------------|
| TlCancelReq   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tla/mod.rs |
| TlConnectConf | nein        | 0 / 0               | 0     | nicht in SapMsgInner            | crates/tetra-saps/src/           |

| Primitive        | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                      | Definition                       |
|------------------|-------------|---------------------|-------|----------------------------------------|----------------------------------|
|                  |             |                     |       | verdrahtet                             | tla/mod.rs                       |
| TlConnectInd     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlConnectReq     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlConnectResp    | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlDisconnectConf | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlDisconnectInd  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlDisconnectReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlReceiveInd     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlReconnectReq   | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlReconnectResp  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlReleaseInd     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlReleaseReq     | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet        | crates/tetra-saps/src/tla/mod.rs |
| TlaTlReportInd   | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/tla/mod.rs |

## TLE: 5 Primitive

| Primitive       | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                       |
|-----------------|-------------|---------------------|-------|---------------------------------|----------------------------------|
| TleCancelReq    | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tle/mod.rs |
| TleReportInd    | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tle/mod.rs |
| TleUnitdataConf | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tle/mod.rs |
| TleUnitdataInd  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tle/mod.rs |
| TleUnitdataReq  | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tle/mod.rs |

## TLMB: 4 Primitive

| Primitive      | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status               | Definition                        |
|----------------|-------------|---------------------|-------|---------------------------------|-----------------------------------|
| TlmbSyncInd    | ja          | 1 / 2               | 0     | bidirektional sichtbar          | crates/tetra-saps/src/tlmb/mod.rs |
| TlmbSyncReq    | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tlmb/mod.rs |
| TlmbSysinfoInd | ja          | 1 / 2               | 0     | bidirektional sichtbar          | crates/tetra-saps/src/tlmb/mod.rs |
| TlmbSysinfoReq | nein        | 0 / 0               | 0     | nicht in SapMsgInner verdrahtet | crates/tetra-saps/src/tlmb/mod.rs |

## TLMC: 18 Primitive

| Primitive             | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status            | Definition                        |
|-----------------------|-------------|---------------------|-------|------------------------------|-----------------------------------|
| TlmcAssessmentInd     | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcAssessmentListReq | ja          | 0 / 1               | 0     | nur Empfang/Handler          | crates/tetra-saps/src/            |

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status            | Definition                        |
|--------------------|-------------|---------------------|-------|------------------------------|-----------------------------------|
|                    |             |                     |       | er sichtbar                  | tlmc/mod.rs                       |
| TlmcCellReadConf   | ja          | 0 / 4               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcCellReadReq    | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcConfigureConf  | ja          | 0 / 4               | 1     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcConfigureInd   | ja          | 0 / 4               | 11    | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcConfigureReq   | ja          | 1 / 4               | 10    | bidirektional sichtbar       | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcMeasurementInd | ja          | 0 / 3               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcMonitorInd     | ja          | 0 / 3               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcMonitorListReq | ja          | 0 / 1               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcReportInd      | ja          | 0 / 6               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcScanConf       | ja          | 0 / 4               | 2     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcScanReportInd  | ja          | 0 / 2               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcScanReq        | ja          | 0 / 1               | 5     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |
| TlmcSelectConf     | ja          | 0 / 4               | 0     | nur Empfang/Handler sichtbar | crates/tetra-saps/src/tlmc/mod.rs |

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                | Definition                                |
|--------------------|-------------|---------------------|-------|----------------------------------|-------------------------------------------|
| TlmcSelectInd      | ja          | 0 / 2               | 0     | nur Empfang/Handl<br>er sichtbar | crates/tetra-<br>saps/src/<br>tlmc/mod.rs |
| TlmcSelectReq      | ja          | 0 / 1               | 0     | nur Empfang/Handl<br>er sichtbar | crates/tetra-<br>saps/src/<br>tlmc/mod.rs |
| TlmcSelectRes<br>p | ja          | 0 / 1               | 0     | nur Empfang/Handl<br>er sichtbar | crates/tetra-<br>saps/src/<br>tlmc/mod.rs |

**TMA: 5 Primitive**

| Primitive          | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                     | Definition                               |
|--------------------|-------------|---------------------|-------|---------------------------------------|------------------------------------------|
| TmaCancelReq       | nein        | 0 / 0               | 0     | nicht in<br>SapMsgInner<br>verdrahtet | crates/tetra-<br>saps/src/<br>tma/mod.rs |
| TmaReleaseInd      | nein        | 0 / 0               | 0     | nicht in<br>SapMsgInner<br>verdrahtet | crates/tetra-<br>saps/src/<br>tma/mod.rs |
| TmaReportInd       | ja          | 1 / 1               | 0     | bidirektional<br>sichtbar             | crates/tetra-<br>saps/src/<br>tma/mod.rs |
| TmaUnitdataIn<br>d | ja          | 8 / 3               | 8     | bidirektional<br>sichtbar             | crates/tetra-<br>saps/src/<br>tma/mod.rs |
| TmaUnitdataRe<br>q | ja          | 5 / 2               | 11    | bidirektional<br>sichtbar             | crates/tetra-<br>saps/src/<br>tma/mod.rs |

**TMD: 2 Primitive**

| Primitive             | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status         | Definition                               |
|-----------------------|-------------|---------------------|-------|---------------------------|------------------------------------------|
| TmdCircuitDat<br>aInd | ja          | 5 / 5               | 0     | bidirektional<br>sichtbar | crates/tetra-<br>saps/src/<br>tmd/mod.rs |
| TmdCircuitDat<br>aReq | ja          | 4 / 1               | 3     | bidirektional<br>sichtbar | crates/tetra-<br>saps/src/<br>tmd/mod.rs |

## TMV: 4 Primitive

| Primitive        | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                      | Definition                       |
|------------------|-------------|---------------------|-------|----------------------------------------|----------------------------------|
| TmvConfigureConf | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/tmv/mod.rs |
| TmvConfigureReq  | ja          | 6 / 4               | 1     | bidirektional sichtbar                 | crates/tetra-saps/src/tmv/mod.rs |
| TmvUnitdataInd   | ja          | 3 / 27              | 29    | bidirektional sichtbar                 | crates/tetra-saps/src/tmv/mod.rs |
| TmvUnitdataReq   | ja          | 1 / 3               | 2     | bidirektional sichtbar                 | crates/tetra-saps/src/tmv/mod.rs |

## TNMM: 2 Primitive

| Primitive        | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status                      | Definition                        |
|------------------|-------------|---------------------|-------|----------------------------------------|-----------------------------------|
| TnmmTestDemand   | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/tnmm/mod.rs |
| TnmmTestResponse | ja          | 0 / 0               | 0     | Variante vorhanden, Runtime-Pfad fehlt | crates/tetra-saps/src/tnmm/mod.rs |

## TP: 1 Primitive

| Primitive     | Verdrahtung | Erzeugt / behandelt | Tests | Statischer Status      | Definition                      |
|---------------|-------------|---------------------|-------|------------------------|---------------------------------|
| TpUnitdataInd | ja          | 1 / 2               | 0     | bidirektional sichtbar | crates/tetra-saps/src/tp/mod.rs |

## 21.3 Explizite State-Typen

Die Suche erfasst Rust-Enums mit `state` im Namen. Sie übersieht implizite Zustände in Maps, Booleans und Ablaufreihenfolgen. Ein hoher Variantenwert sagt nichts über Vollständigkeit der Transitionen.

| Schicht | Typ           | Varianten | Testtreffer | Quellpfad     |
|---------|---------------|-----------|-------------|---------------|
| CMCE    | CcFormalState | 6         | 0           | crates/tetra- |

| Schicht          | Typ                  | Varianten | Testtreffer | Quellpfad                                                     |
|------------------|----------------------|-----------|-------------|---------------------------------------------------------------|
|                  |                      |           |             | entities/src/cmce/subentities/cc_bs/state/mod.rs              |
| CMCE             | GroupCallState       | 2         | 0           | crates/tetra-entities/src/cmce/subentities/cc_bs/state/mod.rs |
| CMCE             | IndividualCallState  | 5         | 0           | crates/tetra-entities/src/cmce/subentities/cc_bs/state/mod.rs |
| MM               | MmClientState        | 3         | 4           | crates/tetra-entities/src/mm/components/client_state.rs       |
| NET_ASTERISK     | DialogState          | 4         | 0           | crates/tetra-entities/src/net_asterisk/entity.rs              |
| NET_AUDIO_PLAYER | AudioPlayerState     | 7         | 0           | crates/tetra-entities/src/net_audio_player/types.rs           |
| NET_CONTROL      | MobilityClientState  | 3         | 0           | crates/tetra-entities/src/net_control/commands.rs             |
| NET_ECHOLINK     | QsoState             | 3         | 0           | crates/tetra-entities/src/net_echolink/mod.rs                 |
| NET_TTS          | TtsState             | 6         | 0           | crates/tetra-entities/src/net_tts/types.rs                    |
| PHY              | SynthesisBufferState | 3         | 0           | crates/tetra-entities/src/phy/components/fcb.rs               |

| Schicht    | Typ                | Varianten | Testtreffer | Quellpfad                                        |
|------------|--------------------|-----------|-------------|--------------------------------------------------|
| SAP/COMMON | ChannelChangeState | 6         | 0           | crates/tetra-saps/src/common/mod.rs              |
| SAP/COMMON | LtpdLinkState      | 10        | 6           | crates/tetra-saps/src/common/mod.rs              |
| SAP/COMMON | MleCellState       | 10        | 0           | crates/tetra-saps/src/common/mod.rs              |
| SAP/COMMON | TlmcScanState      | 5         | 3           | crates/tetra-saps/src/common/mod.rs              |
| SAP/COMMON | TlmcSelectionState | 5         | 0           | crates/tetra-saps/src/common/mod.rs              |
| SAP/COMMON | TxGrantState       | 2         | 0           | crates/tetra-saps/src/common/mod.rs              |
| SNDCP      | PdpState           | 4         | 0           | crates/tetra-entities/src/sndcp/state.rs         |
| UMAC       | DefragBufferState  | 3         | 0           | crates/tetra-entities/src/umac/subcomp/defrag.rs |

**Transitionsprüfung.** Für jede produktiv verwendete Zustandsmaschine Startzustand, erlaubte Trigger, Timeouts, Duplicate-Events, Restart-Persistenz und Fehlerzustand als Testfall erfassen. Beispielhaft bei SIP die Folge CENTRAL\_ACTIVE → FAILOVER\_PENDING → PBX\_DIRECT\_ACTIVE → RECOVERY\_PENDING und zurück mit getrennten Registrierungsbelegen prüfen; bei Edge Fallback online, degraded, isolated und recovering anhand der Matrix-Lease verifizieren.

## 21.4 Priorisierte technische Lücken

| Kategorie          | Treffer |
|--------------------|---------|
| TODO/FIXME         | 206     |
| Todo-Typ           | 240     |
| panic!             | 89      |
| unimplemented!     | 45      |
| unimplemented_log! | 107     |



| Kategorie    | Treffer |
|--------------|---------|
| unreachable! | 33      |

1. aktive `unimplemented!-/todo!`-Pfade in TLMC, TLPD, MLE und den zugehörigen PDU-Codecs entfernen;
2. nicht in `SapMsgInner` verdrahtete TLMC-/TLPD-Primitive typisieren und routen;
3. PDU-Parser mit Runtime-Panics vor Fuzzing und On-Air-Eingaben schützen;
4. für alle bereits produktiv verwendeten PDUs Golden Vectors ergänzen;
5. verbleibende `unimplemented_log!`-Pfade nach Schicht und Roadmap-Phase abarbeiten.

Diese Zählungen enthalten Kommentare, Testcode und nicht zwingend erreichbare Pfade. Priorität haben aus On-Air-Eingaben erreichbare Panics und Parserplatzhalter, danach fehlende SAP-Routen und Golden Vectors. Eine reine Dokumentations-TODO ist anders zu bewerten. Den Matrixstatus erst nach Negativtest und Funkprobe anheben.

## 22 Integrationstests und On-Air-Abnahme

Es gibt drei unterschiedliche Beweisarten: statische Repository-Prüfung, Mock-/LXC-Integrationstest und reale Funkmessung. Ein Erfolg in einer Stufe ersetzt die anderen nicht. Testdaten sind ausdrücklich gekennzeichnet und nach einem Lauf wieder zu entfernen. Für Restart-/Fault-Tests werden ein entbehrliches Labor, freier Speicher, Zugriff auf Units und ein geplanter Wiederanlauf vorausgesetzt.

### 22.1 Szenario `contracts`

**Ziel.** gemeinsame Health-, Status-, Metrics- und OpenAPI-Verträge. Quellfunktion `scenario_contracts` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** jeden Host des Inventories direkt anfragen; Abweichungen am laufenden Build prüfen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

### 22.2 Szenario `node_gateway`

**Ziel.** TBS-WebSocket, Capabilities und Gateway-Kommandos. Quellfunktion `scenario_node_gateway` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Mock-TBS anmelden, Ping/Ack und Knotensicht korrelieren. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

### 22.3 Szenario `edge_fallback_contract`

**Ziel.** Lease, Status und lokale Rückfallentscheidung. Quellfunktion `scenario_edge_fallback_contract` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Matrixrevision und `/api/edge-fallback` bei kontrollierter Degradation prüfen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.4 Szenario `subscriber_group`

**Ziel.** Teilnehmer, Gruppen, Zulassung und Affiliation. Quellfunktion `scenario_subscriber_group` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** markierte Test-ISSI/GSSI anlegen, zulassen/sperrern und aufräumen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.5 Szenario `call_media_recorder`

**Ziel.** Ruf, Floor, Medien und Aufnahme. Quellfunktion `scenario_call_media_recorder` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Call-ID durch Aufbau, Wechsel, Frames und Assetintegrität verfolgen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.6 Szenario `sds`

**Ziel.** Einzelzustellung und Store-and-forward. Quellfunktion `scenario_sds` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Offline-Ziel, Replay, Ack und Deduplizierung prüfen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.7 Szenario `packet_data`

**Ziel.** PDP, IPv4 und IP Gateway. Quellfunktion `scenario_packet_data` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Kontext, Lease, Uplink/Downlink und Release nachweisen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.8 Szenario `observability`

**Ziel.** Metriken, Logs und Traces. Quellfunktion `scenario_observability` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** markierte Correlation ID im zentralen Store suchen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.9 Szenario `control_room_federation`

**Ziel.** aggregierte Lage und Aktualität. Quellfunktion `scenario_control_room_federation` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Origin-Status und Pollzeit im Control Room mit Fachkern vergleichen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.10 Szenario `platform_services`

**Ziel.** weitere Plattformdienste. Quellfunktion `scenario_platform_services` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Management- und Metadatenverträge ohne Funkbeweis prüfen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

### 22.11 Szenario `restart_restore`

**Ziel.** Persistenz nach Unit-Neustart. Quellfunktion `scenario_restart_restore` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** vorher/nachher State und fachliche Konsistenz vergleichen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

### 22.12 Szenario `fault_matrix`

**Ziel.** abhängige Dienste bei Kernausfall. Quellfunktion `scenario_fault_matrix` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Opfer stoppen, Degradation und Recovery in beiden Diensten nachweisen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

### 22.13 Szenario `edge_service_outages`

**Ziel.** TBS-Sicht bei einzelnen Core-Ausfällen. Quellfunktion `scenario_edge_service_outages` in `tests/e2e/netcore_e2e/scenarios.py`; der tatsächliche Runner entscheidet nach Profil, Voraussetzungen und Skip-Regeln, ob es ausgeführt wird.

**Durchführung.** Matrix-Lease, Dienststatus und lokales Weiterarbeiten vergleichen. Vor dem Lauf Inventory, Softwarestände, DNS/Route und Test-ID festhalten. Während des Laufs `report.json`, `junit.xml`, `summary.txt` und die Journale der beteiligten Dienste unter derselben UTC-Zeit sichern.

**Bestehen.** Kein fehlgeschlagener Check, keine unerkannte Skip-Bedingung und keine Test-Fixture nach der Bereinigung. Ein Mock bestätigt die Protokollverträge des Core, nicht den Funkpfad. Bei Fehler zuerst den einzelnen Check mit Dauer/Evidenz im Report lesen, dann die erste fehlerhafte Dienstgrenze isolieren.

## 22.14 Testprofile und Kommandos

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile full --validate-only
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile smoke
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile full --allow-mutations
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml test --
profile fault --allow-mutations --allow-restarts
```

Das eingespielte Inventory muss die reale Adressierung enthalten. Das ältere Docs/OPEN\_LAB\_E2E\_RUNBOOK.md nennt noch 17 Dienste; das aktuelle Inventory umfasst 24. Der Runner und seine Profile sind am aktuellen Code zu prüfen. Das Fault-Profil kann Units stoppen und verlangt einen expliziten Wiederanlauf-Check. Nach einem abgebrochenen Lauf alle betroffenen Services mit `systemctl` kontrollieren.

## 22.15 Funkabnahme mit echten Geräten

Ein Lauf mit echten Funkgeräten erhält ein eigenes Protokoll pro Hersteller, Modell, Firmware und Codeplug. Mindestens zwei Geräte und für Interoperabilität zwei Hersteller prüfen: Zellwahl/Registrierung, Gruppen-PTT mit Sprecherwechsel, Individualruf, SDS, Paketdaten, Handover und Rückkehr nach Core-Ausfall. Pro Fall Start-/Endzeit in UTC, Träger, MCC/MNC/LAC, ISSI/GSSI, beobachtete Funkwirkung sowie Log-/PCAP-/Audio-/Screenshot-Referenzen sichern. Die Vorlage `tests/e2e/on_air_template.json` und der Validator `tests/e2e/validate_on_air_evidence.py --require-complete --require-two-vendors` prüfen Vollständigkeit der Evidenz, nicht die physikalische Messqualität.

Eine durchgehende Testkette lautet: TBS im isolierten Testaufbau starten; ein Gerät anmelden; Gruppenruf A→B und B→A führen; SDS mit Acks zustellen; PDP öffnen und IP-Rückweg messen; Core-Dienst kontrolliert unterbrechen; lokale Funktion und Replay beobachten; Core wiederherstellen; denselben Ruf/SDS/PDP-Pfad erneut prüfen. Jede Abweichung erhält Issue, betroffene Softwareversion und reproduzierbare Schritte.

## 23 Wartung, Änderungsnachweis und Quellnavigation

Das Repository enthält nebeneinander historische Migrationsdokumente, aktive Beispielkonfigurationen und Quellcode. Für die tatsächliche Installation haben ausgerollte Konfiguration, laufendes Binary und Logs Vorrang; für den Sollstand der Ausgabe der gepinnte Commit. Eine Anleitung mit alten IPs oder nur 17 Diensten wird nicht unbesehen auf die 25er-Topologie übertragen.

### 23.1 Regelmäßige Wartung

| Rhythmus          | Kontrolle                                                                  | Nachweis / Maßnahme                                            |
|-------------------|----------------------------------------------------------------------------|----------------------------------------------------------------|
| Täglich im Labor  | TBS-Zelle, Gateway-Health-Matrix, freie Disk, Journale, Clock              | Zeitstempel und offene Alarmer dokumentieren                   |
| Wöchentlich       | Backup/Restore-Probe eines Fachdienstes, Spool- und Recorder-Retention     | Datei, Prüfsumme, Probe-Import und Verantwortlichen festhalten |
| Monatlich         | SIP-Failover, Broker-Reconnect, Packet-Data-Rückweg, lokale Edge-Autonomie | Vorher/Nachher-Status und E2E-Report archivieren               |
| Vor jedem Update  | Commit, Inventory, TOML-Diff, Installer, Schema, Migration                 | Rollback-Paket und Wartungsfenster vorbereiten                 |
| Nach jedem Update | Health, Fachpfade, On-Air-Stichprobe, Backup-Kompatibilität                | Report und Freigabe mit Version/UTC                            |

### 23.2 Reparaturprotokoll

Ein Incident-Eintrag enthält: UTC-Zeit und Zeitzone; betroffene Zelle/Dienst/Host; Software-Commit und Feature-Build; Symptom und erste fehlerhafte Grenze; Rohbeleg (Log, Messung oder Report); getestete Hypothese; **genau eine** Änderung; Vorher/Nachher-Evidenz; Nebenwirkung; Rollback-Pfad; abschließende Prüfung mit einem echten Endgerät, wenn Funk betroffen war. Secrets, Schlüssel und persönliche Kennungen werden vor Weitergabe entfernt. Für wiederkehrende Störungen eine Reproduktion und einen Regressionstest im Repository anlegen.

### 23.3 Pfadweiser Quellenindex

| Quelle                                        | Wofür zuerst lesen                                               |
|-----------------------------------------------|------------------------------------------------------------------|
| deploy/open-lab/inventory.example.toml        | 25 Dienste, Hosts, Ports, Abhängigkeiten, Installer und Units    |
| deploy/open-lab/netcore-deploy.py             | Validieren, Rendern, Anwenden, Status und E2E-Profil             |
| system-backend/<dienst>/config/*.example.toml | Backend-Beispielkonfiguration; nicht automatisch Runtime-Default |

| Quelle                                              | Wofür zuerst lesen                                            |
|-----------------------------------------------------|---------------------------------------------------------------|
| system-backend/<dienst>/src/                        | Route, Serialisierung, Fachlogik und tatsächliche Validierung |
| Docs/<br>basisstation.config.sanitized.example.toml | bereinigte TBS-Beispielwerte und optionale Kommentarzeilen    |
| crates/tetra-config/src/bluestation/                | TBS-Parser und Typen                                          |
| crates/tetra-pdus/src/                              | PDU-Decoder/-Encoder und Feldlogik                            |
| crates/tetra-saps/src/                              | SAP-Primitiven und Routing                                    |
| crates/tetra-entities/src/                          | Funk-/Netz-Entities und Zustandsautomaten                     |
| tests/e2e/                                          | Mock-Verträge, Profile, Resultate und On-Air-Evidenz          |
| Docs/ETSI_SOURCE_REGISTER.md                        | Projektzuordnung normativer Referenzfassungen                 |
| Docs/IMPLEMENTATION_GAPS.md                         | statische technische Risiken, jeweils im Code verifizieren    |

## 23.4 Datenmigration und Rücksicherung

Vor einem Versionswechsel State-Dateien des betroffenen Dienstes, Berechtigungen und Konfigurationsdatei gemeinsam sichern. Ein Restore wird zunächst in einem entbehrlichen LXC mit derselben Version geprobt; dort die Health-Proben und mindestens eine fachliche Read-/Write-/Replay-Runde fahren. Eine neuere Binärdatei kann ein Dateiformat ändern. Darum Rollback nur als konsistentes Set aus Binary, Konfiguration und Datenstand ausführen. TBS-Policy-Cache, Event-Spool, lokale Aufnahmen und freigegebene Medien sind getrennte Zustände; die Sicherung des zentralen Core ersetzt sie nicht.

## 23.5 Editionskontrolle

Für eine neue Handbuchausgabe Commit und Tags neu erfassen, Inventory gegen `services.toml` vergleichen, alle TOML- und OpenAPI-Kataloge erneut extrahieren, generierte PDU-/SAP-Matrizen aktualisieren und bekannte Abweichungen mit realen Testberichten bewerten. Anschließend Seitenzahlen, Querverweise, PDF-Textsuche und visuelle Renderkontrolle erneut durchführen. Ein Dokument mit 400 Seiten beweist keine Vollständigkeit durch Umfang; jede Seite muss beim Installieren, Bedienen, Prüfen oder Reparieren helfen.



## 24 Praxisabläufe: vom Aufbau bis zur Wiederherstellung

Die folgenden Abläufe verbinden die Einzelreferenzen zu einer Ende-zu-Ende-Anleitung. Sie sind als Arbeitsprotokolle gedacht: Der jeweilige Schritt ist erst erledigt, wenn die genannte Evidenz vorliegt. Die Beispieladressen des Inventories und die Funkwerte der bereinigten TBS-Vorlage sind Platzhalter. Vor einem realen Versuch werden Standort, Frequenzberechtigung, Hardwarepfad, Kennungen, Softwarestand und Rückfallkonfiguration festgehalten. Die Quelle für eine konkrete Option bleibt die im Kapitel 23 genannte Datei des gepinnten Commits.

### 24.1 Ein neues Open-Lab-System planen und aufbauen

Am Anfang steht ein Netzplan mit drei voneinander unterscheidbaren Wegen: Managementzugriff auf die Dienste, Steuerung zwischen TBS und Node Gateway und der eigentliche Medien- beziehungsweise Paketdatenweg. Ein Ping zwischen zwei Hosts belegt nur IP-Erreichbarkeit, nicht, dass der WebSocket-Pfad oder ein RTP-Port offen ist. Das Open-Lab-Inventory enthält 25 Backend-Dienste. Provisioning Core kommt als zusätzlicher Verwaltungsdienst hinzu und benötigt einen eigenen Host- und Portplan. Die ältere Installationsdokumentation nennt teils 17 Dienste oder Adressen im 10.0.1.0-Netz; für einen neuen Aufbau wird das tatsächliche Inventory mit dem Standortplan verbunden.

Zuerst einen Versionszettel anlegen: Repository-Commit, Cargo-Lockfile, Debian-/Kernelstand, Proxmox-Host, LXC-Template, SDR-Treiber und TBS-Binary. Dann die Namen/IPs aller Container in einer eigenen `inventory.toml` festlegen. Für jeden Dienst `host`, `port`, `config_target`, `unit`, `install` und `depends_on` prüfen. Keine zweite, ungepflegte Tabelle neben dem Inventory als alleinige Quelle betreiben. Die TBS-Adresse des Node Gateways muss auf denselben Host und den Pfad `/ws/node` zeigen; die Backend-Verbindungen nutzen ihren im TOML angegebenen Pfad. Die bekannte Beispielabweichung zwischen `Repository-Root-config.toml` und `Open-Lab-Inventory` wird vor einem Full-System-Audit bewusst aufgelöst.

Im Proxmox-Netz die Container mit festen Adressen oder reservierten Leases anlegen. Der IP Gateway benötigt die TUN-Voraussetzungen und seine Routing-/Firewall-Regeln; Recorder und Media Library erhalten die geplanten Speicherorte, wobei Livezustand nicht vom optionalen NFS-Archiv abhängen darf. Die Managementoberflächen sind laut Projekt im Open-Lab-Modus ohne Login, Token und TLS. Ein eigenes isoliertes VLAN und enges Routing sind deshalb Teil der Funktion, nicht eine spätere Komfortmaßnahme. Zeitquelle, DNS und MTU werden vor den Fachtests überprüft.

Den Deployment-Host mit schlüsselbasiertem SSH, Zugriff auf die Zielhosts und einem unveränderten Quellcheckout vorbereiten. Nacheinander `validate`, `plan`, `render` und `apply --dry-run` ausführen. Den gerenderten Zielzustand mit dem Netzplan vergleichen, besonders Loopback-Adressen in serviceübergreifenden URLs. Erst danach `apply` im Labor. Ein manueller Installer pro LXC ist möglich, aber der gleiche Konfigurationsstand und die Abhängigkeitsreihenfolge müssen erhalten bleiben. Provisioning Core wird mit seinem separaten Installer und seiner eigenen Unit ergänzt.

Nach jeder installierten Abhängigkeitsgruppe Liveness, Readiness und eine fachliche Lesefunktion testen. Eine 503 bei `ready` wird auf den konkreten Vorgänger zurückgeführt. Vor der TBS-Inbetriebnahme muss der Node Gateway mindestens seine Listener- und Core-Matrix liefern. Der erste Funkversuch erfolgt am vorbereiteten Messplatz; erst nach stabiler

Registrierung werden Ruf, SDS, Paketdaten und Integrationen eingeschaltet. Den Erfolg mit E2E-Report und On-Air-Evidenz festhalten.

| Schritt             | Abnahmebeleg                               | Falls er fehlt                                    |
|---------------------|--------------------------------------------|---------------------------------------------------|
| Inventory validiert | validate ohne Fehler, 25 Einträge          | Host, Port, Unit und depends_on korrigieren       |
| Render geprüft      | Ziel-TOMLs stimmen mit Netzplan überein    | URLs und Loopback-Verweise vor apply anpassen     |
| Core startbereit    | direkte live-/ready-Abfrage pro Dienst     | ersten unbereiten Vorgänger identifizieren        |
| Gateway verbunden   | TBS unter /api/v1/nodes, frische Matrix    | WebSocket-Route, Node-ID und Lease prüfen         |
| Funk nachgewiesen   | Registrierung und bidirektionaler Testcall | SDR, Luftschnittstelle und Policy getrennt prüfen |

## 24.2 SDR, Zelle und Dual-Carrier kontrolliert starten

Der Funkpfad beginnt nicht mit einem WebUI-Schalter, sondern mit der physischen Kette: SDR, Clock, Sende- und Empfangszweig, Duplexer/Filter, Last oder Antenne, Versorgung, Kühlung und Messgerät. Bei einer neuen Hardwarekombination wird zunächst ohne Aussendung geprüft, ob SoapySDR das erwartete Gerät, die Kanäle, Antennenanschlüsse und Gain-Elemente meldet. Die Konfiguration akzeptiert gerätespezifische `rx_gain_<element>`- und `tx_gain_<element>`-Werte; ein frei erfundenes Gain-Element oder ein Textwert kann den Start abbrechen. Für einen ersten Messlauf werden konservative Pegel verwendet und erst nach Messung erhöht.

Die Beispielvorgabe koppelt `phy_io.soapysdr.tx_freq` und `rx_freq` an die Zellparameter `main_carrier`, `secondary_carrier`, Band, Duplexabstand und Offset. Bei einem einzelnen Träger stimmt die SDR-Abstimmung direkt mit dem Kanal überein. Bei benachbarten Trägern sind Center Frequencies und Sample Rate so zu wählen, dass beide 25-kHz-Kanäle im SDR-Passband liegen. Eine falsch gewählte Mitte kann einen Träger scheinbar funktionsfähig lassen und den zweiten durch Filterung, Verzerrung oder Timingfehler beeinträchtigen. Deshalb jeden Träger getrennt im Spektrum und mit einem Endgerät testen; eine abstrakte Konfigurationsprüfung reicht nicht.

Die Zellidentität wird aus `net_info` und `cell_info` gebildet. MCC, MNC, LAC, Colour Code und Funkgeräteprogrammierung müssen dieselbe Zelle meinen. Die Parameter eines bereits laufenden Netzes dürfen nicht nur auf der TBS geändert werden. Vor dem ersten TX Frequenznutzung und Duplexrichtung mit der Genehmigung vergleichen. Beim Test am Dummyload Trägerfrequenz, belegte Bandbreite, spektrale Nebenprodukte, Timing und Vor-/Rücklauf dokumentieren. Danach eine definierte Endgeräteprobe: Zelle finden, registrieren, Daten- und Sprachpfad, zweite Trägerbelegung, Re-Registration nach Unterbrechung.

Bei Fehlern zuerst die Schicht des ersten Ausfalls bestimmen. Sieht das Gerät keine Zelle, ist PHY/Broadcast vorrangig. Sieht es die Zelle, aber der Uplink erscheint nicht, sind Empfangsfrequenz, Antennen-/Duplexpfad und Timing zu prüfen. Wird ein Location Update sichtbar und abgewiesen, folgen Identität und Teilnehmerpolicy. Ein funktionierender

Downlink beweist keinen funktionierenden Uplink. Die TBS-Protokolle, SDR-Probe und Messbild erhalten denselben UTC-Zeitraum.

| Beobachtung             | Nächste Messung                     | Nicht daraus folgern                  |
|-------------------------|-------------------------------------|---------------------------------------|
| SDR-Probe okay          | TX/RX-Pegel und Frequenz unter Last | dass ein Gerät die Zelle decodiert    |
| Träger sichtbar         | Broadcast- und Zellinformation      | dass der Uplink empfangen wird        |
| Gerät zeigt Zelle       | Location Update im TBS-Log          | dass Subscriber-Policy greift         |
| ein Träger funktioniert | zweiter Träger separat mit Last     | dass Dual-Carrier vollständig ist     |
| Registrierung stabil    | Ruf, SDS und Paketdaten             | dass alle Fachdienste integriert sind |

### 24.3 Teilnehmer und Gruppen provisionieren

Teilnehmeridentität, physisches Gerät und aktuelle Serving-Zelle sind drei verschiedene Datenbereiche. Subscriber Core führt die Freigabe der ISSI und ihre Dienstberechtigungen. Group Core hält GSSI, Mitgliedschaften, Affiliation und DGNA-Policy. Asset Management verwaltet Inventarnummer, Seriennummer, Firmware und Ausgabe, ohne die Funkzulassung zu ersetzen. Mobility Core beobachtet, welche TBS den Teilnehmer aktuell bedient. Provisioning Core stellt für Teilnehmer und Gruppen eine gemeinsame Verwaltungsoberfläche bereit, liegt aber außerhalb des 25er-Deploy-Inventories.

Für eine neue Testkennung zuerst ISSI/GSSI-Bereich, Eigentümer und beabsichtigte Funktion dokumentieren. Vor Änderungen einen Export der bisherigen Subscriber- und Group-Daten anlegen. Dann Teilnehmerprofil mit zulässigen Diensten anlegen und die benötigte Gruppe hinzufügen. Die Gruppe bekommt Mitgliedschaft und ggf. eine Policy für Attach, Ruf, SDS, Notruf oder DGNA. Ein Gruppenname im Directory ist eine Anzeigehilfe; die GSSI und ihre Autorität kommen aus Group Core. Nach der Verwaltungsaktion müssen Synchronisationsstand und TBS-Policyrevision überprüft werden. Ein HTTP-Erfolg beim Schreiben beweist nicht, dass eine bereits verbundene TBS die Änderung übernommen hat.

Den Test bewusst positiv und negativ durchführen: zugelassene ISSI registriert sich und darf in einer zugelassenen Gruppe senden; eine gesperrte Testkennung wird abgewiesen; eine nicht zugeordnete GSSI darf den vorgesehenen Ruf nicht auslösen. Wiederholte Änderungen dürfen keine doppelten Mitgliedschaften erzeugen. Nach einem Core-Ausfall überprüft man den Last-known-Policy-Cache der TBS und die lokale Reaktion. Beim Wiederanlauf wird nicht nur ready beobachtet, sondern die Revision der wiederhergestellten Policy und der tatsächliche Gruppenruf.

Beim Löschen eines Teilnehmers zuerst offene Aufgaben, Assets, Gruppenmitgliedschaften, aktive Calls und Spoolnachrichten prüfen. Ein Provisioning-Vorgang kann über zwei autoritative Dienste verteilt sein; bei Teilerfolg wird der Zustand in beiden Quellen verglichen, bevor derselbe Schreibvorgang wiederholt wird. Eine pauschale Neuinitialisierung der TBS kann lokale Calls oder SDS unterbrechen und ist zur Bereinigung einer einzelnen ISSI ungeeignet. Im Abschlussprotokoll stehen Test-ISSI/GSSI, Änderungsrevision, Ziel-TBS und Beleg der Bereinigung.

| Domäne                  | Autorität         | Beleg nach einer Änderung            |
|-------------------------|-------------------|--------------------------------------|
| ISSI und Zulassung      | Subscriber Core   | Profil, Sperrstatus, Sync-Revision   |
| GSSI und Mitgliedschaft | Group Core        | Gruppe, Zuordnung, Affiliation       |
| gemeinsame Bedienung    | Provisioning Core | konsistenter Zustand in beiden Cores |
| physisches Gerät        | Asset Management  | Inventarnummer und Ausgabeakte       |
| aktuelle Zelle          | Mobility Core     | Serving Node der betreffenden ISSI   |

#### 24.4 Einen Gruppenruf über zwei Basisstationen verfolgen

Ein Gruppenruf ist ein besonders guter Gesamttest, weil mehrere Grenzen beteiligt sind: Endgerät A, TBS A, Gruppenpolicy, Call Control, Media Switch, TBS B und Endgerät B. Zuerst beide Geräte einzeln registrieren und die jeweilige Serving-Zelle festhalten. Dann GSSI und Mitgliedschaft auf beiden Seiten lesen. Erst wenn die Policy konsistent ist, den Ruf mit einer eindeutigen Call-ID aufbauen. Die Zeitpunkte von Setup, Alert/Connect, Floor-Zuteilung, Frames, Sprecherwechsel und Release bilden die Testspur.

Die erste Probe beginnt mit A als Sprecher. Am Funkgerät B muss ein verständlicher Anfang und ein vollständiges Ende hörbar sein. Danach Floor freigeben und B sprechen lassen. Der Umkehrweg prüft einen anderen Satz von Media Legs und die Rückrichtung der Luftschnittstelle. Parallel die Zähler des Media Switch für Eingangsframes, Ausgangsframes, Drops und Jitter lesen. Ein Call-Objekt ohne RouteReady oder ohne Framefluss erklärt, warum eine UI einen aufgebauten Ruf zeigt, aber kein Ton ankommt. Ein Framezähler wiederum belegt nicht automatisch hörbares und korrekt codiertes Audio; die Aufzeichnung oder Endgeräteprobe ist erforderlich.

Beim Release sollen beide TBS die Ressource freigeben. Ein hängen gebliebener Floor, ein nicht aufgelöstes Leg oder eine noch aktive Medienroute werden gesondert protokolliert. Nach einem kontrollierten Ausfall von Call Control bleibt ein lokaler Ruf innerhalb einer Zelle nach der Edge-Fallback-Spezifikation möglich; netzweite Vermittlung ist eine andere Fähigkeit. Wird nur Media Switch unterbrochen, muss die lokale Luftschnittstelle weiter bewertet werden, während zentrale Frames fehlen. Das Ergebnis wird nicht allein aus der Health-Matrix abgeleitet.

Für eine Interoperabilitätsprobe zwei Gerätehersteller mit Firmwarestand dokumentieren. Wiederhole Setup/Release bei kurzem Tastendruck, schnellem Sprecherwechsel, Re-Registration und nach Wiederkehr des Core. In jedem Lauf nur einen Störfaktor verändern. Roh- und dekodierte PDU-/SAP-Spuren können eine Fehlinterpretation von Floor- oder Release-Zuständen aufklären; sie ersetzen nicht den hörbaren Test. Ein fehlgeschlagener Call erhält exakt die erste abweichende Grenze und einen reproduzierbaren Zeitstempel.

| Übergang         | Erwartete Evidenz           | Typische Eingrenzung       |
|------------------|-----------------------------|----------------------------|
| A fordert Ruf an | TBS-CMCE und Call-ID        | Zulassung, GSSI, Timeslot  |
| Core bildet Legs | Call Control und RouteReady | Serving Node, Media Switch |

| Übergang          | Erwartete Evidenz               | Typische Eingrenzung            |
|-------------------|---------------------------------|---------------------------------|
|                   |                                 |                                 |
| A spricht zu B    | Framezähler und hörbarer Inhalt | Codec, Route, Jitter, RF        |
| B übernimmt Floor | Freigabe A, Grant B             | Floor-Zustand, Race/Timer       |
| Ruf endet         | Release auf beiden Zellen       | verwaiste Session und Ressource |

## 24.5 Individualruf und SIP/PBX-Fallback abnehmen

Bei Phase 11c bleibt der lokale Asterisk der TBS eine stabile Edge-B2BUA. Die native TBS-Bridge spricht lokal mit ihm; der Asterisk registriert sich im Normalzustand zum zentralen SIP Switch, der zum vorhandenen PBX vermittelt. Die direkte PBX-Registrierung ist der bestätigte Ausfallpfad. Für jeden TBS-Endpunkt wird ein eindeutiger Alias im SIP Switch gebraucht, wenn die Kennung des zentralen Dienstes und die lokale Basisstations-ID unterschiedlich geschrieben sind. Die Zuordnung wird explizit konfiguriert, nicht durch pauschales Ersetzen von Zeichen erraten.

Zuerst den Normalweg testen: Funkgerät ruft PBX-Ziel, PBX ruft ISSI zurück, beide Seiten hören verständliche Sprache, DTMF und Anruferkennung werden geprüft, Release räumt Dialog und RTP auf. Signalisierung und Medien getrennt messen. SIP-200 oder ein Klingelereignis sagt noch nichts über RTP, SDP-Adresse, Codec oder den UDP-Portbereich. Für die Anruferkennung ist P-Asserted-Identity relevant: Kontoname des Trunks und tatsächlich anrufende ISSI sind verschieden. Der lokale Asterisk muss die eingehende Identität vertrauen und weitergeben, sofern die dokumentierte Topologie dies verlangt.

Bei eingehenden PBX-Zielen sind 5102, T5102 und t5102 als Testvarianten im SIP-README genannt. Der optionale T-Marker wird vor den numerischen Prefix-Regeln entfernt. Eine ungültige Zeichenfolge wird nicht still in eine andere ISSI umgewandelt. Wird ein Ruf schon im Asterisk-Context abgewiesen, kann der zentrale Resolve-Zähler leer bleiben; die Diagnose beginnt dann im Dialplan/AGI-Pfad und nicht an der Funkzelle. Nach einer Korrektur die betroffene Konfiguration neu laden und die drei gültigen sowie mindestens eine ungültige Variante testen.

Für den Failover-Test den zentralen Switch im entbehrlichen Labor gezielt unerreichbar machen und die lokale State Machine beobachten. Sie geht über eine pending-Phase zur direkten PBX-Registrierung, erst wenn die konfigurierten Prüfungen fehlgeschlagen sind. Die zentrale Registrierung wird entfernt, bevor die direkte aktiviert wird. Bereits laufende Dialoge werden nicht mitten im Gespräch verschoben; neue Rufe müssen den neuen Weg nehmen. Nach Wiederkehr gilt eine stabile Erholungszeit, bevor die direkte Registrierung abgemeldet und die zentrale wieder geladen wird. Alte Kontakte beim PBX können kurz bis zum Ablauf ihrer Registrierung sichtbar sein; die aktive TBS-Registrierung ist separat zu prüfen.

| Phase          | Nachweis                                                          | Fehlerquelle               |
|----------------|-------------------------------------------------------------------|----------------------------|
| Zentral aktiv  | eine beabsichtigte externe Registrierung, Ruf in beide Richtungen | Alias, Dialplan, PBX-Trunk |
| Switch-Ausfall | definierte Fehlversuche und                                       | Health-Probe und Timer     |

| Phase         | Nachweis                                            | Fehlerquelle               |
|---------------|-----------------------------------------------------|----------------------------|
|               | pending-Zustand                                     |                            |
| Direkt aktiv  | neue Rufe über PBX-Direktweg                        | AstDB-Gate und Kontakt     |
| Wiederkehr    | stabile Zeit, direkte Abmeldung, zentrale Anmeldung | Flapping und Doppelkontakt |
| Medienprüfung | RTP beide Richtungen, DTMF, PAI                     | Firewall, SDP, Codec       |

## 24.6 SDS und Status bei Offline-Zustellung prüfen

SDS hat andere Anforderungen als Sprache: Ein kurzzeitig nicht erreichbarer Empfänger darf eine zulässige Nachricht später erhalten, aber nicht doppelt. Der SDS Router führt die netzweite Zustellung und Store-and-forward; die TBS kann lokal zustellen und begrenzte, dauerhaft gespeicherte Replay-Ereignisse führen. Die Projektdatei Docs/EDGE\_FALLBACK.md beschreibt einen JSONL-Spool mit Eintrags- und Bytebegrenzung sowie einen Marker für lokal schon zugestellte Gruppen-SDS. Hochratige Sprache und RF-Frames werden nicht in diesen Spool geschrieben.

Den Normalweg mit einer eindeutig markierten Einzel-SDS testen. Ursprung, Ziel-ISSI, Message-ID, Zeit und Ack zusammenführen. Danach ein Ziel offline nehmen und eine zweite Nachricht senden. Jetzt müssen Router-Spool und Zustellstatus nachvollziehbar sein; ein HTTP-Ack für die Aufnahme des Auftrags ist noch keine Funkzustellung. Beim Wiederanmelden des Ziels den Replay-Pfad beobachten und nur nach tatsächlicher Zustellung quittieren. Wiederhole die Nachricht mit derselben ID, um die Deduplizierung zu prüfen. Für Gruppen-SDS zusätzlich lokale Zustellung während Isolation und spätere Remote-Legs nach Recovery testen: Der Ursprungsknoten soll die lokal zugestellte Gruppe nicht noch einmal bekommen.

Statusnachrichten und SDS-Kommandos an Integrationen verlangen eine getrennte Zuständigkeitsprüfung. Ein empfangener pre-coded Status kann im Task Workflow eine Zustandsänderung anstoßen. Dort muss die Task-ID, der erlaubte Übergang und die Antwort auf ein doppeltes Kommando sichtbar sein. Ein MQTT-Publish kann die Lage an Home Assistant spiegeln, ist aber nicht gleichzusetzen mit der bestätigten SDS-Zustellung. Für jede Weiterleitung dieselbe Correlation ID verwenden, soweit der betreffende Dienst sie bereitstellt.

Wenn Nachrichten fehlen, den Weg vom ersten TBS-Empfang über Routerentscheidung, Online-/Offline-Ziel, Spool und Replay bis zum Endgerät trennen. Prüfe Größenlimit, TTL, Zielzuordnung und Zustand der Gateway-Verbindung. Spooldateien können noch nicht quittierte Ereignisse enthalten; sie dürfen nicht zur „Reparatur“ einfach gelöscht werden. Vor einem Restore die Datei kopieren, Größe und letzte erfolgreiche Replay-ID notieren und den neuen Lauf auf doppelte lokale Zustellung kontrollieren.

| Ereignis       | Gesuchter Beleg         | Fehlerfrage                                     |
|----------------|-------------------------|-------------------------------------------------|
| SDS angenommen | Message-ID im Ursprung  | wurde sie nur angenommen oder schon zugestellt? |
| Ziel offline   | begründeter Wartestatus | wie lange gilt TTL/Limit?                       |



| Ereignis                    | Gesuchter Beleg              | Fehlerfrage                                   |
|-----------------------------|------------------------------|-----------------------------------------------|
| Ziel online                 | genau ein Delivery-Ack       | ist Replay idempotent?                        |
| Gruppe lokal bedient        | air_fallback_local_delivered | wird der Ursprung beim Replay ausgeschlossen? |
| Integration löst Aktion aus | Task-/MQTT-Ack mit ID        | gibt es einen Loop oder doppelten Auftrag?    |

## 24.7 Paketdaten bis zum IP-Rückweg messen

Der Paketdatenpfad umfasst zunächst die Luftschnittstelle und SMDCP/PDP-Kontexte, dann Packet Core und schließlich einen lokalen oder zentralen IP Gateway mit TUN, Routing, optionalem NAT, Firewall und DNS. Die TBS-Beispielkonfiguration kann eigene Paketdatenwege enthalten; die Core-Vorlage nennt einen `shadow`-Modus und eine `authoritative`-Variante. Vor der Abnahme muss klar sein, welche Instanz die Adresse, Kontextaktion und Route im Test tatsächlich bestimmt. Ein Kontext in der WebUI ist noch keine funktionierende IP-Verbindung.

Mit einem Testgerät eine PDP-Aktivierung auslösen. ISSI, NSAPI, zugewiesene IPv4-Adresse, MTU, Gateway und DNS dokumentieren. Adresspool und vorhandene Standortnetze auf Überschneidung prüfen. Der erste Test ist ein ICMP- oder kleiner UDP-Payload zu einem kontrollierten internen Ziel. Danach den Rückweg aufzeichnen. Eine erfolgreiche Uplink-Capture am TUN ohne Antwort kann auf Firewall, NAT oder fehlende Reverse Route hinweisen. Eine Antwort am TUN ohne Ausgabe zum Funkgerät weist in Richtung Kontext, Fragmentierung, Flow Control oder Air-Interface-Downlink. Bei größeren Payloads MTU und Fragmentierungsgrenzen getrennt testen.

Der IP Gateway braucht im LXC das TUN-Device und passende Namespace-Rechte; ein `ready`-Status des HTTP-Prozesses darf die tatsächliche TUN- und Routingprobe nicht ersetzen. Der NAT-Modus muss zum externen Netz passen. Im gerouteten Modus benötigt die Gegenstelle einen Rückweg in den Adresspool. DNS ist ein separater Test: Eine IP-Adresse kann erreichbar sein, während Namensauflösung fehlschlägt. WAP hat darüber hinaus eigene Inhaltstypen und Terminalgrenzen. Für einen belastbaren Befund jeweils eine interne IP, einen DNS-Namen und eine WAP-Seite getrennt dokumentieren.

Bei Deaktivierung des PDP-Kontextes oder Abmeldung des Endgeräts müssen Lease und NSAPI-Zustand entsprechend freigegeben oder nach dokumentierter Policy erhalten werden. Einen Packet-Core-Ausfall im Labor simulieren und lokale TBS-Kontexte anhand des vorgesehenen Fallbacks prüfen. Nach Recovery dürfen keine doppelten Adressen oder verwaisten Kontexte entstehen. Alle Captures auf persönliche Daten minimieren und auf den Testzeitraum begrenzen.

| Messstelle     | Erwartung                        | Deutet bei Abweichung auf        |
|----------------|----------------------------------|----------------------------------|
| Endgerät       | PDP akzeptiert, Adresse sichtbar | MM/SMDCP/Policy                  |
| TBS            | N-PDU mit NSAPI und Richtung     | Luftschnittstelle/Fragmentierung |
| Packet Core    | Kontext und Lease eindeutig      | Pool/Sync/Autorität              |
| IP Gateway TUN | Paket im Namespace sichtbar      | Device und Routing               |

| Messstelle    | Erwartung            | Deutet bei Abweichung auf    |
|---------------|----------------------|------------------------------|
|               |                      |                              |
| externes Ziel | Antwort kommt zurück | NAT, Firewall, Reverse Route |

## 24.8 MQTT und Home Assistant ohne Zustandsloops anbinden

Der IoT Gateway übersetzt ausgewählte TETRA- und Core-Ereignisse in MQTT-Topics und kann Home Assistant Discovery sowie eine Homematic-Anbindung im Sandboxmodus bedienen. Ein Broker auf Port 1883 im Beispiel ist keine Sicherheitsgrenze. Vor der Integration wird ein Topic-Namensraum für Testgeräte definiert, einschließlich Quelle, Entity-ID, retained State, Verfügbarkeit und Command/Ack. Das Open-Lab-Netz darf keine unkontrollierte Brücke in eine produktive Hausautomation erhalten.

Der erste Test verwendet ein registrierendes Funkgerät. Die TBS erzeugt ein Ereignis, Gateway/Core nehmen es auf, IoT Gateway normalisiert es und der Broker liefert es an einen unabhängigen Subscriber. Erst danach wird geprüft, ob Home Assistant eine Discovery-Entity anlegt und den aktuellen State aktualisiert. Ein retained State ist nach Neustart sofort sichtbar, kann aber veraltet sein; deshalb Zeitstempel und Availability unabhängig testen. Bei Deregistrierung muss der Endzustand erscheinen. Eine beschädigte oder doppelte Discovery-Konfiguration wird gezielt am Test-Topic bereinigt, nicht durch pauschales Löschen aller Brokerdaten.

Ein virtueller Command ist eine zweite Richtung: Broker → IoT Gateway → berechtigte Node-/Fachaktion → Funkwirkung → Ack. Dem Command wird eine eindeutige ID gegeben. Vor dem Senden muss klar sein, welche ISSI/GSSI betroffen ist und ob die Aktion im isolierten Zustand zulässig ist. Nach der Ausführung darf derselbe Command bei einem Broker-Reconnect nicht erneut wirken. Ein Ack nur vom MQTT-Publish ist nicht der Ack der TBS. Die beiden Belege werden getrennt geführt. Loop-Schutz ist besonders wichtig, wenn eine Core-Statusänderung wiederum als neues MQTT-Kommando interpretiert werden könnte.

Bei fehlendem HA-State zuerst den Broker direkt mit Subscriber prüfen, danach Topic-/Prefix-Mapping, Discovery-Payload und Entity-ID. Bei einer ausbleibenden Funkaktion die Command-ID über Gateway-Ack und TBS-Log verfolgen. Verschiedene Richtungen dürfen nicht zu einer vermeintlichen „MQTT-Störung“ zusammengefasst werden. Nach Update des IoT Gateways Registrierung, Deregistrierung, Broker-Ausfall und Wiederkehr erneut testen.

| Grenze         | Probe                       | Nachweis                          |
|----------------|-----------------------------|-----------------------------------|
| Funk zu Core   | Registrierung der Test-ISSI | frisches Ereignis mit Zeitpunkt   |
| Core zu Broker | Subscribe auf Test-Topic    | Payload, Retain und Availability  |
| Broker zu HA   | Discovery und Entity-State  | aktuelle Entity ohne Dublette     |
| Broker zu Funk | einmaliges Command          | berechtigte Funkwirkung           |
| Funk zu Broker | Command-Ack                 | gleiche ID und keine Wiederholung |



## 24.9 TTS, Media Library und Playout als Medienkette prüfen

Eine Textansage durchläuft mehrere Zustände: Text oder Vorlage, TTS-Synthese, importiertes oder erzeugtes Asset, Formatvorbereitung, Genehmigung, Cache auf der TBS und tatsächliche Aussendung. Piper ist im Projekt ein eigener Dienst-/Hostpfad. Media Library verwaltet Import, Preview, Processing, Freigabe, Archiv und Dispatch. Der lokale TBS-Pfad kann vorbereitete und freigegebene Inhalte nach einem Core-Ausfall weiter nutzen. Eine erfolgreiche Synthese sagt noch nichts über Codec, Länge oder Lautstärke auf dem Funkgerät.

Für einen Test eine kurze, neutral formulierte Ansage mit eindeutiger ID erstellen. Vor dem Dispatch TTS-Stimme, Text, Ergebnisdatei und Dauer prüfen. Dann das Asset in Media Library importieren, Preview anhören und Processingstatus bis `ready` verfolgen. Die Genehmigung ist ein eigener Schritt; im Test nur ein klar identifiziertes Asset freigeben. Danach Format und Cache-Zustand auf der Ziel-TBS prüfen. Ein NFS-Pfad kann für Archivierung dienen, aber die laufende Ansage sollte nicht von einem zufällig abgehängten Archivmount abhängen.

Der Playout-Test hat eine explizite Zielzelle, Zielgruppe und ein Zeitfenster. Vorher aktiven Ruf-/Floorzustand prüfen, damit die Ansage nicht einen laufenden Sprecher unkontrolliert verdrängt. Nachher am Funkgerät den Anfang, die vollständige Länge, das Ende und den hörbaren Pegel bestätigen. TBS-Logs und Media-Switch-/Library-Job-ID werden korreliert. Bei abgebrochener Ansage zuerst unterscheiden, ob nur Preview, nur Asset-Transfer, nur Cache oder die eigentliche Luftschnittstelle scheitert. Wiederholungen mit derselben Asset-ID dürfen nicht mehrere gleichzeitige Playouts erzeugen.

Für Wartung die Quelle des Assets, Freigabezeitpunkt, verarbeitete Variante, Cacheversion und Archivort zusammen sichern. Ein Austausch des Codecs oder der TTS-Stimme verlangt eine erneute Hörprobe. Ein altes lokales Cachefile kann sonst den Eindruck erwecken, eine neue zentrale Version sei aktiv. Bei Core-Ausfall im Labor nur ein bereits freigegebenes und gecachtes Asset verwenden; neue zentrale Freigaben sind ohne Core nicht vorzusetzen.

| Station       | Statusbeleg                    | Fehlerfolge                        |
|---------------|--------------------------------|------------------------------------|
| Piper/TTS     | Syntheseantwort, Stimme, Dauer | falscher Text oder fehlende Stimme |
| Media Library | Import, ready, approved        | Asset nicht sendefähig             |
| TBS Cache     | passende Version und Format    | altes oder fehlendes File          |
| Dispatch      | Job- und Zielkennung           | falsche Gruppe/Zelle               |
| Funkgerät     | verständliche komplette Ansage | Codec, RF oder Playout-Gate        |

## 24.10 Aufnahme, Vorschau und Archiv wiederherstellen

Es gibt einen lokalen Aufnahmeweg der TBS und einen Recorder-Tap am zentralen Media Switch. Sie sind unterschiedliche Quellen, auch wenn beide später als Assets im Media-Library-/Archivpfad auftauchen können. Bei einem Ruf werden daher Call-ID, Recorder-Session, lokale Recording-ID und Archiv-Asset-ID getrennt notiert. Ein Ausfall des Core-Recorders darf die lokale TBS-Aufnahme nicht still als „ebenfalls ausgefallen“ klassifizieren. Umgekehrt beweist eine lokale WAV nicht, dass der zentrale Tap alle Call Legs erfasst hat.

Mit einem kurzen Testcall beginnen und zwei deutlich unterscheidbare Sprachabschnitte aufnehmen. Danach Metadaten, Dateigröße, Dauer, Audioinhalt und Prüfsumme der Originaldatei kontrollieren. Für die lokale TBS-Aufnahme liegen WAV und JSON im konfigurierten Recording-Verzeichnis. Den zentralen Recorder über seine Status-/Asset-Endpunkte und den Media-Switch-Tap prüfen. Erst wenn beide Rohquellen bekannt sind, Import, Processing, Preview, Freigabe und NFS-Archiv der Media Library verfolgen. Das Archiv kann optional sein; ein lokales Original wird nicht gelöscht, bevor Kopie und Wiederlesbarkeit bestätigt sind.

Für einen Restore eine Kopie in einem entbehrlichen Testverzeichnis verwenden. Dateirechte, Asset-ID und Metadatenreferenzen wiederherstellen, dann Preview und Integrität vergleichen. Ein reiner Dateikopie-Test reicht nicht, wenn die WebUI einen Index oder eine JSON-Zuordnung benötigt. Aufbewahrungszeit, Löschregel und Backupgröße müssen zum verfügbaren Speicher passen; volle Platte kann sowohl Aufnahme als auch andere Dienste stören. Vor einer Bereinigung die ältesten, bereits erfolgreich archivierten Testassets auswählen und jeden Löschlauf protokollieren.

Beim Fehlerbild „Recording sichtbar, Audio leer“ den Medienfluss während des Calls prüfen, nicht nur den späteren Archivjob. Bei „Audio lokal vorhanden, zentral fehlt“ Tap und Import trennen. Bei „Preview geht, Archiv fehlt“ NFS-Mount, Eigentümer und Jobstatus überprüfen. Eine Reparatur schließt erst mit einer neuen Testaufnahme und einer tatsächlich abgespielten Restore-Kopie ab.

| Quelle        | Primärbeleg               | Wiederanlaufbeleg                   |
|---------------|---------------------------|-------------------------------------|
| TBS lokal     | WAV, JSON, lokale ID      | Datei abspielbar, Metadaten passend |
| Recorder Core | Session und Tap-Asset     | vollständige beide Sprachabschnitte |
| Media Library | verarbeitete Asset-ID     | Preview und Status konsistent       |
| NFS/Archiv    | kopierte Datei, Prüfsumme | Rücklesen an einem zweiten Pfad     |

### 24.11 Edge-Autonomie und Recovery ohne Split Authority testen

Die TBS bleibt bei Verlust des zentralen Netzes lokal für die Funkzelle handlungsfähig. Der Projektpfad unterscheidet *online*, *degraded*, *isolated* und *recovering*. Ein einzelner ausgefallener Fachdienst führt zur dienstspezifischen Degradation; ein verlorener Gateway oder eine abgelaufene Health-Matrix-Lease führt zur vollen Isolation. Der Gateway veröffentlicht die komplette Matrix wiederholt. Die TBS akzeptiert eine ältere Revision nicht als neue Lease. Diese Regel verhindert, dass eine offene WebSocket-Verbindung mit stehen gebliebenem Monitor fälschlich einen gesunden Core vortäuscht.

Vor dem Test den gesunden Zustand belegen: Gateway-Knotensicht, `/api/v1/core-services`, TBS `/api/edge-fallback`, Matrixrevision und Empfangszeit. Eine Test-ISSI/GSSI, ein kurzer lokaler Ruf und eine markierte SDS bilden die Baseline. Nun genau einen Fachdienst stoppen, etwa Subscriber Core. Die TBS soll die betreffende Funktion nach der dokumentierten lokalen Policy behandeln, während andere gesunde Dienste verfügbar bleiben. Für die Teilnehmerpolicy den Last-known-Cache und eine zuvor gesperrte Kennung prüfen; der Ausfall darf keine stille Sicherheitsöffnung bewirken.

Danach den Gateway-Pfad unterbrechen. Nach der eingestellten Eintrittsfrist wird Isolation sichtbar. Lokale Registrierung und lokale Calls werden an einem realen Gerät geprüft, ebenso die Behandlung einer nicht lokal zustellbaren SDS im begrenzten JSONL-Spool. Sprechframes werden nicht nachträglich wiederholt. Die Größe und das Alter des Spools werden vor und nach dem Test kontrolliert. Bei Wiederkehr darf der Dienst erst nach der konfigurierten Hysterese zurück zur zentralen Autorität wechseln. Replay-Acks und Policyrevisionen werden abgewartet; ein bloßes `ready=200` ist noch nicht das Ende des Recovering-Zustands.

Ein besonders wichtiger Negativtest hält den WebSocket offen, lässt aber die Health-Matrix nicht mehr erneuern. Nach Lease-Ablauf muss die TBS zentrale Dienste als nicht verfügbar behandeln. Umgekehrt darf eine alte Matrixrevision den Timer nicht verlängern. Diese Probe ist im Labor an einem kontrollierten Testpunkt zu fahren, weil sie Managementzustand und eventuell laufende Calls beeinflusst. Nach Abschluss alle gestoppten Units starten, Testnachrichten bereinigen und dieselbe fachliche Baseline erneut nachweisen.

| Zustand    | Auslöser                                    | Beobachtete Autorität                  |
|------------|---------------------------------------------|----------------------------------------|
| online     | frische Matrix, erforderliche Dienste ready | zentrale Dienste plus lokale Funkkante |
| degraded   | einzelner Dienst fehlt                      | servicebezogener lokaler Fallback      |
| isolated   | Gateway weg oder Lease abgelaufen           | lokale Zelle und begrenzter Spool      |
| recovering | Core wieder gesund, Hysterese/Replay läuft  | Übergang mit Revision und Acks         |

## 24.12 Kontrolliertes Update und belastbarer Rollback

Ein Update ist ein Daten- und Protokollwechsel, nicht nur der Austausch eines Binaries. Für jede betroffene Komponente werden vorher Quellcommit, aktive Konfigurationsdatei, Unit-Definition, persistenter Zustand, Portbelegung und Fachtest festgehalten. Ein Bundle umfasst den vorherigen Binary-/Containerstand, die dazu passende TOML und die Datenkopie. Wenn sich das Dateiformat geändert hat, ist ein altes Binary mit einem neuen State möglicherweise nicht startfähig. Darum ein Restore vor dem eigentlichen Wartungsfenster in einem separaten Test-LXC ausprobieren.

Das Deployment-Inventory bestimmt die Abhängigkeitsreihenfolge. Einen zentralen Gateway oder Autoritätsdienst nicht gleichzeitig mit allen abhängigen Diensten ändern, wenn danach kein Vergleichspunkt bleibt. Zuerst `plan/render` des neuen Commits prüfen. Beispielwerte aus dem Repository nicht über standortspezifische Secrets, IPs oder Dateipfade kopieren. Danach einen Dienst aktualisieren, `live` und `ready` prüfen, eine echte Fachaktion durchführen und die nachgelagerten Dienste beobachten. Die TBS wird erst mit passender Gateway-/Backendversion getestet, wenn deren Vertrag verändert wurde.

Der Rollback wird schon vor dem Update in kurzen Schritten aufgeschrieben: Unit anhalten, altes Paket/Binary und alte TOML einspielen, passenden State zurückspielen, Rechte kontrollieren, Unit starten, Health und Fachtest durchführen. Falls Daten seit dem Backup verändert wurden, zuerst entscheiden, ob ein Restore diese Änderungen verlieren würde. Besonders Subscriber-/Group-Policy, Recorder-Assets, SDS-Spool und Taskstatus erfordern

eine fachliche Entscheidung; ein ungeprüftes Überschreiben kann die Störung verschlimmern. Bei Funkproblemen TBS-config.toml.fallback als unabhängige bekannte Startoption bereithalten.

Nach erfolgreichem Update mindestens ein Negativereignis und einen Wiederanlauf testen: beispielsweise kurzzeitiger Broker-/Gatewayverlust oder Neustart eines nicht kritischen Fachdienstes. Artefakte des E2E-Runners werden mit Commit, Inventory-Hash und UTC archiviert. Für on-air-relevante Änderungen gelten Gerätehersteller/Firmware und Messdaten als zusätzlicher Abnahmebeleg. Ein Review-Protokoll benennt auch Tests, die wegen fehlender Live-Hardware nicht möglich waren, statt sie still als bestanden zu zählen.

| Zeitpunkt      | Pflichtnachweis                              | Entscheidung            |
|----------------|----------------------------------------------|-------------------------|
| Vorher         | Commit, Backup, Restore-Probe, Fach-Baseline | Updatefenster freigeben |
| Nach Rendern   | IPs, Pfade, Secrets, Diff                    | Bundle konsistent?      |
| Nach Ausrollen | Unit, live, ready, Fachpfad                  | fortsetzen oder stoppen |
| Bei Regression | altes Set aus Binary/TOML/State              | Rollback ausführen      |
| Abschluss      | E2E-/On-Air-Bericht, UTC, offene Risiken     | Ausgabe dokumentieren   |

### 24.13 Security Core und KMF im Laborausbau behandeln

Security Core und KMF sind getrennte Zuständigkeiten. Security Core bewertet Admission, Security-Class-Policy, Sperren und Audit; KMF hält Schlüssel-Lebenszyklus und orchestriert OTAR-Aktionen. Ein im Code vorhandener Dienst ist kein Beweis für kryptographische Interoperabilität oder die sichere Verwahrung echter Schlüssel. Die Open-Lab-Beispielumgebung hat keine Management-Anmeldung oder TLS und gehört nicht in ein untrusted Netz. Schlüsselmaterial und Zugangsdaten werden weder in dieses Handbuch übernommen noch in Testberichte oder Journalauszüge kopiert.

Für die Prüfung nur dafür vorgesehene Testkennungen und Testschlüssel verwenden. Vor einer Policyänderung die aktuelle Security-Klasse, betroffene ISSIs, TBS-Revision und Rückfallregel notieren. Eine erlaubte und eine gesperrte Kennung testen. Danach den Security Core kontrolliert ausfallen lassen und verifizieren, dass die TBS die letzte bekannte Policy beibehält und nicht unbemerkt auf eine offenere Klasse fällt. Bei Wiederkehr Audit, Revision und tatsächliches Verhalten erneut abgleichen. Die dokumentierte Grenze ist wichtiger als ein pauschales „ready“ des KMF.

Für einen OTAR-Test getrennt protokollieren: Auftrag, Ziel, Schlüsselmetadaten ohne Geheimwert, Version/Revision, Versand, Bestätigung und Endgerätewirkung. Abgebrochene oder doppelte Jobs dürfen nicht blind wiederholt werden. Vor einer Rotation muss ein Rückweg für den vorherigen Schlüsselzustand existieren, einschließlich der betroffenen Geräte und ihrer Erreichbarkeit. Eine Reproduktion mit zwei Herstellergeräten ist erforderlich, wenn man Interoperabilität behaupten möchte.

Incident-Logs zu Security-Vorgängen werden auf IDs und Status reduziert. Eine Übertragung der Open-Lab-Konfiguration in den regulären Betrieb verlangt eigene Architektur für Authentisierung, TLS, Segmentierung, Secret-Verwaltung, Rollen und Audit. Bis diese

Maßnahmen nachweislich implementiert und getestet sind, ist die sichere Grenze das isolierte Labor.

#### 24.14 Control Room und Observability als Sicht auf die Fachwahrheit nutzen

Control Room bündelt Zustände und Operatoraktionen. Observability sammelt Scrapes, Logs, Traces, Alerts, Silences und Diagnosepakete. Beide vereinfachen Betrieb, erzeugen aber nicht selbst die fachliche Autorität für ISSI, GSSI, Call-ID oder PDP-Kontext. Eine grüne Kachel kann einen alten Poll enthalten; eine fehlende Metrik kann ein Scrapeproblem sein. Deshalb steht bei einer Abweichung immer der direkte Endpunkt des fachlich zuständigen Dienstes neben dem aggregierten Wert.

Für die Erstinbetriebnahme einen markierten Vorgang erzeugen, etwa eine Test-SDS oder einen kurzen Call. Die Correlation ID wird von Gateway über Fachdienst und ggf. Application Gateway verfolgt. In Observability den aktuellen Scrapezeitpunkt, Logeingang, Trace-Spans und Alertregel prüfen. Im Control Room dieselbe Lage mit Pollzeit und Origin anzeigen. Eine Operatoraktion muss am Ziel den erwarteten Effekt auslösen und im Audit nachvollziehbar sein. Bei offenem Managementnetz sind Rollen- und Login-Funktionen einzelner Oberflächen nicht als Ersatz für eine durchgängige Zugriffssicherung aller 25 Backends zu betrachten.

Ein Alarmtest löst denselben Hardware- oder RF-Grenzwert zweimal aus. Alarm Workflow soll die Ereignisse deduplizieren, eine Alarmakte führen und Eskalation/Ack nachvollziehbar machen. Gleichzeitig wird geprüft, ob die zugrunde liegende Messquelle kalibriert ist: RF-Monitor kann DSP-Werte und optionale Hardwaremessungen anzeigen, die nicht dieselbe Genauigkeit haben. Eine Silence in Observability ist kein Reparaturbeleg; nach ihrem Ablauf muss die Ursache behoben oder als offenes Risiko dokumentiert sein.

Im Tagesbetrieb drei Sichtweisen nebeneinander halten: direkte Fachprobe, aggregiertes Dashboard und Endgerätewirkung. Wenn sie sich widersprechen, hat die konkrete fachliche Transaktion mit passender Zeit und ID Vorrang. Die Abweichung ist selbst ein Fehlerfall für Polling, Caching oder Mapping. Ein exportiertes Diagnosepaket wird auf Secrets und Teilnehmerdaten geprüft, bevor es außerhalb des isolierten Labors geteilt wird.

| Ansicht       | Kann zuverlässig belegen          | Benötigt zusätzlich             |
|---------------|-----------------------------------|---------------------------------|
| Dienst-Health | Prozess-/Bereitschaftszustand     | Fachtransaktion und Gegenstelle |
| Control Room  | aggregierte, zeitgestempelte Lage | direkten Origin-Status          |
| Observability | Mess- und Ereignisspur            | Endgeräte-/Funkwirkung          |
| RF-Monitor    | Sensor-/DSP-Werte                 | kalibrierte Messquelle          |
| On-Air-Test   | tatsächliches Geräteverhalten     | Versions- und Logkorrelation    |

#### 24.15 Nach einer Störung systematisch wieder freigeben

Eine Reparatur ist abgeschlossen, wenn der betroffene Dienstpfad wieder funktioniert, abhängige Funktionen stabil bleiben und der Fehler bei den ursprünglichen Bedingungen nicht erneut auftritt. Der erste Schritt ist deshalb die präzise Ausgangslage: Welche Funktion, welcher Host, welche TBS, welche ISSI/GSSI/Call-ID, welcher Zeitpunkt und welcher Softwarestand waren betroffen? Danach den ersten fehlerhaften Übergang aus Kapitel 20

festlegen. Unabhängige Ursachen wie Clockdrift, volle Disk oder fehlender NFS-Mount dürfen nicht durch das Neustarten sämtlicher Dienste verdeckt werden.

Bei einem Containerfehler zuerst Unit, Journal, Bindung, Datei- und Mountrechte, freien Speicher sowie direkte Abhängigkeiten prüfen. Bei einer Funkstörung PHY, Broadcast, Uplink, Policy und fachliche Vermittlung getrennt messen. Bei Medien- und Datenproblemen beide Richtungen betrachten. Für jeden Reparaturschritt wird vorher die betroffene Konfiguration gesichert. Nur eine Variable ändern und dann den ursprünglichen Test wiederholen. Bleibt das Symptom, Änderung zurücknehmen oder als neue Hypothese dokumentieren; keine Kette unprotokollierter „Versuche“ anlegen.

Nach Wiederherstellung zunächst den kleinsten erfolgreichen Fachtest, dann den vollständigen Ablauf und schließlich einen kontrollierten Wiederanlauf prüfen. Bei Policy-, Cache- oder Replay-Reparaturen ist eine negative Probe wichtig: Gesperrte Teilnehmer bleiben gesperrt, doppelte Nachrichten bleiben einmalig, alte Matrixrevisionen verlängern keine Lease. Bei SIP und Media beide Richtungen, beim IP Gateway Uplink und Rückweg, bei RF beide Träger und den Uplink kontrollieren. Erst danach die Anlage für den vorgesehenen Testbetrieb freigeben.

Das Abschlussblatt enthält eine kurze Ursachenbeschreibung, den korrigierten Quell- oder Konfigurationsstand, Vorher/Nachher-Evidenz, Auswirkungen auf bestehende Daten, einen Rollbackbeleg und einen Regressionstest. Wenn die Ursache im Code liegt, den kleinsten reproduzierbaren Fall als Test in das passende Modul oder E2E-Szenario übernehmen. So wird aus einer einmaligen Reparatur eine überprüfbare Verbesserung des nächsten Builds.



## 25 Installationsbuch und Standortanpassung

Dieses Kapitel ist für einen neuen Laboraufbau oder einen geordneten Neuaufbau nach einem Schaden bestimmt. Es führt die konkreten Befehle des aktuellen Deployment-Helpers mit den notwendigen Entscheidungen am Standort zusammen. Beispiele enthalten absichtlich keine realen Zugangsdaten. Die Verzeichnisse unter `/etc` und `/var/lib` sind vor Ausführung mit den `systemd-Units` und Installern des gepinnten Commits abzugleichen. Die Befehle laufen jeweils auf dem angegebenen Host; ein `127.0.0.1` im LXC bezeichnet diesen LXC und nicht die Basisstation.

### 25.1 Vorbereitungsblatt für den Installationsverantwortlichen

Vor dem ersten Container wird eine Standortmappe erstellt. Sie nennt Proxmox-Host, Management-VLAN, Gateway, DNS, NTP, IP-Bereich, SSH-Zugang, Storage, Backupziel und Namensschema. Für die Funkseite kommen Standortberechtigung, Frequenzen, Träger, Duplexabstand, SDR-Typ/Seriennummer, Antennen-/Lastaufbau und Messgeräte hinzu. Für Anwendungen werden Test-ISSIs/GSSIs, PBX-Trunk, MQTT-Broker, NFS-Share und Piper-Host als getrennte Gegenstellen aufgeführt. Eine fehlende Gegenstelle erhält einen bewusst deaktivierten oder lokal begrenzten Modus, nicht eine scheinbar gültige Loopback-Adresse.

Das Repository wird auf einen konkreten Commit fixiert. `git rev-parse HEAD`, `git status --short` und `cargo --version` werden vor der Installation notiert. Für ein Wiederherstellungsszenario liegt ein Quellbundle oder Paketcache unabhängig vom Internet bereit. Die ausgewählte Version muss zum `Cargo.lock` und zu den zum Zeitpunkt der Installation vorhandenen Dienstschema-Versionen passen. Tag und Branchname allein genügen nicht, weil sie sich ändern können. Der Quellstand dieser Ausgabe ist `086a81fa8820ef579c475a65a38e3d23644c52f0` auf `main`.

Die Verantwortung für gefährliche Änderungen wird vorab geklärt. Das Open-Lab-Backend ist über seine Management-APIs ohne durchgängige Authentisierung/TLS erreichbar. Das VLAN darf nur autorisierte Testhosts enthalten. Fault-Tests stoppen Dienste absichtlich; RF-Tests erfordern den dafür vorgesehenen Mess- und Frequenzrahmen. Eine personell zugeordnete Testleitung bestätigt den Start jedes dieser Schritte und hält einen sofortigen Stop-/Rollbackweg bereit.

| Feld        | Einzutragen                         | Gegenprüfung                                |
|-------------|-------------------------------------|---------------------------------------------|
| Repository  | Commit, Lockfile, Builddatum        | auf TBS und LXC's identisch oder kompatibel |
| Management  | VLAN, CIDR, Gateway, DNS, NTP       | jeder Host nur aus Testnetz erreichbar      |
| Funk        | SDR, Clock, TX/RX, Träger, Messlast | Frequenzplan und Messung                    |
| Identität   | MCC/MNC/LAC, Test-ISSI/GSSI         | Endgeräteprogrammierung                     |
| Integration | PBX, Broker, NFS, Piper             | Ports, Pfade, Zugangsdaten separat          |
| Rückweg     | Binary, TOML, State, Backupport     | Restore-Probe vor Betrieb                   |

| Feld | Einzutragen | Gegenprüfung |
|------|-------------|--------------|
|      |             |              |

## 25.2 LXC-Basis und Ressourcen anlegen

Das Projekt sieht einen eigenen unprivilegierten Debian-LXC pro deploybarem Backend-Dienst vor. Das ist ein Isolations- und Betriebsmodell, kein Muss für jeden Entwicklerlaptop. Im Proxmox-Host werden zunächst Template und Storage geprüft; die konkreten CTIDs sind standortabhängig. Jedem Container eine eindeutige Adresse, einen Hostnamen und Autostart geben. Grundressourcen können gemäß altem Installationsguide mit zwei Kernen, 2 GiB RAM und 512 MiB Swap beginnen, müssen aber für Recorder, Media Library, SIP/Asterisk und Observability anhand echter Last und Retention angepasst werden. Disk-Reserve ist für Logs und Aufnahmen wichtiger als ein theoretischer Minimalwert.

Im LXC Grundpakete, Zeitquelle und SSH vorbereiten. Der Deployer nutzt `ssh_user` und `ssh_options` aus dem Inventory, im Beispiel Root mit BatchMode und Hostkey-Policy. Wer einen anderen Nutzer oder Schlüssel nutzt, trägt das ausdrücklich dort ein und testet eine nichtinteraktive Verbindung zu jedem Ziel. Das Deployment soll nicht beim 17. von 25 Hosts an einem erstmaligen Hostkey-Prompt hängen. Ein `ssh <host> hostname` und `sudo -n true` (bei Nicht-Root-Nutzern) sind sinnvolle Vorprüfungen. Hostkeys gehören zum Standortinventar.

Für den IP Gateway `/dev/net/tun` und nötige Rechte im Container gesondert bereitstellen. Der Proxmox-Eintrag aus der Projektanleitung ist eine Beispielkonfiguration, deren Wirkung nach LXC-Neustart im Container mit `ls -l /dev/net/tun` und einer kurzlebigen TUN-Testinstanz verifiziert wird. Nicht wahllos zusätzliche Hostrechte freigeben. Beim Recorder/Media-Library-LXC NFS als optionale Archivschicht anlegen; Livezustand und Cache bleiben lokal. NFS-Mount, Eigentümer und Schreibprobe müssen auch nach einem Containerstart funktionieren. Die Mount-Optionen `_netdev`, `nofail` und `Automount` verhindern, dass ein fehlendes Archiv unbegrenzt den Systemstart blockiert.

Auf allen Hosts denselben Zeitsynchronisationsdienst und die gleiche Zeitzone-Strategie verwenden. Logs und E2E-Artefakte werden in UTC korreliert. Eine falsch gehende TBS kann scheinbar veraltete Matrix-Leases, Zertifikate oder SDS-TTLs produzieren; Zeit ist daher eine funktionale Abhängigkeit. Nach dem Grundimage einen Snapshot erstellen, bevor Rust-/Python- und Dienstpakete installiert werden.

## 25.3 Inventory kopieren und technisch validieren

Eine lokale Standortdatei wird aus `deploy/open-lab/inventory.example.toml` abgeleitet und außerhalb einer versehentlichen Übernahme in `main` gepflegt. Zuerst alle `host`- und `port`-Werte setzen, dann `ssh_user`, `ssh_options` und `remote_source_root`. Alle 25 Dienste müssen eindeutige Host-/Port-Ziele besitzen; `depends_on` bildet einen gerichteten, zyklenfreien Graph. URLs innerhalb der Dienst-TOMLs werden vom Renderer anhand des Inventories umgesetzt, aber die ausgegebene Datei muss weiterhin visuell auf unpassende Loopback- oder historische `10.0.1.x`-Ziele geprüft werden.

```
cp deploy/open-lab/inventory.example.toml deploy/open-lab/inventory.toml
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml validate
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml plan
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml render
```



Der Validator prüft Dateipfade, Ports und Abhängigkeitsgraph. Das Ergebnis OK: 25 services ist ein statischer Strukturbeleg, noch kein Netzwerk- oder Runtime-Test. `plan` zeigt die aufgelöste Reihenfolge, `render` erzeugt Konfigurationen und Katalog unter `deploy/open-lab/generated`. Das Material wird mit einem Diff gegen die Vorlagen und gegen den Standortplan kontrolliert. Insbesondere `server.bind` muss zur gewünschten Netzbindung passen; eine Backend-URL mit `127.0.0.1` ist nur korrekt, wenn die Gegenstelle tatsächlich im selben Container läuft.

Im geprüften Inventory steht für Control Room als `config_target /etc/netcore/control-room.toml`, während die eingetragene systemd-Unit `--config /etc/netcore-control-room/control-room.toml` verwendet. Diese Abweichung ist vor `apply` am gewählten Commit aufzulösen und nach dem Start mit `systemctl cat netcore-control-room.service` zu verifizieren. Eine grüne Unit kann sonst eine andere Datei lesen als die gerenderte. Ein historisches Dokument enthält ein korrigiertes Inventory-Beispiel; trotzdem wird dessen übrige 17er-Topologie nicht ungeprüft übernommen. Ebenso muss die TBS-Gateway-URL gegen das aktuelle Node-Gateway-Hostziel geprüft werden; der statische Full-System-Audit war im Quellstand gerade wegen einer abweichenden `Root-config.toml`-Adresse nicht erfolgreich.

## 25.4 Bundle und Dry Run vor dem Ausrollen

Der Deployer kann aus dem gepinnten Checkout ein deterministisches Quellbundle ohne PDFs, Git-Historie, Buildausgabe und Caches erstellen. Das ist hilfreich für Reproduktion und Archivierung. Der Dry Run erzeugt keine laufenden Dienste, zeigt aber die beabsichtigten Hostaktionen. Beide Stufen laufen vom Deployment-Host. Den Digest des Bundles zusammen mit dem Commit und der gerenderten Inventory-Datei aufbewahren; nur so lässt sich später ein Update oder Rollback eindeutig zuordnen.

```
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml bundle
python3 deploy/open-lab/netcore-deploy.py --inventory deploy/open-lab/inventory.toml apply --dry-run
```

Die Ausgabe des Dry Runs gegen den geplanten Wartungsumfang lesen. Bei einem selektiven `apply` ergänzt der Deployer transitive Abhängigkeiten. Das kann mehr Container umfassen als der einzelne angegebene Dienstname; deshalb die tatsächliche Reihenfolge in `plan <dienst>` vorab ansehen. Keine Installer blind als „idempotent“ annehmen. Der Node-Gateway-Installer stoppt seine Unit, entfernt eine alte Binärdatei und Teile des Release-Buildverzeichnisses, baut neu und startet wieder. Ein unerwarteter Buildfehler kann deshalb eine zuvor laufende Instanz betreffen. Dafür liegt das alte Binary bereit und das Wartungsfenster berücksichtigt den Ausfall.

Vor einem echten `apply` auf jedem Host Config und persistenten State sichern. Der Deployer schreibt gerenderte TOMLs an `config_target`, während Installer teils nur bei fehlender Zielfeld eine Beispielkonfiguration anlegen. Der konkrete Reihenfolgeeffekt wird am Skript des Commits geprüft. Zugangsdaten für PBX, Broker oder Webhooks werden getrennt in der aktiven Konfiguration gehalten und nach dem Rendern nicht versehentlich durch einen leeren Beispielwert ersetzt. SSH-Log, Bundle-Digest und Ausgabe von `apply` sind Teil der Installationsakte.

## 25.5 Backend in Abhängigkeitsreihenfolge starten

Der Einstieg ist Node Gateway. Nach `apply` wird seine Unit direkt geprüft: `systemctl is-active`, `journalctl -u`, Bindeadresse, `/health/live`, `/health/ready` und `/api/v1/core-services`. Die Matrix kann zunächst andere Dienste als nicht bereit zeigen; das ist vor deren Installation erwartbar.

Als Nächstes folgen die autoritativen Teilnehmer-, Gruppen-, Mobility- und Call-Dienste entsprechend `plan`. Bei jedem Dienst dieselbe Reihenfolge: Prozess, Listener, Ready, fachliche GET-Route, dann erst eine markierte schreibende Testaktion. Der Health-Endpunkt beantwortet die Frage nach Betriebsbereitschaft, nicht die nach Funkinteroperabilität.

Die medienabhängigen Dienste Media Switch und Recorder werden mit einem kontrollierten Call erst dann fachlich getestet, wenn Call Control und TBS verfügbar sind. SDS Router erhält eine Einzel-SDS mit Delivery-Ack. Packet Core und IP Gateway erhalten einen echten PDP-/IP-Test, wobei der IP Gateway seine TUN- und NAT-/Routingvoraussetzungen haben muss. Media Library erhält ein kurzes Asset und eine Preview. Control Room und Observability werden zuletzt gegen die direkten Fachendpunkte verglichen. Ein Dashboard, das alle Kacheln rendert, aber alte Pollzeiten anzeigt, besteht die Abnahme nicht.

Die sieben später ergänzten Dienste IoT Gateway, Hardware Gateway, RF Monitor, Alarm Workflow, Task Workflow, Asset Management und SIP Switch können Python, Rust oder Asterisk als Runtime verwenden; deshalb die jeweilige Installerdatei und `systemctl cat pro` Dienst prüfen. Ein pauschaler `cargo build` für alle ist falsch. Der SIP-Switch-Installer prüft die Hostrolle, installiert Python/Asterisk-Komponenten, rendert Asterisk-Includes und startet Asterisk sowie die Switch-Unit. Er ist nicht als generischer Open-Lab-Rust-Dienst zu behandeln. Für Control Room ist zusätzlich die tatsächliche Config-Datei der Unit zu kontrollieren.

| Gruppe                    | Vorbedingung              | Fachprobe                           |
|---------------------------|---------------------------|-------------------------------------|
| Gateway                   | Netz, Zeit, SSH, Konfig   | TBS-Knoten und Core-Matrix          |
| Subscriber/Group/Mobility | Gateway                   | ISSI/GSSI, Sync, Serving Node       |
| Call/Media/Recorder       | Autoritätsdienste, TBS    | Ruf, beide Richtungen, Aufnahme     |
| SDS/Packet/IP             | Gateway, TBS              | Ack/Replay und IP-Rückweg           |
| Anwendungen               | Fachkerne, Broker/PBX/NFS | echte Connector-/Operatoraktion     |
| NMS/Leitstelle            | direkte Dienste bereit    | Pollzeit, Trace und Originvergleich |

## 25.6 TBS-Binary bauen und Dienststart sichern

Die TBS verwendet das Cargo-Paket `bluestation-bs` unter `bins/bluestation-bs`. Vor dem Build die Hardware- und Featureanforderungen des ausgewählten Commits lesen. SoapySDR-Library, gerätespezifischer Treiber, Compiler und eine zum Lockfile passende Rust-Toolchain müssen vorhanden sein. Das Beispiel im alten Komplettguide enthält historische Arbeitsverzeichnisse wie `/opt/netcore-tetra-swmi` und eine Vorlage ohne ihren heutigen `Docs/-`Pfad. Für diese Ausgabe werden die tatsächlich ausgecheckten Pfade verwendet; ein Installationskommando wird erst nach `ls` und `cargo metadata` ausgeführt.

```
cd /opt/netcore-tetra
git rev-parse HEAD
cargo build --locked --release -p bluestation-bs
SoapySDRUtil --find
SoapySDRUtil --probe="driver=<TREIBER>"
```

Das Binary wird nicht während eines unbekannten Rufzustands ersetzt. Alte Binärdatei und Unitdefinition separat sichern. Die bereinigte Vorlage

`Docs/basisstation.config.sanitized.example.toml` nach `/etc/netcore/config.toml` kopieren, standortgerecht bearbeiten und mit restriktiven Rechten ablegen. Die Datei enthält Beispielwerte für Frequenz, MCC/MNC, IPs und optionale Integrationen. Die `.fallback` wird als bekannte, zuvor erfolgreich gestartete Konfiguration unabhängig gepflegt; sie wird nicht bei jedem Fehler automatisch überschrieben. Danach zuerst einen manuellen Parser-/SDR-Start am Messplatz durchführen und erst bei stabilem Start die `systemd`-Unit aktivieren.

`systemctl cat tetra.service` zeigt den tatsächlich verwendeten Binary- und Config-Pfad. `journalctl -u tetra.service -b` und das TBS-Dashboard liefern die ersten Statushinweise; die RF-Probe erfolgt mit Messgerät und Endgerät. Ein Serviceprozess im Status `active` kann noch einen falschen Träger oder unzureichenden Uplink haben. Die Abnahme schließt daher Broadcast, Registrierung, Gruppenruf und Rückfall nach Gateway-Verlust ein. Die vorhandene `.fallback` wird einmal durch einen kontrollierten Parsefehler im isolierten Test verifiziert, dann die Primärdatei repariert.

## 25.7 Lokale Medien, TTS und Archivfade installieren

Media Library, Recorder, lokales Audio und Piper haben verschiedene Pfade. Ein gemeinsames NFS-Verzeichnis darf nicht ungeprüft als Datenbank, Livecache und Archiv zugleich dienen. Die TBS benötigt lokale Verzeichnisse für Aufnahmen, TTS-Vorlagen und vorbereitete Audiofiles. Dienstbenutzer, Schreibrechte und freier Speicher werden vor dem Playout getestet. `ffmpeg` und der tatsächlich konfigurierte Codecpfad werden an einem kurzen Testasset geprüft. Ein Audiofile mit korrekter Dateiendung kann dennoch ein falsches Format, falsche Abtastrate oder leere Samples haben.

Piper kann nach der Projektanleitung über `system-backend/tts/install-piper.sh` eingerichtet werden; die lokale API wird beispielhaft auf Port 5005 mit `/voices` geprüft. Nach dem Start zunächst Stimme und kurze Synthese testen, dann den Übergang zur Media Library. Erst nach Import, Processing, Preview, Freigabe und Cachetest wird eine Ansage an eine Zielzelle gesendet. Ein lokaler TTS-Erfolg wird nicht als On-Air-Erfolg gemeldet. Für Archiv/NFS Testdatei schreiben, wieder lesen und ihre Prüfsumme mit dem lokalen Original vergleichen. Ein fehlender Mount darf die lokale TBS-Aufnahme nicht unbemerkt stoppen.

Im Betrieb werden Retention und Diskbudget festgelegt: maximale Anzahl/Größe der Aufnahmen, Dauer von Caches, Zeitpunkt von Import-/Archivjobs und Restore-Stichprobe. Eine Backuproutine kopiert nicht nur WAV, sondern auch zugehörige JSON-Metadaten und Asset-Indexzustände. Bei einem Rollback werden diese zusammenpassend wiederhergestellt. Das ist besonders wichtig, wenn Media Library nach einem Update neue ID- oder Formatbeziehungen geschrieben hat.

## 25.8 SIP-Switch und lokalen TBS-Asterisk einrichten

Der zentrale SIP-Switch und der lokale TBS-Fallback haben getrennte Installer. Der zentrale Host benötigt Asterisk, Python-Runtime, AGI-Skript, generierte PJSIP-/Dialplan-/RTP-Includes und eine Konfiguration mit PBX- sowie TBS-Endpunkten. Die eingetragene zentrale Installerdatei prüft die Hostrolle, legt `/etc/netcore/sip-switch.toml` an, rendert Asterisk-Konfiguration und startet `asterisk.service` sowie `netcore-sip-switch.service`. Vor der Ausführung PBX-Daten, Endpoint-IDs, RTP-Bereich und Firewallregeln in einer geschützten Standortdatei vorbereiten. Die Beispiel-TOML enthält keine einsatzfertigen Geheimnisse.

Auf jeder TBS bleibt die native Bridge zum lokalen Asterisk stabil. Der lokale Phase-11c-Fallback verwaltet die Umschaltung zwischen zentralem Switch und PBX-Direktweg. Vor dem Start die beiden externen Registrierungsobjekte und die AstDB-Sperre des Direkt-Dialplans lesen. Im Normalzustand soll nur die zentrale Registrierung aktiv sein. Der Ausfalltest führt über definierte Fehlversuche und eine pending-Phase zum Direktweg; die Rückkehr hat eine stabile Hysterese. Den zentralen und lokalen Host in den jeweiligen Installer-/Update-Skripten nicht vertauschen.

Zur Abnahme gehören REGISTER-/Kontaktstatus, eingehender und ausgehender Ruf, korrektes ISSI-Routing, P-Asserted-Identity, DTMF und beide RTP-Richtungen. Ein Portscan auf 5060 ersetzt keinen RTP-Test. Den konfigurierten UDP-Medienbereich in der Firewall auf beiden Seiten betrachten. Nach einem Failover nur neue Calls über den neuen Weg erwarten; laufende Dialoge werden nicht mitten im Gespräch verschoben. Bei doppeltem PBX-Kontakt zuerst unterscheiden, ob ein alter Kontakt nur bis zur Ablaufzeit sichtbar ist oder ob die TBS aktiv zwei Registrierungen hält.

### 25.9 Provisioning Core außerhalb des Inventories ergänzen

Provisioning Core ist im Workspace vorhanden, jedoch weder in `system-backend/services.toml` noch in `deploy/open-lab/inventory.example.toml`. Deshalb erhält er einen eigenen LXC oder einen klar abgegrenzten Testhost, ohne die 25er-Deploy-Reihenfolge fälschlich als vollständig für ihn auszugeben. Seine Beispiel-API/WebUI nutzt Port 8125 und spricht Subscriber Core und Group Core. Vor dem Start müssen deren aktuellen Hosts/Ports in seiner TOML stehen und von diesem Container aus erreichbar sein.

Die Datei `Docs/PROVISIONING_CORE_COMPLETE_INSTALL.md` enthält einen historischen Git-Patch- und Branchablauf. Die genannten Branches sind kein Installationspfad für diesen gepinnten main-Commit, in dem der Dienst bereits vorhanden ist. Den aktuellen Installer `system-backend/provisioning-core/install/install.sh`, die Unit `netcore-provisioning-core.service` und `/etc/netcore/provisioning-core.toml` verwenden. Die tatsächlichen Pfade mit `systemctl cat` und dem Installerinhalt bestätigen. Beim ersten Start das Dashboard und den Status lesen, dann mit einer ausschließlich dafür reservierten ISSI/GSSI eine Mitgliedschaft anlegen und entfernen.

Der End-to-End-Nachweis verlangt zwei autoritative Antworten: Teilnehmer im Subscriber Core und Gruppe/Mitgliedschaft im Group Core. Danach Sync zur TBS und eine reale Zulassungsprobe. Wird nur eine Seite aktualisiert, handelt es sich um Teilerfolg, nicht um eine abgeschlossene Provisionierung. Vor erneutem Schreibversuch die aktuellen Daten beider Cores exportieren und die Differenz analysieren. Das UI ist im Open Lab keine Sicherheitsschranke für die dahinterliegenden ungeschützten APIs.

### 25.10 Netzports und Firewall als Richtungsmatrix prüfen

Die Porttabelle in Kapitel 15 nennt Beispiele, aber eine Firewall wird nach Kommunikationsrichtung gebaut. Für jede Verbindung sind Quelle, Ziel, Protokoll, Portbereich und Begründung zu notieren. TBS → Node Gateway nutzt den konfigurierten WebSocket-Pfad `/ws/node` am Gateway-Listener. Backend-Dienste können einen eigenen WebSocket-/HTTP-Weg zum Gateway verwenden. Der Deployment-Host braucht SSH zu den LXCs. Observability fragt die Health-/Metrics-Endpunkte der Dienste ab. SIP benutzt Signalisierung und einen getrennten RTP-UDP-Bereich. MQTT nutzt im Open-Lab-Beispiel 1883/TCP. TTS/Piper, Directory und Brew sind zusätzliche Gegenstellen, die nicht aus der 25er-HTTP-Tabelle folgen.

Eine Firewallprobe beginnt mit dem tatsächlichen Zielprozess: `ss -ltnup` am Ziel, dann eine Anfrage vom vorgesehenen Quellhost. Ein Test direkt am Ziel mit `127.0.0.1` prüft nur den Listener und kann eine Netzsperrung verdecken. Für WebSocket neben TCP-Erreichbarkeit den Upgrade-/Anmeldepfad und einen Ack prüfen. Bei SIP neben REGISTER/INVITE auch RTP in beiden Richtungen. Bei IP Gateway neben dem HTTP-Managementport TUN, Forwarding und Rückroute. ICMP darf als Hilfsmittel dienen, aber sein Fehlen ist kein Beweis für eine gesperrte TCP-Verbindung.

| Quelle → Ziel           | Transport          | Prüffunktion                 |
|-------------------------|--------------------|------------------------------|
| Deployment → LXC        | SSH/TCP            | Bundle, Installer, Status    |
| TBS → Node Gateway      | WebSocket/TCP      | Node-Anmeldung, Matrix, Ack  |
| Fachkern → Gateway      | HTTP/WebSocket/TCP | Ereignis/Steuerung nach TOML |
| Observability → Dienste | HTTP/TCP           | Health und Metrics           |
| TBS/PBX ↔ SIP Switch    | SIP und RTP/UDP    | Signalisierung plus Medien   |
| IoT Gateway ↔ Broker    | MQTT/TCP           | Topic, State, Command/Ack    |

### 25.11 Statische Prüfungen gegen echte Laufzeit abgrenzen

Nach dem Deployment werden zunächst statische Prüfungen ausgeführt: Inventory-Validator, `tools/check_full_system_integration.py`, ggf. generierte Protokollinventur und `test --profile full --validate-only`. Eine statische Prüfung kann mit einem bekannten Konfigurationsmismatch fehlschlagen, obwohl der Code gebaut werden kann. Diese Abweichung muss geklärt werden, bevor aus einem Dashboard ein Gesamturteil abgeleitet wird. Umgekehrt kann ein statischer PASS keine reale SDR-Clock, Firewall, NFS-Mount oder Endgeräte-Firmware nachweisen.

Danach folgt Smoke gegen laufende Dienste, anschließend der mutierende Full-Test mit eigens markierten ISSIs/GSSIs. Das Fault-Profil stoppt Units und wird erst nach erfolgreichem Full-Test in einem entbehrlichen Testsystem gefahren. Seine Reportdateien enthalten einzelne Checks, Dauer und Evidenz; ein `failed=0` ohne Blick auf Skips und aufgeräumte Fixtures ist unvollständig. Beim Abbruch alle betroffenen Units explizit auf `active` und `ready` prüfen. Mit `--list-scenarios` lässt sich der tatsächlich verfügbare Szenariensatz des Runner-Builds anzeigen.

Die On-Air-Abnahme wird separat durchgeführt. Der Mock-TBS des E2E-Runners belegt JSON-/WebSocket-Verträge und fachliche Zustandsübergänge, nicht HF, MAC, LLC oder Endgeräteinteroperabilität. Pro Funkgerät Hersteller, Modell, Firmware, Codeplug und Testzeit dokumentieren. Die Evidenzvorlage wird mit `validate_on_air_evidence.py` auf Vollständigkeit geprüft. Ein normativer Konformitätsbericht benötigt darüber hinaus die einschlägige ETSI-Ausgabe, definierte Messmethodik, Golden Vectors und reale Testergebnisse.

### 25.12 Backup, Restore und Neuaufbau nach Hostverlust

Ein Backupplan unterscheidet Configuration, Binärstand, Fach-State, temporären Cache, durable Spools und Archiv. Die wichtigste Wiederherstellungsfrage lautet: Welche dieser Daten sind nach dem letzten Backup fachlich neu? Ein pauschales Zurückkopieren aller alten Dateien kann eine aktuelle Teilnehmerpolicy, einen bereits quitierten SDS-Status oder neue Aufnahmen überschreiben. Für jeden Dienst in Kapitel 17 den `storage`-Block und den



tatsächlichen Dateipfad der Unit prüfen. Bei der TBS `config.toml`, `.fallback`, Edge-Policy-Cache, Event-Spool, lokale Aufnahmen und Audiofiles getrennt sichern.

Nach einem verlorenen LXC zuerst Grundimage mit derselben OS-/Runtimebasis wiederherstellen. Dann den gepinnten Quellstand oder das archivierte Bundle verwenden, passende Unit und TOML installieren, State mit korrektem Eigentümer einspielen und erst danach starten. `live/ready` sind die erste Prüfung; die Fachprobe liest und verändert ein Testobjekt und kontrolliert die Replikation zur TBS. Wird ein anderer Softwarestand genutzt, vor dem Produktivstart eine Migration in einem Test-LXC durchführen. Beim IP Gateway zusätzlich TUN-/Route-/NAT-Zustand des neuen Containers nachweisen; beim SIP Switch Asterisk-Includes und Registrierung; bei Media Library Archivmount und Assetindex.

Ein Restore der TBS beginnt ohne Aussendung mit Parser, SDR-Probe und Frequenz-/Identitätsvergleich. Die `.fallback` bleibt unabhängig vom restaurierten Primärstand. Nach dem Start erst Broadcast und Registrierung, dann Ruf/SDS/Paketdaten, anschließend Core-Reconnect und Replay prüfen. Alte, noch nicht quittierte Spool-Einträge können doppelte Aktionen bewirken, wenn Empfänger und Ackzustand nicht konsistent restauriert wurden. Deshalb Message-IDs und letzte Revision vor und nach dem Restore vergleichen.

Die Backupqualität wird nicht an der Existenz einer Archivdatei gemessen, sondern an einer wiederholten Restore-Probe. Mindestens eine Probe pro Dienstklasse durchführen: autoritativer Datenkern, Medien-/Recordingpfad, Gateway/Routing, Anwendung und TBS. Die Proben erhalten einen Zeitstempel, Quellversion, Prüfsumme und einen echten Fachtest. Ein ungetestetes Backup ist nur eine Vermutung über Wiederherstellbarkeit.

### 25.13 Bekannte Dokumentations- und Projektgrenzen beim Ausrollen

Der aktuelle Commit enthält mehrere Dokumentationsgenerationen. Der historische Kompletthandbuch und das E2E-Runbook beschreiben an Stellen 17 Backends; das aktuelle Inventory enthält 25. Das Provisioning-Installationsdokument nennt ältere Branch-/Patch-Schritte, obwohl der Dienst im betrachteten Workspace vorhanden ist. Die Quell-TOMLs, Installer und Units haben bei Widersprüchen Vorrang. Gerade bei Control Room ist der `Beispiel-config_target` gegen den Unit-Pfad zu prüfen. Diese Punkte sind keine bloßen Fußnoten: Eine Installation kann technisch „erfolgreich“ enden und trotzdem eine veraltete Konfigurationsdatei lesen.

Weitere Grenzen sind funktional. Die TBS kann lokal weiterarbeiten, aber zentrale Call-/Medien-/Transitfunktionen sind bei Isolation nicht verfügbar. Der Security-Core-/KMF-Code und eine konfigurierte Klasse beweisen keine vollständig interoperable Schlüsselverteilung. Ein PDU-Parser mit vorhandenem Einstiegspunkt kann noch offene Runtime-Pfade oder fehlende Golden-Vector-Tests haben. Die generierten Matrizen nennen deshalb Status und Datei; sie sind keine ETSI-Zertifizierung. MQTT/HA, SIP/PBX, WAP, TTS und Recorder benötigen je eigene externe Gegenstellen und End-to-End-Proben.

Die Handbuchausgabe selbst basiert auf einem Repository-Snapshot und enthält keine Messung einer live laufenden Anlage. Für einen konkreten Standort werden vor dem ersten Sendebetrieb die aktuelle Remote-Version, lokale Änderungen, reale Hardware und regulatorischer Rahmen erneut geprüft. Jede Abweichung zum hier dokumentierten Commit wird als Delta im Standortprotokoll festgehalten. So bleibt eine umfangreiche Referenz anwendbar, obwohl sich der Code nach dieser Ausgabe weiterentwickelt.

## 26 Technische Datenblätter für Schnittstellen und Zustände

Dieses Kapitel fasst die betrieblichen Verträge der wichtigsten Protokollgrenzen zusammen. Zahlen aus den Beispiel-TOMLs sind Standortbeispiele; dynamische Ports, IP-Adressen und Zeitlimits sind nach dem ausgerollten Build und der aktiven Konfiguration zu lesen. Eine offene TCP-Verbindung beweist keine erfolgreiche Fachtransaktion. Für jede Grenze werden daher Transport, Identität, Nutzlast, Bestätigung und Fehlerbild getrennt betrachtet.

### 26.1 TBS-Luftschnittstelle und Zeittakt

Die Basisstation erzeugt und empfängt TETRA-Bursts über den konfigurierten SDR. PHY, MAC, LLC, MLE, MM und CMCE sind unterschiedliche Schichten: PHY benötigt passende Frequenz und Timing; MAC weist Ressourcen zu; LLC transportiert logische Nachrichten; MLE behandelt Zelle und Mobilität; MM verwaltet Registrierung; CMCE führt Ruf- und SDS-Signalisierung. Ein Fehler in einer unteren Schicht kann oben wie eine Policy- oder Call-Störung aussehen. Deshalb die erste sichtbar abweichende PDU beziehungsweise SAP-Grenze bestimmen.

Die TBS-Vorlage nennt `stack_mode = "Bs"`, `SoapySDR`, `Sample Rate`, `TX-/RX-Frequenz`, `Center Frequency` und `Kanalnummern`. `cell_info` enthält `Träger`, `Band`, `Duplexrichtung` und `Offset`. Diese Angaben dürfen nicht unabhängig voneinander verändert werden. Bei zwei benachbarten Trägern muss das SDR-Passband beide Kanäle abdecken. Die effektiven Werte werden an SDR und Funkgerät gemessen, nicht nur aus TOML gelesen. Ein Lasttest umfasst Uplink, Downlink und Ruf mit wechselndem Sprecher.

Technische Evidenz: SDR-Probe; Spektrum; TBS-Log mit Zellidentität; Endgerätehersteller, Firmware und Codeplug; registrierte ISSI; hörbarer Testcall. Der PDU-Katalog in Kapitel 21 gibt Parser-/Encoder- und Teststatus des Quellstands an, keine pauschale Funkkonformität. Für einen Normbezug die im ETSI-Quellenregister festgelegte Fassung und eine definierte Messmethode verwenden.

### 26.2 WebSocket zwischen TBS und Node Gateway

Der TBS-Verwaltungspfad verbindet die Basisstation mit `/ws/node` am Node Gateway. Darüber werden Node-Identität, Capabilities, Steuerereignisse und die Core-Health-Matrix transportiert. Backends benutzen ihren in der Konfiguration genannten Gateway-Pfad, im Open-Lab-Beispiel häufig `/ws/backend`. HTTP auf demselben Listener und ein funktionierender WebSocket sind getrennte Tests; ein 200 auf `/health/live` des Gateway sagt nichts über die angemeldete TBS.

Wichtige Zustandswerte sind Verbindung, letzte Matrixrevision, Empfangszeit, Lease-Frische und Ack zu einer Aktion. Der Gateway sendet vollständige Snapshots regelmäßig; die TBS soll veraltete Revisionen nicht zur Verlängerung der Lease nutzen. Nach Ausfall des Monitors kann der TCP-Socket noch offen sein, während zentrale Dienstverfügbarkeit bereits abgelaufen sein muss. TBS `/api/edge-fallback` und Gateway `/api/v1/core-services` werden deshalb nebeneinander gelesen.

Bei einer Störung zuerst Host/Port, dann Pfad/Upgrade, dann Node-ID und Payload/Revision prüfen. Nicht vorschnell einen Gateway-Neustart ausführen, wenn nur ein einzelner Core-Dienst `ready=503` meldet. Der Fallback wechselt in diesem Fall dienstbezogen; bei Gateway-Ausfall oder Matrix-Lease-Ablauf ist Isolation der erwartete Zustand.

## 26.3 HTTP-Management, Health und OpenAPI

Jeder LXC-Dienst hat im Inventory einen Beispiel-TCP-Port. Viele Dienste bieten `/health/live`, `/health/ready`, `/metrics`, `/openapi.json` und `/api/v1/...` `live` beantwortet, ob der Prozess lebt; `ready` kann von Fachabhängigkeiten abhängen. `/metrics` ist Text im Prometheus-Format. `/openapi.json` beschreibt deklarierte Pfade des laufenden Dienstes, aber dynamische Handler oder ältere Pfade können außerhalb des statischen Katalogs liegen. Kapitel 16 verzeichnet 426 methodenbezogene Quellpfade des festgehaltenen Commits.

Management-APIs im Open Lab sind überwiegend ohne durchgängigen Login, Token und TLS. Ein Client im gleichen Netz kann damit schreibende Aktionen auslösen. Deshalb keine ungeschützte Portfreigabe in ein normales Client- oder Internetsegment. Für Fehlersuche GET-Methoden zuerst, POST/PUT/PATCH/DELETE nur mit markiertem Testobjekt, Sicherung und dokumentierter Wirkung. HTTP-Status, JSON-Fehler, Audit und nachgelagerter Ack gehören zusammen; ein 202 Accepted kann lediglich eine Aktion in die Queue aufgenommen haben.

Eine Versionsdiagnose vergleicht `git rev-parse HEAD`, aktives Binary/Unit, Quell-OpenAPI und tatsächlich abgerufenes `/openapi.json`. Ein 404 auf einer Handbuchroute ist ein Hinweis auf Build- oder Pfaddifferenz, nicht automatisch eine Netzwerkstörung. Ein 503 bei `ready` wird über Inventory-Abhängigkeiten eingegrenzt. Ein 500 verlangt Journal und die konkrete Request-ID.

## 26.4 SIP-Signalisierung und RTP-Medien

SIP und RTP sind getrennte Transportwege. Die Beispiele führen den zentralen SIP-Switch über Port 8300/TCP für Management, 5060 für SIP und einen konfigurierbaren UDP-Bereich für RTP. Die lokale TBS-Bridge und der lokale Asterisk können weitere Ports belegen; der genaue Listener steht in TBS-/Switch-TOML und im gerenderten Asterisk. Eine erfolgreiche SIP-Registrierung oder ein klingelnder Ruf belegt noch keinen bidirektionalen RTP-Pfad.

Bei einem Ruf die INVITE-/Antwortfolge, Dialog-ID, SDP-Adressen, gewählten Codec, RTP-Quell-/Zielports, Paketzähler und Release prüfen. PAI überträgt im vorgesehenen Aufbau die Anruferidentität, während Authentisierungsname und Trunk-Konto eine andere Aufgabe haben. DTMF benötigt eine eigene Probe. Bei NAT wird die Adresse im SDP gegen die tatsächliche Route und Firewall geprüft. Einseitiges Audio deutet oft auf genau eine fehlende Richtung; der Gegentest wechselt Sprecher und ruft auch aus der Gegenrichtung an.

Der Phase-11c-Fallback hält die native TBS-Bridge lokal, schaltet aber die externe Registrierung des Asterisk zwischen zentralem Switch und PBX-Direktweg. Pending- und Recovery-Timer verhindern hektisches Umschalten. Für den Failoverbeleg stehen Status der State Machine, aktive Registrierung, neuer Call über den gewünschten Weg und Rückkehr nach stabiler Zeit im Bericht. Bereits bestehende Gespräche werden nicht still auf einen neuen Pfad migriert.

## 26.5 MQTT, retained State und Command-Acks

MQTT verbindet IoT Gateway, Broker und Empfänger wie Home Assistant. Das Open-Lab-Beispiel nutzt 1883/TCP. Topic-Namen und Prefixe sind frei konfigurierbar; ein „richtiger“ Port mit falschem Topic ist kein funktionierender Anschluss. Der Eventpfad kann retained State und Discovery-Nachrichten veröffentlichen. Ein retained Wert wird bei Subscription sofort wiedergegeben und kann nach Ausfall veraltet sein; Availability und Zeitstempel müssen deshalb mitgelesen werden.



Ein Commandpfad wird separat geprüft: Publish, Annahme des IoT Gateways, Berechtigung und Weiterleitung, Ausführung durch Gateway/TBS, Ack, Zustandspublish. Eine Command-ID oder Correlation ID verbindet die Schritte. Broker-Publish-Ack ist nicht gleich Funk-Ack. Nach Reconnect darf eine retained Command-Nachricht keine zweite reale Aktion auslösen. Topics für Status und Kommandos werden getrennt, damit eine Rückmeldung keinen Loop erzeugt.

Die konkrete HA-Entity wird im Broker mit einem unabhängigen Subscriber gegengetestet. Fehlt sie nur in Home Assistant, liegt der erste Fehler wahrscheinlich bei Discovery oder Mapping. Fehlt schon das Brokerereignis, beginnt die Diagnose beim IoT Gateway und der TBS-/Core-Quelle. Zugangsdaten und produktive Automationen bleiben aus dem offenen Labor heraus.

## 26.6 SDS, Spool und Zustellsemantik

SDS und Status können lokal, netzweit oder über Anwendungen zugestellt werden. Ein SDS Router führt Zielentscheidung und Offline-Store-and-forward. Die TBS kann während Isolation lokal zustellen und nach Wiederkehr ein begrenztes, dauerhaftes Ereignis-Replay ausführen. Der `edge-event-spool.jsonl` ist daher kein bloßer Cache: vor Löschen oder Restore sind Message-ID, Quittung und bereits erfolgte lokale Zustellung zu klären. Sprache und RF-Frames sind zeitkritisch und gehören nicht hinein.

Die fachliche Zustellkette unterscheidet Annahme eines Auftrags, Queueing, Funkversuch, Delivery-Ack und ggf. Anwendungseffekt. Ein 200/202 am REST-Endpunkt kann nur die erste Stufe belegen. Für Gruppen-SDS markiert `air_fallback_local_delivered` eine bereits lokal erreichte Teilmenge; beim Replay werden entfernte Legs bedient, ohne denselben Ursprung erneut zu adressieren. Bei Task Workflow ist ein SDS-Kommando darüber hinaus ein Zustandswechsel einer Task und verlangt idempotente Verarbeitung.

Grenzwerte für Payload, Queue und TTL stehen in den jeweiligen TOMLs. Unter Last werden maximale Anzahl, Bytes, Diskplatz und Wiederanlaufzeit gemessen. Zu große Nachricht oder abgelaufene TTL muss nachvollziehbar abgelehnt oder als Fehler ausgewiesen werden. Eine verlorene Nachricht wird mit genau einer Test-ID durch die gesamte Kette verfolgt.

## 26.7 SNDCP, PDP, TUN, Routing und NAT

Paketdaten werden vom Funkgerät über SNDCP und einen PDP-/NSAPI-Kontext zur TBS beziehungsweise zum Packet Core geführt. Der IP Gateway setzt danach TUN, Route, Firewall und optional NAT/DNS um. Die Adresse des Endgeräts muss im Pool eindeutig sein, die Netzroute zur Gegenstelle und der Rückweg stimmen. Ein TUN-Interface im Container braucht Device-Durchreichung und Rechte; das reine HTTP-Management des IP Gateway liefert keinen Beweis für diese Voraussetzung.

Für den Test drei Pakete unterscheiden: kleines IP-Paket zu einer bekannten Adresse, größeres Paket nahe der MTU und DNS-Anfrage. Beim kleinen Paket die Capture-Positionen TBS, Packet Core, TUN und externes Ziel vergleichen. Beim großen Paket Fragmentierung, Reassembly-Limits und Queue-Zähler prüfen. Bei DNS Fehler zwischen Erreichbarkeit des Servers und Namensantwort trennen. NAT-Modus erwartet eine passende SNAT-Regel; gerouteter Modus eine explizite Rückroute. In beiden Fällen werden Forwarding und Firewall in zwei Richtungen kontrolliert.

Der Kontext-Lebenszyklus umfasst Aktivierung, Adresse, Traffic, Inaktivität, Abmeldung und Freigabe. Nach einem Neustart dürfen alte Leases nicht still mit neuen Teilnehmern

kollidieren. Bei Core-Ausfall die konfigurierte lokale TBS-Autorität prüfen und nach Recovery auf doppelte Kontexte achten. Paketmitschnitte sind auf Testverkehr zu begrenzen.

## 26.8 Medienformate, TTS und Aufnahmen

Media Switch transportiert codierte Sprachframes zwischen Call Legs. Recorder kann einen Tap außerhalb des zeitkritischen Rufpfads erhalten. Media Library importiert, verarbeitet, genehmigt und verteilt Assets; Piper erzeugt TTS-Audio. Die TBS kann lokal aufnehmen und freigegebene Audiodateien cachen. Diese Funktionen haben unterschiedliche Zustände und dürfen im Fehlerbericht nicht zu einem einzigen „Audio funktioniert“ zusammengezogen werden.

Bei einem Asset stehen Quelle, Abtastrate/Format, Dauer, Lautstärke, verarbeitete Variante und Genehmigung im Vordergrund. Preview ist eine lokale Medienprobe; On-Air-Playout ist ein weiterer Schritt mit Zell-/Gruppenressource. Bei Aufnahmen werden Rohdatei, JSON-Metadaten, zentrale Tap-Session, Archiv-ID und Prüfsumme unterschieden. Die Archivkopie zählt erst, wenn sie wieder gelesen und abgespielt wurde.

Die Codec-Grenze wird durch einen hörbaren Zweiwege-Test mit echtem Gerät und dokumentierter Firmware abgenommen. Ein Framezähler kann bei falsch formatiertem Audio trotzdem steigen. Bei TTS sind Synthese und Funkansage getrennte Acks. Ein fehlendes NFS-Archiv darf nicht unbemerkt den lokalen Aufnahme- und Playoutpfad blockieren.

## 26.9 Persistenz, Dateisysteme und Backups

Die Dienste benutzen unterschiedliche Zustandsorte aus ihren `storage`-Blöcken. Einige TOMLs enthalten Datenbank- und Backup-Pfad, andere Spools, Caches oder Archivverzeichnisse. Der operative Unterschied ist wichtig: Ein Cache darf bei Bedarf aus autoritativen Daten neu entstehen; ein nicht quittierter Spool oder eine Originalaufnahme kann unwiederbringliche Vorgänge enthalten. Die TBS-`.fallback` ist eine bekannte Startkonfiguration, nicht bloß eine automatische Kopie des aktuellen möglicherweise fehlerhaften Stands.

Ein Backup erfasst Binary-/Schema-Version, TOML, Dateirechte, State, Prüfsumme und Zeitpunkt. Bei Restore werden alle zusammenpassend zurückgespielt und anschließend eine Fachtransaktion ausgeführt. NFS ist im Projekt für Medienarchivierung optional; der Live-State bleibt lokal. Ein Dateisystem mit ausreichend freiem Platz und geprüften Eigentümern verhindert Fehler, die sonst als API- oder Funkstörung erscheinen.

Retention für Logs, Aufnahmen, Diagnosepakete und Captures wird nach tatsächlichem Datenvolumen geplant. Ein voller Datenträger kann einen Backenddienst startfähig erscheinen lassen, während neue Ereignisse nicht persistieren. Daher Diskplatz und Schreibprobe in die Readiness-/Wartungsroutine aufnehmen, soweit der konkrete Dienst sie nicht bereits abdeckt.

## 26.10 Metriken, Logs und Traces

Observability fragt die Dienstendpunkte ab und kann Logs/Traces aufnehmen. Prometheus-Metriken haben Namen, Labels, Zeitpunkte und Zählersemantik; ein fehlender Scrape ist keine Null. Für jede Alertregel den Ursprung, den erwarteten Schwellwert, Deduplizierung und die Reaktion dokumentieren. Eine Silence unterdrückt Benachrichtigung, nicht den zugrunde

liegenden Fehler. Alarm Workflow kann zusätzlich eine fachliche Alarmakte und Eskalation führen.

Die Korrelation eines Vorfalls beginnt mit UTC-Zeit und einer Test-ID. Am Gateway wird die Node-/Core-Matrix gelesen, am Fachdienst der konkrete Zustand, in Observability Log-/Trace-Eintrag und am Endgerät die Wirkung. Aggregierte Kacheln nennen idealerweise den letzten Pollzeitpunkt. Bei widersprüchlichem Status die Origin-Quelle direkt abfragen und eine neue Testtransaktion auslösen.

Ein Diagnosepaket kann Teilnehmerkennungen, SIP-Nummern, URLs, Payloads oder Zugangsdaten enthalten. Vor Weitergabe nur die für den Vorfall erforderlichen Ausschnitte mit entfernten Geheimnissen verwenden. Für eine reproduzierbare Reparatur gehören relevante Zählerstände vor/nach Test, Logs mit UTC und der genaue Softwarestand in die Akte.

### 26.11 Directory, WAP und Anwendungsintegration

Directory liefert Namen und Labels für Geräte, Gruppen, Basisstationen und Status; es ist nicht die autoritative Zulassungsquelle. Eine im Control Room sichtbare Bezeichnung kann fehlen oder veraltet sein, während ISSI/GSSI im Subscriber-/Group-Core korrekt bleibt. Bei einem Anzeigeproblem zuerst IDs und Rohantwort vergleichen, dann Directory-Sync oder Cache. Eine Änderung des Namens darf keine neue Funkidentität erzeugen.

WAP- und Formularpfade dienen älteren Geräten und dem Task Workflow. XHTML Basic und WML werden je nach Endgerät unterstützt; die Pfade /x und /w im Task Workflow sind im Open Lab ohne echte Authentisierung der issi-Query. Deshalb ist diese Angabe ein Testkontext, kein Identitätsbeweis. Ein erfolgreicher HTTP-Test auf dem Desktop ersetzt keine Darstellung und Eingabe am Funkgerät. MIME-Typ, Seitenlänge, Zeichenkodierung und Navigationsrückweg werden auf dem konkreten Gerät geprüft.

Application Gateway bindet Webhooks, Connectoren, Regeln und Vorlagen an. Ein Event kann dort mehrfach versucht werden. Correlation ID, Zielantwort, Retry und deduplizierter Anwendungseffekt sind die vier entscheidenden Belege. Externe Ziele werden im Labor mit einer kontrollierten Testinstanz verbunden; ein fehlgeschlagener Webhook darf nicht unbemerkt eine Funkaktion erneut auslösen.

### 26.12 Protokollabnahme als durchgehende Beweiskette

Ein technischer Bericht sollte nicht nur einen Port als „offen“ markieren. Für jeden Pfad werden dieselben fünf Fragen beantwortet: **Wer** hat mit welcher Identität gesendet? **Was** wurde kodiert oder angefordert? **Wo** wurde die Nachricht angenommen? **Welche Bestätigung** kam vom tatsächlichen Ziel? **Was geschah nach Ausfall und Wiederholung?** Diese Struktur gilt für WebSocket-Commands, SIP/RTP, SDS, MQTT, Medienassets und PDP/IP gleichermaßen.

Die statische PDU-/SAP-Inventur liefert den Codepfad. Der Open-Lab-E2E-Test liefert Vertrags- und Zustandsbelege mit Mock-TBS. Eine reale On-Air-Probe liefert Verhalten der Funkgeräte und Hardware. Nur die Kombination dieser Ebenen erlaubt eine belastbare Aussage zum konkreten Build und Testaufbau. Für ETSI-Konformität sind darüber hinaus Normfassung, Testmethode, Messgrenzen und dokumentierte Ergebnisse notwendig.

| Beweisart                | Geeignet für                     | Nicht ausreichend für              |
|--------------------------|----------------------------------|------------------------------------|
| Quellcode/Schema         | vorhandene Pfade und Typen       | Laufzeit, RF und Interoperabilität |
| Unit-/Golden-Vector-Test | definierte Parser-/Encoderfälle  | vollständigen Funkablauf           |
| Mock-E2E                 | Core-Verträge und Recovery       | SDR, Endgeräte und Spektrum        |
| On-Air mit Gerät         | reale Zell-/Ruf-/Datenwirkung    | alle optionalen Codepfade          |
| Normprüfung              | definierten Konformitätsanspruch | unbelegte Produktivfreigabe        |

Für eine reproduzierbare Entscheidung wird zu jedem Test zusätzlich angegeben, was **nicht** geprüft wurde. Ein Mock-Lauf ohne SDR nennt ausdrücklich die fehlende RF-Probe. Ein On-Air-Test mit nur einem Hersteller nennt die fehlende Interoperabilitätsprobe. Ein erfolgreicher SIP-Dialog ohne RTP-Capture nennt die offene Medienrichtung. Diese Grenzen stehen im Ergebnisfeld und werden bei der nächsten Abnahme gezielt geschlossen.

Die Reihenfolge der Beweise ist praktisch: zuerst Konfiguration und Codepfad, dann kleiner Unit-/Vertragstest, danach vollständiger Dienstverbund und zuletzt Gerät/Messplatz. Bei einem Fehler zurück zur ersten abweichenden Stufe gehen. Ein Debug-Fix auf einer späteren Stufe kann sonst ein tieferes Problem verdecken, etwa einen alten Policy-Cache hinter einer grünen UI oder einen einseitigen RTP-Pfad hinter einem erfolgreichen SIP-REGISTER. Nach jeder Reparatur die nächsthöhere Stufe erneut fahren und Rohbelege mit derselben Test-ID verbinden.

## 27 Kopierbare Prüf- und Einsatzblätter

Die Vorlagen in diesem Kapitel werden für einen konkreten Standort kopiert und ausgefüllt. Ein leeres Feld gilt nie als bestanden. Jede Messung erhält UTC-Zeit, Softwarestand und einen Link auf Rohbelege; Geheimnisse und echte Schlüssel werden nicht eingetragen. Die Formblätter ersetzen die ausführlichen Anleitungen nicht, sondern machen eine Schichtübergabe oder spätere Reproduktion möglich.

### 27.1 Blatt für neue Basisstation und Funkzelle

Vor dem Start die Hardwarekette physisch abnehmen. Ein Foto des Aufbaus, die Bezeichnung des Messgeräts und dessen Kalibrierstand gehören zur Akte. Die Testleitung trägt den genehmigten Frequenzrahmen ein und vergleicht ihn mit effektiver TBS-Konfiguration und Messung. Die Tabelle nimmt Soll- und Istwert getrennt auf; bei einer Abweichung bleibt die Anlage bis zur Klärung ohne reguläre Aussendung.

| Feld                              | Soll am Standort         | Gemessen / geprüft | Beleg                        |
|-----------------------------------|--------------------------|--------------------|------------------------------|
| TBS-Host, Binary, Commit          | aus Standortplan         | einzutragen        | git rev-parse, systemctl cat |
| SDR-Modell, Seriennummer, Treiber | aus Aufbauplan           | einzutragen        | SoapySDR-Probe               |
| Clock und Sample Rate             | aus RF-Plan              | einzutragen        | SDR-/Messprotokoll           |
| TX/RX, Träger und Duplex          | genehmigte Werte         | einzutragen        | Spektrum/<br>Frequenzzähler  |
| MCC/MNC/LAC/Colour Code           | programmierter Zellplan  | einzutragen        | TBS und Endgerät             |
| Gain, Antenne/Dummyload           | Messaufbau               | einzutragen        | Pegel und Rücklauf           |
| Gerät A/B mit Firmware            | Testliste                | einzutragen        | Foto/Codeplug-ID             |
| Registrierung und Wiederanmeldung | erfolgreich              | einzutragen        | TBS-MM-Log                   |
| Gruppenruf beide Richtungen       | hörbar, sauberer Release | einzutragen        | Audio/Call-ID                |
| SDS und Paketdaten                | Ack und IP-Rückweg       | einzutragen        | Message-ID/Capture           |

Bei Dual-Carrier einen zweiten Satz für Träger, Spektrum, Belegung und Endgeräteverhalten ergänzen. Ein bestandener Test auf dem Hauptträger kann einen falsch abgestimmten Nebenträger verdecken. Ergebnis: **freigegeben, mit Einschränkung** oder **nicht freigegeben**; Einschränkungen benennen genaue Funktion, betroffene Geräte und Ablaufdatum für Nachtest.

## 27.2 Blatt für einen Backend-LXC

Dieses Blatt wird je Dienst einmal ausgefüllt. Die Sollwerte stammen aus dem standortangepassten Inventory und der gerenderten TOML. Die tatsächliche Unit kann von einem alten Dokument abweichen; deshalb `systemctl cat` und aktive Config-Datei zusammen lesen. Bei Control Room den bekannten Pfadunterschied zwischen Inventory-Beispiel und Unit ausdrücklich prüfen.

| Feld            | Eintrag                                 | Rohbeleg / Ergebnis        |
|-----------------|-----------------------------------------|----------------------------|
| Dienst und Host | Name, IP, CTID                          | Inventory-Revision         |
| Softwarestand   | Commit, Binary/Script, Paketversion     | Git, Hash, Unit            |
| Abhängigkeiten  | Namen und erreichbare Ziele             | plan, direkte Health-Probe |
| Config          | Zieldatei, Eigentümer, gesicherte Kopie | systemctl cat, Dateirechte |
| Listener/Port   | Bindung, Firewall-Quelle                | ss, Quellhost-Anfrage      |
| live / ready    | HTTP-Status mit Zeitpunkt               | Antwort/Journal            |
| Fachprobe       | Test-ID, Route, Zielwirkung             | API, Gegenstelle, Ack      |
| Persistenz      | Pfad, Backup, Restore-Probe             | Prüfsumme und Testfall     |
| Ausfallprobe    | degradiert und erholt                   | zwei Statuszeitpunkte      |
| Abschluss       | Tester, UTC, offene Punkte              | Berichtlink                |

Eine Liveness-Probe mit 200 und Readiness mit 503 ist kein pauschaler Ausfall und kein Erfolg: den fehlenden Vorgänger und die fachliche Auswirkung notieren. Ein Test mit schreibender Route erhält ein einzigartiges Testobjekt und einen belegten Cleanup. Nach einem Dienstneustart den State erneut lesen, bevor der LXC als abgenommen gilt.

## 27.3 Blatt für einen netzweiten Ruf oder eine SDS

Für eine Ende-zu-Ende-Störung eine einzelne Transaktion auswählen. Mehrere gleichzeitige Testgespräche oder Nachrichten erschweren die Zuordnung von Logs, Countern und Medien. Die Tabelle ist die Zeitlinie vom Ursprung zum Ziel. Bei jedem Übergang steht die erste Abweichung; spätere Fehler sind häufig nur Folgeeffekte.

| Grenze             | Ruf: Call-ID, Floor, Frame | SDS: Message-ID, Ack    | UTC/Beleg   |
|--------------------|----------------------------|-------------------------|-------------|
| Gerät A → TBS A    | Setup/UL beobachtet        | SDS/Status empfangen    | einzutragen |
| TBS A → Gateway    | Node- und Sessionereignis  | Versand-/Replayeintrag  | einzutragen |
| Gateway → Fachkern | Call-Control-Zustand       | Router-Zielentscheidung | einzutragen |
| Fachkern → TBS B   | RouteReady/Frame           | Delivery-Leg            | einzutragen |

| Grenze          | Ruf: Call-ID, Floor, Frame | SDS: Message-ID, Ack     | UTC/Beleg   |
|-----------------|----------------------------|--------------------------|-------------|
| TBS B → Gerät B | hörbarer Inhalt            | tatsächlich zugestellt   | einzutragen |
| Rückweg/Ack     | Floor-Wechsel/Release      | Delivery-/Anwendungs-Ack | einzutragen |
| Nach Recovery   | keine verwaiste Session    | kein Duplikat            | einzutragen |

Für einen Ruf wird zusätzlich die Medienrichtung B→A getestet. Für SDS im Offlinefall werden Queueaufnahme, Wiederanmeldung, Replay und Deduplizierung einzeln markiert. Das Ergebnis enthält den ersten defekten Übergang, eine überprüfbare Hypothese und den kleinsten nächsten Test.

## 27.4 Blatt für Incident, Reparatur und Rollback

Dieses Protokoll begleitet eine konkrete Änderung. Es ist vor dem Eingriff zu beginnen, nicht erst nach einem geglückten Neustart. Eine „Reparatur“ ohne Vorherbeleg, Rückweg und Abschlussprobe bleibt ein Versuch. Besonders bei Security-Policy, Teilnehmerverwaltung, SIP-Registrierung und Spoolzustand können unbedachte Eingriffe Folgefehler erzeugen.

| Zeitpunkt / Feld | Eintrag                                  | Pflichtnachweis       |
|------------------|------------------------------------------|-----------------------|
| Meldung          | Symptom, betroffene Funktion, Beginn UTC | erste Beobachtung     |
| Umfeld           | TBS/LXC, Commit, Configrevision          | Unit und Git/Hash     |
| Umfang           | Endgeräte, Gruppen, Calls, Integrationen | direkte Fachprobe     |
| Hypothese        | erster fehlerhafter Übergang             | Log/Messung mit ID    |
| Sicherung        | Binary, TOML, State, Spool               | Ort und Prüfsumme     |
| Eingriff         | genau eine Änderung                      | Diff und Ausführer    |
| Wiederholung     | ursprünglicher Fehlerfall                | Vorher/Nachher-Belege |
| Nebenwirkung     | andere Dienste/Richtungen                | kurze Regression      |
| Rückweg          | altes konsistentes Set                   | Restore-Probe         |
| Freigabe         | Tester, UTC, offene Risiken              | E2E-/On-Air-Bericht   |

Wenn der Eingriff fehlschlägt, den vorbereiteten Rückweg ausführen und den dadurch entstandenen Datenunterschied bewerten. Ein altes Backup kann neuere, bereits bestätigte Zustellungen oder Provisionierungsänderungen verlieren. Bei unklarer Datenlage betroffene Fachfunktion im Labor belassen und die Rohdateien sichern, bis eine konsistente Entscheidung möglich ist.

## 27.5 Blatt für Versionswechsel und neue Handbuchausgabe

Vor einem neuen Release werden die automatisch abgeleiteten Tabellen erneut erzeugt. Das Inventory kann Dienste, Ports, Abhängigkeiten oder Pfade ändern; OpenAPI kann Methoden hinzufügen oder entfernen; TOML-Schlüssel können umbenannt werden. Auch ein im Code vorhandener PDU-Parser kann neue Testbelege oder offene Stellen haben. Deshalb werden die Kapitel 14 bis 18 und 21 nicht per Hand aus dem alten PDF abgeschrieben.

| Bereich        | Vergleich                                   | Ergebnis    |
|----------------|---------------------------------------------|-------------|
| Git            | alter/neuer Commit und Tags                 | einzutragen |
| Inventory      | Dienstzahl, Ports, Hosts, Units, depends_on | einzutragen |
| Config         | Backend-TOMLs, TBS-Parser, optionale Werte  | einzutragen |
| Schnittstellen | OpenAPI, WebSocket-/SIP-/MQTT-Vertrag       | einzutragen |
| Funk           | PDU/SAP-Matrizen, zwei Gerätehersteller     | einzutragen |
| Sicherheit     | Auth/TLS-Modus, Policy und KMF              | einzutragen |
| Betrieb        | E2E-Profil, Fault- und Restore-Test         | einzutragen |
| Dokument       | TOC, PDF-Suche, Tabellenrender, Seitenzahl  | einzutragen |

Die Ausgabe trägt danach ein neues Datum und einen neuen Quellenstand. Ein offener Testpunkt bleibt sichtbar; er wird nicht durch eine größere Seitenzahl oder eine grüne Komponentenkachel ersetzt.



## 28 Bekannte Abweichungen im untersuchten Quellstand

Eine „bekannte Abweichung“ in diesem Kapitel ist am gepinnten Repository-Stand statisch nachvollziehbar oder als Grenze in den Projektquellen dokumentiert. Sie ist nicht automatisch eine Störung jeder laufenden Anlage. Vor einer Korrektur den lokalen Build, die ausgerollte Konfiguration und die tatsächlich geladene Unit prüfen. Die Maßnahmen nennen den kleinsten verifizierbaren Schritt, nicht einen pauschalen Neustart des Gesamtsystems.

### 28.1 Control Room: Inventory-Ziel und Unit-Pfad

Im Beispiel-Inventory steht `config_target = "/etc/netcore/control-room.toml"`. Die eingeecheckte Unit `system-backend/control-room/systemd/netcore-control-room.service` startet mit `--config /etc/netcore-control-room/control-room.toml`. Der Deployer schreibt eine gerenderte Datei an das Inventory-Ziel. Dadurch kann der Control Room eine andere Konfiguration lesen, selbst wenn beide Dateien syntaktisch gültig sind. Ein altes Projektdokument nennt einen korrigierten Inventory-Entwurf; für den aktuellen 25er-Aufbau ist die Korrektur an der lokalen Standortdatei und der Unit zu verifizieren.

**Behebung.** Vor `apply` einen einzigen autoritativen Zielpfad festlegen, den Installer und die Unit damit abgleichen und das Inventory entsprechend ändern. Dann `render`, `Dry Run` und `systemctl cat` ausführen. Nach dem Start eine bewusst geänderte, ungefährliche Einstellung gegen die effektive API/UI prüfen. Alte verwaiste Konfigurationsdateien erst nach erfolgreichem Rollbacktest bereinigen. Der Nachweis besteht aus genau dem gelesenen Pfad, Dateihash, Unit-Kommando und einer frischen Fachaktion.

### 28.2 Root-TBS-Beispiel und Gateway-Inventory widersprechen sich

Der Full-System-Audit hat drei Befunde: eine noch auf 24 Dienste begrenzte Prüfliste, fehlende `/metrics//openapi.json`-Marker in `alert-service` sowie die Gateway-Abweichung zwischen `Root-config.toml` (10.0.1.179) und `Inventory` (10.0.20.10). OK: 25 services validiert das Inventory, repariert aber keinen dieser Befunde.

**Behebung.** Für den Laboraufbau TBS-`control_room`-URL, Gateway-Host, Port und `/ws/node` aus einem einheitlichen Netzplan setzen. Dann die tatsächlich gestartete TBS-Datei und die Node-Gateway-Knotensicht vergleichen. Erst danach den Full-System-Audit und einen echten WebSocket-/Ack-Test wiederholen. Eine bloße Änderung der Testdatei, während die laufende TBS eine andere Datei liest, gilt nicht als Reparatur.

### 28.3 Dokumentationen mit 17 statt 25 Diensten

`Docs/NetCore-Tetra-Komplettguide.md` und `Docs/OPEN_LAB_E2E_RUNBOOK.md` beschreiben Abschnitte mit 17 Backend-LXCs. Das aktuelle `deploy/open-lab/inventory.example.toml` enthält 25. Bei einer Installation nach der alten Liste fehlen die später hinzugekommenen IoT-, Hardware-, RF-, Alarm-, Task-, Asset- und SIP-Komponenten. Der E2E-Runner kann je nach Profil dennoch einen historischen Kernvertrag prüfen; daraus folgt keine Vollabnahme aller 25 Dienste.

**Behebung.** Die `plan`-Ausgabe der aktuellen lokalen `inventory.toml` als Soll-Liste verwenden, jeden fehlenden Dienst mit Host, Unit, Port und Abhängigkeiten ergänzen und danach die fachliche Probe aus Kapitel 19 fahren. Die historische Anleitung darf für Installationsdetails einzelner alter Dienste helfen, aber ihre Diensthzahl und IP-Beispiele sind nicht der aktuelle

Sollstand. Im Abschlussbericht genau nennen, welche Dienste getestet und welche bewusst nicht aufgebaut wurden.

## 28.4 Provisioning Core fehlt im 25er-Autodeployment

Provisioning Core liegt im Workspace, jedoch weder in `system-backend/services.toml` noch im Open-Lab-Inventory. Sein Port 8125 und die Abhängigkeit zu Subscriber und Group Core müssen zusätzlich geplant werden. Wer nur `apply` über das 25er-Inventory ausführt, erhält daher nicht automatisch diese gemeinsame Verwaltungsoberfläche. Das ältere Provisioning-Installationsdokument erwähnt Patch- und Branchschritte, die am geprüften `main`-Stand keine verlässliche Anweisung für den Neuaufbau sind.

**Behebung.** Einen getrennten Provisioning-Host/Container mit dem aktuellen Installer und der aktuellen TOML aufsetzen. `systemctl cat, /health/live, /health/ready` und das Dashboard prüfen. Anschließend eine Test-ISSI/GSSI über beide autoritativen Cores anlegen, zur TBS synchronisieren und entfernen. Den Dienst im lokalen Betriebsinventar als **zusätzlich** kennzeichnen, bis eine spätere Projektversion ihn offiziell in den Deployer aufnimmt.

## 28.5 Offenes Managementnetz ohne durchgängige Sicherung

Die Open-Lab-Fachdienste deklarieren in ihren Beispielkonfigurationen und APIs vielfach keinen Login, keine Management-Tokens und kein TLS. Das gilt nicht als versteckter Fehler, sondern als ausdrücklicher Laborstatus. Für einen erreichbaren Client im Managementnetz können schreibende HTTP-Routen sonst unmittelbar wirksam sein. Einzelne Oberflächen mit Rollenmodell sichern nicht automatisch alle dahinterliegenden Dienstports.

**Behebung.** Management-VLAN physisch/logisch abgrenzen, Firewallregeln nach Richtung setzen, nur Testhosts zulassen und keine Portweiterleitung ins Internet oder normale Clientnetz aktivieren. Zugangsdaten für SIP/MQTT/Webhooks getrennt schützen. Für einen späteren regulären Einsatz ist eine projektspezifische Prüfung von Authentisierung, TLS, Autorisierung, Secret-Lebenszyklus und Audit nötig; die bloße Isolation eines einzelnen WebUI-Ports reicht nicht.

## 28.6 OpenAPI-Katalog ist nicht für jeden Dienst vollständig verfügbar

Der Quelltext enthält auswertbare `paths`-Objekte für die Mehrzahl der Dienste. Für Control Room und Hardware Gateway wurde in der statischen Extraktion kein entsprechender Pfadkatalog gefunden. Auch bei Diensten mit OpenAPI können dynamisch zusammengesetzte oder ältere Handler existieren, die nicht in `paths` stehen. Kapitel 16 führt nur deklarierte Methoden/Pfade und nennt seine Grenze ausdrücklich. Eine exakte Requeststruktur kann sich zwischen Quellstand und laufendem Binary unterscheiden.

**Behebung.** Bei einem betroffenen Dienst die laufende `/openapi.json`-Ausgabe versuchen und den tatsächlichen Handler-/UI-Code des Builds lesen. Eine neue Route erst in ein Skript übernehmen, wenn Methode, Requestkörper, Fehlercode und Seiteneffekt in einer Testumgebung nachgewiesen sind. Für die Projektdokumentation den OpenAPI-Katalog im Code ergänzen und mit einem Handler-Vertragstest synchron halten.

## 28.7 Statische PDU-/SAP-Matrizen zeigen offene Pfade

Die generierten Projektmatrizen zählen 77 PDU-Implementierungen und 130 SAP-Primitiven. Sie markieren unter anderem 15 PDUs mit fehlendem oder teilweisem Parser-/Encoderpfad, 62 nicht in `SapMsgInner` verdrahtete Primitive und zahlreiche fehlende Testverweise. Die `IMPLEMENTATION_GAPS`-Datei enthält außerdem TODO-/FIXME-, panic!- und unimplemented!-Treffer. Diese Zahlen sind Suchergebnisse und enthalten nicht zwingend erreichbare Laufzeitpfade, aber sie verhindern eine pauschale Konformitätsbehauptung.

**Behebung.** Für einen konkreten Funkfehler den erreichten PDU-/SAP-Pfad im Quellcode und Trace identifizieren. Dann gültigen und fehlerhaften Rohpuffer als Testfall hinzufügen, Parser/Encoder, Routing und Endgerätewirkung prüfen. Eine Matrixzeile erst ändern, wenn der Test den korrigierten Pfad tatsächlich ausführt. Normkonformität benötigt eine definierte ETSI-Fassung und Mess-/Testverfahren zusätzlich zur statischen Reparatur.

## 28.8 E2E-Mock ist kein On-Air-Nachweis

Der Open-Lab-E2E-Runner kann Gateway, Backend-Verträge, Persistenz und definierte Ausfälle gegen Mock-TBS prüfen. Er beweist weder SDR-Timing und Spektrum noch Interoperabilität eines konkreten Sepura-, Motorola- oder anderen Funkgeräts. Die separate On-Air-Vorlage verlangt Hersteller, Firmware, Zellparameter und Belegreferenzen. Ohne diese Daten bleibt eine Aussage wie „registriert zuverlässig“ für den konkreten Standort unbewiesen.

**Behebung.** Nach erfolgreichem Smoke/Full/Fault-Test reale Funkfälle mit mindestens zwei dokumentierten Gerätevarianten fahren. Registrierung, Gruppenruf, Individualruf, SDS, Paketdaten und Recovery erhalten jeweils UTC, Rohbeleg und Ergebnis. Den Validator der Evidenzvorlage ausführen und fehlende Messungen als offen markieren. Ein korrigierter Mock-Test darf eine fehlgeschlagene Funkprobe nicht überschreiben.

## 28.9 Installer können laufende Dienste vor dem Build stoppen

Der geprüfte Node-Gateway-Installer stoppt die Unit und entfernt eine alte Binärdatei vor dem `cargo build`. Solche Skripte sind bequem für einen frischen LXC, können bei einem Buildfehler aber eine zuvor funktionierende Instanz stilllegen. Auch ein selektives Deployment ergänzt Abhängigkeiten und kann mehr Hosts berühren als der genannte Dienst. Das ist bei jeder Releaseplanung als Ausfallrisiko einzubeziehen.

**Behebung.** Installer des neuen Commits vor Ausführung diffen, altes Binary/TOML/State sichern und den Build möglichst vor dem Stop auf einem gleichartigen Testhost prüfen. `plan <dienst>` und `apply --dry-run` zeigen die Reichweite. Im Fehlerfall das konsistente alte Set zurückspielen, Unit starten und Fachtest durchführen. Ein erfolgreicher neuer Build ohne Tests des abhängigen Funkpfads bleibt nur eine technische Zwischenstufe.

## 28.10 Beispielwerte können gefährliche Standortannahmen enthalten

Die bereinigte TBS-TOML enthält konkrete Frequenzen, MCC/MNC, Dual-Carrier-Beispiele und Netzwerk-URLs. Backend-TOMLs enthalten Lab-Bindungen wie `0.0.0.0`, Loopback-Gegenstellen und Open-Lab-Modi. Diese Werte illustrieren Syntax und Zusammenhang; sie sind keine freigegebene Standortkonfiguration. Ein kopiertes `127.0.0.1` in einem eigenen LXC verbindet den Dienst zu sich selbst, nicht zum Node Gateway auf einem anderen Host.

**Behebung.** Vor dem ersten Start ein Standort-Diff über alle aktiven Konfigurationen erstellen. RF-Werte gegen Berechtigung und Messplan prüfen; Backend-Hosts und Ports gegen Inventory; Datenspeicher gegen Mounts und Eigentümer; Secrets gegen gesicherten lokalen Vorrat. Den gerenderten Output des Deployers lesen und anschließend die effektive Runtime-Datei mit der Unit vergleichen. Jede Abweichung erhält eine bewusste Freigabe und einen Test.

### 28.11 Alte lokale Zustände können einen gesunden Core falsch darstellen

Policy-Cache, Event-Spool, retained MQTT-State und gecachte Media-Assets überleben zeitweise eine Core-Störung. Das ist für Edge-Autonomie gewollt, kann nach einem fehlerhaften Restore oder einer unklaren Revision aber alte Daten sichtbar machen. Ein grünes Dashboard, eine vorhandene Audio-Datei oder eine Broker-Entity beweisen dann nicht, dass die zentrale Autorität aktuell ist.

**Behebung.** Quelle, Revision, Empfangszeit und Ack jedes fraglichen Zustands vergleichen. Beim Edge-Fallback Matrix-Lease und `service_matrix_fresh` prüfen; bei MQTT Availability und retained-Zeit; bei Medien Asset-Version und Cache; bei SDS Message-ID und Delivery-Ack. Nur den nachweislich verwaisten Testzustand bereinigen, nicht pauschal alle Caches oder Spools. Danach einen neuen Vorgang mit eindeutiger ID erzeugen und dessen Weg verfolgen.

### 28.12 Die Grenze dieser Ausgabe selbst

Dieses Handbuch basiert auf einem festgehaltenen Quellcommit, Beispielkonfigurationen und bereitgestellten Normreferenzen. Es hatte für die Redaktion keinen direkten Zugriff auf laufende LXCs, SDR-Messungen, Funkgeräte oder Produktionslogs. Die Tabellen sind daher ein sehr umfangreicher Soll- und Arbeitskatalog; sie können einen standortspezifischen On-Air- und Restore-Bericht nicht ersetzen. Spätere Commits können Routen, Ports, State-Dateien oder den Status einer Lücke ändern.

**Behebung bei neuer Ausgabe.** Commit neu pinnen, Inventory/Services abgleichen, TOMLs und OpenAPI extrahieren, die generierten PDU-/SAP-Matrizen aktualisieren und echte E2E-/On-Air-Evidenz einarbeiten. Jede bislang offene Abweichung bekommt ein Ergebnis mit Testbeleg oder bleibt ausdrücklich offen. So ist die Seitenzahl ein Navigationsmittel, aber kein Ersatz für technische Wahrheit.

## 29 Einsatzreferenz für die ersten fünf Minuten

Diese Kurzreferenz ist für den Moment gedacht, in dem eine laufende Laboranlage ein Symptom zeigt. Sie ersetzt die Ursachenanalyse aus Kapitel 20 nicht. Pro Vorfall zuerst UTC, Host, Dienst, Testkennung und letzte Änderung notieren. Danach nur lesende Proben ausführen. Jede Zeile benennt eine erste fachliche Grenze; die eigentliche Reparatur erfolgt erst nach Backup und reproduzierter Ursache.

### 29.1 Funkzelle verschwunden

**Sofortbild.** Ist die TBS-Unit aktiv? Meldet `journalctl -u tetra.service -b` einen Parse-, SDR- oder Clockfehler? Ist ein TX-Träger am Messplatz vorhanden, und sieht ein bekanntes Endgerät die Zelle? Diese vier Beobachtungen trennen Prozess, Hardware, Aussendung und Endgeräte-Decode. Eine grüne Core-Matrix hat für einen ausgefallenen PHY-Pfad keinen Beweiswert.

**Nächster Schritt.** Bei Parsefehler `config.toml` gegen die unabhängig gepflegte `.fallback` diffen. Bei SDR-Fehler `SoapySDRUtil --probe` sowie Kanal/Gain/Clock kontrollieren. Bei Träger ohne Registrierung Uplink, Broadcastparameter und Policy weiterverfolgen. Kein wiederholtes TX-On/Off ohne gemessenen Last- und Frequenzzustand. Nach Behebung zuerst Messplatz, dann Registrierung und Gruppenruf erneut abnehmen.

### 29.2 Zelle sendet, Gerät registriert nicht

**Sofortbild.** Gerät sieht Zellkennung? Zeigt die TBS einen empfangenen Location-Update-Versuch? Ist die Test-ISSI im Subscriber Core zugelassen und hat die TBS die passende Policyrevision? Ein fehlender Uplink weist zu RX-/Duplex-/Timingpfad; ein sichtbarer Reject zu Identität, Zulassung oder MM-Verfahren.

**Nächster Schritt.** Genau eine ISSI und ein Zeitfenster wählen. Endgeräte-MCC/MNC/LAC und TBS-Broadcast vergleichen; danach Policy und lokale Cacheversion. Eine Sperre bleibt während Core-Ausfall gesperrt. Bei Wiederanmeldung nicht nur den WebUI-Eintrag prüfen, sondern die tatsächliche Registrierung nach einem kontrollierten Neustart.

### 29.3 Lokaler Ruf geht, zweite Zelle bleibt stumm

**Sofortbild.** Gateway kennt beide TBS? Mobility Core nennt für beide ISSIs die erwarteten Serving Nodes? Call Control besitzt eine logische Call-ID und zwei Legs? Media Switch zeigt RouteReady und Frames in beide Richtungen? Diese Reihenfolge verhindert, dass ein fehlender Medienweg als RF-Fehler der Zielzelle behandelt wird.

**Nächster Schritt.** Mit Testcall A→B, dann B→A und eindeutigem Sprecherwechsel arbeiten. Wenn Call-ID fehlt, Gruppenpolicy und Gateway prüfen. Wenn Route/Frames fehlen, Call Control/Media Switch. Wenn Frames an der TBS ankommen, aber kein Ton hörbar ist, Codec/Playout und Funkgerät. Nach Release müssen Legs und Ressourcen auf beiden Seiten verschwinden.

## 29.4 Core gesund, TBS meldet Isolation

**Sofortbild.** TBS /api/edge-fallback mit Gateway /api/v1/core-services vergleichen: WebSocket, service\_matrix\_fresh, letzte Empfangszeit, Revision und erforderliche Dienste. Ein offener Socket mit alter Matrix reicht nicht. Eine ältere Revision darf die Lease nicht erneuern.

**Nächster Schritt.** Zuerst Monitor-/Gatewaypfad und Clock prüfen, dann abhängige Dienste. Während Isolation lokale Funkfunktion und begrenzten SDS-Spool kontrollieren; Security-Policy nicht öffnen. Nach Recovery Hysterese, Replay-Acks und neue Revision abwarten. Erst mit einer neuen Fachtransaktion gilt die zentrale Autorität als wiederhergestellt.

## 29.5 SIP verbindet, Sprache nur in einer Richtung

**Sofortbild.** SIP-Dialog und SDP-Adressen/Ports aufzeichnen. Danach RTP-Pakete an zentralem Switch, lokalem Asterisk und TBS-Bridge für beide Richtungen zählen. Signalisierungsport und RTP-Bereich nicht verwechseln. PAI/Anruferkennung und DTMF sind weitere, getrennt zu prüfende Eigenschaften.

**Nächster Schritt.** Bei fehlendem RTP die konkrete Firewall-/NAT-/Rückroute reparieren. Bei vorhandenen Paketen Codec und Bridge/Playout prüfen. Den normalen zentralen Weg und den direkten PBX-Fallback in separaten neuen Calls testen; laufende Gespräche werden nicht mitten im Dialog umgeschaltet. Externe Registrierungsobjekte nach dem Test auf genau den beabsichtigten Modus zurückführen.

## 29.6 SDS oder MQTT zeigt doppelte Aktion

**Sofortbild.** Eine Message-/Command-ID und alle Acks sammeln: TBS-Annahme, Router-/Spool-Entscheidung, Brokerpublish, tatsächliche Funk- oder Taskwirkung. Bei Gruppen-SDS den Marker für lokale Vorzustellung prüfen. Bei MQTT retained Command und Reconnectzeitpunkt beachten.

**Nächster Schritt.** Nicht den gesamten Spool oder Broker löschen. Den ersten doppelten Übergang identifizieren und den zugehörigen Deduplizierungs-/Ackzustand reparieren. Danach denselben Auftrag erneut senden und nach Recovery beweisen, dass er genau einmal wirkt. Test-Fixtures und retained Testtopics gezielt aufräumen.

## 29.7 PDP aktiv, aber Ziel-IP antwortet nicht

**Sofortbild.** Endgerätadresse und NSAPI notieren. Paket an der TBS, am Packet Core, im IP-Gateway-TUN und am Ziel suchen. Kleine IP-Adresse zuerst, danach DNS und größere MTU-nahe Payloads. Ein funktionierender PDP-Dialog ohne Rückweg ist ein typischer Routing-/NAT-/Firewallfall.

**Nächster Schritt.** Die erste Capture-Position ohne Paket bestimmt die Schicht. Im gerouteten Modus Rückroute zum Adresspool; im NAT-Modus SNAT/Forwarding prüfen. Bei Antwort am TUN ohne Endgerät prüfen Downlink-Kontext, Fragmentierung und Flow Control. Nach Reparatur Kontextfreigabe und erneute Zuweisung kontrollieren, um Adresskollisionen auszuschließen.



## 29.8 Nach Update: Prozess läuft, Fachfunktion fehlt

**Sofortbild.** Aktives Binary, Commit, Unit-ExecStart, gelesene Config-Datei und /openapi.json vergleichen. Beim Control Room besonders Inventory- und Unit-Pfad prüfen. Der Health-Endpunkt kann 200 liefern, während eine abhängige Ressource oder ein altes Datenformat den Fachpfad stört.

**Nächster Schritt.** Den kleinsten reproduzierbaren Test aus dem Vorherprotokoll ausführen. Ist eine Route verschwunden, Build und Handler prüfen. Ist eine neue State-Version unvereinbar, nicht nur das Binary zurückkopieren: das passende Set aus Binary, TOML und State wiederherstellen. Nach Rollback ready, Fachaktion, abhängige Dienste und eine Endgeräteprobe erneut ausführen.

## 29.9 Eskalation und Übergabe

Wenn der erste Fehlerpfad innerhalb von fünf Minuten nicht klar ist, die Diagnose nicht mit planlosen Neustarts überdecken. Eine Übergabe enthält UTC-Zeit, betroffenen Host/Unit, Version, erste abweichende Grenze, Test-ID, Status von TBS/Gateway/Fachdienst, aktuelle Rückfallfunktion und bereits gesicherte Daten. Ein laufender Funkruf, noch nicht quitierte SDS oder eine aktive SIP-Registrierung werden ausdrücklich genannt.

Die nächste Person kann damit gezielt in Kapitel 20 (Symptom), Kapitel 19 (Dienst-Arbeitsblatt), Kapitel 16 (API), Kapitel 17/18 (Konfiguration) und Kapitel 21 (Protokollcode) springen. Nach der Reparatur dokumentiert sie den Rückweg, die vollständige Fachprobe und offene Grenzen im Einsatzblatt aus Kapitel 27. Die Anlage gilt erst als freigegeben, wenn die ursprüngliche Störung und mindestens eine angrenzende Funktion überprüft sind.

## 29.10 Wann ein Test kontrolliert angehalten wird

Ein Laborlauf wird angehalten, wenn RF-Pegel, Frequenz oder ausgestrahlte Bandbreite nicht zum freigegebenen Messaufbau passen; wenn ein Gerät oder Duplexer ungewöhnlich warm wird; wenn unklare Schlüssel-/Policyzustände eine ungewollte Zulassung erzeugen; oder wenn ein Fault-Test einen nicht entbehrlichen Dienst stoppt. Gleiches gilt bei voller Disk auf einem Host mit unquitiertem Spool oder Originalaufnahmen. In diesen Fällen bleibt die Anlage in einem dokumentierten sicheren Zustand, während Log, Konfiguration und Messwerte gesichert werden.

Bei einem SIP-Failover ist eine mögliche doppelte **aktive** externe Registrierung ein Stoppsignal für neue Testcalls. Bei einem Packet-Data-Test mit falscher Rückroute können Pakete in ein fremdes Netz gelangen; zuerst Route und NAT begrenzen. Bei MQTT-Commands mit unklarer Zielzuordnung keine wiederholten Publishes absetzen. Ein 202 Accepted kann eine verzögerte Aktion bereits in die Queue gelegt haben, auch wenn am Endgerät noch nichts sichtbar ist.

Der Anhaltevorgang selbst wird protokolliert: UTC, wer entschied, welche Unit/Verbindung gestoppt wurde, welche Daten noch ausstehen und welche Wiederanlaufbedingungen gelten. Nach der Ursachenklärung mit der kleinstmöglichen Probe beginnen. Erst danach die ursprünglich geplante Ausfall- oder Lastmatrix fortsetzen.

## 29.11 Freigabe nach einem Neustart oder Restore

Ein Dienst, der nach einem Restore `active` meldet, kann dennoch eine alte Policy, verwaiste Session oder inkonsistente Referenz geladen haben. Deshalb wird nach dem Neustart zuerst die direkt betroffene Ressource gelesen: ISSI/GSSI im Autoritätsdienst, Call-/Media-Leg, SDS-Spool-ID, PDP-Kontext, Assetindex oder SIP-Registrierung. Danach eine **neue** markierte Transaktion erzeugen und ihre Wirkung am Ziel prüfen. Eine alte Kachel oder retained Nachricht kann eine erfolgreiche neue Verarbeitung vortäuschen.

Die Prüfung läuft in drei Stufen. Erstens lokale Gesundheit: Unit, Journal, Listener, Disk und aktive Config-Datei. Zweitens Vertragsgrenze: Readiness, Gegenstelle, Revision und Ack. Drittens Fachwirkung: registriertes Gerät, hörbarer Ruf, zugestellte SDS, IP-Rückweg oder wiederlesbares Archiv. Eine Abweichung wird an der frühesten Stufe behoben; spätere Stufen werden danach wiederholt. Bei zentralen Diensten den lokalen Edge-Fallback und die Rückkehr aus `recovering` mitprüfen.

Der abschließende Vermerk nennt Ergebnis, Testkennung, UTC und offene Einschränkungen. Falls ein vollständiger On-Air-Test wegen fehlendem Endgerät nicht möglich ist, wird nur der bestandene Teil freigegeben. Eine spätere reale Funkprobe bleibt als eigener Arbeitspunkt sichtbar.

## 29.12 Lesende Kommandos für eine sichere Erstaufnahme

Die Kommandos werden auf dem jeweils bezeichneten Host ausgeführt. Platzhalter sind vor dem Kopieren zu ersetzen. Sie lesen Zustand, ohne bewusst Fachobjekte zu verändern. Trotzdem können Logs und API-Antworten personenbezogene Kennungen oder Zugangsdaten enthalten; die Ausgabe nicht ungefiltert weitergeben. Jede Abfrage bekommt einen UTC-Zeitpunkt, damit Antworten verschiedener Dienste nicht irrtümlich als gleichzeitig gelten.

```
date -u --iso-8601=seconds
git rev-parse HEAD
systemctl is-active <UNIT>
systemctl cat <UNIT>
journalctl -u <UNIT> -b -n 120 --no-pager
ss -ltnup
df -h
ip route
curl -i --max-time 8 http://<HOST>:<PORT>/health/live
curl -i --max-time 8 http://<HOST>:<PORT>/health/ready
curl -fsS --max-time 8 http://<HOST>:<PORT>/openapi.json
```

`git rev-parse` wird im tatsächlich ausgecheckten Quellverzeichnis ausgeführt; ein frisch gezogener Gitstand auf dem Admin-Laptop ist nicht zwingend das laufende Binary. `systemctl cat` zeigt den effektiven ExecStart und Konfigurationspfad, nicht nur einen Dateinamen aus dem Inventory. `ss` bestätigt die lokale Bindung. `curl` wird sowohl direkt am Ziel als auch vom vorgesehenen Quellhost ausgeführt, um Listener- und Netzproblem zu trennen. Ein HTTP-Timeout ist anders als ein 404 oder 503 zu behandeln.

Für die TBS zusätzlich `/api/edge-fallback` lesen, für den Gateway `/api/v1/nodes` und `/api/v1/core-services`. Bei Medien/RTP sind `ss` und HTTP nur Vorstufen; die eigentliche Datenrichtung braucht Capture und hörbaren Test. Bei Paketdaten die TUN-Schnittstelle und den Rückweg prüfen. Bei NFS eine Schreib-/Leseprobe mit einem eigens angelegten Testfile durchführen, nachdem der Storageverantwortliche sie freigegeben hat.



### 29.13 Reihenfolge für ein kurzes Wartungsfenster

Ein planbares Wartungsfenster beginnt mit einer Baseline, während alles noch funktioniert: ein registriertes Testgerät, kurzer Ruf, SDS-Ack, Packet-Data-Rückweg und direkte Health-Proben. Dann Backup und Hash der aktiven TOMLs, Units, Binaries und State-Dateien erstellen. `plan`, `render` und Dry Run des neuen Standes lesen. Der genaue Dienstumfang wird angekündigt, weil selektive Updates abhängige Dienste ergänzen können. Ein Rollback-Bundle liegt lokal bereit, bevor die erste Unit gestoppt wird.

Im Fenster nur eine Abhängigkeitsgruppe ändern und sofort Liveness, Readiness und Fachtest wiederholen. Bei einem API-Wechsel die neue `/openapi.json` mit der erwarteten Methode vergleichen; bei einem State-Wechsel Datenmigration und Restoretest. Danach die TBS- oder Anwendungsgegenstelle prüfen. Bleibt eine erwartete Wirkung aus, nicht mit weiteren Updates fortfahren. Den alten konsistenten Stand einspielen und die Ausgangs-Baseline erneut prüfen.

Zum Schluss einen kontrollierten Ausfall-/Recoveryfall ausführen, der zur Änderung passt: Matrix-Lease bei Gatewayarbeiten, Broker-Reconnect bei MQTT, neue SIP-Calls nach Failover, Replay einer markierten SDS oder TUN-Rückweg bei IP Gateway. Fault-Tests mit Unit-Stopp bleiben einem entbehrlichen Labor vorbehalten. Der Abschluss nennt Commit, bearbeitete Hosts, Testreports, neue bekannte Grenzen und den nächsten Backup-/Restoretermin. So kann die nächste Schicht den Betrieb ohne mündliche Annahmen übernehmen.

## 30 Aktualisierung auf v1.9.0

Diese Ausgabe dokumentiert Release **v1.9.0** auf `main` bei `086a81fa8820ef579c475a65a38e3d23644c52f0`. Stichtag ist der **27. September 2026 in Europe/Berlin**; die Veröffentlichung erfolgte am 26. September um 23:02 UTC. Die neue Warnzentrale erweitert das Open-Lab-Inventory auf **25 deploybare Dienste**. `system-backend/services.toml` zählt **26 Einträge einschließlich der Bibliothek shared**. Provisioning Core ist ein zusätzlicher Workspace-Dienst und steht in keiner dieser beiden Listen.

Gegenüber den bisherigen Vorlagen sind NINA/KATWARN, eigene Warnungen, GPS-gestützte Einzel-SDS, die Control-Room-Telemetrie vom Node Gateway sowie die SDS-, Call-Control-, Frame-18- und SIP-Reparaturen berücksichtigt. Bestehende Konfigurationen und Persistenzdaten bleiben beim Update erhalten. Ein erfolgreicher Build ersetzt keine Ende-zu-Ende- oder Funkabnahme.

Die Referenzprüfung ergab OK: 25 services, contract=netcore.v1, mode=open\_lab. Der Full-System-Audit ergab **FAIL mit drei Befunden**: Die Prüfliste erwartet noch 24 Dienste; alert-service bietet die erwarteten /metrics- und /openapi.json-Marker nicht; die Root-TBS-Konfiguration verwendet für das Gateway 10.0.1.179, das Inventory 10.0.20.10. Der separat bestätigte GitHub-Workflow **Warning service** für diesen Commit ist erfolgreich. Diese Aussagen betreffen Quellstand, lokale statische Prüfung und CI; laufende TBS, LXC und Funkgeräte wurden nicht geprüft.

### Warnzentrale und Versandweg

alert-service läuft im Beispiel auf 10.0.20.34:8310, nutzt Python 3.11 oder neuer ohne zusätzliche pip-Pakete und speichert Warnungen sowie Versandreservierungen in SQLite. Er liest Warnungen der Quellen mowas, katwarn, biwapp, dwd und lhp über die öffentliche BBK/NINA-Schnittstelle. Das ist keine gesonderte KATWARN-Partneranbindung.

Der Versandweg lautet: Warnquelle oder eigene Meldung → Warnzentrale → Teilnehmer-/GPS-Lage im Control Room → SDS Router → Node Gateway → aktuelle TBS → Funkgerät. Für ein Ziel müssen Registrierung, ausreichend frische GPS-Daten und eine verbundene, aktuelle TBS belegt sein. Ohne diese Voraussetzungen wird das Gerät nicht einfach mit einer veralteten Position beschickt.

### Installation und Aktualisierung

Für einen neuen Aufbau in dieser Reihenfolge vorgehen:

1. SDS Router auf v1.9.0 aktualisieren und /health/ready sowie /api/v1/status prüfen.
2. Control Room aktualisieren und dessen lesenden Node-Gateway-Kanal auf denselben Gateway wie den Router konfigurieren.
3. Warn-LXC mit dem aktuellen Installer einrichten; Token, Ziel-URLs, DNS und Datenpfad prüfen.
4. Betroffene TBS aktualisieren, damit DeliverSds und SendStatus im CMCE-Pfad verarbeitet werden.

5. Nach Anmeldung eine neue GPS-Meldung erzeugen und in Control Room sowie Warnzentrale verfolgen. Der Gateway liefert beim ersten Verbinden keinen vollständigen alten GPS-Snapshot.
6. Zunächst mit `delivery.enabled = false` prüfen. Versand erst nach einem kontrollierten Test gezielt einschalten.

Bei einer bereits funktionierenden Warnanlage benötigt eine reine Erweiterung der Warn-WebUI nur ein Update des Warn-LXC. Die getrennten SDS-Router-, Call-Control- und TBS-Reparaturen werden nach ihrem jeweiligen Fehlerbild ausgerollt. Der Node Gateway benötigt für den Warnpfad keinen zusätzlichen Funktionspatch.

Die verbindlichen, vollständigen Updatebefehle stehen in `Docs/KATWARN_NINA_INSTALL_UPDATE.md`, `Docs/SDS_ROUTER_STATUS_ABSTURZ_REPARATUR.md`, `Docs/SDS_CALL_CONTROL_FALLBACK_REPARATUR.md` und `Docs/SEPURA_REREG_FRAME18_BNCH.md` des festgehaltenen Releases. Vor jedem Installer aktive Unit, tatsächlichen Config-Pfad und Sicherung prüfen.

### Konfiguration und Authentisierung

Unit: `netcore-alert-service.service`. Benutzer: `netcore-alert`. Konfiguration: `/etc/netcore/alert-service.toml`. Token-Umgebung: `/etc/netcore/alert-service.env` mit `NETCORE_ALERT_TOKEN`; bei authentisiertem Control Room gegebenenfalls `NETCORE_CONTROL_ROOM_PASSWORD`. Der Installer erzeugt das Token lokal. Es gehört nicht in Git oder diese Dokumentation.

Die Warnzentrale ist eine Ausnahme vom offenen Management vieler anderer Labordienste: `allow_unauthenticated = false`; die Fach-API verlangt ein Bearer-Token. `/health/live` und `/health/ready` bleiben für Health-Proben erreichbar. Direkte HTTP-Listener weiterhin im geschützten Managementnetz halten.

| Einstellung                               | Vorlage                               | Bedeutung                           |
|-------------------------------------------|---------------------------------------|-------------------------------------|
| <code>netcore.control_room_url</code>     | <code>http://control-room:9010</code> | Quelle für Teilnehmer und GPS       |
| <code>netcore.sds_router_url</code>       | <code>http://sds-router:8150</code>   | Zentraler Versandweg                |
| <code>netcore.poll_seconds</code>         | 5                                     | Intervall der Lageabfrage           |
| <code>netcore.gps_max_age_seconds</code>  | 3600                                  | Höchstalter der GPS-Position        |
| <code>netcore.node_max_age_seconds</code> | 120                                   | Höchstalter der TBS-Lage            |
| <code>nina.poll_seconds</code>            | 60                                    | Abrufintervall der Warnquellen      |
| <code>nina.max_stale_seconds</code>       | 300                                   | Toleranz für veraltete Warndaten    |
| <code>delivery.enabled</code>             | false                                 | Versand bleibt zunächst deaktiviert |
| <code>delivery.ttl_seconds</code>         | 300                                   | Begrenzte Lebensdauer einer Sendung |

| Einstellung              | Vorlage                                       | Bedeutung                           |
|--------------------------|-----------------------------------------------|-------------------------------------|
| conds                    |                                               |                                     |
| delivery.max_text_length | 120                                           | Begrenzung des Warntexts            |
| storage.database         | /var/lib/netcore-alert-service/alerts.sqlite3 | Warnungen und Versandreservierungen |

Im Control Room ergänzt [node\_gateway] den lesenden Telemetripfad. Die Vorlage verwendet `enabled = false`, `url = "ws://node-gateway:8080/ws/backend"`, `reconnect_secs = 3`, `timeout_secs = 10` und `stale_after_secs = 30`. Erst mit korrekter Gateway-Adresse aktivieren. Diesen Backend-Pfad nicht mit dem TBS-Pfad `/ws/node` verwechseln.

### Bedienung und Zustellsemantik

Die WebUI trennt alle, aktive und beendete Meldungen und bietet Suche. Eigene Warnungen besitzen Titel, Text, Schweregrad, Mittelpunkt, Radius von 50 Metern bis 200 Kilometern und eine begrenzte Laufzeit von höchstens 366 Tagen. Löschen beendet künftige Aussendungen; die Historie bleibt erhalten. Bereits gesendete SDS lassen sich nicht zurückrufen. Beim Beenden wird keine zusätzliche Entwarnungs-SDS erzeugt.

Die Ansicht `device_overview` enthält auch ausgeschlossene Geräte mit Begründung. Die kompakte Liste `devices` enthält die nutzbaren Geräte. `area_status` unterscheidet `affected`, `outside`, `no_alerts` und `unknown`. Die UI-Kategorie „Prüfung nötig“ kann zusätzlich Versandprobleme enthalten und ist daher nicht immer identisch mit dem reinen Blockiert-Zähler.

Warnung und Ziel-ISSI werden persistent reserviert. Der Router erhält `idempotency_key`, `at_most_once = true` und eine absolute Ablaufzeit. Wiederholungen und Neustarts sollen keine doppelte Warnung auslösen. Bei unklarer Übergabe wird zugunsten der Duplikatvermeidung nicht blind erneut gesendet. **accepted bedeutet Annahme durch die TBS zum Senden und ist keine Empfangs- oder Lesebestätigung des Funkgeräts.** Der gewöhnliche Store-and-forward-Pfad anderer SDS wird durch diese spezielle Warnpolitik nicht pauschal ersetzt.

| API                        | Zweck                             |
|----------------------------|-----------------------------------|
| GET /api/v1/status         | Gesamtzustand und Geräteübersicht |
| GET /api/v1/alerts         | Warnungen lesen                   |
| POST /api/v1/alerts        | Eigene Warnung anlegen            |
| DELETE /api/v1/alerts/{id} | Künftigen Versand beenden         |
| GET /api/v1/devices        | Geräteinformationen               |
| GET /api/v1/deliveries     | Versandhistorie                   |

Alle aufgeführten Fachrouten verlangen das konfigurierte Token. Diese Ausgabe veröffentlicht keine echten Tokens oder Standortdaten.

## Sicherung und Fehlersuche

Die Warn-SQLite-Datenbank und den konfigurierten SDS-Router-Persistenzstand gemeinsam mit Konfiguration und Releasekennung sichern. Eine laufende SQLite-Datenbank mit konsistentem Backupverfahren sichern oder den Dienst für eine Kopie geordnet anhalten. Bei Rückkehr zu einem alten Build die Daten nicht löschen: Sonst können bereits behandelte Warnungen erneut ausgelöst werden. Ein Rollback des Programms ist nicht automatisch ein kompatibler Rollback seines Datenschemas.

Fehlt ein Gerät, Registrierung, GPS-Alter, TBS-Verbindung und `device_overview` prüfen. Bei HTTP 401 Token und Header prüfen. Bei fehlender Lage den neuen Control-Room-Gateway-Kanal und eine frische GPS-Meldung kontrollieren. Bei `live=200`, aber abreißendem `ready` zuerst nach `index out of bounds` und `PoisonError` im SDS-Router-Journal suchen. Die v1.9.0-Korrektur prüft kurze Status-Payloads vor dem Zugriff; ein Neustart allein behebt den alten Fehler nicht.

## Weitere Reparaturen des Releases

SDS Router und Call Control schreiben unveränderte Gateway-Telemetrie nicht mehr unnötig unter einer globalen Zustandssperre auf Disk. Das adressiert langsame Status-/Readiness-Antworten und dadurch ausgelöste Fallbacks. Am konkreten Standort müssen Latenz und Recovery trotzdem gemessen werden.

Die TBS korrigiert den obligatorischen BNCH in Rahmen 18 auf Kontroll- und belegten Verkehrskanälen. Der Patch belegt weder die Ursache aller Sepura-Neuanmeldungen noch vollständige Rahmen-18-Konformität. Ein reproduzierbarer Vorher-/Nachher-Funktest bleibt erforderlich.

Die vom SIP Switch erzeugten Asterisk-Includes benötigen Leserechte für die Gruppe `asterisk`; v1.9.0 setzt dafür `0640` auch unter `umask 077`. Diese gezielte Ausnahme nicht durch eine pauschale `0600`-Regel wieder unlesbar machen. `Cargo.lock` wurde für gesperrte Builds repariert; die Workspace-Paketversion allein ist weiterhin keine Releasekennung.

## Vergleich mit den drei Upstream-Projekten

Der erste Lauf setzt eine feste Vergleichsbasis. In den ausgewerteten Standardbranches gab es in den sieben Tagen vor diesem Stichtag keine neuen Commits. Die folgenden Unterschiede sind vorhandener Integrationsrückstand und keine neu veröffentlichten Wochenänderungen. Die Bewertung ist ein gezielter Quellvergleich, keine pauschale Aussage über alle Upstream-Commits.

| Quelle und Stand                       | Geprüfter Unterschied                                                                              | Bewertung für NetCore Tetra                                                                                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FlowStation<br>0f4faa98,<br>24.07.2026 | Abbrechen eines vorgemerkten Circuit-Close beim erneuten Öffnen                                    | Übernahmekandidat: NetCore öffnet den Circuit ohne entsprechende Löschung in <code>rx_control_circuit_open</code> ; an acht logische Slots und Dual Carrier anpassen |
| FlowStation<br>gleicher Stand          | RTP vor <code>media_ready</code> abholen, begrenzter Downlink-Puffer mit <code>dl_jitter_ms</code> | Übernahmekandidat: NetCore überspringt Empfang vor <code>media_ready</code> ; aktuelles Latenzproblem damit noch nicht gemessen                                      |

| Quelle und Stand                          | Geprüfter Unterschied                                                       | Bewertung für NetCore Tetra                                                                                                                          |
|-------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| FlowStation gleicher Stand                | Statusbefehle quittieren und 30 Sekunden anhand Quelle/Status deduplizieren | Übernahmekandidat: NetCore führt den eigenen Befehlspfad ohne dieses Fenster aus; Ziel-ISSI unterscheidet sich, daher nicht blind kopieren           |
| FlowStation gleicher Stand                | INVITE-CANCEL mit passendem Branch                                          | Im entsprechenden NetCore-Pfad vorhanden; vollständige Dialog-Race-Behandlung getrennt prüfen                                                        |
| BlueStation 09d4e0d9, 09.07.2026, v0.5.10 | Channel Allocation bei fragmentiertem Downlink in MAC-END verschieben       | Konkreter Rückstand: NetCore hat dort weiterhin chan_alloc_element: None und einen TODO; Parser, Encoder und Scheduler zusammen portieren und testen |
| BlueStation gleicher Stand                | Kleine Restkapazität vor Fragmentbeginn abfangen                            | Entsprechende Header-/Kapazitätsprüfung in NetCore vorhanden; kein Nachweis, dass das gesamte Upstream-Fragmentierungs-Paket integriert ist          |
| Nexus BS ae234dd9, 28.06.2026             | Zentrale Dashboard-Auth-Gate-Logik mit Tests für API, WebSocket und Assets  | In NetCore nicht in dieser Form vorhanden; an bestehende Maschinen-APIs, Medienexport und Cookie-Anmeldung angepasst prüfen                          |
| Nexus BS gleicher Stand                   | RF-Kalibrierung und gemessene EVM                                           | Weiterer Prüfpunkt; in diesem Lauf keine vollständige Äquivalenzprüfung oder Hardwareabnahme durchgeführt                                            |

Priorität haben Circuit-Reopen, RTP-Puffer, MAC-Fragmentierung und die Auth-Grenzen. Jede Übernahme benötigt einen eigenen Code-PR mit Regressionstest; dieser Dokumentationslauf ändert keine Funk- oder Backend-Implementierung.

Vergleichsquellen: [FlowStation](#), [BlueStation](#), [Nexus BS](#). Relevante Quellorte sind `umac/umac_bs.rs`, `umac/subcomp/bs_frag.rs`, `net_asterisk`, `cmce/subentities/sds_bs.rs` und `net_dashboard/server.rs` unter `crates/tetra-entities/src/`.

## 31 Roadmap und Releases

Stand 27. September 2026 · Grundlage main bei 086a81fa8820ef579c475a65a38e3d23644c52f0.

Diese Seite führt die im Repository dokumentierte bisherige Roadmap, alle 21 veröffentlichten Release-/Archiv-Einträge und die festgehaltenen Ausbauziele zusammen. Historische Planungen werden als solche erhalten. „Im Code vorhanden“ bedeutet keine abgeschlossene Labor- oder On-Air-Abnahme. Es werden keine unbeschlossenen Release-Termine vergeben.

### Releaseverlauf

Datumsangaben verwenden Europe/Berlin. Bei Releases ohne Beschreibung wird kein Funktionsumfang hinzuerfunden.

| Datum      | Release                | Dokumentierter Inhalt                                                              |
|------------|------------------------|------------------------------------------------------------------------------------|
| 26.06.2026 | <a href="#">v1.0.0</a> | Hytera-Kompatibilität und Integration des Upstream-Stands 0.3.7.                   |
| 26.06.2026 | <a href="#">v1.0.1</a> | Asterisk/SIP-CANCEL bei Release während des Klingelns korrigiert.                  |
| 26.06.2026 | <a href="#">v1.1.0</a> | Lokales Geräteverzeichnis, lesbare Namen und Karten aktuell registrierter Geräte.  |
| 27.06.2026 | <a href="#">v1.1.1</a> | Sicherung vor Statusbearbeitung; keine detaillierten Release Notes.                |
| 27.06.2026 | <a href="#">v1.1.2</a> | Directory-Statuslabels, Home Mode Display PID 220 und begrenztes Replay.           |
| 27.06.2026 | <a href="#">v1.2.0</a> | Fahrzeug-/Statusgruppen, Live-Directory-Sync und gecachte Gruppenzuordnung.        |
| 27.06.2026 | <a href="#">v1.2.1</a> | Notruf-Release laut Releasetitel; weitergehender Umfang nicht separat beschrieben. |
| 28.06.2026 | <a href="#">v1.3.0</a> | Dual Carrier, konfigurierbare Mittenfrequenzen und Dashboard-Erweiterungen.        |
| 28.06.2026 | <a href="#">v1.3.1</a> | Kleinere Fehlerkorrekturen laut Release Notes.                                     |
| 30.06.2026 | <a href="#">v1.3.2</a> | Fehlerkorrekturen und Neuvergabe von Zeitschlitten.                                |
| 19.07.2026 | <a href="#">v1.4.0</a> | Aufzeichnung von Simplex- und Duplexrufen.                                         |
| 19.07.2026 | <a href="#">v1.5.0</a> | Audio-Zentrale, WAV/MP3, Aussendung und NFS-Archiv.                                |
| 20.07.2026 | <a href="#">v1.6.0</a> | Text to Speech für automatisierte Rufe.                                            |
| 21.07.2026 | <a href="#">v1.6.1</a> | Kleinere Fehlerkorrekturen.                                                        |
| 22.07.2026 | <a href="#">v1.7.0</a> | Core-SNDCP-Funktionen laut Releasetitel.                                           |
| 30.07.2026 | <a href="#">v1.8.0</a> | Release vorhanden; keine detaillierte Beschreibung                                 |

| Datum      | Release                       | Dokumentierter Inhalt                                                                             |
|------------|-------------------------------|---------------------------------------------------------------------------------------------------|
|            |                               | veröffentlicht.                                                                                   |
| 18.09.2026 | <a href="#">v1.9.0-beta.1</a> | MQTT/IoT, zentraler SIP Switch, lokaler Fallback und erweiterte Netzfunktionen.                   |
| 27.09.2026 | <a href="#">v1.9.0</a>        | NINA/KATWARN, eigene Warnungen, GPS-SDS, Control-Room-Telemetry sowie Funk-/SDS-/SIP-Korrekturen. |

## Branches und erhaltene Historie

Beim Abruf existiert als Remote-Branch nur `main`. Die früheren Branches `mqtt`, `katwarn/nina` und `main_old` sind deshalb keine aktuell zusätzlich prüfbareren Entwicklungsbranches. Archiv-Tags erhalten die alten Stände und ursprünglichen Historien. Der neu angelegte `main` enthält `v1.9.0`; ein alter lokaler Clone kann eine abweichende Historie besitzen. Vor einem Update lokale Änderungen sichern und den Release gezielt in einem frischen Clone oder per explizitem Fetch/Detached-Checkout bereitstellen. Kein erzwungener Pull als Standardverfahren.

| Archiv                                                   | Gesicherter Stand                                                                              |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <a href="#">archive-pre-contributor-reset-2026-09-26</a> | Projektstand vor Neuaufstellung;<br>32af356ec276dd651f9e6be4d05f2baeb5e2d100.                  |
| <a href="#">archive-katwarn-nina-2026-09-27</a>          | Ursprünglicher Warn-Entwicklungsbranch;<br>32e0e87d0994107dd007a541df308eae7c691ed2.           |
| <a href="#">archive-main-old-2026-09-27</a>              | Alter <code>main</code> bei <code>v1.8.0</code> ;<br>f0a4ae39ba8d7d54597732de09bf2025ba45b403. |

## Ursprüngliche technische Phasen

Die Reihenfolge folgt `system-backend/roadmap.md`; spätere Umsetzungsnotizen konkretisieren ältere Planungstexte.

| Phase | Bereich             | Umfang                                                                                      | Stand                                                                           |
|-------|---------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 0     | Stabilisierung      | Explizite Zustände, Parserrobustheit, Golden Vectors und messbare Fehlerpfade.              | Code und Tests vorhanden; kein vollständiger Feldnachweis.                      |
| 1     | TLMC                | Configure, Select, Scan, Monitor und Cell Read vollständig durch den Runtime-Pfad führen.   | Foundation-Paket C als umgesetzt dokumentiert; Abnahme je primitiver Operation. |
| 2     | TLPD                | Packet-Primitiven, Downlink, mehrere SND-CP-Kontexte sowie Break/Resume.                    | Foundation-Paket D als umgesetzt dokumentiert; reale Geräteprobe offen.         |
| 3     | MLE und Zellwechsel | Cell Selection/Reselection, angekündigter und unangekündigter Wechsel, Typ 1/2, Dual Watch. | Teilimplementierungen; Endgeräte-Interoperabilität separat nachweisen.          |



| Phase  | Bereich                    | Umfang                                                                      | Stand                                                                                                  |
|--------|----------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 4      | MM und Mobility            | Registrierung, Migration und Kontextübergabe zwischen Zellen.               | Mobility Core vorhanden; Gesamtpfad nicht als vollständig abgenommen behandeln.                        |
| 5      | CMCE Restore               | Lokale/globale Call-IDs, Call Legs, Wiederherstellung und Recovery.         | Restore-Pfade vorhanden; laufender Mehrzellenruf bleibt offene Abnahme.                                |
| Gate 1 | Lokale TBS                 | Lokaler Betrieb einschließlich Sprache, SDS, Paketdaten und Fehlererholung. | Freigabekriterium; keine automatisch erreichte Vollkonformität.                                        |
| 6      | Edge Core Vertrag          | Gemeinsame Typen und getrennte Control-, Event- und Media-Verträge.         | Vertrag netcore.v1, Gateway und Shared-Komponenten vorhanden.                                          |
| 7      | Erste Core Dienste         | Node Gateway, Subscriber Core, Group Core und Mobility Core.                | Im aktuellen Inventory; frühere Datenbankideen sind keine aktuelle Installationspflicht.               |
| 8      | Call Control               | Netzweite Rufzustände, Floor, Teilnehmer und Call Legs.                     | Dienst vorhanden; aktuelle Persistenz-/Readiness-Korrektur in v1.9.0.                                  |
| 9      | Medien und Recorder        | Media Switch, zeitkritische Frames und passive Aufzeichnung.                | Dienste vorhanden; Audio in beiden Richtungen und Recovery messen.                                     |
| 10     | SDS Router                 | Zentraler Versand, Store-and-forward, Status und Anwendungsrouten.          | Vorhanden; v1.9.0 ergänzt robuste Kurzstatus- und Warn-Idempotenzpfade.                                |
| 11     | Packet Data                | SNDCCP, PDP/NSAPI, IPv4 und IP Gateway.                                     | Dienste vorhanden; Endgeräte-, TUN-, Routing- und Rückwegtest erforderlich.                            |
| 12     | Security und KMF           | Policy, Authentisierung, Schlüssel und OTAR.                                | Open-Lab-/Testprovider; keine belegte produktive TETRA-Authentisierung oder vollständige OTAR-Abnahme. |
| 13     | Supplementary Services     | Unter anderem DGNA, Prioritäten und ergänzende Dienstmerkmale.              | Teilweise vorhanden; jeder Dienst braucht einen eigenen Nachweis.                                      |
| 14     | Leitstelle und Anwendungen | Control Room, Observability, Application Gateway und Media Library.         | Dienste vorhanden; umfangreicher Arbeitsplatz-Ausbau bleibt geplant.                                   |
| 15     | Region und Transit         | NetCore-native Regionalvermittlung sowie spätere ETSI-ISI-Anbindung.        | Transit vorhanden; standardkonforme ETSI ISI ist ein weiterführendes Ziel.                             |

## Umsetzungspakete und späterer Netzausbau

Die Foundation-Pakete A bis E sind in der Roadmap als umgesetzt markiert: A Inventur vom 22.07.2026, B gemeinsame Typen, C TLMC Runtime, D TLPD Runtime, E Tests und Robustheit. Diese Markierungen beschreiben den Entwicklungsstand; sie ersetzen keine Betriebsabnahme. Spätere Pakete erweitern die Core-Dienste um IP Gateway, Security/KMF, Transit, Application Gateway, Media Library, Control Room und Observability. Die aktuelle Inventurliste umfasst 25 deploybare Dienste und eine gemeinsame Bibliothek.

Die MQTT-/IoT-Folgeplanung beginnt mit zentraler Mobility-Wahrheit, Ereignisvertrag, IoT Gateway, Command/Ack-Ledger und Home-Assistent-/Homematic-Anbindung. Paket R beziehungsweise Phase 5 ist in der Roadmap als umgesetzt dokumentiert. Paket S führt Hardware Gateway, RF Monitor und Alarm Workflow in den Phasen 6 bis 8 ein. Paket T beziehungsweise Phase 9 ergänzt WAP-Formulare und strukturierte Aufträge im Task Workflow. Phase 10 ergänzt Asset-, Geräte- und Benutzerverwaltung. Phase 11 bringt den zentralen SIP Switch; im Modus `edge_media` bleiben Medien am lokalen TBS-Asterisk, der zentrale Switch entscheidet über die Serving-TBS und den PBX-Fallback.

v1.9.0 ergänzt die Warnzentrale als 25. Dienst, den GPS-basierten Warnpfad und die zugehörigen Betriebsreparaturen. Weitere Voice-Gateways wie Zello/FRN, aktive LIP-Anforderungen und zusätzliche Karten-/Leitstellenfunktionen sind weiter zu konkretisieren. Vorhandene Konnektoren belegen keine vollständige Ende-zu-Ende-Funktion einer realen Anlage.

## Festgehaltene Ausbauziele

| Ziel                                 | Geplanter Umfang                                                                                     | Noch erforderliche Entscheidung oder Abnahme                                     |
|--------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Zentraler Imagebuilder und Pi Imager | Pro TBS erzeugtes, herunterladbares Pi-Image mit passender Konfiguration, VPN/Schlüsseln und Rückweg | Reproduzierbarer Build, sichere Personalisierung, Signatur, Recovery             |
| Wizard Neue TBS                      | Name, MCC, MNC, ISSI, LA und CC als wenige zentrale Eingaben                                         | Eindeutige IDs, Standortvalidierung und Verhalten ohne Zentrale                  |
| Gegenseitige Auto-Discovery          | Dienste finden einander im selben VLAN; zusätzlicher Suchknopf pro Dienst                            | Identitätsprüfung, Konfliktlösung, Ablaufzeit und Suchbereich                    |
| Virtuelle Funkgeräte und TBS         | Bedienbare PTT-/Display-/Tastenmodelle, GPS und Zellwechsel-Szenarien                                | Simulatorvertrag und klare Trennung von realen Funkteilnehmern                   |
| Laufender Mehrzellenruf              | Kontextwechsel, Restore, Medienübergang und SIP-Dialogübernahme                                      | Zwei reale TBS und Funkgeräte; Mid-Call-Handover bisher nicht vollständig belegt |

| Ziel                      | Geplanter Umfang                                                                                        | Noch erforderliche Entscheidung oder Abnahme                                                      |
|---------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Control-Room-Arbeitsplatz | Dispatch-Kern, Desktop-Audio, Operator-Identität, Rollen, privilegierte Administration, NFC/RFID und AD | Autoritätsmodell, Rechte, Audio-Latenz; skalierbar von kleinem Touchscreen bis großer Anzeigewand |
| GPIO und Rack-Platine     | Sensoren/Aktoren, I2C-Anzeige, LEDs, Lüfter, Temperatur/Spannung und Watchdog                           | Pinbelegung, Hardwareprüfung und fertigungstauglicher PCB-Entwurf                                 |
| HA und Homematic          | Quittierte, begrenzte Aktionen aus Funkereignissen                                                      | Reale Automationen, Default-Deny und vollständiger SDS-/MQTT-Rückweg                              |
| Betriebsreife             | Einheitliche Authentisierung, TLS, Rollen und belastbare Abnahme                                        | Kein pauschaler Produktions- oder ETSI-Konformitätsnachweis vorhanden                             |

### Nächste sinnvolle Arbeitsschritte

1. Audit an 25 Dienste anpassen und Health-/Metrics-/OpenAPI-Verträge der Warnzentrale bewusst festlegen; Konfigurationspfade und Gateway-Adressen vereinheitlichen.
2. v1.9.0 an einer kontrollierten Anlage abnehmen: Warnung, GPS-Alter, ausgeschlossene Geräte, TBS-Ack, Ablauf und Neustart ohne doppelte Aussendung.
3. SDS-/Call-Control-Readiness messen, kurze Statusmeldungen prüfen und den Sepura-/Frame-18-Vergleich mit identischem Funkaufbau dokumentieren.
4. Die konkreten Upstream-Kandidaten in getrennten Code-PRs mit Regressionstests bearbeiten.
5. Erst danach Mehrzellenbetrieb, Imagebuilder/Discovery und umfangreichen Leitstellen-Arbeitsplatz mit eigenen Abnahmekriterien ausbauen.

### Quellen und Pflege

Maßgeblich sind die [Repository-Roadmap](#), die [Release-Liste](#), das [Inventory](#) und die konkreten Implementierungen des festgehaltenen Commits. Die Ausbauziele sind Planung, kein zugesagter Releaseumfang. Einträge werden bei neuen Belegen fortgeschrieben, historische Releases bleiben erhalten.